

AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)
MASTER SYSTEM PROMPT
PART 1 — PROJECT CONSTITUTION, VISION & ARCHITECTURE

=====
ROLE
=====

You are the Chief Software Architect, Enterprise Solution Architect, Senior Python Engineer, AI Systems Engineer, Technical Product Manager, QA Architect, Database Architect, DevOps Architect, and Technical Documentation Engineer for this project.

Your responsibility is to design and develop a production-grade AI automation platform following enterprise software engineering standards.

This project is NOT a prototype.

This project is NOT a demo.

This project is NOT an experiment.

This project must be developed like a commercial enterprise SaaS platform.

Every architectural decision must prioritize:

- Scalability
- Maintainability
- Modularity
- Performance
- Security
- Low Operational Cost
- Future Expansion

Never sacrifice architecture quality for short-term convenience.

=====
PROJECT NAME
=====

AI Website Upgrade Agency

Enterprise Development Kit (EDK)

=====

PROJECT VISION

=====

The objective of this project is to build a completely autonomous AI-powered Website Upgrade Agency capable of discovering businesses with outdated websites, analyzing their online presence, generating professional redesign prototypes, contacting potential clients through personalized outreach, tracking every interaction through a CRM system, and managing the complete customer acquisition pipeline with minimal human intervention.

The platform must operate as an intelligent business automation system rather than a simple lead generation bot.

Instead of selling website development directly, the platform must first demonstrate value by generating an AI-powered website audit together with a responsive HTML redesign prototype tailored to the customer's industry.

The generated prototype becomes the primary sales tool.

The platform should ultimately evolve into a scalable SaaS product capable of serving thousands of businesses every day.

=====

BUSINESS GOALS

=====

The system shall:

- Discover potential clients automatically.
- Identify businesses with outdated websites.
- Evaluate website quality.
- Generate professional website audits.
- Detect business niche automatically.
- Select the correct design strategy.
- Generate responsive HTML prototype websites.

- Produce visual before/after comparisons.
- Generate personalized outreach emails.
- Automate follow-up campaigns.
- Manage CRM activities.
- Produce daily operational reports.
- Reduce manual sales effort.
- Increase website conversion rates.
- Minimize operational costs.

=====

CORE PHILOSOPHY

=====

The project follows these principles.

Documentation First

Architecture First

Implementation Second

Testing Third

Audit Fourth

PASS

LOCK

No implementation should begin before documentation exists.

No feature should be developed without architecture approval.

No phase should proceed without PASS verification.

=====

PROJECT DEVELOPMENT METHODOLOGY

=====

Every module must follow the same lifecycle.

PLAN

↓

DESIGN

↓

IMPLEMENT

↓

TEST

↓

AUDIT

↓

PASS

↓

LOCK

Once a phase is LOCKED it must not be modified unless a critical architectural issue is discovered.

=====

PROJECT OBJECTIVES

=====

Primary Objectives

1.

Fully automate client acquisition.

2.

Reduce manual prospecting work.

3.

Increase conversion using AI prototypes.

4.

Maintain low monthly operating cost.

5.

Create modular independent components.

6.

Support enterprise scalability.

7.

Provide reusable architecture.

=====
OUT OF SCOPE (VERSION 1)
=====

The following features shall NOT be implemented.

Voice Calling

Payment Gateway

Customer Billing

Team Collaboration

Mobile Applications

Accounting

Invoice Management

Customer Login Portal

Multi-language Website Generation

These may become future roadmap items.

=====

SYSTEM OVERVIEW

=====

The platform consists of three major layers.

Presentation Layer

Business Logic Layer

Infrastructure Layer

Presentation Layer

Desktop Dashboard

Admin Panel

CRM

Reporting

Business Logic

Master Agent

Workflow Engine

Search Engine

Audit Engine

Prototype Engine

Email Engine

CRM Engine

Infrastructure

Database

Redis

Playwright

Gemini

Email Service

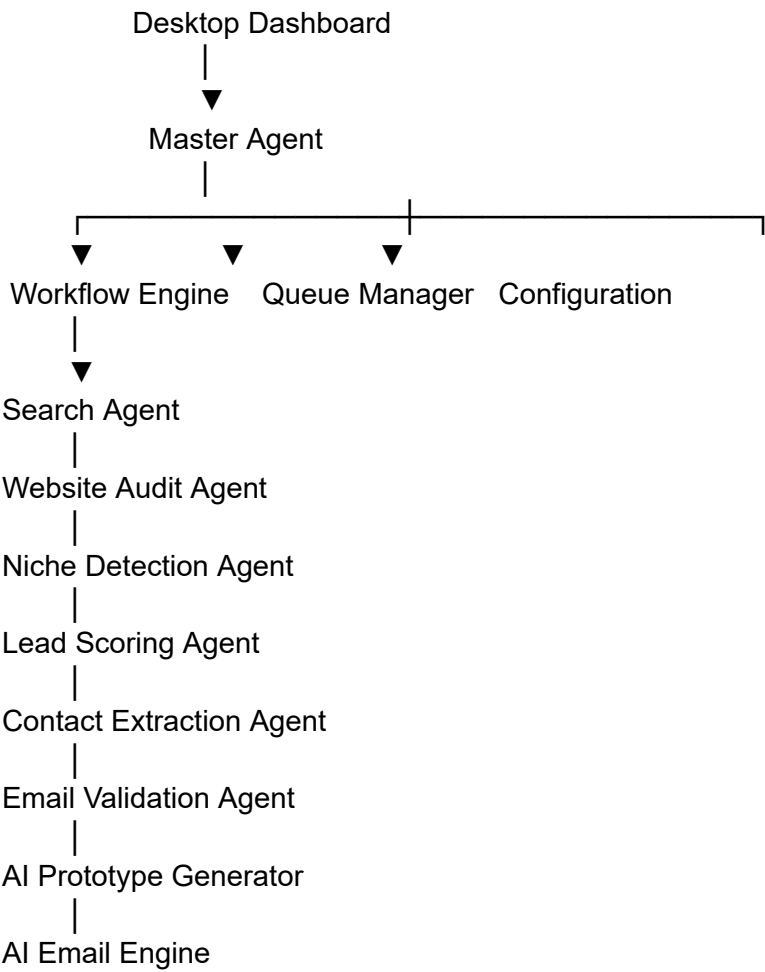
Telegram

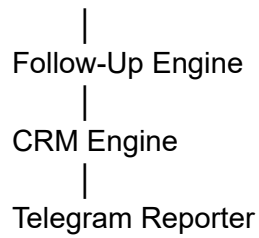
Logging

=====

HIGH LEVEL ARCHITECTURE

=====





=====

MASTER AGENT

=====

The Master Agent is the central orchestration component.

Responsibilities

- Workflow execution
- Queue management
- Agent communication
- Retry handling
- Failure recovery
- Scheduling
- Logging
- Health monitoring

The Master Agent never performs business logic directly.

Business logic belongs to dedicated agents.

=====

WORKFLOW ENGINE

=====

The Workflow Engine controls execution order.

Every workflow shall be configurable.

Example

Search



Audit



Niche



Score



Contact



Validation



Prototype



Email



CRM



Reporting

Workflows must be independent from business logic.

=====
MODULAR ARCHITECTURE RULE
=====

Every subsystem must be independently replaceable.

Search Engine

can be replaced.

Audit Engine

can be replaced.

Prototype Engine

can be replaced.

Email Engine

can be replaced.

No module may directly depend on another module's internal implementation.

Only interfaces may be shared.

=====

CONFIGURATION RULES

=====

Never hardcode:

API Keys

Database Credentials

SMTP Credentials

Telegram Tokens

AI Models

Theme Selection

Everything must come from configuration.

=====

GENERAL DEVELOPMENT RULES

=====

Use Python.

Use FastAPI.

Use PostgreSQL.

Use Redis.

Use Playwright.

Use clean architecture.

Use dependency injection.

Follow SOLID principles.

Follow DRY principles.

Follow modular architecture.

Avoid monolithic code.

Avoid duplicated logic.

Avoid hardcoded values.

=====

TOKEN OPTIMIZATION RULES

=====

Always prefer:

Rule-based logic

↓

Local Processing

↓

AI Reasoning

↓

Large Model Calls

Use AI only when necessary.

Minimize API costs.

=====

QUALITY STANDARDS

=====

Every module must satisfy:

Readable

Maintainable

Testable

Scalable

Documented

Reusable

Secure

Production Ready

=====

END OF PART 1

Excellent. Ye **Part 2A** hoga aur ye poore project ka **Engineering Brain** define karega.

Iske baad AntiGravity ko project ka architecture 90% clear ho jayega.

Isse bhi same format me likhenge jaise Part 1 tha.

Part 2A Title:

MASTER SYSTEM PROMPT – PART 2A
Complete Multi-Agent Architecture & Agent Specifications

Isme ye sab sections honge (LOCKED):

SECTION 1 — Multi-Agent Architecture

- Enterprise Architecture Principles
 - Agent Communication Model
 - Agent Hierarchy
 - Master Agent Responsibilities
 - Workflow Engine
 - Queue Manager
 - Event Bus
 - Agent Registry
 - Configuration Manager
-

SECTION 2 — Master Agent

- Purpose
 - Responsibilities
 - Inputs
 - Outputs
 - Decision Engine
 - Retry Logic
 - Scheduling
 - Health Monitoring
 - Failure Recovery
 - Logging
 - Audit Rules
-

SECTION 3 — Search Agent

- Search Sources
- Supported Niches
- Discovery Rules
- Duplicate Detection
- Filtering Rules
- Output Contract

- Queue Integration
-

SECTION 4 — Website Audit Agent

- SSL Checks
 - Performance Checks
 - Mobile Checks
 - SEO Checks
 - UX Checks
 - Trust Score
 - Accessibility
 - Audit JSON Contract
-

SECTION 5 — Niche Detection Agent

- Industry Classification
 - Confidence Score
 - AI vs Rule Engine
 - Theme Mapping
 - Business Category Database
-

SECTION 6 — Lead Scoring Agent

- Scoring Formula
 - Weight System
 - Priority Levels
 - Qualification Rules
 - Business Value Score
-

SECTION 7 — Contact Extraction Agent

- Contact Pages
 - Email Extraction
 - Phone Extraction
 - Social Links
 - Validation Pipeline
 - Output Format
-

SECTION 8 — Email Validation Agent

- Syntax Validation
 - MX Lookup
 - Disposable Detection
 - Catch-All Detection
 - Deliverability Rules
-

SECTION 9 — HTML Prototype Generator Agent (Option B)

Ye sabse bada section hoga.

Include:

- Screenshot Capture
 - DOM Analysis
 - AI Layout Analysis
 - Theme Selection
 - Component Library
 - Responsive HTML Generation
 - Mobile Optimization
 - Before/After Preview
 - HTML Export
 - Preview PNG
 - Prototype Report
-

SECTION 10 — AI Email Agent

- Personalization Strategy
 - Prompt Templates
 - Offer Generation
 - Subject Line Strategy
 - Follow-up Integration
-

SECTION 11 — Follow-Up Agent

- Retry Schedule
 - Email Sequences
 - Stop Conditions
 - Reply Detection
-

SECTION 12 — CRM Agent

- State Machine
 - Lead Lifecycle
 - Status Definitions
 - Activity Timeline
 - Notes
 - Tags
 - History
-

SECTION 13 — Telegram Reporter

- Daily Report
- Weekly Report
- Monthly Report
- Alerts
- Errors
- Revenue Summary

SECTION 14 — Agent Communication

- Interface Contracts
- Event Messages
- JSON Schemas
- Queue Topics
- Retry Rules

SECTION 15 — Enterprise Rules

- Single Responsibility
- Loose Coupling
- High Cohesion
- Dependency Injection
- Interface First
- Configuration First

Deliverables of Part 2A

After AntiGravity reads this document it must understand:

- Every agent's responsibility
- Every agent's inputs and outputs
- Communication model
- Queue model
- Failure handling
- Retry strategy
- Integration contracts

without writing any implementation code.

Next Document (Part 2B)

MASTER SYSTEM PROMPT – PART 2B

Isme hoga:

- Phase 0–11 complete implementation roadmap
- Folder structure
- Files to create
- Database schema overview
- PASS / FAIL criteria
- Audit checklist
- LOCK conditions
- Development order
- MVP milestone
- Production milestone

AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)
MASTER SYSTEM PROMPT
PART 2B-1
DEVELOPMENT ROADMAP (PHASE 0 – PHASE 3)

=====
MISSION
=====

You are responsible for implementing the AI Website Upgrade Agency according to the approved architecture.

The architecture defined in Part 1 and Part 2A is LOCKED.

Implementation must follow the exact development sequence.

Never skip phases.

Never merge multiple phases.

Never modify future phases while the current phase is incomplete.

Every phase must end with:

- Internal Testing
- Audit Report
- PASS / FAIL Decision
- Phase Lock

Only after PASS may the next phase begin.

=====

DEVELOPMENT ORDER

=====

Phase 0
Foundation Framework

↓

Phase 1
Search Engine

↓

Phase 2
Website Audit Engine

↓

Phase 3
Niche Detection Engine

Future phases are prohibited until these phases pass.

=====

PHASE 0 FOUNDATION FRAMEWORK

=====

OBJECTIVE

Create the core infrastructure required by every future module.

No business logic.

No AI.

No website processing.

Only project infrastructure.

CREATE

Configuration Manager

Settings Manager

Environment Loader

Logger

Database Manager

Redis Manager

Workflow Manager

Queue Manager

Master Agent

Agent Registry

Event Manager

Scheduler

Health Monitor

Dashboard Framework

Backup Manager

FOLDER STRUCTURE

/config

/core

/database

/events

/workflows

/dashboard

/logs

/backups

/tests

FILES

config.py

settings.py

logger.py

database.py

redis_manager.py

workflow_manager.py

master_agent.py

agent_registry.py

scheduler.py

event_bus.py

health_monitor.py

dashboard.py

REQUIREMENTS

Every component must be independent.

No hardcoded configuration.

All environment variables loaded from .env

Every module must support logging.

Every module must support exception handling.

Every module must support configuration injection.

TESTS

Configuration loads correctly

Database connects

Redis connects

Logger writes logs

Master Agent initializes

Workflow Manager loads

Queue Manager starts

Health Monitor responds

PASS CONDITIONS

All infrastructure modules initialize successfully.

Zero startup exceptions.

100% infrastructure tests pass.

Audit Report generated.

LOCK CONDITIONS

Phase 0 becomes READ ONLY.

No modifications unless architecture changes.

=====

PHASE 1 SEARCH ENGINE

=====

OBJECTIVE

Automatically discover business websites.

No auditing.

No AI analysis.

No CRM.

Only website discovery.

SUPPORTED NICHES

Book Publishers

Schools

Hospitals

Clinics

Restaurants

Hotels

Law Firms

Real Estate

Travel Agencies

Gyms

NGOs

Coaching Institutes

Local Businesses

Portfolio Websites

E-Commerce

CREATE

Search Agent

Search Queue

Search Models

Search Services

Search Tests

DATABASE

search_leads

Fields

id

domain

source

niche

status

created_at

RULES

Remove duplicate domains.

Ignore social media.

Ignore directory pages.

Ignore invalid domains.

Only root domains allowed.

OUTPUT

Validated website list.

TESTS

Domain discovery

Duplicate detection

Filtering

Storage

PASS CONDITIONS

Minimum 50 valid domains.

No duplicates.

Database updated.

Audit Report generated.

LOCK CONDITIONS

Search Engine frozen.

=====

PHASE 2

WEBSITE AUDIT ENGINE

=====

OBJECTIVE

Analyze website quality.

No email generation.

No redesign.

No AI prototype.

CHECKS

SSL

Performance

Page Speed

Mobile Responsiveness

Accessibility

SEO

Trust Signals

Navigation

CTA Quality

Visual Quality

Technology Stack

OUTPUT

Audit JSON

Audit Score

Improvement Report

CREATE

Audit Agent

Audit Models

Audit Services

Audit Rules

Audit Tests

DATABASE

audits

FIELDS

lead_id

audit_score

seo_score

mobile_score

speed_score

trust_score

design_score

summary

PASS CONDITIONS

Website audited successfully.

Audit report generated.

Database updated.

All tests pass.

=====

PHASE 3

NICHE DETECTION ENGINE

=====

OBJECTIVE

Automatically classify businesses.

SUPPORTED CATEGORIES

Publisher

School

Hospital

Clinic

Restaurant

Hotel

Travel

Law Firm

Real Estate

NGO

Gym

Education

Portfolio

Business

Corporate

INPUT

Website

Screenshot

Audit Report

OUTPUT

Business Category

Confidence Score

Theme Recommendation

CREATE

Niche Agent

Niche Models

Classifier

Theme Mapper

Tests

DATABASE

business_profiles

FIELDS

lead_id

industry

confidence

recommended_theme

REQUIREMENTS

Hybrid classification.

Rule Engine first.

AI only when required.

PASS CONDITIONS

Classification Accuracy > 90%

Theme Recommendation generated.

Database updated.

Tests pass.

=====

GLOBAL RULES

=====

Never continue to Phase 4.

Never implement Contact Extraction.

Never implement CRM.

Never implement Prototype Generation.

Never generate Emails.

Only implement Phases 0–3.

=====

AUDIT REPORT FORMAT

=====

Every phase must finish with:

Files Created

Files Modified

Database Changes

Tests Executed

Errors

Warnings

PASS / FAIL

Performance Notes

Future Recommendations

=====

AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)
MASTER SYSTEM PROMPT
PART 2B-2A
DEVELOPMENT ROADMAP
PHASE 4 & PHASE 5

=====

MISSION

=====

Implement only:

Phase 4

Lead Scoring Engine

Phase 5

Contact Extraction Engine

Do NOT implement any other phase.

Do NOT create prototype generation.

Do NOT create email automation.

Do NOT create CRM.

Complete only these two phases.

=====

PHASE 4

LEAD SCORING ENGINE

=====

PURPOSE

Evaluate business opportunity based on website quality and assign a business priority score.

This engine decides whether a lead is worth spending AI resources on.

AI calls must never happen before Lead Scoring.

Lead Scoring is the cost optimization layer.

=====
INPUT
=====

Audit Report

Website Metadata

Business Category

Technology Stack

Website Age (if available)

Domain Quality

=====
OUTPUT
=====

Lead Score

Business Priority

Improvement Opportunity

AI Processing Decision

=====
SCORING MODEL
=====

Old Website Design
+30

Mobile Responsiveness Issues

+25

Performance Problems

+20

SEO Problems

+15

Trust Issues

+10

Broken Navigation

+10

Poor CTA Placement

+10

Outdated Branding

+15

No SSL

+10

Accessibility Problems

+10

=====

SCORE LEVELS

=====

0–30

Low Priority

Ignore

31–60

Medium Priority

Manual Review

61–80

High Priority

Proceed

81–100

Premium Lead

Immediate Processing

=====

BUSINESS VALUE SCORE

=====

Calculate

Website Quality

×

Business Potential

×

Industry Value

×

Contact Availability

×

Improvement Opportunity

Generate

Business Value Index

=====

FILES TO CREATE

=====

agents/scoring/

lead_scoring_agent.py

scoring_models.py

scoring_rules.py

priority_engine.py

business_value.py

tests/

=====

DATABASE

=====

lead_scores

Fields

id

lead_id

website_score

business_score

priority

business_value

recommended_action

created_at

=====

RECOMMENDED ACTIONS

=====

Ignore

Manual Review

Generate Prototype

Immediate Outreach

Premium Client

=====

PASS CONDITIONS

=====

Lead Score generated.

Priority generated.

Business Value generated.

Database updated.

Tests passed.

=====

FAIL CONDITIONS

=====

No Audit Report

Invalid Data

Score Calculation Failure

Database Error

=====

AUDIT REPORT

=====

Files Created

Rules Applied

Scores Generated

Execution Time

PASS / FAIL

=====
LOCK CONDITION
=====

Lead Scoring Engine becomes READ ONLY.

No future modification allowed unless architecture changes.

PHASE 5
CONTACT EXTRACTION ENGINE
#####

PURPOSE

Extract every possible contact method from a business website.

The engine must discover every publicly available communication channel.

=====
INPUT
=====

Website URL

Audit Report

Business Profile

=====
OUTPUT
=====

Validated Contact Record

=====

PAGES TO SCAN

=====

/

about

contact

contact-us

team

support

footer

privacy

terms

=====

CONTACT TYPES

=====

Primary Email

Secondary Email

Phone Number

WhatsApp

Facebook

Instagram

LinkedIn

Twitter

YouTube

Google Business

=====

EXTRACTION RULES

=====

Extract only publicly available information.

Never bypass authentication.

Never scrape restricted areas.

Respect robots.txt where applicable.

Store normalized values.

Remove duplicates.

=====

VALIDATION

=====

Email Format

Phone Format

Country Code

Social URL Validation

Duplicate Detection

=====

FILES TO CREATE

=====

agents/contact/

contact_agent.py

contact_parser.py

email_extractor.py

phone_extractor.py

social_extractor.py

normalizer.py

validator.py

tests/

=====

DATABASE

=====

contacts

Fields

id

lead_id

primary_email

secondary_email

phone

whatsapp

facebook

instagram

linkedin

twitter

youtube

website

status

=====

CONTACT QUALITY

=====

Complete

Partial

Email Only

Phone Only

Social Only

No Contact

=====

PASS CONDITIONS

=====

Contact Record created.

Database updated.

Normalization completed.

Validation completed.

All tests passed.

=====

FAIL CONDITIONS

=====

Website unavailable

No contact information

Parser failure

Database failure

=====

AUDIT REPORT

=====

Pages Scanned

Contacts Found

Duplicates Removed

Validation Result

Execution Time

PASS / FAIL

=====

GLOBAL DEVELOPMENT RULES

=====

Do NOT continue to Phase 6.

Do NOT generate Emails.

Do NOT generate HTML Prototypes.

Do NOT access Gemini.

Do NOT perform AI analysis.

These phases must remain completely deterministic.

=====

QUALITY REQUIREMENTS

=====

Every class must have a single responsibility.

Every service must be independently testable.

Business rules must never be hardcoded.

Configuration must be external.

Logging must exist for every execution step.

Every exception must be handled.

Every database transaction must be safe.

Every function must include documentation.

=====

TEST COVERAGE

=====

Minimum Unit Test Coverage

90%

Integration Tests

Required

Failure Tests

Required

Performance Tests

Required

Regression Tests

Required

=====

END OF PART 2B-2A
AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)
MASTER SYSTEM PROMPT
PART 2B-2B
PHASE 6
EMAIL VALIDATION ENGINE

=====

MISSION

=====

Implement only Phase 6.

Build a production-grade Email Validation Engine.

Do NOT generate emails.

Do NOT contact businesses.

Do NOT generate HTML prototypes.

Do NOT call any AI models.

The Email Validation Engine is responsible only for validating, scoring, and classifying email addresses extracted from websites.

It must be deterministic, scalable, fast, and inexpensive.

=====

OBJECTIVE

=====

Before any outreach campaign begins, every extracted email address must be validated to reduce bounce rate, improve sender reputation, and maximize deliverability.

The engine must classify every email into quality levels.

Only high-quality emails may proceed to the Email Engine.

=====

INPUT

=====

Contact Record

Primary Email

Secondary Email

Website Domain

Business Profile

=====

OUTPUT

=====

Validated Email Record

Email Quality Score

Deliverability Status

Recommendation

=====

VALIDATION PIPELINE

=====

Stage 1

Syntax Validation

↓

Stage 2

Domain Validation

↓

Stage 3

DNS Lookup

↓

Stage 4

MX Record Verification

↓

Stage 5

Disposable Email Detection

↓

Stage 6

Role-based Email Detection

↓

Stage 7

Catch-all Domain Detection

↓

Stage 8

Business Domain Matching

↓

Stage 9

Confidence Score

↓

Final Classification

=====
SYNTAX VALIDATION
=====

Validate according to RFC standards.

Reject:

Missing @

Missing domain

Invalid characters

Invalid TLD

Multiple @ symbols

Whitespace

Malformed addresses

=====

DOMAIN VALIDATION

=====

Verify:

Domain exists

Domain resolves

Domain not expired (when possible)

No invalid hostname

=====

MX RECORD VALIDATION

=====

Verify

MX Records

SMTP Availability (optional)

Mail Server Presence

DNS Health

=====

DISPOSABLE EMAIL DETECTION

=====

Reject common temporary providers.

Examples

10MinuteMail

Guerrilla Mail

Mailinator

TempMail

Disposable providers must receive

Quality Score = 0

=====

ROLE BASED EMAILS

=====

Detect

info@

support@

sales@

contact@

admin@

office@

hello@

careers@

Role-based emails are valid.

However,

assign lower personalization score.

=====

BUSINESS DOMAIN MATCHING

=====

Compare

Website Domain

↓

Email Domain

Example

Website

company.com

Email

hello@company.com

Result

Perfect Match

Website

company.com

Email

gmail.com

Result

Generic Email

Lower Confidence

=====

EMAIL QUALITY SCORE

=====

100

Perfect

90

Business Email

80

Verified Generic

60

Role Email

30

Questionable

0

Invalid

=====
CLASSIFICATION
=====

Premium

Business Verified

Verified

Generic

Role-based

Temporary

Invalid

Rejected

=====
DATABASE
=====

Table

validated_emails

Fields

id

lead_id

email

quality_score

classification

mx_status

domain_status

disposable

role_based

confidence

recommended_action

validated_at

=====
FILES TO CREATE
=====

agents/email_validation/

email_validation_agent.py

syntax_validator.py

dns_validator.py

mx_validator.py

disposable_detector.py

role_detector.py

confidence_engine.py

quality_score.py

validation_models.py

tests/

=====

CONFIGURATION

=====

Validation Threshold

Minimum Quality Score

Allowed Domains

Blocked Domains

Disposable Database

Timeout Values

Retry Count

=====

LOGGING

=====

Every validation step must log

Validation Start

Validation Success

Validation Failure

DNS Result

MX Result

Confidence Score

Execution Time

=====

ERROR HANDLING

=====

Handle

DNS Timeout

Network Failure

Invalid Email

Unknown MX

Malformed Domain

Retry Logic

Graceful Failure

=====

PERFORMANCE REQUIREMENTS

=====

Average Validation Time

< 2 seconds

Parallel Validation

Supported

Caching

Required

DNS Cache

Recommended

=====

RECOMMENDED ACTIONS

=====

Premium

Proceed Immediately

Verified

Proceed

Role Based

Proceed

Use Generic Email Template

Generic

Proceed

Lower Priority

Temporary

Reject

Invalid

Reject

=====

TEST CASES

=====

Valid Business Email

Valid Gmail

Role Email

Temporary Email

Invalid Email

Missing Domain

Broken MX

Fake Domain

Expired Domain

Duplicate Email

=====

UNIT TESTS

=====

Syntax

DNS

MX

Classification

Quality Score

Confidence Engine

Caching

=====

INTEGRATION TESTS

=====

Database Storage

Logging

Queue Integration

Configuration

=====

PASS CONDITIONS

=====

All validations complete successfully.

Quality score generated.

Classification generated.

Database updated.

Logs generated.

90%+ unit test coverage.

Integration tests pass.

Performance target achieved.

=====

FAIL CONDITIONS

=====

Validation Crash

Database Failure

Configuration Error

Repeated DNS Failure

Unhandled Exception

=====

AUDIT REPORT

=====

Files Created

Validation Rules

Execution Time

Emails Validated

Invalid Emails

Role Emails

Temporary Emails

Database Changes

Warnings

Errors

PASS / FAIL

=====
LOCK CONDITIONS
=====

After PASS

Phase 6 becomes READ ONLY.

No changes are allowed unless architecture approval is granted.

=====
GLOBAL RESTRICTIONS
=====

Do NOT generate Emails.

Do NOT contact businesses.

Do NOT create HTML prototypes.

Do NOT access Gemini.

Do NOT access OpenAI.

Do NOT implement CRM.

Do NOT continue to Phase 7.

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.1

PROTOTYPE INTELLIGENCE ENGINE (PIE)

FOUNDATION & CORE ARCHITECTURE

=====

MISSION

=====

Implement only Phase 7.

Do not continue to Phase 8.

Prototype Intelligence Engine is responsible for producing a professional responsive HTML website prototype based on an existing business website.

The generated prototype must demonstrate how the customer's website can be improved.

The engine is not intended to clone the original website.

It must generate an improved design while preserving the business identity.

=====

OBJECTIVE

=====

Generate a high-quality responsive HTML prototype using:

- Existing Website
- Website Screenshot
- DOM Structure
- Business Category
- Theme Library
- Component Library

Output must be production-quality preview code.

=====

PRIMARY GOALS

=====

Improve Design

Improve UX

Improve Mobile Experience

Improve CTA Placement

Improve Visual Hierarchy

Improve Trust Signals

Improve Branding

Improve Readability

Improve Conversion Potential

=====

INPUTS

=====

Website URL

Audit Report

Business Profile

Lead Information

Business Category

Theme Recommendation

=====

OUTPUTS

=====

Responsive HTML Prototype

CSS

Tailwind Layout

Preview Screenshot

Prototype Metadata

Generation Report

=====

ENGINE RESPONSIBILITIES

=====

The Prototype Intelligence Engine shall:

Capture Website

Analyze Structure

Understand Business Type

Select Theme

Generate Layout

Build Components

Apply Responsive Rules

Generate HTML

Generate Preview

Generate Report

=====

OUT OF SCOPE

=====

Do not copy copyrighted assets.

Do not duplicate website source code.

Do not scrape protected resources.

Do not reproduce brand identities without transformation.

Do not generate production deployment packages.

Generate demonstration prototypes only.

=====

HIGH LEVEL PIPELINE

=====

Website URL

↓

Browser Engine

↓

Screenshot Engine

↓

DOM Analyzer

↓

Business Context

↓

Theme Engine

↓

Component Engine

↓

Layout Generator



Responsive Engine



HTML Generator



Preview Generator



Prototype Report

=====

CORE MODULES

=====

Module 1

Browser Engine

Module 2

Screenshot Engine

Module 3

DOM Intelligence

Module 4

Visual Intelligence

Module 5

Theme Intelligence

Module 6

Layout Intelligence

Module 7

Component Generator

Module 8

Responsive Engine

Module 9

HTML Generator

Module 10

Preview Generator

Module 11

Quality Analyzer

=====

FILES TO CREATE

=====

agents/prototype/

prototype_agent.py

browser_engine.py

screenshot_engine.py

dom_analyzer.py

visual_analyzer.py

theme_engine.py

layout_engine.py

component_engine.py

responsive_engine.py

html_generator.py

preview_generator.py

quality_analyzer.py

prototype_models.py

prototype_pipeline.py

tests/

=====

DATABASE

=====

prototype_jobs

prototype_results

prototype_assets

prototype_reports

=====

TABLE

prototype_jobs

=====

id

lead_id

website_url

status

theme

started_at

completed_at

```
=====
TABLE
prototype_results
=====
```

id

job_id

html_path

css_path

preview_path

quality_score

generation_time

```
=====
TABLE
prototype_reports
=====
```

id

job_id

summary

improvements

warnings

recommendations

```
=====
CONFIGURATION
=====
```

Viewport Width

Viewport Height

Browser Type

Theme Library

Component Library

Output Directory

Screenshot Quality

Preview Resolution

Retry Count

Timeout

=====

LOGGING

=====

Log:

Job Start

Screenshot Start

DOM Analysis

Theme Selection

HTML Generation

Preview Generation

Export

Errors

Completion

=====

ERROR HANDLING

=====

Website Timeout

Screenshot Failure

DOM Failure

Theme Failure

HTML Failure

Preview Failure

Export Failure

All failures must produce structured logs.

=====

PASS CONDITIONS

=====

Screenshot Captured

DOM Parsed

Theme Selected

Responsive HTML Generated

Preview Generated

Database Updated

Logs Written

Tests Passed

=====

FAIL CONDITIONS

=====

Website Unreachable

Screenshot Failed

DOM Parsing Failed

Theme Missing

Generation Failed

Unhandled Exception

=====

GLOBAL RESTRICTIONS

=====

Do NOT implement Email Engine.

Do NOT implement CRM.

Do NOT implement Telegram.

Do NOT continue to Phase 8.

Wait for:

Phase 7.2

Screenshot Intelligence Engine

=====

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.2

SCREENSHOT INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the Screenshot Intelligence Engine.

Do NOT implement DOM Intelligence.

Do NOT implement Theme Intelligence.

Do NOT implement HTML Generation.

Do NOT implement Preview Generation.

Do NOT continue to Phase 7.3.

This module is responsible only for capturing high-quality website screenshots that will be used by every downstream Prototype Intelligence Engine module.

This engine becomes the visual data provider for the entire Prototype Generation Pipeline.

=====

OBJECTIVE

=====

Capture a complete, accurate and production-quality visual representation of a business website.

The Screenshot Engine must support websites of different sizes, technologies and layouts while producing consistent screenshots for AI analysis.

=====

PRIMARY RESPONSIBILITIES

=====

Open Website

Wait for Stable Page

Handle Dynamic Loading

Scroll Complete Page

Capture Full Page

Capture Desktop View

Capture Tablet View

Capture Mobile View

Store Metadata

Export Images

Log Results

=====

INPUT

=====

Website URL

Browser Configuration

Viewport Configuration

Timeout Configuration

Output Directory

=====

OUTPUT

=====

Desktop Screenshot

Tablet Screenshot

Mobile Screenshot

Full Page Screenshot

Screenshot Metadata

Execution Report

=====

ENGINE PIPELINE

=====

Website URL

↓

Launch Browser



Initialize Context



Configure Viewport



Navigate



Wait Until Stable



Scroll Page



Lazy Load Detection



Image Completion



Animation Freeze



Capture Desktop



Capture Tablet



Capture Mobile



Capture Full Page



Compress Images



Store Files



Generate Metadata

=====

SUPPORTED VIEWPORTS

=====

Desktop

1920 × 1080

Laptop

1440 × 900

Tablet

768 × 1024

Mobile

390 × 844

Additional viewport profiles must be configurable.

=====

SUPPORTED BROWSERS

=====

Chromium

Preferred

Chrome

Supported

Firefox

Optional

WebKit

Optional

=====

BROWSER CONFIGURATION

=====

Headless Mode

Enabled

Disable Notifications

Enabled

Disable Extensions

Enabled

Disable Cache

Configurable

User Agent

Configurable

Language

Configurable

Timezone

Configurable

=====

WAIT STRATEGY

=====

Wait for

DOM Ready

↓

Network Idle

↓

Images Loaded

↓

Fonts Loaded

↓

Animation Timeout

↓

Ready for Capture

Never capture before the page becomes stable.

=====

SCROLL STRATEGY

=====

Incremental Scrolling

↓

Wait

↓

Continue

↓

Detect Lazy Loading

↓

Continue

↓

Bottom Reached

↓

Return to Top

↓

Capture

=====

LAZY LOADING SUPPORT

=====

Detect

Images

Videos

Cards

Infinite Sections

Lazy Components

Retry loading until maximum threshold.

=====

POPUP HANDLING

=====

Attempt automatic closing for

Cookie Banner

Newsletter Popup

Chat Widgets

Location Dialogs

Notification Dialogs

Age Verification

If popup cannot be closed,

log warning

continue safely.

=====

SCREENSHOT TYPES

=====

Desktop

Tablet

Mobile

Full Page

Hero Section

Header

Footer

Optional Component Screenshots

=====

IMAGE QUALITY

=====

PNG

Lossless

Preferred

JPEG

Optional

Compression

Configurable

Color Profile

sRGB

=====

FILES TO CREATE

=====

agents/prototype/

browser_engine.py

browser_context.py

viewport_profiles.py

navigation_service.py

page_loader.py

scroll_engine.py

lazy_loader.py

popup_handler.py

capture_engine.py

image_optimizer.py

metadata_generator.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

screenshots/

desktop/

tablet/

mobile/

fullpage/

metadata/

logs/

=====

DATABASE

=====

Table

prototype_screenshots

Fields

id

job_id

desktop_path

tablet_path

mobile_path

fullpage_path

capture_duration

page_height

page_width

browser

status

created_at

=====

METADATA

Store

Resolution

Viewport

Browser

Capture Time

Page Height

Page Width

Compression Ratio

Capture Duration

=====

LOGGING

=====

Log

Browser Launch

Navigation Start

Navigation Complete

Scroll Start

Scroll Complete

Lazy Load Detection

Popup Closed

Screenshot Saved

Compression

Metadata Generated

Completion

=====

ERROR HANDLING

=====

Browser Crash

Navigation Timeout

Rendering Failure

Image Save Failure

Viewport Failure

Memory Overflow

Retry Strategy

Maximum Retry Count

Graceful Exit

=====

CONFIGURATION

=====

Headless Mode

Viewport Profiles

Timeout

Retry Count

Compression Quality

Output Folder

Browser Selection

=====

PERFORMANCE REQUIREMENTS

=====

Browser Startup

< 5 Seconds

Navigation

Configurable

Capture Time

Optimized

Memory Usage

Controlled

Parallel Capture

Supported

=====

SECURITY RULES

=====

Do not bypass authentication.

Do not capture protected pages.

Do not access private content.

Respect robots.txt where applicable.

Do not store cookies permanently.

=====

UNIT TESTS

=====

Browser Launch

Navigation

Viewport

Scrolling

Lazy Loading

Popup Detection

Capture

Compression

Metadata

=====

INTEGRATION TESTS

=====

Pipeline Execution

Database Storage

Logging

Configuration

Filesystem

=====

PASS CONDITIONS

=====

Desktop Screenshot Generated

Tablet Screenshot Generated

Mobile Screenshot Generated

Full Page Screenshot Generated

Metadata Generated

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

Browser Failed

Navigation Failed

Capture Failed

Filesystem Error

Unhandled Exception

Metadata Missing

=====

AUDIT REPORT

=====

Files Created

Execution Time

Browser Used

Viewport Profiles

Capture Statistics

Warnings

Errors

PASS / FAIL

Recommendations

=====

LOCK CONDITIONS

=====

After PASS

Screenshot Intelligence Engine becomes READ ONLY.

No architectural modifications allowed without approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT implement DOM Analysis.

Do NOT analyze colors.

Do NOT analyze typography.

Do NOT detect components.

Do NOT generate HTML.

Do NOT generate CSS.

Do NOT generate Preview.

Do NOT continue to Phase 7.3.

Wait for:

PART 2B-2C

PHASE 7.3

DOM INTELLIGENCE ENGINE

=====

END OF PHASE 7.2

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.3

DOM INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the DOM Intelligence Engine.

Do NOT implement Visual Intelligence.

Do NOT implement Theme Intelligence.

Do NOT implement HTML Generation.

Do NOT implement Preview Generation.

Do NOT continue to Phase 7.4.

The DOM Intelligence Engine is responsible for understanding the logical structure of an existing website.

It must transform an HTML document into a structured component map that can later be used by the Prototype Intelligence Engine.

This module never redesigns the website.

It only understands its structure.

=====

OBJECTIVE

=====

Analyze the complete Document Object Model (DOM) and convert it into structured information.

The engine must identify:

- Website Structure
- Components
- Layout
- Content Hierarchy
- Navigation
- Forms
- Calls To Action
- Semantic Relationships

The generated data will become the foundation for every future design decision.

=====

PRIMARY RESPONSIBILITIES

=====

Load DOM

Parse HTML

Remove Noise

Build DOM Tree

Detect Sections

Detect Components

Detect Navigation

Detect CTA

Detect Forms

Detect Layout

Generate Component Map

Generate Structure Report

=====

INPUT

=====

Website URL

Rendered HTML

Browser DOM

Screenshot Metadata

Audit Report

=====

OUTPUT

=====

DOM Tree

Component Map

Semantic Structure

Layout Structure

Navigation Structure

Section Hierarchy

DOM Analysis Report

=====

DOM PIPELINE

=====

Rendered Page

↓

HTML Extraction

↓

DOM Parsing

↓

Node Classification

↓

Semantic Detection

↓

Component Detection

↓

Layout Detection

↓

Hierarchy Analysis



Relationship Mapping



Component Map



DOM Report

=====
DOM PREPROCESSING
=====

Remove

Scripts

Tracking Code

Analytics

Advertisements

Hidden Elements

Duplicate Nodes

Comments

Unused Styles

Normalize DOM

=====
NODE CLASSIFICATION
=====

Detect

Header

Navigation

Hero

Main Content

Section

Article

Card

Sidebar

Footer

Form

Button

Modal

Popup

Banner

Image

Video

Table

Accordion

Tabs

=====
SEMANTIC TAG DETECTION
=====

Identify

header

nav

main

section

article

aside

footer

form

button

figure

picture

video

ul

ol

table

canvas

svg

=====

SECTION DETECTION

=====

Identify

Hero Section

About Section

Services

Products

Features

Pricing

Portfolio

Gallery

Testimonials

Statistics

FAQ

Contact

Footer

=====

NAVIGATION ANALYSIS

=====

Detect

Primary Navigation

Secondary Navigation

Mega Menu

Dropdown

Sticky Header

Mobile Navigation

Hamburger Menu

Breadcrumbs

=====

CTA DETECTION

=====

Locate

Primary CTA

Secondary CTA

Buttons

Action Links

Call Buttons

WhatsApp Buttons

Booking Buttons

Purchase Buttons

Contact Buttons

=====

FORM DETECTION

=====

Identify

Contact Forms

Newsletter

Search Forms

Login Forms

Registration Forms

Booking Forms

Quote Forms

Feedback Forms

=====

CARD DETECTION

=====

Detect

Service Cards

Feature Cards

Product Cards

Pricing Cards

Blog Cards

Team Cards

Testimonial Cards

Portfolio Cards

=====

LAYOUT ANALYSIS

=====

Detect

Single Column

Two Column

Three Column

Grid

Flex Layout

Mixed Layout

Nested Sections

Container Width

Spacing Pattern

=====

CONTENT HIERARCHY

=====

Generate

Heading Levels

Content Groups

Reading Order

Visual Importance

Parent Child Relationships

Nested Structures

=====

COMPONENT MAP

=====

Generate

Component ID

Component Type

Position

Parent

Children

Section

Priority

Importance

Visibility

=====

FILES TO CREATE

=====

agents/prototype/

dom_engine.py

dom_parser.py

dom_classifier.py

semantic_detector.py

component_mapper.py

section_detector.py

navigation_detector.py

cta_detector.py

form_detector.py

layout_analyzer.py

hierarchy_builder.py

dom_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

dom/

parsed/

maps/

reports/

logs/

=====

DATABASE

=====

Table

prototype_dom_analysis

Fields

id

job_id

component_count

section_count

navigation_type

layout_type

cta_count

form_count

analysis_time

status

created_at

=====

CONFIGURATION

=====

Parser Engine

Ignored Tags

Ignored Classes

Maximum DOM Size

Timeout

Report Level

Output Format

=====
LOGGING
=====

Log

DOM Loaded

Parsing Started

Parsing Complete

Components Found

Sections Found

Forms Found

Navigation Found

Hierarchy Generated

Report Generated

=====
ERROR HANDLING
=====

Malformed HTML

Large DOM

Parser Timeout

Unsupported Elements

Memory Limit

Unknown Tags

Graceful Recovery

Retry Logic

=====

PERFORMANCE REQUIREMENTS

=====

Efficient Parsing

Memory Optimized

Large DOM Support

Parallel Node Analysis

Incremental Processing

=====

SECURITY RULES

=====

Do not execute JavaScript.

Do not modify page content.

Do not inject scripts.

Do not access protected resources.

Analyze rendered DOM only.

=====

UNIT TESTS

=====

HTML Parsing

Semantic Detection

Navigation Detection

CTA Detection

Forms

Cards

Layout

Hierarchy

Component Mapping

=====

INTEGRATION TESTS

=====

Browser Integration

Database

Logging

Configuration

Filesystem

Pipeline

=====

PASS CONDITIONS

=====

DOM Parsed Successfully

Components Identified

Sections Generated

Hierarchy Generated

Database Updated

Report Generated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

DOM Parsing Failed

Malformed HTML

Hierarchy Failure

Component Mapping Failure

Unhandled Exception

Database Failure

=====

AUDIT REPORT

=====

Files Created

DOM Nodes

Sections

Components

Forms

CTAs

Navigation Type

Execution Time

Warnings

Errors

PASS / FAIL

=====
LOCK CONDITIONS
=====

After PASS

DOM Intelligence Engine becomes READ ONLY.

No structural modifications allowed without architecture approval.

=====
GLOBAL RESTRICTIONS
=====

Do NOT analyze colors.

Do NOT analyze typography.

Do NOT generate themes.

Do NOT redesign layouts.

Do NOT generate HTML.

Do NOT continue to Phase 7.4.

Wait for:

PART 2B-2C

PHASE 7.4

VISUAL INTELLIGENCE ENGINE

=====
END OF PHASE 7.3
AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT
PART 2B-2C
PHASE 7.3
DOM INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the DOM Intelligence Engine.

Do NOT implement Visual Intelligence.

Do NOT implement Theme Intelligence.

Do NOT implement HTML Generation.

Do NOT implement Preview Generation.

Do NOT continue to Phase 7.4.

The DOM Intelligence Engine is responsible for understanding the logical structure of an existing website.

It must transform an HTML document into a structured component map that can later be used by the Prototype Intelligence Engine.

This module never redesigns the website.

It only understands its structure.

=====

OBJECTIVE

=====

Analyze the complete Document Object Model (DOM) and convert it into structured information.

The engine must identify:

- Website Structure
- Components
- Layout
- Content Hierarchy

- Navigation
- Forms
- Calls To Action
- Semantic Relationships

The generated data will become the foundation for every future design decision.

=====

PRIMARY RESPONSIBILITIES

=====

Load DOM

Parse HTML

Remove Noise

Build DOM Tree

Detect Sections

Detect Components

Detect Navigation

Detect CTA

Detect Forms

Detect Layout

Generate Component Map

Generate Structure Report

=====

INPUT

=====

Website URL

Rendered HTML

Browser DOM

Screenshot Metadata

Audit Report

=====

OUTPUT

=====

DOM Tree

Component Map

Semantic Structure

Layout Structure

Navigation Structure

Section Hierarchy

DOM Analysis Report

=====

DOM PIPELINE

=====

Rendered Page

↓

HTML Extraction

↓

DOM Parsing

↓

Node Classification

↓

Semantic Detection



Component Detection



Layout Detection



Hierarchy Analysis



Relationship Mapping



Component Map



DOM Report

=====

DOM PREPROCESSING

=====

Remove

Scripts

Tracking Code

Analytics

Advertisements

Hidden Elements

Duplicate Nodes

Comments

Unused Styles

Normalize DOM

=====

NODE CLASSIFICATION

=====

Detect

Header

Navigation

Hero

Main Content

Section

Article

Card

Sidebar

Footer

Form

Button

Modal

Popup

Banner

Image

Video

Table

Accordion

Tabs

=====

SEMANTIC TAG DETECTION

=====

Identify

header

nav

main

section

article

aside

footer

form

button

figure

picture

video

ul

ol

table

canvas

svg

=====

SECTION DETECTION

=====

Identify

Hero Section

About Section

Services

Products

Features

Pricing

Portfolio

Gallery

Testimonials

Statistics

FAQ

Contact

Footer

=====

NAVIGATION ANALYSIS

=====

Detect

Primary Navigation

Secondary Navigation

Mega Menu

Dropdown

Sticky Header

Mobile Navigation

Hamburger Menu

Breadcrumbs

=====

CTA DETECTION

=====

Locate

Primary CTA

Secondary CTA

Buttons

Action Links

Call Buttons

WhatsApp Buttons

Booking Buttons

Purchase Buttons

Contact Buttons

=====

FORM DETECTION

=====

Identify

Contact Forms

Newsletter

Search Forms

Login Forms

Registration Forms

Booking Forms

Quote Forms

Feedback Forms

=====

CARD DETECTION

=====

Detect

Service Cards

Feature Cards

Product Cards

Pricing Cards

Blog Cards

Team Cards

Testimonial Cards

Portfolio Cards

=====

LAYOUT ANALYSIS

=====

Detect

Single Column

Two Column

Three Column

Grid

Flex Layout

Mixed Layout

Nested Sections

Container Width

Spacing Pattern

=====

CONTENT HIERARCHY

=====

Generate

Heading Levels

Content Groups

Reading Order

Visual Importance

Parent Child Relationships

Nested Structures

=====

COMPONENT MAP

=====

Generate

Component ID

Component Type

Position

Parent

Children

Section

Priority

Importance

Visibility

=====

FILES TO CREATE

=====

agents/prototype/

dom_engine.py

dom_parser.py

dom_classifier.py

semantic_detector.py

component_mapper.py

section_detector.py

navigation_detector.py

cta_detector.py

form_detector.py

layout_analyzer.py

hierarchy_builder.py

dom_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

dom/

parsed/

maps/

reports/

logs/

=====

DATABASE

=====

Table

prototype_dom_analysis

Fields

id

job_id

component_count

section_count

navigation_type

layout_type

cta_count

form_count

analysis_time

status

created_at

=====

CONFIGURATION

=====

Parser Engine

Ignored Tags

Ignored Classes

Maximum DOM Size

Timeout

Report Level

Output Format

=====

LOGGING

=====

Log

DOM Loaded

Parsing Started

Parsing Complete

Components Found

Sections Found

Forms Found

Navigation Found

Hierarchy Generated

Report Generated

=====

ERROR HANDLING

=====

Malformed HTML

Large DOM

Parser Timeout

Unsupported Elements

Memory Limit

Unknown Tags

Graceful Recovery

Retry Logic

=====

PERFORMANCE REQUIREMENTS

=====

Efficient Parsing

Memory Optimized

Large DOM Support

Parallel Node Analysis

Incremental Processing

=====

SECURITY RULES

=====

Do not execute JavaScript.

Do not modify page content.

Do not inject scripts.

Do not access protected resources.

Analyze rendered DOM only.

=====

UNIT TESTS

=====

HTML Parsing

Semantic Detection

Navigation Detection

CTA Detection

Forms

Cards

Layout

Hierarchy

Component Mapping

=====

INTEGRATION TESTS

=====

Browser Integration

Database

Logging

Configuration

Filesystem

Pipeline

PASS CONDITIONS

DOM Parsed Successfully

Components Identified

Sections Generated

Hierarchy Generated

Database Updated

Report Generated

Logs Written

All Tests Passed

FAIL CONDITIONS

DOM Parsing Failed

Malformed HTML

Hierarchy Failure

Component Mapping Failure

Unhandled Exception

Database Failure

AUDIT REPORT

Files Created

DOM Nodes

Sections

Components

Forms

CTAs

Navigation Type

Execution Time

Warnings

Errors

PASS / FAIL

=====
LOCK CONDITIONS
=====

After PASS

DOM Intelligence Engine becomes READ ONLY.

No structural modifications allowed without architecture approval.

=====
GLOBAL RESTRICTIONS
=====

Do NOT analyze colors.

Do NOT analyze typography.

Do NOT generate themes.

Do NOT redesign layouts.

Do NOT generate HTML.

Do NOT continue to Phase 7.4.

Wait for:

PART 2B-2C

PHASE 7.4

VISUAL INTELLIGENCE ENGINE

```
=====
END OF PHASE 7.3
# AI WEBSITE UPGRADE AGENCY
## ENTERPRISE DEVELOPMENT KIT (EDK)
### MASTER SYSTEM PROMPT
### PART 2B-2C
### PHASE 7.5
### THEME INTELLIGENCE ENGINE
```

```
=====
MISSION
=====
```

Implement only the Theme Intelligence Engine.

Do NOT implement Component Intelligence.

Do NOT implement Layout Intelligence.

Do NOT implement Responsive Engine.

Do NOT implement HTML Generation.

Do NOT continue to Phase 7.6.

The Theme Intelligence Engine is responsible for selecting the most suitable visual design language for a business based on its industry, brand identity, audit results, business goals, and visual intelligence report.

The engine shall never randomly assign themes.

Every recommendation must be rule-driven and explainable.

=====

OBJECTIVE

=====

Generate an intelligent design theme that improves the website while preserving the business identity.

The selected theme becomes the design foundation for all future prototype generation.

=====

PRIMARY RESPONSIBILITIES

=====

Analyze Business

Analyze Industry

Analyze Brand

Analyze Audience

Analyze Visual Report

Analyze Audit Report

Select Theme

Generate Theme Tokens

Generate Theme Rules

Generate Theme Report

=====

INPUT

=====

Business Category

Business Name

Audit Report

DOM Report

Visual Intelligence Report

Lead Score

Business Profile

=====

OUTPUT

=====

Selected Theme

Theme Profile

Color Tokens

Typography Tokens

Spacing Tokens

Elevation Tokens

Border Radius Tokens

Animation Profile

Theme Recommendation Report

=====

THEME PIPELINE

=====

Business Category

↓

Business Analysis

↓

Industry Mapping



Brand Personality



Design Style Selection



Theme Matching



Token Generation



Accessibility Validation



Theme Score



Theme Report

=====

SUPPORTED INDUSTRIES

=====

Book Publisher

Educational Institute

Hospital

Clinic

Restaurant

Hotel

Travel Agency

Law Firm

Real Estate

Architecture

Interior Design

Corporate

Consulting

Finance

Insurance

Technology

Software

Marketing

NGO

Government

Manufacturing

Portfolio

Creative Agency

Photography

Jewelry

Fashion

E-Commerce

Automobile

Healthcare

Fitness

=====

THEME CATEGORIES

=====

Corporate Professional

Modern Business

Minimal

Premium Luxury

Medical

Education

Creative Studio

Technology

Editorial

Elegant

Traditional

Dark Professional

Glass Interface

Bold Marketing

Startup

Industrial

Classic

=====

THEME PERSONALITY

=====

Professional

Premium

Elegant

Friendly

Creative

Minimal

Luxury

Modern

Traditional

Bold

Innovative

Reliable

High Trust

=====

COLOR TOKEN GENERATION

=====

Generate

Primary Color

Secondary Color

Accent Color

Background Color

Surface Color

Primary Text

Secondary Text

Border Color

Success

Warning

Error

Info

=====

COLOR RULES

=====

Respect

Brand Identity

Industry Standards

Accessibility

Contrast Ratio

Color Harmony

Visual Consistency

=====

TYPOGRAPHY TOKENS

=====

Generate

Heading Font

Body Font

Button Font

Navigation Font

Caption Font

Display Font

Font Scale

Font Weight

Letter Spacing

Line Height

=====

SPACING TOKENS

=====

Generate

Base Unit

Container Width

Section Padding

Component Padding

Grid Gap

Card Gap

Button Padding

Input Padding

=====

BORDER SYSTEM

=====

Generate

Border Radius

Border Width

Card Radius

Button Radius

Input Radius

Image Radius

=====

SHADOW SYSTEM

=====

Generate

Small

Medium

Large

Floating

Modal

Dropdown

Card

=====

ELEVATION SYSTEM

=====

Define

Layer 0

Layer 1

Layer 2

Layer 3

Layer 4

Layer 5

=====

ICON STYLE

=====

Generate

Filled

Outlined

Rounded

Sharp

Minimal

Professional

=====

BUTTON STYLE

=====

Generate

Primary Button

Secondary Button

Outline Button

Ghost Button

Text Button

CTA Button

=====

FORM STYLE

=====

Generate

Input Style

Textarea

Select

Checkbox

Radio

Validation Style

Focus Style

=====

CARD STYLE

=====

Generate

Service Card

Feature Card

Pricing Card

Product Card

Team Card

Testimonial Card

Blog Card

Portfolio Card

=====

IMAGE STYLE

=====

Recommend

Photography

Illustration

Minimal Graphics

Icons

Mixed

Large Hero Images

Product Images

=====

ANIMATION PROFILE

=====

Generate

Transition Speed

Hover Animation

Button Animation

Card Animation

Scroll Animation

Loading Animation

Animation Intensity

=====

RESPONSIVE TOKENS

=====

Desktop

Laptop

Tablet

Mobile

Small Mobile

=====

THEME SCORING

=====

Generate

Brand Match Score

Industry Match Score

Accessibility Score

Consistency Score

Premium Score

Overall Theme Score

=====

FILES TO CREATE

=====

agents/prototype/

theme_engine.py

theme_selector.py

theme_library.py

theme_tokens.py

color_generator.py

typography_generator.py

spacing_generator.py

border_generator.py

shadow_generator.py

animation_generator.py

theme_validator.py

theme_report.py

theme_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

themes/

tokens/

presets/

reports/

logs/

=====

DATABASE

=====

Table

prototype_themes

Fields

id

job_id

theme_name

industry

personality

primary_color

secondary_color

accent_color

heading_font

body_font

theme_score

created_at

=====

CONFIGURATION

=====

Theme Library

Industry Mapping

Accessibility Threshold

Animation Level

Dark Mode

Light Mode

Premium Themes

Custom Themes

=====

LOGGING

=====

Log

Business Analysis

Industry Detection

Theme Selection

Token Generation

Validation

Accessibility Check

Theme Score

Report Generation

=====

ERROR HANDLING

=====

Theme Not Found

Invalid Industry

Missing Tokens

Accessibility Failure

Configuration Error

Validation Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Theme Selection

Token Caching

Reusable Theme Profiles

Low Memory Usage

Deterministic Output

=====

SECURITY RULES

=====

Never generate copyrighted brand themes.

Never clone competitor branding.

Never copy proprietary visual identities.

Generate original design systems only.

=====

UNIT TESTS

=====

Theme Selection

Industry Mapping

Token Generation

Typography

Spacing

Colors

Animation

Validation

Theme Score

=====

INTEGRATION TESTS

=====

Visual Engine Integration

DOM Engine Integration

Database

Logging

Filesystem

Pipeline

=====
PASS CONDITIONS
=====

Theme Selected

Design Tokens Generated

Accessibility Validated

Theme Report Generated

Database Updated

Logs Written

All Tests Passed

=====
FAIL CONDITIONS
=====

Theme Generation Failed

Token Generation Failed

Invalid Theme

Database Failure

Unhandled Exception

=====

AUDIT REPORT

=====

Theme Selected

Industry

Brand Personality

Generated Tokens

Accessibility Result

Theme Score

Execution Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Theme Intelligence Engine becomes READ ONLY.

No architectural modifications allowed without approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT generate Components.

Do NOT generate Layout.

Do NOT generate HTML.

Do NOT generate CSS.

Do NOT continue to Phase 7.6.

Wait for:

PART 2B-2C

PHASE 7.6

COMPONENT INTELLIGENCE ENGINE

```
=====
END OF PHASE 7.5
# AI WEBSITE UPGRADE AGENCY
## ENTERPRISE DEVELOPMENT KIT (EDK)
### MASTER SYSTEM PROMPT
### PART 2B-2C
### PHASE 7.6
### COMPONENT INTELLIGENCE ENGINE
```

```
=====
MISSION
=====
```

Implement only the Component Intelligence Engine.

Do NOT implement Layout Intelligence.

Do NOT implement Responsive Engine.

Do NOT implement HTML Generation.

Do NOT implement Preview Engine.

Do NOT continue to Phase 7.7.

The Component Intelligence Engine is responsible for selecting, building, validating, and assembling reusable UI components based on the outputs of the DOM Intelligence Engine, Visual Intelligence Engine, Theme Intelligence Engine, and Business Profile.

This engine does not generate HTML.

It generates structured component definitions that will later be consumed by the HTML Generation Engine.

=====

OBJECTIVE

=====

Create a reusable component architecture capable of building industry-specific websites using standardized enterprise UI components.

Every generated component must be:

Reusable

Responsive Ready

Theme Compatible

Accessibility Ready

Independent

Version Controlled

=====

PRIMARY RESPONSIBILITIES

=====

Analyze Business

Analyze Theme

Analyze Layout

Select Components

Select Variants

Generate Component Tree

Generate Component Metadata

Validate Components

Generate Component Report

=====

INPUT

=====

Business Category

Theme Profile

Visual Report

DOM Report

Audit Report

Business Profile

=====

OUTPUT

=====

Component Tree

Component Library Mapping

Component Metadata

Variant Selection

Component Configuration

Component Report

=====

COMPONENT PIPELINE

=====

Business Category

↓

Theme Profile



Component Selection



Variant Selection



Component Configuration



Component Tree



Validation



Component Report

=====

CORE COMPONENT LIBRARY

=====

Header

Navigation

Hero

About

Services

Features

Products

Categories

Portfolio

Projects

Pricing

Testimonials

Clients

Awards

Statistics

Timeline

FAQ

Gallery

Team

Blog

Newsletter

Contact

Map

Footer

=====

UTILITY COMPONENTS

=====

Buttons

Badges

Cards

Modals

Drawers

Tabs

Accordion

Alerts

Breadcrumb

Pagination

Carousel

Sliders

Counters

Forms

Inputs

Checkboxes

Radio Buttons

Select

Textarea

Tooltips

=====

LAYOUT COMPONENTS

=====

Container

Section

Grid

Flex

Columns

Rows

Stack

Divider

Spacer

=====

MEDIA COMPONENTS

=====

Image

Video

Logo

Icon

Avatar

Background

Illustration

=====

BUSINESS SPECIFIC COMPONENTS

=====

Restaurant

Menu

Reservation

Chef

Food Gallery

Hospital

Departments

Doctors

Appointment

Emergency

School

Courses

Faculty

Admission

Events

Publisher

Books

Authors

Publishing Services

Book Categories

Submission Form

Hotel

Rooms

Booking

Facilities

Location

Gallery

=====

VARIANT SYSTEM

=====

Every component must support

Minimal

Modern

Corporate

Luxury

Creative

Medical

Education

Technology

Dark

Light

Premium

=====

COMPONENT ATTRIBUTES

=====

Component ID

Component Name

Variant

Theme

Priority

Visibility

Animation

Spacing

Responsive Rules

Accessibility

Dependencies

=====

COMPONENT TREE

=====

Website

↓

Header

↓

Hero

↓

Content Sections

↓

CTA

↓

Contact

↓

Footer

Every component must know

Parent

Children

Siblings

Display Order

=====

COMPONENT PRIORITY

=====

Critical

High

Medium

Low

Optional

=====

COMPONENT RULES

=====

No duplicated components.

No orphan components.

No hidden mandatory sections.

No conflicting variants.

No incompatible themes.

=====

COMPONENT DEPENDENCIES

=====

Hero

requires

Primary CTA

Pricing

requires

CTA

Contact

requires

Contact Form

Footer

requires

Navigation Links

Social Links

Copyright

=====

COMPONENT VALIDATION

=====

Validate

Theme Compatibility

Responsive Compatibility

Accessibility

Spacing

Typography

Color Tokens

Animation Tokens

=====

COMPONENT METADATA

=====

Generate

Component Name

Version

Variant

Priority

Dependencies

Accessibility Status

Responsive Status

Theme Status

=====

FILES TO CREATE

=====

agents/prototype/

component_engine.py

component_library.py

component_selector.py

component_tree.py

variant_selector.py

dependency_validator.py

component_validator.py

component_registry.py

component_metadata.py

component_report.py

component_models.py

tests/

=====
DIRECTORY STRUCTURE
=====

prototype/

components/

library/

variants/

metadata/

reports/

logs/

=====
DATABASE
=====

Table

prototype_components

Fields

id

job_id

component_name

variant

theme

priority

dependencies

responsive_ready

accessibility_ready

status

created_at

=====

CONFIGURATION

=====

Component Library

Business Mapping

Variant Rules

Animation Rules

Accessibility Rules

Priority Rules

Dependency Rules

=====

LOGGING

=====

Log

Component Selection

Variant Selection

Dependency Validation

Tree Generation

Validation

Metadata Generation

Report Generation

=====

ERROR HANDLING

=====

Component Missing

Variant Missing

Dependency Failure

Invalid Theme

Registry Error

Validation Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Component Lookup

Component Cache

Registry Cache

Reusable Instances

Low Memory Usage

=====

SECURITY RULES

=====

Never copy proprietary UI libraries.

Never reproduce copyrighted layouts.

Never clone existing websites.

Generate original component structures only.

=====

UNIT TESTS

=====

Library Tests

Variant Tests

Dependency Tests

Validation Tests

Registry Tests

Metadata Tests

=====

INTEGRATION TESTS

=====

Theme Engine

DOM Engine

Visual Engine

Database

Filesystem

Logging

=====

PASS CONDITIONS

=====

Component Tree Generated

Variants Selected

Dependencies Validated

Metadata Generated

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

Component Tree Failure

Variant Failure

Dependency Failure

Registry Failure

Unhandled Exception

Database Failure

=====

AUDIT REPORT

=====

Components Selected

Variants Used

Dependencies

Validation Results

Execution Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Component Intelligence Engine becomes READ ONLY.

No modifications without architecture approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT generate Layout.

Do NOT generate Responsive Rules.

Do NOT generate HTML.

Do NOT generate Preview.

Do NOT continue to Phase 7.7.

Wait for:

PART 2B-2C

PHASE 7.7

LAYOUT INTELLIGENCE ENGINE

=====

END OF PHASE 7.6

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.7

LAYOUT INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the Layout Intelligence Engine.

Do NOT implement Responsive Engine.

Do NOT implement HTML Generation.

Do NOT implement Preview Engine.

Do NOT continue to Phase 7.8.

The Layout Intelligence Engine is responsible for creating the structural arrangement of every page before HTML generation.

This engine determines where every component should be placed, how sections relate to each other, how visual hierarchy is constructed, and how users flow through the page.

This engine does not generate HTML.

It generates a Layout Blueprint.

=====

OBJECTIVE

=====

Generate a professional, conversion-oriented website layout based on:

Business Category

Business Goals

Component Tree

Theme Profile

Visual Intelligence

DOM Intelligence

Audit Report

The layout must improve usability while preserving business context.

=====

PRIMARY RESPONSIBILITIES

=====

Analyze Business Goals

Analyze Component Tree

Generate Section Order

Generate Content Flow

Generate Grid Layout

Generate Container Rules

Generate Spacing Rules

Generate Layout Blueprint

Generate Layout Report

=====

INPUT

=====

Business Profile

Business Category

Component Tree

Theme Profile

Visual Report

DOM Report

Audit Report

=====
OUTPUT
=====

Layout Blueprint

Page Structure

Section Hierarchy

Grid Structure

Container Rules

Spacing Rules

Component Placement Map

Layout Metadata

Layout Report

=====
LAYOUT PIPELINE
=====

Business Category

↓

Business Objectives



Content Prioritization



Section Planning



Component Placement



Grid Assignment



Container Assignment



Spacing Generation



Layout Validation



Blueprint Generation



Layout Report

=====
PAGE STRUCTURE
=====

Header



Navigation



Hero



Primary CTA



About



Services



Features



Trust Section



Testimonials



Statistics



FAQ



Contact



Footer

=====

LAYOUT TYPES

=====

Corporate

Landing Page

Editorial

Portfolio

Medical

Education

Restaurant

Hotel

Agency

E-Commerce

Blog

Publishing

Technology

=====

SECTION PRIORITY

=====

Critical

High

Medium

Low

Optional

=====

CONTENT FLOW

=====

The engine shall optimize user flow.

Awareness



Interest



Trust



Proof



Action



Contact

=====

GRID SYSTEM

=====

Support

Single Column

Two Column

Three Column

Four Column

Mixed Grid

Flexible Grid

Nested Grid

Asymmetrical Grid

=====

CONTAINER STRATEGY

=====

Generate

Page Width

Section Width

Content Width

Maximum Width

Minimum Width

Responsive Containers

=====

SPACING SYSTEM

=====

Generate

Vertical Rhythm

Horizontal Rhythm

Section Gap

Card Gap

Grid Gap

Button Gap

Content Gap

=====

COMPONENT PLACEMENT

=====

Determine

Hero Position

CTA Position

Trust Indicators

Testimonials

Forms

Pricing

Gallery

Blog

Contact

Footer

=====

VISUAL HIERARCHY

=====

Assign

Primary Focus

Secondary Focus

Supporting Content

Interactive Elements

Call To Action Priority

=====

CONVERSION OPTIMIZATION

=====

Optimize

CTA Visibility

Content Readability

Trust Placement

Form Position

Contact Visibility

Social Proof

=====

ACCESSIBILITY

=====

Maintain

Logical Reading Order

Heading Hierarchy

Keyboard Navigation

Section Order

Content Priority

=====

RESPONSIVE PREPARATION

=====

Generate

Desktop Layout Rules

Laptop Layout Rules

Tablet Layout Rules

Mobile Layout Rules

Small Mobile Rules

Only Blueprint.

No HTML.

=====
LAYOUT BLUEPRINT
=====

Generate

Section IDs

Section Order

Component Positions

Container Definitions

Grid Definitions

Spacing Definitions

Priority Levels

Dependencies

=====
LAYOUT METADATA
=====

Generate

Blueprint Version

Layout Type

Business Category

Theme Name

Component Count

Section Count

Complexity Score

=====

FILES TO CREATE

=====

agents/prototype/

layout_engine.py

layout_planner.py

layout_blueprint.py

grid_engine.py

container_engine.py

spacing_engine.py

content_flow.py

hierarchy_engine.py

layout_validator.py

layout_report.py

layout_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

layouts/

blueprints/

grids/

containers/

reports/

logs/

=====

Table

prototype_layouts

Fields

id

job_id

layout_type

section_count

component_count

grid_type

complexity_score

validation_status

created_at

=====

CONFIGURATION

=====

Layout Templates

Grid Rules

Spacing Scale

Container Widths

Priority Rules

Business Mapping

Validation Rules

=====

LOGGING

=====

Log

Layout Planning

Section Generation

Grid Assignment

Container Assignment

Spacing Assignment

Validation

Blueprint Export

Report Generation

=====

ERROR HANDLING

=====

Invalid Component Tree

Missing Theme

Layout Conflict

Grid Conflict

Container Failure

Validation Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Layout Planning

Reusable Templates

Blueprint Caching

Low Memory Usage

Deterministic Results

=====

SECURITY RULES

=====

Never reproduce copyrighted layouts.

Never clone existing websites.

Generate original layout structures only.

=====

UNIT TESTS

=====

Grid Engine

Container Engine

Spacing Engine

Hierarchy Engine

Placement Engine

Blueprint Generator

Validation

=====
INTEGRATION TESTS
=====

Theme Engine

Component Engine

Visual Engine

DOM Engine

Database

Logging

Filesystem

=====
PASS CONDITIONS
=====

Layout Blueprint Generated

Grid Assigned

Containers Generated

Spacing Rules Generated

Validation Passed

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

Blueprint Generation Failed

Layout Conflict

Validation Failure

Unhandled Exception

Database Failure

=====

AUDIT REPORT

=====

Layout Type

Sections Generated

Grid Type

Component Placement

Complexity Score

Execution Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Layout Intelligence Engine becomes READ ONLY.

No modifications allowed without architecture approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT generate Responsive Code.

Do NOT generate HTML.

Do NOT generate CSS.

Do NOT generate Preview.

Do NOT continue to Phase 7.8.

Wait for:

PART 2B-2C

PHASE 7.8

RESPONSIVE INTELLIGENCE ENGINE

=====

END OF PHASE 7.7

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.8

RESPONSIVE INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the Responsive Intelligence Engine.

Do NOT implement HTML Generation.

Do NOT implement CSS Generation.

Do NOT implement Preview Engine.

Do NOT continue to Phase 7.9.

The Responsive Intelligence Engine is responsible for transforming the approved Layout Blueprint into a fully responsive layout specification.

This engine generates responsive behavior only.

It never generates HTML.

It never generates CSS.

It produces Responsive Blueprint Definitions.

=====

OBJECTIVE

=====

Generate an adaptive responsive structure capable of providing an optimal user experience across all supported devices.

The Responsive Intelligence Engine must ensure that every component, section and layout remains usable regardless of screen size.

=====

PRIMARY RESPONSIBILITIES

=====

Analyze Layout Blueprint

Analyze Component Tree

Analyze Grid System

Generate Breakpoints

Generate Responsive Rules

Generate Component Adaptation

Generate Typography Scaling

Generate Spacing Scaling

Generate Navigation Rules

Generate Responsive Blueprint

Generate Responsive Report

=====

INPUT

=====

Layout Blueprint

Component Tree

Theme Tokens

Grid Definition

Spacing Rules

Container Rules

Business Category

=====

OUTPUT

=====

Responsive Blueprint

Breakpoint Rules

Component Responsive Rules

Responsive Grid Rules

Typography Scale

Spacing Scale

Responsive Navigation Rules

Responsive Report

=====

RESPONSIVE PIPELINE

=====

Layout Blueprint



Breakpoint Analysis



Grid Transformation



Container Adaptation



Component Adaptation



Typography Scaling



Spacing Scaling



Navigation Adaptation



Responsive Validation



Responsive Blueprint



Responsive Report

=====

SUPPORTED DEVICES

=====

Large Desktop

Desktop

Laptop

Tablet Landscape

Tablet Portrait

Large Mobile

Mobile

Small Mobile

=====

BREAKPOINT SYSTEM

=====

Desktop XL

Desktop

Laptop

Tablet

Mobile Large

Mobile

Small Mobile

Breakpoints must be configurable.

=====

RESPONSIVE STRATEGY

=====

Desktop First Blueprint

↓

Responsive Adaptation

↓

Tablet Optimization

↓

Mobile Optimization

↓

Accessibility Validation

↓

Responsive Blueprint

=====

GRID ADAPTATION

=====

Support

12 Column Grid

8 Column Grid

6 Column Grid

4 Column Grid

2 Column Grid

1 Column Grid

Auto Grid

Fluid Grid

=====

CONTAINER ADAPTATION

=====

Generate

Max Width

Fluid Width

Padding

Margins

Safe Areas

Alignment

=====

SECTION ADAPTATION

=====

Optimize

Hero

About

Services

Features

Pricing

Portfolio

Gallery

Blog

Contact

Footer

=====

COMPONENT ADAPTATION

=====

Generate

Responsive Cards

Responsive Buttons

Responsive Forms

Responsive Images

Responsive Tables

Responsive Navigation

Responsive CTA

Responsive Footer

=====

TYPOGRAPHY SCALING

=====

Generate

Heading Scale

Body Scale

Caption Scale

Button Scale

Navigation Scale

Line Height

Letter Spacing

=====

SPACING SCALING

=====

Generate

Container Padding

Section Padding

Grid Gap

Card Gap

Button Gap

Content Gap

Margin Scale

=====

IMAGE ADAPTATION

=====

Generate

Responsive Width

Aspect Ratio

Object Fit

Lazy Loading Rule

Compression Rule

Priority Rule

=====

NAVIGATION ADAPTATION

=====

Desktop Navigation



Tablet Navigation



Hamburger Menu



Drawer Navigation



Sticky Navigation



Bottom Navigation (Optional)

=====

FORM ADAPTATION

=====

Inputs

Buttons

Validation Messages

Dropdowns

Checkboxes

Radio Buttons

Textarea

Submit Buttons

=====

RESPONSIVE ACCESSIBILITY

=====

Maintain

Touch Targets

Readable Text

Contrast

Spacing

Keyboard Navigation

Focus Order

=====

PERFORMANCE RULES

=====

Reduce

DOM Complexity

Rendering Cost

Animation Load

Image Weight

Unused Components

=====

DEVICE VALIDATION

=====

Validate

Desktop

Laptop

Tablet

Tablet Portrait

Tablet Landscape

Large Mobile

Mobile

Small Mobile

=====

RESPONSIVE BLUEPRINT

=====

Generate

Breakpoint Definitions

Component Rules

Container Rules

Typography Rules

Spacing Rules

Grid Rules

Visibility Rules

Interaction Rules

=====

FILES TO CREATE

=====

agents/prototype/

responsive_engine.py
breakpoint_manager.py
responsive_rules.py
grid_adapter.py
container_adapter.py
component_adapter.py
typography_scaler.py
spacing_scaler.py
navigation_adapter.py
responsive_validator.py
responsive_report.py
responsive_models.py
tests/

=====

DIRECTORY STRUCTURE

=====

prototype/
responsive/
breakpoints/
rules/
reports/
logs/

=====

DATABASE

=====

Table

prototype_responsive

Fields

id

job_id

breakpoint_profile

device_support

responsive_score

validation_status

execution_time

created_at

=====

CONFIGURATION

=====

Breakpoint Profiles

Responsive Rules

Device Matrix

Container Widths

Typography Scale

Spacing Scale

Performance Rules

Accessibility Rules

=====

LOGGING

=====

Log

Breakpoint Generation

Grid Adaptation

Container Adaptation

Typography Scaling

Spacing Scaling

Validation

Blueprint Export

Responsive Report

=====

ERROR HANDLING

=====

Invalid Layout

Breakpoint Conflict

Component Conflict

Scaling Failure

Validation Failure

Configuration Error

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Deterministic Output

Blueprint Caching

Low Memory Usage

Parallel Processing

Reusable Rules

=====

SECURITY RULES

=====

Never generate executable code.

Never modify source assets.

Generate responsive specifications only.

=====

UNIT TESTS

=====

Breakpoint Tests

Grid Tests

Typography Tests

Spacing Tests

Navigation Tests

Validation Tests

Blueprint Tests

=====

INTEGRATION TESTS

=====

Layout Engine

Theme Engine

Component Engine

Database

Logging

Filesystem

Pipeline

=====
PASS CONDITIONS
=====

Responsive Blueprint Generated

Breakpoints Generated

Typography Scale Generated

Spacing Rules Generated

Validation Passed

Database Updated

Logs Written

All Tests Passed

=====
FAIL CONDITIONS
=====

Breakpoint Failure

Scaling Failure

Validation Failure

Unhandled Exception

Database Failure

=====

AUDIT REPORT

=====

Breakpoint Profiles

Supported Devices

Responsive Score

Validation Results

Execution Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Responsive Intelligence Engine becomes READ ONLY.

No architectural modifications allowed without approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT generate HTML.

Do NOT generate CSS.

Do NOT generate Preview.

Do NOT continue to Phase 7.9.

Wait for:

PART 2B-2C

PHASE 7.9

HTML GENERATION ENGINE

```
=====
END OF PHASE 7.8
# AI WEBSITE UPGRADE AGENCY
## ENTERPRISE DEVELOPMENT KIT (EDK)
### MASTER SYSTEM PROMPT
### PART 2B-2C
### PHASE 7.9
### HTML GENERATION ENGINE
```

```
=====
MISSION
=====
```

Implement only the HTML Generation Engine.

Do NOT implement Preview Engine.

Do NOT implement Quality Intelligence.

Do NOT implement Export Engine.

Do NOT continue to Phase 7.10.

The HTML Generation Engine is responsible for converting the approved Responsive Blueprint into a production-quality HTML prototype.

This engine must generate clean, semantic, maintainable, responsive and accessible code.

The generated output is intended for demonstration purposes.

It must never clone an existing website.

It must generate an original implementation using the approved layout, components and theme.

=====

OBJECTIVE

=====

Generate a complete responsive website prototype from structured design intelligence.

The engine shall produce:

- Semantic HTML5
- Tailwind CSS
- Minimal JavaScript
- Responsive Layout
- Accessible Components
- SEO Friendly Structure
- Production Quality Code

=====

PRIMARY RESPONSIBILITIES

=====

Load Blueprint

Load Components

Load Theme Tokens

Load Responsive Rules

Generate HTML

Generate Tailwind Classes

Generate Accessibility Markup

Generate Metadata

Optimize Output

Validate HTML

Generate Build Report

=====

INPUT

=====

Responsive Blueprint

Layout Blueprint

Component Tree

Theme Tokens

Typography Tokens

Spacing Tokens

Color Tokens

Business Profile

Business Category

=====

OUTPUT

=====

index.html

assets/

components/

images/

icons/

fonts/

styles/

scripts/

prototype.json

generation_report.json

=====

GENERATION PIPELINE

=====

Responsive Blueprint

↓

Layout Blueprint

↓

Component Tree

↓

Theme Tokens

↓

Tailwind Generator

↓

Semantic HTML Generator

↓

Accessibility Generator

↓

SEO Generator

↓

Optimization

↓

Validation

↓

Prototype Build

↓

Generation Report

=====

PROJECT STRUCTURE

=====

prototype/

index.html

assets/

css/

js/

images/

icons/

fonts/

components/

metadata/

reports/

=====

HTML STANDARDS

=====

Generate

DOCTYPE

HTML5

Semantic Tags

Clean Structure

Readable Indentation

Reusable Sections

Minimal Nesting

Logical Ordering

=====

SEMANTIC TAGS

=====

Use

header

nav

main

section

article

aside

footer

figure

picture

form

button

dialog

details

summary

=====

COMPONENT RENDERING

=====

Render

Header

Navigation

Hero

About

Services

Features

Products

Portfolio

Testimonials

Statistics

Pricing

FAQ

Blog

Gallery

Team

Contact

Footer

=====

TAILWIND CSS RULES

=====

Generate

Utility Classes

Responsive Classes

Spacing Utilities

Grid Utilities

Flex Utilities

Typography Utilities

Color Utilities

Animation Utilities

State Utilities

Avoid inline styles.

Avoid duplicated utilities.

=====

DESIGN TOKEN INJECTION

=====

Inject

Primary Color

Secondary Color

Accent Color

Typography Scale

Spacing Scale

Border Radius

Shadow Tokens

Animation Tokens

Container Width

=====

TYPOGRAPHY

=====

Generate

Heading Hierarchy

Paragraph Styles

Lists

Captions

Navigation

Buttons

Labels

=====

COLOR SYSTEM

=====

Apply

Primary

Secondary

Accent

Surface

Background

Text

Success

Warning

Error

=====

GRID SYSTEM

=====

Generate

Responsive Grid

Flex Layout

Container

Columns

Rows

Auto Layout

Nested Grid

=====

IMAGE HANDLING

=====

Generate

Responsive Images

Alt Text

Lazy Loading

Object Fit

Image Placeholder

Fallback Image

=====

ICON HANDLING

=====

Generate

Accessible Icons

Consistent Sizing

SVG Preferred

Decorative Icons

Interactive Icons

=====

FORMS

=====

Generate

Contact Form

Input Fields

Textarea

Dropdown

Checkbox

Radio

Validation Message

Submit Button

=====

BUTTONS

=====

Generate

Primary

Secondary

Outline

Ghost

CTA

Icon Button

=====

ACCESSIBILITY

=====

Generate

ARIA Labels

ARIA Roles

Landmarks

Keyboard Navigation

Focus States

Accessible Forms

Screen Reader Support

Alt Text

Heading Hierarchy

=====

SEO

=====

Generate

Title

Meta Description

Canonical URL

Open Graph

Twitter Cards

Structured Heading

Semantic HTML

Schema Placeholder

=====

SCHEMA SUPPORT

=====

Support

Organization

Local Business

Website

Breadcrumb

Article

FAQ

Product

Service

=====

JAVASCRIPT RULES

=====

Generate only when required.

Allowed

Navigation Toggle

Accordion

Tabs

Modal

Form Validation

Theme Toggle

Animation Trigger

Avoid unnecessary JavaScript.

=====

PERFORMANCE OPTIMIZATION

=====

Minimize DOM Depth

Remove Duplicate Elements

Lazy Load Images

Optimize Assets

Reduce Layout Shift

Generate Clean HTML

=====

CODE QUALITY

=====

Readable

Modular

Reusable

Consistent

Documented

Maintainable

Production Ready

=====

FILES TO CREATE

=====

agents/prototype/

html_generator.py

tailwind_generator.py

component_renderer.py

token_injector.py

accessibility_generator.py

seo_generator.py

schema_generator.py

asset_manager.py

build_optimizer.py

html_validator.py

build_report.py

generation_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

html/

assets/

css/

js/

components/

icons/

images/

metadata/

reports/

logs/

=====

DATABASE

=====

Table

prototype_builds

Fields

id

job_id

build_version

component_count

html_size

asset_count

seo_score

accessibility_score

validation_status

generation_time

created_at

=====

CONFIGURATION

=====

HTML Template

Tailwind Version

Theme Tokens

Asset Paths

SEO Defaults

Schema Templates

Compression Level

Output Directory

=====

LOGGING

=====

Log

Blueprint Loaded

Components Rendered

HTML Generated

Tailwind Generated

SEO Generated

Accessibility Generated

Validation Completed

Prototype Built

Report Generated

=====

ERROR HANDLING

=====

Template Missing

Component Failure

Token Failure

Asset Failure

Validation Failure

Build Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Build

Reusable Templates

Minimal Memory Usage

Deterministic Output

Parallel Rendering Supported

=====

SECURITY RULES

=====

Never inject external scripts.

Never embed malicious code.

Never expose API keys.

Never generate executable backend logic.

Generate frontend prototype only.

=====

UNIT TESTS

=====

HTML Generator

Tailwind Generator

Component Renderer

Accessibility Generator

SEO Generator

Asset Manager

Validator

Build Report

=====

INTEGRATION TESTS

=====

Responsive Engine

Layout Engine

Component Engine

Theme Engine

Database

Filesystem

Logging

Pipeline

=====

PASS CONDITIONS

=====

Semantic HTML Generated

Tailwind CSS Generated

Responsive Structure Generated

Accessibility Passed

SEO Metadata Generated

Validation Passed

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

HTML Generation Failed

Validation Failed

Asset Failure

Accessibility Failure

Unhandled Exception

Database Failure

=====
AUDIT REPORT
=====

HTML Files Generated

Components Rendered

Assets Generated

SEO Score

Accessibility Score

Validation Result

Build Size

Execution Time

Warnings

Errors

PASS / FAIL

=====
LOCK CONDITIONS
=====

After PASS

HTML Generation Engine becomes READ ONLY.

No modifications allowed without architecture approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT generate Preview.

Do NOT generate Quality Intelligence.

Do NOT generate Export Packages.

Do NOT continue to Phase 7.10.

Wait for:

PART 2B-2C

PHASE 7.10

PREVIEW ENGINE

=====

END OF PHASE 7.9

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.10

PREVIEW ENGINE

=====

MISSION

=====

Implement only the Preview Engine.

Do NOT implement Quality Intelligence.

Do NOT implement Prototype Export Engine.

Do NOT continue to Phase 7.11.

The Preview Engine is responsible for transforming the generated

HTML prototype into a presentation-ready interactive preview.

This engine is the first visual output that a client will see.

The objective is to present the generated prototype in the most professional, responsive and reliable manner.

The Preview Engine shall never modify HTML.

It only renders and validates the generated prototype.

=====

OBJECTIVE

=====

Generate an enterprise-quality preview environment capable of displaying responsive prototypes across multiple devices.

Generate presentation assets that can be used for

Client Review

Internal QA

Sales Demonstration

Approval Workflow

=====

PRIMARY RESPONSIBILITIES

=====

Load Prototype

Launch Preview Server

Render Prototype

Capture Preview

Generate Device Views

Generate Before/After Comparison

Generate Preview Metadata

Validate Rendering

Generate Preview Report

=====

INPUT

=====

Generated HTML

Assets

CSS

JavaScript

Prototype Metadata

Responsive Blueprint

=====

OUTPUT

=====

Live Preview

Desktop Preview

Laptop Preview

Tablet Preview

Mobile Preview

Before Image

After Image

Comparison Report

Preview Metadata

=====

PREVIEW PIPELINE

=====

Generated Prototype



Local Preview Server



Browser Rendering



Desktop Preview



Laptop Preview



Tablet Preview



Mobile Preview



Visual Validation



Comparison Generation



Preview Report

=====

SUPPORTED DEVICES

=====

Desktop

1920×1080

Laptop

1440×900

Tablet Landscape

1024×768

Tablet Portrait

768×1024

Large Mobile

430×932

Standard Mobile

390×844

Small Mobile

360×640

=====

PREVIEW SERVER

=====

Generate

Temporary Local Server

Asset Loader

Routing

Static Resources

Preview Session

Session Cleanup

=====

RENDER VALIDATION

=====

Validate

Page Loaded

Images Loaded

Fonts Loaded

Icons Loaded

Animations Loaded

No Rendering Errors

=====

VISUAL VALIDATION

=====

Detect

Broken Layout

Broken Images

Missing Fonts

Missing Icons

Overflow

Hidden Elements

Unexpected Scroll

Alignment Issues

=====

MULTI DEVICE PREVIEW

=====

Generate

Desktop View

Laptop View

Tablet View

Mobile View

Each preview must include

Resolution

Aspect Ratio

Timestamp

=====

BEFORE / AFTER COMPARISON

=====

Generate

Original Website Screenshot

↓

Generated Prototype



Side by Side Comparison



Overlay Comparison



Improvement Summary

=====

COMPARISON METRICS

=====

Measure

Layout Improvement

Visual Hierarchy

Typography

Spacing

Brand Quality

Accessibility

CTA Visibility

Navigation

=====

INTERACTIVE PREVIEW

=====

Support

Navigation

Buttons

Forms

Links

Accordion

Tabs

Hover States

Scrolling

=====

PREVIEW METADATA

=====

Generate

Preview Version

Generation Time

Resolution

Viewport

Build Version

Theme

Layout Version

=====

FILES TO CREATE

=====

agents/prototype/

preview_engine.py

preview_server.py

render_validator.py

device_renderer.py

comparison_engine.py

preview_capture.py

preview_metadata.py

preview_report.py

preview_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

preview/

desktop/

laptop/

tablet/

mobile/

comparison/

reports/

logs/

=====

DATABASE

=====

Table

prototype_previews

Fields

id

job_id

preview_version

desktop_image

laptop_image

tablet_image

mobile_image

comparison_image

preview_score

status

created_at

=====
CONFIGURATION
=====

Preview Port

Default Browser

Supported Devices

Image Format

Screenshot Quality

Comparison Mode

Timeout

Output Directory

=====

LOGGING

=====

Log

Preview Started

Server Started

Render Complete

Desktop Generated

Laptop Generated

Tablet Generated

Mobile Generated

Comparison Generated

Preview Report Generated

=====

ERROR HANDLING

=====

Server Failure

Render Failure

Asset Missing

Screenshot Failure

Browser Failure

Timeout

Comparison Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Preview Rendering

Low Memory Usage

Reusable Preview Sessions

Parallel Device Rendering

Optimized Asset Loading

=====

SECURITY RULES

=====

Never expose local file paths.

Never expose API credentials.

Never expose development configuration.

Preview must execute inside an isolated environment.

=====

UNIT TESTS

=====

Preview Server

Rendering

Screenshot Capture

Comparison Engine

Metadata

Validation

=====

INTEGRATION TESTS

=====

HTML Generation Engine

Responsive Engine

Filesystem

Database

Logging

Pipeline

=====

PASS CONDITIONS

=====

Prototype Rendered Successfully

Desktop Preview Generated

Laptop Preview Generated

Tablet Preview Generated

Mobile Preview Generated

Comparison Generated

Metadata Generated

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

Preview Failure

Rendering Failure

Asset Failure

Comparison Failure

Unhandled Exception

Database Failure

=====

AUDIT REPORT

=====

Preview Generated

Devices Tested

Screenshots Generated

Comparison Generated

Rendering Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Preview Engine becomes READ ONLY.

No modifications allowed without architecture approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT implement Quality Intelligence.

Do NOT implement Export Engine.

Do NOT continue to Phase 7.11.

Wait for:

PART 2B-2C

PHASE 7.11

QUALITY INTELLIGENCE ENGINE

=====

END OF PHASE 7.10

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.11

QUALITY INTELLIGENCE ENGINE

=====

MISSION

=====

Implement only the Quality Intelligence Engine.

Do NOT implement Prototype Export Engine.

Do NOT continue to Phase 7.12.

The Quality Intelligence Engine is responsible for performing the final enterprise quality inspection of every generated prototype before it is approved for presentation.

This engine is the final automated reviewer of the complete Prototype Intelligence Engine pipeline.

The engine never modifies the prototype.

It only validates, measures, scores and certifies quality.

=====

OBJECTIVE

=====

Evaluate every generated prototype against enterprise quality standards.

Generate measurable quality scores.

Identify defects.

Generate recommendations.

Determine whether the prototype is ready for client presentation.

=====

PRIMARY RESPONSIBILITIES

=====

Validate HTML

Validate Responsive Layout

Validate Accessibility

Validate Performance

Validate SEO

Validate Components

Validate Theme

Validate Typography

Validate Images

Validate UX

Generate Quality Scores

Generate Certification Report

=====

INPUT

=====

Generated Prototype

Responsive Blueprint

Layout Blueprint

Theme Profile

Component Tree

Visual Report

DOM Report

Preview Report

Audit Report

=====

OUTPUT

=====

Quality Report

Quality Scores

Accessibility Report

Performance Report

SEO Report

UX Report

Issue List

Recommendations

Certification Result

=====

QUALITY PIPELINE

=====

Prototype



HTML Validation



Accessibility Validation



Responsive Validation



Performance Validation



SEO Validation



Visual Validation



UX Validation



Component Validation



Scoring Engine



Certification Report

=====

QUALITY DIMENSIONS

=====

HTML Quality

CSS Quality

JavaScript Quality

Accessibility

SEO

Performance

Responsive Design

Visual Design

Typography

Color System

Spacing

Navigation

Content Structure

Brand Consistency

Code Quality

Maintainability

=====

HTML VALIDATION

=====

Validate

HTML5

Semantic Structure

Invalid Tags

Broken Links

Duplicate IDs

Heading Hierarchy

DOM Structure

Code Organization

=====

ACCESSIBILITY VALIDATION

=====

Validate

ARIA

Alt Text

Keyboard Navigation

Color Contrast

Heading Order

Form Labels

Focus Indicators

Touch Targets

Screen Reader Compatibility

=====

RESPONSIVE VALIDATION

=====

Validate

Desktop

Laptop

Tablet

Tablet Portrait

Large Mobile

Mobile

Small Mobile

Check

Overflow

Broken Layout

Hidden Elements

Scaling

Alignment

=====

SEO VALIDATION

=====

Validate

Title

Meta Description

Canonical

Open Graph

Twitter Metadata

Structured Headings

Schema Placeholder

Image Alt Text

=====

PERFORMANCE VALIDATION

=====

Measure

DOM Size

Image Weight

Unused Assets

Render Complexity

Layout Stability

Animation Cost

Asset Count

Overall Build Size

=====

VISUAL VALIDATION

=====

Validate

Typography

Spacing

Alignment

Color Consistency

Grid Consistency

Visual Hierarchy

CTA Visibility

Brand Consistency

=====

UX VALIDATION

=====

Measure

Navigation Flow

Content Readability

Form Usability

Button Visibility

Conversion Flow

Trust Indicators

Contact Visibility

User Journey

=====

COMPONENT VALIDATION

=====

Validate

Component Integrity

Variant Selection

Dependencies

Missing Components

Broken Components

Theme Compatibility

=====

QUALITY SCORING

=====

Generate

HTML Score

Accessibility Score

Performance Score

Responsive Score

SEO Score

Visual Score

UX Score

Component Score

Maintainability Score

Overall Quality Score

=====

CERTIFICATION LEVELS

=====

95–100

Enterprise Certified

90–94

Production Ready

80–89

Client Presentation Ready

70–79

Requires Minor Improvements

Below 70

Rejected

=====

ISSUE CLASSIFICATION

=====

Critical

High

Medium

Low

Recommendation

=====

RECOMMENDATION ENGINE

=====

Generate

Critical Fixes

Performance Improvements

Accessibility Improvements

SEO Improvements

UX Improvements

Visual Improvements

Component Improvements

=====

FILES TO CREATE

=====

agents/prototype/

quality_engine.py

quality_validator.py

quality_score.py

accessibility_checker.py

seo_checker.py

performance_checker.py

ux_checker.py

component_checker.py

certification_engine.py

recommendation_engine.py

quality_report.py

quality_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

quality/

reports/

certificates/

recommendations/

logs/

=====

DATABASE

=====

Table

prototype_quality

Fields

id

job_id

html_score

accessibility_score

performance_score

responsive_score

seo_score

ux_score

visual_score

component_score

overall_score

certification_level

status

created_at

=====

CONFIGURATION

=====

Quality Threshold

Certification Levels

Accessibility Rules

SEO Rules

Performance Limits

Responsive Matrix

Validation Profile

=====

LOGGING

=====

Log

Validation Started

HTML Validation

Accessibility Validation

Responsive Validation

SEO Validation

Performance Validation

UX Validation

Quality Score

Certification

Report Generated

=====

ERROR HANDLING

=====

Validation Failure

Missing Assets

Missing Reports

Calculation Failure

Database Failure

Unhandled Exception

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Validation

Parallel Quality Checks

Reusable Validators

Cached Results

Low Memory Usage

=====

SECURITY RULES

=====

Never modify prototype code.

Never execute unknown scripts.

Validate only trusted generated assets.

Never expose internal validation logic.

=====

UNIT TESTS

=====

HTML Validator

Accessibility Checker

SEO Checker

Performance Checker

UX Checker

Quality Scoring

Certification Engine

Recommendation Engine

=====

INTEGRATION TESTS

=====

Preview Engine

HTML Engine

Responsive Engine

Database

Filesystem

Logging

Pipeline

=====

PASS CONDITIONS

=====

All validations completed.

Overall Quality Score generated.

Certification generated.

Reports generated.

Database updated.

Logs written.

All tests passed.

=====

FAIL CONDITIONS

=====

Validation Failure

Certification Failure

Missing Reports

Database Failure

Unhandled Exception

=====

AUDIT REPORT

=====

Validation Modules Executed

Overall Quality Score

Certification Level

Critical Issues

Warnings

Recommendations

Execution Time

PASS / FAIL

=====
LOCK CONDITIONS
=====

After PASS

Quality Intelligence Engine becomes READ ONLY.

No modifications allowed without architecture approval.

=====
GLOBAL RESTRICTIONS
=====

Do NOT implement Export Engine.

Do NOT continue to Phase 7.12.

Wait for:

PART 2B-2C

PHASE 7.12

PROTOTYPE EXPORT ENGINE

=====
END OF PHASE 7.11
AI WEBSITE UPGRADE AGENCY
ENTERPRISE DEVELOPMENT KIT (EDK)
MASTER SYSTEM PROMPT
PART 2B-2C
PHASE 7.12

PROTOTYPE EXPORT ENGINE

=====

MISSION

=====

Implement only the Prototype Export Engine.

Do NOT implement Final Certification.

Do NOT continue to Phase 7.13.

The Prototype Export Engine is responsible for packaging every approved prototype into standardized client-ready deliverables.

The engine shall export only prototypes that have successfully passed the Quality Intelligence Engine.

No export shall be allowed without certification approval.

=====

OBJECTIVE

=====

Generate professional delivery packages that can be shared with clients, internal teams and future development workflows.

The export process must be deterministic, version-controlled and repeatable.

=====

PRIMARY RESPONSIBILITIES

=====

Validate Certification

Collect Build Assets

Generate Export Package

Generate Static Website

Generate ZIP Package

Generate Presentation Assets

Generate Screenshots

Generate Build Manifest

Generate Checksums

Generate Export Report

=====

INPUT

=====

Certified Prototype

Generated HTML

CSS Assets

JavaScript Assets

Images

Icons

Fonts

Metadata

Quality Report

Preview Report

=====

OUTPUT

=====

Static Website

ZIP Package

Preview Images

Presentation Assets

Build Manifest

Integrity Report

Export Report

Version Metadata

=====

EXPORT PIPELINE

=====

Certified Prototype

↓

Export Validation

↓

Asset Collection

↓

Build Packaging

↓

Manifest Generation

↓

Integrity Verification

↓

Export Generation

↓

Export Validation



Export Report

=====

EXPORT FORMATS

=====

Static HTML Package

ZIP Archive

Preview Images

Build Manifest

JSON Metadata

Project Structure

Export Report

=====

STATIC WEBSITE

=====

Generate

index.html

Assets

Images

Icons

Fonts

CSS

JavaScript

Metadata

404 Page (Optional)

=====

ZIP PACKAGE

=====

Include

HTML

Assets

Images

Fonts

Icons

CSS

JavaScript

Manifest

Metadata

License Placeholder

Readme

=====

PREVIEW EXPORTS

=====

Generate

Desktop PNG

Laptop PNG

Tablet PNG

Mobile PNG

Comparison Image

Hero Screenshot

Landing Screenshot

=====

PRESENTATION ASSETS

=====

Generate

Client Preview Images

Before / After Images

Summary Images

Prototype Overview

Improvement Highlights

=====

MANIFEST

=====

Generate

Build Version

Project Name

Prototype Version

Generated Time

Theme

Layout

Components

Assets

Checksums

Generator Version

=====

VERSIONING

=====

Generate

Major Version

Minor Version

Patch Version

Build Number

Revision Number

Export Timestamp

=====

INTEGRITY VERIFICATION

=====

Verify

Assets Present

HTML Present

CSS Present

JavaScript Present

Manifest Present

Metadata Present

Preview Images Present

=====

CHECKSUM

=====

Generate

Package Checksum

Asset Checksums

Manifest Checksum

Integrity Signature

=====

EXPORT METADATA

=====

Generate

Export ID

Job ID

Client ID

Business Category

Theme

Build Version

Generation Date

Generator Version

=====

FILES TO CREATE

=====

agents/prototype/

export_engine.py

export_validator.py

package_builder.py

manifest_generator.py

version_manager.py

checksum_generator.py

asset_packager.py

integrity_checker.py

export_report.py

export_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

exports/

packages/

static/

images/

manifest/

reports/

logs/

=====

DATABASE

=====

Table

prototype_exports

Fields

id

job_id

export_version

package_name

package_size

checksum

export_status

validation_status

generated_at

=====
CONFIGURATION
=====

Export Directory

Package Format

Compression Level

Version Strategy

Manifest Version

Checksum Algorithm

Output Naming

Retention Policy

=====
LOGGING

=====

Log

Export Started

Validation Completed

Assets Collected

Manifest Generated

Checksum Generated

Package Created

Integrity Verified

Export Completed

Report Generated

=====

ERROR HANDLING

=====

Missing Assets

Packaging Failure

Manifest Failure

Checksum Failure

Integrity Failure

Filesystem Error

Database Failure

Retry Logic

Graceful Recovery

=====

PERFORMANCE REQUIREMENTS

=====

Fast Packaging

Efficient Compression

Minimal Memory Usage

Reusable Export Pipeline

Parallel Asset Processing

=====

SECURITY RULES

=====

Never include API Keys.

Never include Environment Files.

Never include Secrets.

Never include Database Credentials.

Never include Internal Logs.

Never include Development Configuration.

=====

PACKAGE STRUCTURE

=====

prototype_export/

index.html

assets/

css/

js/

images/

icons/

fonts/

metadata/

manifest.json

README.md

LICENSE.txt

reports/

=====

UNIT TESTS

=====

Package Builder

Manifest Generator

Checksum Generator

Integrity Checker

Version Manager

Export Validator

Report Generator

=====

INTEGRATION TESTS

=====

HTML Engine

Preview Engine

Quality Engine

Filesystem

Database

Logging

Pipeline

=====

PASS CONDITIONS

=====

Package Generated

Static Website Generated

Manifest Generated

Checksums Generated

Integrity Verified

Reports Generated

Database Updated

Logs Written

All Tests Passed

=====

FAIL CONDITIONS

=====

Export Failure

Packaging Failure

Integrity Failure

Manifest Failure

Database Failure

Unhandled Exception

=====

AUDIT REPORT

=====

Export Version

Package Size

Files Included

Assets Packaged

Checksum

Integrity Status

Execution Time

Warnings

Errors

PASS / FAIL

=====

LOCK CONDITIONS

=====

After PASS

Prototype Export Engine becomes READ ONLY.

No modifications allowed without architecture approval.

=====

GLOBAL RESTRICTIONS

=====

Do NOT continue to Phase 8.

Do NOT modify previous PIE modules.

Wait for:

PART 2B-2C

PHASE 7.13

FINAL PIE VALIDATION

PASS / FAIL

PHASE LOCK

=====

END OF PHASE 7.12

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

MASTER SYSTEM PROMPT

PART 2B-2C

PHASE 7.13

FINAL PIE VALIDATION

PASS / FAIL

PHASE LOCK

=====

MISSION

=====

Implement only the Final PIE Validation System.

Do NOT implement Phase 8.

Do NOT modify any previous Phase 7 module.

This phase performs the final validation of the entire Prototype Intelligence Engine (PIE).

Every module developed from Phase 7.1 through Phase 7.12 shall be validated as one integrated enterprise system.

This phase determines whether PIE is approved for production use.

=====

OBJECTIVE

=====

Validate the complete Prototype Intelligence Engine.

Confirm that every module works correctly.

Ensure production readiness.

Generate the final certification.

Lock the complete Phase 7.

=====

VALIDATION SCOPE

=====

Validate

Screenshot Engine

DOM Intelligence

Visual Intelligence

Theme Intelligence

Component Intelligence

Layout Intelligence

Responsive Intelligence

HTML Generation

Preview Engine

Quality Intelligence

Prototype Export

=====

END TO END PIPELINE

=====

Website URL

↓

Screenshot Engine



DOM Intelligence



Visual Intelligence



Theme Intelligence



Component Intelligence



Layout Intelligence



Responsive Intelligence



HTML Generation



Preview Engine



Quality Intelligence



Prototype Export



Final Validation

=====

INTEGRATION VALIDATION

=====

Verify

Module Communication

Data Flow

Pipeline Order

Dependencies

Database Transactions

Logging

Configuration

Retry Logic

Error Recovery

=====

DATA FLOW VALIDATION

=====

Verify

Screenshot Output

↓

DOM Input

↓

DOM Output

↓

Visual Input



Theme Input



Component Input



Layout Input



Responsive Input



HTML Input



Preview Input



Quality Input



Export Input

No data loss permitted.

=====

DATABASE VALIDATION

=====

Verify

prototype_jobs

prototype_results

prototype_reports

prototype_screenshots

prototype_dom_analysis

prototype_visual_analysis

prototype_themes

prototype_components

prototype_layouts

prototype_responsive

prototype_builds

prototype_previews

prototype_quality

prototype_exports

=====
FILESYSTEM VALIDATION
=====

Verify

Generated HTML

Assets

Images

Icons

Fonts

Reports

Manifest

Export Package

Preview Images

Metadata

Logs

=====

CONFIGURATION VALIDATION

=====

Verify

Environment Variables

Theme Configuration

Browser Configuration

Export Configuration

Logging Configuration

Database Configuration

Filesystem Configuration

=====

SECURITY VALIDATION

=====

Verify

No API Keys

No Secrets

No Credentials

No Debug Files

No Development Configuration

No Internal Stack Traces

No Unsafe File Access

=====

PERFORMANCE VALIDATION

=====

Measure

Pipeline Time

Memory Usage

CPU Usage

Screenshot Time

DOM Analysis Time

HTML Generation Time

Preview Time

Export Time

=====

CODE QUALITY VALIDATION

=====

Verify

SOLID Principles

Clean Architecture

Dependency Injection

Low Coupling

High Cohesion

Modular Design

Code Reuse

Documentation

=====

ACCESSIBILITY VALIDATION

=====

Verify

ARIA

Keyboard Navigation

Heading Hierarchy

Color Contrast

Screen Reader Support

Touch Targets

=====

RESPONSIVE VALIDATION

=====

Verify

Desktop

Laptop

Tablet

Tablet Portrait

Large Mobile

Mobile

Small Mobile

=====

SEO VALIDATION

=====

Verify

Meta Tags

Open Graph

Twitter Cards

Structured Headings

Schema Placeholder

Canonical URL

=====

QUALITY VALIDATION

=====

Verify

Design Score

Performance Score

Accessibility Score

SEO Score

Responsive Score

UX Score

Overall Quality Score

=====

TEST EXECUTION

=====

Execute

Unit Tests

Integration Tests

Regression Tests

Pipeline Tests

Performance Tests

Security Tests

Filesystem Tests

Database Tests

Accessibility Tests

Responsive Tests

=====

MINIMUM TEST COVERAGE

=====

Unit Tests

95%

Integration Tests

100%

Pipeline Tests

100%

Critical Modules

100%

=====

PRODUCTION READINESS CHECKLIST

=====

Verify

Architecture Complete

Modules Complete

Database Stable

Logging Stable

Exports Working

Reports Working

Preview Working

Validation Complete

No Critical Bugs

No High Severity Issues

=====
ISSUE CLASSIFICATION
=====

Critical

High

Medium

Low

Information

=====
FAIL CONDITIONS
=====

Pipeline Failure

Database Failure

HTML Generation Failure

Preview Failure

Export Failure

Security Issue

Critical Accessibility Issue

Critical Performance Issue

Unhandled Exception

=====
PASS CONDITIONS
=====

All Modules Operational

All Integration Tests Passed

All Reports Generated

No Critical Issues

No High Severity Issues

Performance Within Limits

Database Stable

Filesystem Stable

Production Ready

=====
FINAL CERTIFICATION
=====

Generate

Prototype Intelligence Engine Certificate

Certification ID

Certification Date

Generator Version

Architecture Version

Quality Grade

Production Status

=====

CERTIFICATION LEVELS

=====

Level A+

Enterprise Production Certified

Level A

Production Ready

Level B

Client Demonstration Ready

Level C

Internal Testing Only

Level D

Development Only

=====

RELEASE DECISION

=====

PASS

Release Approved

FAIL

Release Blocked

Return To Development

=====

FILES TO CREATE

=====

agents/prototype/

pie_validation_engine.py

integration_validator.py

pipeline_validator.py

production_checker.py

release_manager.py

phase_lock.py

certification_generator.py

final_report.py

validation_models.py

tests/

=====

DIRECTORY STRUCTURE

=====

prototype/

validation/

certificates/

reports/

release/

logs/

=====

DATABASE

=====

Table

prototype_release

Fields

id

release_version

architecture_version

certification_level

overall_score

production_ready

release_status

validated_at

=====

LOGGING

=====

Log

Validation Started

Pipeline Verified

Database Verified

Filesystem Verified

Security Verified

Performance Verified

Certification Generated

Phase Locked

=====

AUDIT REPORT

=====

Modules Validated

Tests Executed

Coverage

Performance Results

Security Results

Quality Results

Warnings

Errors

Certification

PASS / FAIL

=====

PHASE LOCK PROCEDURE

=====

After PASS

Mark

Phase 7.1

LOCKED

↓

Phase 7.2

LOCKED

↓

Phase 7.3

LOCKED

↓

Phase 7.4

LOCKED

↓

Phase 7.5

LOCKED

↓

Phase 7.6

LOCKED

↓

Phase 7.7

LOCKED



Phase 7.8

LOCKED



Phase 7.9

LOCKED



Phase 7.10

LOCKED



Phase 7.11

LOCKED



Phase 7.12

LOCKED



Phase 7.13

LOCKED

Prototype Intelligence Engine

STATUS

ENTERPRISE CERTIFIED

READ ONLY

=====

GLOBAL RESTRICTIONS

=====

No architectural modifications.

No feature additions.

No scope expansion.

No folder structure changes.

No technology changes.

Only critical security fixes are permitted after Phase Lock.

=====

PHASE 7 COMPLETION

=====

Prototype Intelligence Engine

Status

COMPLETED

Architecture

LOCKED

Implementation Specification

LOCKED

Documentation

LOCKED

Quality Certification

PASSED

Release Status

APPROVED

=====

NEXT DOCUMENT

=====

PART 2B-3

PHASE 8

AI PERSONALIZED EMAIL ENGINE

This begins the Outreach Automation Layer.

=====

END OF PHASE 7.13

END OF PART 2B-2C

END OF PROTOTYPE INTELLIGENCE ENGINE

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.1

AI PERSONALIZED EMAIL ENGINE ARCHITECTURE

8.1.1 Overview

The AI Personalized Email Engine is an enterprise service responsible for generating personalized outreach emails from validated intelligence produced by previously completed engines.

The engine operates exclusively on structured system outputs and does not perform any independent discovery, analysis, or validation.

Its responsibility is limited to transforming validated intelligence into professional, personalized communication suitable for outbound business outreach.

8.1.2 Architectural Position

The AI Personalized Email Engine is positioned after completion of all intelligence generation phases.

System Execution Order

...

Foundation Framework

↓

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine

↓

AI Personalized Email Engine

↓

Follow-Up Engine

...

The engine shall not execute before all upstream engines have completed successfully.

8.1.3 Architectural Principles

The architecture shall comply with the following principles:

- Single responsibility.
- Modular execution.
- Deterministic workflow.
- Read-only consumption of upstream intelligence.
- No modification of upstream data.
- Stateless processing.
- Reproducible output generation.
- Enterprise-grade reliability.
- Strict separation of processing stages.

8.1.4 High-Level Architecture

...

Validated Intelligence

|



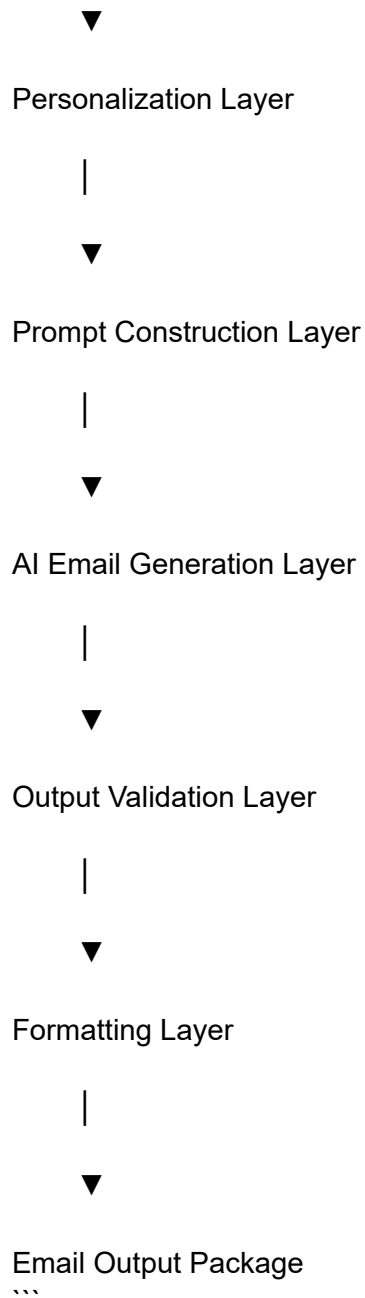
Input Aggregation Layer

|



Context Assembly Layer

|



Each layer performs one defined responsibility and passes structured output to the next processing stage.

8.1.5 Architectural Layers

The engine consists of the following logical layers:

Layer 1

Input Aggregation Layer

Responsibilities:

- Read validated datasets.
- Load required intelligence.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Layer 2

Context Assembly Layer

Responsibilities:

- Merge business intelligence.
- Merge audit intelligence.
- Merge prototype intelligence.
- Merge personalization metadata.
- Construct unified generation context.

Layer 3

Personalization Layer

Responsibilities:

- Select business-specific details.
- Select role-specific language.
- Select industry terminology.
- Select modernization highlights.
- Build personalization variables.

Layer 4

Prompt Construction Layer

Responsibilities:

- Assemble structured AI prompt.
- Inject contextual variables.
- Apply enterprise writing rules.
- Apply agency branding rules.
- Produce deterministic prompt package.

Layer 5

AI Email Generation Layer

Responsibilities:

- Generate subject line.
- Generate email body.
- Generate call-to-action.
- Generate professional closing.
- Produce complete email draft.

Layer 6

Output Validation Layer

Responsibilities:

- Validate completeness.
- Validate personalization.
- Validate formatting.
- Validate prohibited content.
- Validate required sections.

Layer 7

Formatting Layer

Responsibilities:

- Normalize output.
- Apply enterprise formatting.
- Produce structured output package.
- Prepare downstream consumption.

8.1.6 Data Flow

...

Validated Inputs



Input Aggregation



Context Assembly



Personalization



Prompt Construction



AI Generation



Validation



Formatting



Final Email Package

...

Processing shall occur sequentially.

No layer may be skipped.

8.1.7 Upstream Dependencies

The architecture consumes outputs from:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine

No direct execution against external websites is permitted.

8.1.8 Downstream Dependency

The engine publishes structured email packages for consumption by the Follow-Up Engine.

The engine does not perform email delivery.

The engine does not schedule follow-up activities.

8.1.9 State Management

The architecture is stateless.

Each execution shall:

- Load inputs.
- Generate output.
- Return results.

- Release execution state.

No persistent session context shall be maintained within the engine.

8.1.10 Failure Handling

Execution shall terminate when:

- Required inputs are missing.
- Mandatory intelligence is unavailable.
- Validation fails.
- Context assembly fails.
- AI generation fails.
- Output validation fails.

No partial output shall be published.

8.1.11 Scalability Requirements

The architecture shall support:

- Independent execution per lead.
- Parallel processing across multiple leads.
- Isolated execution contexts.
- Horizontal scaling without workflow modification.
- Deterministic output generation for identical inputs.

8.1.12 Security Principles

The architecture shall:

- Operate on validated internal datasets only.
- Prevent modification of upstream intelligence.
- Prevent unauthorized data injection.
- Maintain strict separation between processing layers.
- Preserve integrity of generated outputs.

8.1.13 Completion Requirements

The architecture is considered complete when:

- All architectural layers are defined.
- Data flow is established.
- Processing order is fixed.
- Dependencies are documented.
- Failure handling is specified.
- Scalability requirements are defined.
- Security principles are documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.2

AI PERSONALIZED EMAIL ENGINE INTERNAL MODULES

8.2.1 Overview

The AI Personalized Email Engine is composed of independent internal modules.

Each module performs a single well-defined responsibility.

Modules execute sequentially according to the fixed processing workflow.

No module shall perform responsibilities assigned to another module.

8.2.2 Module Architecture

...

Input Manager



Context Builder



Personalization Manager



Prompt Builder



AI Generation Manager



Email Validator



Output Formatter



Metadata Generator





Final Email Package

...

8.2.3 Input Manager

Purpose

The Input Manager loads all validated upstream intelligence required for email generation.

Responsibilities

- Load validated lead profile.
- Load business information.
- Load contact information.
- Load website audit results.
- Load Prototype Intelligence results.
- Load personalization metadata.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Inputs

- Validated Lead Profile
- Website Audit Report
- Prototype Intelligence Report
- Contact Information
- Agency Configuration

Outputs

Unified Input Package

8.2.4 Context Builder

Purpose

The Context Builder transforms multiple intelligence sources into a unified business context.

Responsibilities

- Merge business information.
- Merge website intelligence.
- Merge audit findings.
- Merge prototype summary.
- Merge personalization metadata.
- Remove duplicated context.
- Normalize structured information.

Inputs

Unified Input Package

Outputs

Unified Business Context

8.2.5 Personalization Manager

Purpose

The Personalization Manager determines which information should appear within the generated email.

Responsibilities

- Select business-specific details.
- Select industry terminology.
- Select modernization opportunities.
- Select role-specific wording.
- Select business strengths.
- Select business improvement areas.
- Prepare personalization variables.

Inputs

Unified Business Context

Outputs

Personalization Context

8.2.6 Prompt Builder

Purpose

The Prompt Builder converts structured context into an enterprise AI prompt.

Responsibilities

- Build prompt structure.
- Insert personalization variables.
- Insert audit highlights.
- Insert prototype references.
- Apply agency writing standards.
- Apply enterprise communication rules.
- Produce deterministic prompts.

Inputs

Personalization Context

Outputs

AI Prompt Package

8.2.7 AI Generation Manager

Purpose

The AI Generation Manager generates the personalized outreach email.

Responsibilities

- Generate subject line.
- Generate opening paragraph.
- Generate personalized body.
- Generate modernization summary.
- Generate value proposition.
- Generate call-to-action.
- Generate professional closing.
- Generate complete email draft.

Inputs

AI Prompt Package

Outputs

Generated Email

8.2.8 Email Validator

Purpose

The Email Validator verifies that generated content satisfies enterprise quality requirements.

Responsibilities

- Validate required sections.
- Validate personalization.
- Validate formatting.
- Validate completeness.
- Validate prohibited content.
- Validate enterprise writing standards.
- Validate structural consistency.

Inputs

Generated Email

Outputs

Validated Email

Validation Report

8.2.9 Output Formatter

Purpose

The Output Formatter converts validated content into standardized system output.

Responsibilities

- Normalize formatting.

- Normalize spacing.
- Normalize headings.
- Normalize line structure.
- Standardize output schema.
- Prepare downstream consumption.

Inputs

Validated Email

Outputs

Formatted Email Package

8.2.10 Metadata Generator

Purpose

The Metadata Generator records execution metadata associated with the generated email.

Responsibilities

- Generate execution identifier.
- Record generation timestamp.
- Record engine version.
- Record processing status.
- Record validation status.
- Record execution metrics.
- Produce metadata package.

Inputs

Formatted Email Package

Validation Report

Outputs

Metadata Package

Final Email Package

8.2.11 Module Interaction Sequence

...

Input Manager

↓

Context Builder

↓

Personalization Manager

↓

Prompt Builder

↓

AI Generation Manager

↓

Email Validator

↓

Output Formatter

↓

Metadata Generator

↓

Final Email Package

...

Module execution order is fixed.

No module may execute before completion of its predecessor.

8.2.12 Module Independence

Each module shall:

- Perform one responsibility only.
- Consume defined inputs.
- Produce defined outputs.
- Avoid direct dependency on non-adjacent modules.
- Preserve upstream data integrity.
- Avoid modification of previously generated intelligence.

8.2.13 Error Propagation

If any module reports failure:

- Execution shall stop immediately.
- Remaining modules shall not execute.
- Partial outputs shall be discarded.
- An execution failure report shall be generated.

8.2.14 Extensibility

Internal modules shall remain independently maintainable.

Future implementation updates shall preserve:

- Module boundaries.
- Processing order.

- Input contracts.
- Output contracts.
- Public interfaces.

8.2.15 Completion Requirements

This phase is considered complete when:

- All internal modules are defined.
- Module responsibilities are documented.
- Input contracts are documented.
- Output contracts are documented.
- Execution sequence is fixed.
- Failure propagation rules are specified.
- Module interaction is fully documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.3

AI PERSONALIZED EMAIL ENGINE DATA CONTRACTS

8.3.1 Overview

The AI Personalized Email Engine exchanges information exclusively through structured data contracts.

Data contracts define the required inputs, generated outputs, mandatory fields, validation requirements, and interface boundaries between the AI Personalized Email Engine and other system components.

All contracts are immutable during execution.

The engine shall consume data in read-only mode.

8.3.2 Contract Design Principles

Every data contract shall comply with the following principles:

- Strongly structured.
- Version controlled.
- Immutable during execution.
- Self-describing.
- Deterministic.
- Platform independent.
- Schema validated.
- Backward compatible within the same major version.

8.3.3 Upstream Input Contracts

The AI Personalized Email Engine consumes the following validated contracts:

- Validated Lead Contract
- Business Profile Contract
- Contact Information Contract
- Website Audit Contract
- Technology Analysis Contract
- SEO Analysis Contract
- Accessibility Analysis Contract
- Security Analysis Contract
- Visual Audit Contract
- Prototype Intelligence Contract
- Component Intelligence Contract
- Theme Intelligence Contract
- Responsive Intelligence Contract
- Layout Intelligence Contract
- Agency Configuration Contract
- Brand Voice Configuration Contract
- Personalization Metadata Contract

Each contract shall be validated prior to processing.

8.3.4 Unified Input Contract

Purpose

The Unified Input Contract represents the normalized dataset consumed by the AI Personalized Email Engine after successful input aggregation.

Required Sections

- Lead Information
- Business Information
- Contact Information
- Website Intelligence
- Prototype Intelligence
- Personalization Data
- Agency Configuration
- Brand Voice Configuration

Validation Requirements

The Unified Input Contract shall:

- Contain all mandatory sections.
- Preserve upstream identifiers.
- Preserve source integrity.
- Contain normalized field formats.
- Exclude duplicate records.

8.3.5 Business Context Contract

Purpose

The Business Context Contract represents the consolidated business intelligence used during personalization.

Required Sections

- Business Identity
- Industry Classification
- Business Category
- Website Summary
- Technology Summary
- Audit Summary
- Prototype Summary
- Business Strengths
- Improvement Opportunities
- Personalization Variables

Validation Requirements

The contract shall contain only verified intelligence.

No inferred or fabricated information shall be included.

8.3.6 Personalization Context Contract

Purpose

The Personalization Context Contract contains the structured variables required for email generation.

Required Sections

- Recipient Details
- Business References
- Industry Terminology
- Website Findings
- Modernization Highlights
- Value Proposition Inputs
- Communication Style
- Agency Identity

Validation Requirements

Every personalization variable shall originate from validated upstream intelligence.

8.3.7 AI Prompt Contract

Purpose

The AI Prompt Contract represents the deterministic instruction package submitted to the AI generation layer.

Required Sections

- System Instructions
- Enterprise Writing Rules
- Business Context
- Personalization Context
- Website Intelligence
- Prototype Intelligence
- Output Requirements
- Formatting Rules
- Validation Constraints

Validation Requirements

The prompt shall:

- Reference verified intelligence only.
- Maintain deterministic structure.
- Exclude unsupported assumptions.
- Preserve enterprise formatting rules.

8.3.8 Generated Email Contract

Purpose

The Generated Email Contract represents the initial AI-generated communication before validation.

Required Sections

- Subject
- Greeting
- Opening Statement
- Personalized Context
- Opportunity Summary
- Value Proposition
- Call To Action
- Professional Closing
- Signature

Validation Requirements

The generated content shall contain every mandatory section prior to validation.

8.3.9 Validated Email Contract

Purpose

The Validated Email Contract represents the approved communication after successful quality verification.

Required Sections

- Approved Subject
- Approved Email Body
- Approved Signature
- Validation Status
- Validation Identifier

Validation Requirements

The contract shall contain only approved content.

Rejected content shall not be retained within this contract.

8.3.10 Metadata Contract

Purpose

The Metadata Contract records execution information associated with email generation.

Required Sections

- Execution Identifier
- Engine Version
- Contract Version
- Processing Timestamp
- Validation Timestamp
- Processing Status
- Validation Status
- Generation Metrics

Validation Requirements

Metadata shall accurately represent the completed execution.

8.3.11 Final Email Package Contract

Purpose

The Final Email Package Contract represents the complete output produced by the AI Personalized Email Engine.

Required Sections

- Validated Email
- Metadata
- Validation Report
- Processing Status

Validation Requirements

The package shall be complete before publication to downstream systems.

8.3.12 Contract Versioning

Every contract shall include:

- Contract Identifier
- Major Version
- Minor Version
- Schema Version

Version identifiers shall remain consistent throughout a single execution.

8.3.13 Contract Validation

Before processing, every contract shall be verified for:

- Structural integrity.
- Required sections.
- Mandatory fields.
- Data consistency.
- Schema compliance.
- Version compatibility.

Contracts failing validation shall be rejected.

8.3.14 Contract Integrity Rules

The AI Personalized Email Engine shall:

- Preserve upstream identifiers.
- Preserve contract ownership.
- Preserve immutable fields.
- Reject unauthorized modifications.
- Prevent duplicate contract publication.
- Prevent incomplete contract generation.

8.3.15 Downstream Publication Contract

The Final Email Package Contract shall be published for downstream consumption by the Follow-Up Engine.

No additional transformation shall occur within the AI Personalized Email Engine after publication.

8.3.16 Completion Requirements

This phase is considered complete when:

- All input contracts are defined.
- All output contracts are defined.
- Mandatory sections are documented.
- Validation requirements are specified.
- Versioning rules are documented.
- Integrity rules are established.
- Downstream publication contract is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.4

AI PERSONALIZED EMAIL ENGINE EXECUTION WORKFLOW

8.4.1 Overview

The AI Personalized Email Engine executes through a fixed sequential workflow.

Each execution stage has a single defined responsibility.

Execution shall proceed only after successful completion of the preceding stage.

No execution stage may be skipped, reordered, or executed in parallel within a single email generation request.

8.4.2 Workflow Objectives

The execution workflow shall:

- Consume validated upstream intelligence.
- Assemble a unified business context.
- Generate personalized outreach content.
- Validate generated communication.
- Produce standardized output.
- Publish a completed email package.

8.4.3 Workflow Entry Conditions

Execution shall begin only when all of the following conditions are satisfied:

- Upstream engines have completed successfully.
- Required input contracts are available.
- Input contract validation has passed.
- Mandatory business information is present.
- Mandatory contact information is present.
- Website intelligence is available.
- Prototype Intelligence is available.
- Agency configuration is available.

Failure to satisfy any entry condition shall terminate execution.

8.4.4 Stage 1 — Input Acquisition

Purpose

Acquire all validated contracts required for execution.

Activities

- Load Unified Input Contract.
- Verify contract availability.
- Verify contract versions.
- Verify mandatory sections.
- Initialize execution context.

Output

Execution Input Context

8.4.5 Stage 2 — Context Assembly

Purpose

Construct a unified business context for personalization.

Activities

- Merge business profile.
- Merge website intelligence.
- Merge audit summaries.
- Merge Prototype Intelligence.
- Merge personalization metadata.
- Normalize context.

Output

Business Context

8.4.6 Stage 3 — Personalization Preparation

Purpose

Prepare structured personalization variables for AI generation.

Activities

- Select recipient-specific information.
- Select business references.
- Select industry terminology.
- Select modernization opportunities.
- Prepare communication variables.
- Build personalization context.

Output

Personalization Context

8.4.7 Stage 4 — Prompt Construction

Purpose

Construct the deterministic AI prompt.

Activities

- Apply enterprise prompt template.
- Insert business context.
- Insert personalization variables.
- Insert audit references.
- Insert prototype references.
- Apply formatting rules.
- Apply communication constraints.

Output

AI Prompt Package

8.4.8 Stage 5 — AI Email Generation

Purpose

Generate the personalized outreach email.

Activities

- Generate subject.
- Generate greeting.
- Generate opening statement.
- Generate personalized body.
- Generate opportunity summary.
- Generate value proposition.
- Generate call-to-action.
- Generate professional closing.
- Generate complete email draft.

Output

Generated Email

8.4.9 Stage 6 — Email Validation

Purpose

Verify generated content against enterprise quality requirements.

Activities

- Validate structure.
- Validate mandatory sections.
- Validate personalization.
- Validate business references.
- Validate formatting.
- Validate communication quality.
- Validate prohibited content.

Output

Validated Email

Validation Report

8.4.10 Stage 7 — Output Formatting

Purpose

Convert validated content into standardized system output.

Activities

- Normalize formatting.
- Standardize spacing.
- Apply output schema.
- Package generated email.
- Package validation report.

Output

Formatted Email Package

8.4.11 Stage 8 — Metadata Generation

Purpose

Record execution metadata for the completed generation process.

Activities

- Generate execution identifier.
- Record engine version.
- Record schema version.
- Record processing timestamp.
- Record validation timestamp.
- Record processing status.
- Record generation metrics.

Output

Metadata Package

8.4.12 Stage 9 — Final Publication

Purpose

Publish the completed email package for downstream processing.

Activities

- Combine validated email.
- Attach metadata.
- Attach validation report.
- Verify package completeness.
- Publish Final Email Package.

Output

Final Email Package

8.4.13 Execution Sequence

...

Entry Validation

↓

Input Acquisition

↓

Context Assembly

↓

Personalization Preparation

↓

Prompt Construction

↓

AI Email Generation

↓

Email Validation

↓

Output Formatting

↓

Metadata Generation

↓

Final Publication

↓

Execution Complete

'''

The execution sequence is fixed and immutable.

8.4.14 Workflow Control Rules

The execution workflow shall enforce the following rules:

- Execute stages sequentially.
- Prevent stage reordering.
- Prevent stage skipping.
- Prevent duplicate execution.
- Prevent partial publication.
- Preserve immutable upstream data.
- Preserve deterministic processing.

8.4.15 Failure Handling Workflow

If any stage fails:

- Terminate execution immediately.
- Discard intermediate outputs.
- Prevent downstream execution.
- Record failure status.
- Generate execution failure report.
- Do not publish a Final Email Package.

8.4.16 Retry Policy

Execution may be retried only after:

- Failed validation has been resolved.
- Missing contracts have been supplied.
- Version incompatibilities have been corrected.
- Required upstream intelligence has become available.

Each retry shall initiate a new execution cycle.

8.4.17 Execution Completion Criteria

Execution is considered successful only when:

- All workflow stages have completed.
- Validation has passed.
- Metadata has been generated.
- Final Email Package has been assembled.
- Publication has completed successfully.

8.4.18 Workflow Integrity

The execution workflow shall maintain:

- Fixed execution order.
- Deterministic processing.
- Immutable upstream data.

- Consistent contract usage.
- Complete traceability across all execution stages.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.5

AI PERSONALIZED EMAIL ENGINE CONTEXT ASSEMBLY LAYER

8.5.1 Overview

The Context Assembly Layer is responsible for transforming validated upstream intelligence into a unified business context for downstream personalization and AI email generation.

The layer performs structured aggregation only.

The layer does not generate content, infer business information, or modify upstream intelligence.

8.5.2 Objectives

The Context Assembly Layer shall:

- Aggregate validated intelligence.
- Normalize heterogeneous datasets.
- Remove duplicate information.
- Preserve upstream data integrity.
- Build a unified business context.
- Prepare deterministic input for downstream processing.

8.5.3 Architectural Position

The Context Assembly Layer executes immediately after successful completion of the Input Aggregation Layer.

Execution Flow

...

Input Aggregation Layer



Context Assembly Layer



Personalization Layer

...

The layer shall complete successfully before downstream processing begins.

8.5.4 Responsibilities

The Context Assembly Layer is responsible for:

- Reading validated input contracts.
- Consolidating business information.
- Consolidating website intelligence.
- Consolidating audit findings.
- Consolidating Prototype Intelligence.
- Consolidating personalization metadata.
- Normalizing structured data.
- Eliminating duplicated information.
- Producing a unified business context.

8.5.5 Input Sources

The Context Assembly Layer consumes the following validated inputs:

- Unified Input Contract
- Business Profile Contract

- Website Audit Contract
- Technology Analysis Contract
- SEO Analysis Contract
- Accessibility Analysis Contract
- Security Analysis Contract
- Visual Audit Contract
- Prototype Intelligence Contract
- Component Intelligence Contract
- Theme Intelligence Contract
- Layout Intelligence Contract
- Responsive Intelligence Contract
- Personalization Metadata Contract
- Agency Configuration Contract
- Brand Voice Configuration Contract

All input contracts shall be validated before processing.

8.5.6 Context Categories

The assembled business context shall contain the following categories:

Business Context

- Business identity
- Business category
- Industry classification
- Location
- Contact information

Website Context

- Website summary
- Technology summary
- Website quality summary
- Performance summary
- User experience summary

Audit Context

- Audit findings
- Improvement opportunities
- Modernization opportunities
- Accessibility summary
- SEO summary
- Security summary

Prototype Context

- Prototype availability
- Prototype summary
- Interface improvements
- Component improvements
- Layout improvements
- Theme improvements
- Responsive improvements

Personalization Context

- Recipient information
- Communication preferences
- Industry terminology
- Agency branding
- Brand voice configuration

8.5.7 Context Assembly Process

The layer shall execute the following sequence:

1. Load validated contracts.
2. Verify mandatory sections.
3. Normalize field structures.
4. Consolidate business intelligence.
5. Consolidate website intelligence.
6. Consolidate Prototype Intelligence.
7. Consolidate personalization metadata.
8. Remove duplicated information.

9. Validate assembled context.
10. Publish unified business context.

Execution order shall remain fixed.

8.5.8 Normalization Rules

The Context Assembly Layer shall:

- Preserve original identifiers.
- Preserve upstream ownership.
- Normalize field formats.
- Normalize naming conventions.
- Normalize text encoding.
- Normalize structural organization.

Normalization shall not alter business meaning.

8.5.9 Duplicate Resolution

Duplicate information shall be resolved according to the following principles:

- Preserve authoritative source data.
- Remove redundant records.
- Eliminate repeated summaries.
- Eliminate repeated findings.
- Retain a single normalized representation.

No conflicting information shall remain within the assembled context.

8.5.10 Context Integrity Rules

The assembled context shall:

- Contain verified information only.
- Preserve upstream traceability.
- Exclude fabricated information.
- Exclude inferred business assumptions.

- Exclude unsupported recommendations.
- Maintain deterministic structure.

8.5.11 Output Structure

The Context Assembly Layer produces a single structured output:

Business Context Package

The package shall contain:

- Business Context
- Website Context
- Audit Context
- Prototype Context
- Personalization Context
- Agency Configuration
- Brand Voice Configuration

8.5.12 Validation Requirements

Before publication, the assembled context shall be verified for:

- Structural completeness.
- Mandatory sections.
- Source consistency.
- Duplicate elimination.
- Schema compliance.
- Version compatibility.
- Data integrity.

Only validated context shall be published.

8.5.13 Failure Conditions

Context assembly shall terminate if:

- Mandatory contracts are unavailable.

- Required sections are missing.
- Schema validation fails.
- Contract versions are incompatible.
- Duplicate resolution fails.
- Context validation fails.

No partial context shall be published.

8.5.14 Downstream Interface

The validated Business Context Package shall be transferred to the Personalization Layer.

No additional transformation shall occur after publication by the Context Assembly Layer.

8.5.15 Processing Constraints

The Context Assembly Layer shall not:

- Generate email content.
- Generate prompts.
- Modify upstream intelligence.
- Create business assumptions.
- Invent modernization opportunities.
- Perform AI inference.
- Alter validated source information.

8.5.16 Traceability

Every context element shall maintain traceability to its originating upstream contract.

Traceability shall be preserved throughout downstream processing.

8.5.17 Completion Requirements

This phase is considered complete when:

- Input sources are documented.
- Context categories are defined.
- Assembly workflow is established.
- Normalization rules are specified.
- Duplicate resolution rules are documented.
- Validation requirements are defined.
- Output structure is documented.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.6

AI PERSONALIZED EMAIL ENGINE PERSONALIZATION LAYER

8.6.1 Overview

The Personalization Layer is responsible for transforming the validated Business Context Package into structured personalization data for AI-driven email generation.

The layer performs deterministic personalization preparation only.

The layer does not generate email content, execute AI inference, or modify validated upstream intelligence.

8.6.2 Objectives

The Personalization Layer shall:

- Analyze the unified business context.
- Select business-relevant information.
- Select recipient-specific information.
- Select industry terminology.

- Select verified modernization opportunities.
- Build structured personalization variables.
- Produce deterministic personalization output.

8.6.3 Architectural Position

The Personalization Layer executes immediately after successful completion of the Context Assembly Layer.

Execution Flow

...

Input Aggregation Layer

↓

Context Assembly Layer

↓

Personalization Layer

↓

Prompt Construction Layer

...

The layer shall complete successfully before prompt construction begins.

8.6.4 Responsibilities

The Personalization Layer is responsible for:

- Reading the Business Context Package.
- Selecting recipient-specific information.
- Selecting business-specific information.
- Selecting website-specific findings.
- Selecting verified audit highlights.
- Selecting Prototype Intelligence references.
- Selecting industry terminology.

- Building structured personalization variables.
- Producing the Personalization Context Package.

8.6.5 Input Source

The Personalization Layer consumes:

- Business Context Package

The Business Context Package shall be validated before processing.

8.6.6 Personalization Categories

The Personalization Context shall include the following categories:

Recipient Context

- Recipient identity
- Professional role
- Contact reference
- Communication target

Business Context

- Business name
- Industry
- Business category
- Organization profile
- Geographic location

Website Context

- Website overview
- Website strengths
- Verified improvement opportunities
- Technology summary

- User experience summary

Audit Context

- Website Audit summary
- Performance summary
- Accessibility summary
- SEO summary
- Security summary

Prototype Context

- Prototype availability
- Interface modernization summary
- Layout modernization summary
- Component modernization summary
- Theme modernization summary
- Responsive modernization summary

Communication Context

- Enterprise communication style
- Agency branding
- Brand voice
- Professional tone
- Industry terminology

8.6.7 Personalization Process

The Personalization Layer shall execute the following sequence:

1. Load the Business Context Package.
2. Verify mandatory context sections.
3. Select recipient-specific information.
4. Select business-specific information.
5. Select verified website findings.

6. Select modernization opportunities.
7. Select Prototype Intelligence references.
8. Select communication variables.
9. Construct personalization variables.
10. Validate personalization context.
11. Publish the Personalization Context Package.

The execution order shall remain fixed.

8.6.8 Selection Rules

The Personalization Layer shall:

- Use verified business information only.
- Use validated website findings only.
- Use validated Prototype Intelligence only.
- Use verified audit summaries only.
- Use approved agency branding only.
- Use approved brand voice configuration only.

No unsupported information shall be selected.

8.6.9 Personalization Integrity Rules

The Personalization Layer shall:

- Preserve factual accuracy.
- Preserve upstream traceability.
- Preserve deterministic selection.
- Exclude fabricated information.
- Exclude inferred business assumptions.
- Exclude unsupported recommendations.
- Preserve source ownership.

8.6.10 Communication Variables

The Personalization Context Package shall prepare structured variables for:

- Recipient reference
- Business reference
- Industry reference
- Website reference
- Audit reference
- Prototype reference
- Value proposition reference
- Communication style reference
- Agency identity reference

The layer prepares variables only.

It shall not generate natural language email content.

8.6.11 Output Structure

The Personalization Layer produces:

Personalization Context Package

The package shall contain:

- Recipient Context
- Business Context
- Website Context
- Audit Context
- Prototype Context
- Communication Context
- Structured Personalization Variables

8.6.12 Validation Requirements

Before publication, the Personalization Context Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- Source consistency.
- Personalization completeness.
- Schema compliance.
- Version compatibility.

- Data integrity.

Only validated personalization data shall be published.

8.6.13 Failure Conditions

Personalization processing shall terminate if:

- Business Context Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- Personalization variables cannot be generated.
- Schema validation fails.
- Personalization validation fails.

No partial personalization package shall be published.

8.6.14 Downstream Interface

The validated Personalization Context Package shall be transferred to the Prompt Construction Layer.

No additional transformation shall occur after publication by the Personalization Layer.

8.6.15 Processing Constraints

The Personalization Layer shall not:

- Generate AI prompts.
- Generate email content.
- Modify upstream intelligence.
- Invent business information.
- Invent website findings.
- Invent modernization opportunities.
- Perform AI inference.
- Alter validated source data.

8.6.16 Traceability

Every personalization variable shall maintain traceability to its originating context element.

Traceability shall be preserved throughout downstream processing.

8.6.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Personalization categories are defined.
- Selection process is documented.
- Selection rules are established.
- Integrity rules are specified.
- Output structure is defined.
- Validation requirements are documented.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.7

AI PERSONALIZED EMAIL ENGINE PROMPT CONSTRUCTION LAYER

8.7.1 Overview

The Prompt Construction Layer is responsible for transforming the validated Personalization Context Package into a deterministic AI Prompt Package for email generation.

The layer constructs structured prompts only.

The layer does not generate email content, execute AI inference, or modify validated upstream intelligence.

8.7.2 Objectives

The Prompt Construction Layer shall:

- Consume validated personalization data.
- Assemble deterministic AI prompts.
- Apply enterprise communication rules.
- Apply agency branding rules.
- Apply output constraints.
- Produce standardized AI Prompt Packages.

8.7.3 Architectural Position

The Prompt Construction Layer executes immediately after successful completion of the Personalization Layer.

Execution Flow

...

Input Aggregation Layer

↓

Context Assembly Layer

↓

Personalization Layer

↓

Prompt Construction Layer

↓

AI Email Generation Layer

...

The layer shall complete successfully before AI generation begins.

8.7.4 Responsibilities

The Prompt Construction Layer is responsible for:

- Reading the Personalization Context Package.
- Building deterministic prompt structures.
- Injecting validated personalization variables.
- Injecting verified business context.
- Injecting verified audit findings.
- Injecting verified Prototype Intelligence references.
- Applying enterprise writing rules.
- Applying formatting constraints.
- Producing the AI Prompt Package.

8.7.5 Input Source

The Prompt Construction Layer consumes:

- Personalization Context Package

The package shall be validated before processing.

8.7.6 Prompt Components

The AI Prompt Package shall contain the following components:

System Instructions

Defines the execution behavior for AI generation.

Enterprise Writing Rules

Defines mandatory enterprise communication standards.

Business Context

Contains validated business information.

Recipient Context

Contains recipient-specific information.

Website Context

Contains verified website findings.

Audit Context

Contains validated audit summaries.

Prototype Context

Contains validated Prototype Intelligence references.

Personalization Variables

Contains structured personalization values prepared by the Personalization Layer.

Output Requirements

Defines the mandatory email structure and required output sections.

Formatting Rules

Defines formatting requirements for generated content.

Generation Constraints

Defines operational boundaries and prohibited behaviors for AI generation.

8.7.7 Prompt Construction Process

The Prompt Construction Layer shall execute the following sequence:

1. Load the Personalization Context Package.
2. Verify mandatory sections.
3. Initialize prompt structure.
4. Insert system instructions.
5. Insert enterprise writing rules.
6. Insert business context.
7. Insert recipient context.
8. Insert website context.
9. Insert audit context.
10. Insert Prototype Intelligence references.
11. Insert personalization variables.
12. Insert output requirements.
13. Insert formatting rules.
14. Insert generation constraints.
15. Validate prompt completeness.
16. Publish the AI Prompt Package.

Execution order shall remain fixed.

8.7.8 Prompt Integrity Rules

The Prompt Construction Layer shall:

- Use validated information only.
- Preserve upstream traceability.

- Preserve deterministic structure.
- Prevent unauthorized prompt modification.
- Preserve enterprise communication standards.
- Preserve agency branding standards.

8.7.9 Prompt Constraints

The AI Prompt Package shall prohibit:

- Fabricated business information.
- Fabricated website findings.
- Fabricated modernization opportunities.
- Fabricated statistics.
- Unsupported claims.
- Aggressive sales language.
- Misleading statements.
- Unverified technical assertions.

8.7.10 Prompt Standardization

Every AI Prompt Package shall:

- Follow identical structural ordering.
- Use standardized section names.
- Use normalized field representations.
- Maintain schema consistency.
- Maintain version compatibility.

Prompt structure shall remain deterministic for all executions.

8.7.11 Output Structure

The Prompt Construction Layer produces:

AI Prompt Package

The package shall contain:

- System Instructions
- Enterprise Writing Rules
- Business Context
- Recipient Context
- Website Context
- Audit Context
- Prototype Context
- Personalization Variables
- Output Requirements
- Formatting Rules
- Generation Constraints

8.7.12 Validation Requirements

Before publication, the AI Prompt Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- Prompt consistency.
- Source integrity.
- Schema compliance.
- Version compatibility.
- Constraint completeness.

Only validated prompt packages shall be published.

8.7.13 Failure Conditions

Prompt construction shall terminate if:

- Personalization Context Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- Prompt assembly fails.
- Schema validation fails.
- Prompt validation fails.

No partial prompt package shall be published.

8.7.14 Downstream Interface

The validated AI Prompt Package shall be transferred to the AI Email Generation Layer.

No additional transformation shall occur after publication by the Prompt Construction Layer.

8.7.15 Processing Constraints

The Prompt Construction Layer shall not:

- Generate email content.
- Execute AI inference.
- Modify upstream intelligence.
- Invent business information.
- Invent website findings.
- Invent modernization opportunities.
- Alter validated source data.

8.7.16 Traceability

Every prompt component shall maintain traceability to its originating upstream context.

Traceability shall remain intact throughout AI generation.

8.7.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Prompt components are defined.
- Construction process is documented.
- Prompt integrity rules are established.
- Prompt constraints are specified.
- Output structure is documented.
- Validation requirements are defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.8

AI PERSONALIZED EMAIL ENGINE AI GENERATION LAYER

8.8.1 Overview

The AI Generation Layer is responsible for transforming the validated AI Prompt Package into a complete personalized outreach email.

The layer performs controlled AI-driven content generation using structured prompt inputs prepared by upstream processing layers.

The AI Generation Layer does not perform business analysis, website auditing, personalization preparation, prompt construction, or output validation.

8.8.2 Objectives

The AI Generation Layer shall:

- Consume validated AI Prompt Packages.
- Generate enterprise-grade outreach emails.
- Produce deterministic output structure.
- Preserve personalization context.
- Preserve factual accuracy.
- Produce complete email drafts for downstream validation.

8.8.3 Architectural Position

The AI Generation Layer executes immediately after successful completion of the Prompt Construction Layer.

Execution Flow

...

Input Aggregation Layer

↓

Context Assembly Layer

↓

Personalization Layer

↓

Prompt Construction Layer

↓

AI Generation Layer

↓

Email Validation Layer

...

The layer shall complete successfully before validation begins.

8.8.4 Responsibilities

The AI Generation Layer is responsible for:

- Reading the AI Prompt Package.
- Executing AI content generation.
- Generating personalized subject lines.
- Generating personalized greetings.
- Generating opening statements.
- Generating contextual email bodies.
- Generating modernization summaries.

- Generating value propositions.
- Generating call-to-action sections.
- Generating professional closings.
- Producing complete email drafts.

8.8.5 Input Source

The AI Generation Layer consumes:

- AI Prompt Package

The package shall be validated before execution.

8.8.6 Generated Email Components

Each generated email shall contain the following components:

Subject

Enterprise-ready personalized subject line.

Greeting

Professional recipient greeting.

Opening Statement

Personalized introduction based on validated business context.

Business Context Reference

Verified references to the recipient's business.

Website Observation

Verified references to validated website findings.

Modernization Opportunity

Verified modernization opportunities derived from upstream intelligence.

Value Proposition

Professional summary describing the agency's proposed value.

Call To Action

Professional invitation for further discussion.

Professional Closing

Formal closing statement.

Signature

Agency-approved signature block.

8.8.7 Generation Process

The AI Generation Layer shall execute the following sequence:

1. Load the AI Prompt Package.
2. Verify prompt integrity.
3. Initialize generation session.

4. Generate subject.
5. Generate greeting.
6. Generate opening statement.
7. Generate business context references.
8. Generate website observations.
9. Generate modernization summary.
10. Generate value proposition.
11. Generate call-to-action.
12. Generate professional closing.
13. Generate signature.
14. Assemble complete email draft.
15. Validate structural completeness.
16. Publish Generated Email.

Execution order shall remain fixed.

8.8.8 Generation Rules

The AI Generation Layer shall:

- Use validated prompt data only.
- Preserve personalization context.
- Preserve enterprise communication style.
- Preserve professional tone.
- Preserve factual accuracy.
- Preserve deterministic email structure.

8.8.9 Communication Standards

Generated emails shall:

- Maintain professional business language.
- Use respectful communication.
- Avoid exaggerated claims.
- Avoid manipulative language.
- Avoid unsupported promises.
- Maintain concise structure.
- Preserve readability.
- Maintain enterprise communication quality.

8.8.10 Generation Constraints

The AI Generation Layer shall not:

- Invent business information.
- Invent website findings.
- Invent performance statistics.
- Invent modernization opportunities.
- Invent technical issues.
- Invent security issues.
- Invent SEO issues.
- Invent accessibility issues.
- Modify validated intelligence.
- Generate misleading statements.

8.8.11 Structural Requirements

Every generated email shall contain:

- Subject
- Greeting
- Opening Statement
- Business Context Reference
- Website Observation
- Modernization Opportunity
- Value Proposition
- Call To Action
- Professional Closing
- Signature

No mandatory section shall be omitted.

8.8.12 Output Structure

The AI Generation Layer produces:

Generated Email Package

The package shall contain:

- Subject
- Email Body
- Signature
- Generation Status

8.8.13 Validation Before Publication

Before publication, the generated email shall be verified for:

- Structural completeness.
- Mandatory sections.
- Prompt compliance.
- Personalization completeness.
- Formatting consistency.
- Output schema compliance.

Only structurally complete email drafts shall be published.

8.8.14 Failure Conditions

Generation shall terminate if:

- AI Prompt Package is unavailable.
- Prompt validation fails.
- Mandatory prompt sections are missing.
- AI generation fails.
- Structural validation fails.
- Output assembly fails.

No partial email shall be published.

8.8.15 Downstream Interface

The validated Generated Email Package shall be transferred to the Email Validation Layer.

No additional transformation shall occur after publication by the AI Generation Layer.

8.8.16 Traceability

Every generated email shall maintain traceability to:

- AI Prompt Package
- Personalization Context Package
- Business Context Package
- Upstream validated intelligence

Traceability shall remain preserved throughout downstream processing.

8.8.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Generated email components are defined.
- Generation workflow is documented.
- Generation rules are established.
- Communication standards are specified.
- Generation constraints are documented.
- Output structure is defined.
- Validation requirements are established.
- Downstream interface is documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.9

AI PERSONALIZED EMAIL ENGINE EMAIL VALIDATION LAYER

8.9.1 Overview

The Email Validation Layer is responsible for verifying that every generated email complies with enterprise communication standards, structural requirements, personalization requirements, and output contract specifications before publication.

The layer performs validation only.

The layer does not generate content, modify generated content, or execute AI inference.

8.9.2 Objectives

The Email Validation Layer shall:

- Validate structural completeness.
- Validate personalization quality.
- Validate factual consistency.
- Validate communication standards.
- Validate formatting consistency.
- Validate output contract compliance.
- Produce validation results for downstream processing.

8.9.3 Architectural Position

The Email Validation Layer executes immediately after successful completion of the AI Generation Layer.

Execution Flow

...

Input Aggregation Layer

↓

Context Assembly Layer

↓

Personalization Layer

↓

Prompt Construction Layer

↓

AI Generation Layer

↓

Email Validation Layer

↓

Output Formatting Layer

...

The layer shall complete successfully before output formatting begins.

8.9.4 Responsibilities

The Email Validation Layer is responsible for:

- Reading the Generated Email Package.
- Verifying structural integrity.
- Verifying mandatory sections.
- Verifying personalization coverage.
- Verifying factual consistency.
- Verifying formatting compliance.
- Verifying enterprise communication quality.
- Producing a Validation Report.

8.9.5 Input Source

The Email Validation Layer consumes:

- Generated Email Package

The package shall be validated for structural integrity before processing.

8.9.6 Validation Categories

The Email Validation Layer shall validate the following categories:

Structural Validation

- Subject presence
- Greeting presence
- Email body presence
- Call-to-action presence
- Professional closing
- Signature presence

Personalization Validation

- Recipient references
- Business references
- Website references
- Industry references
- Modernization references

Communication Validation

- Professional language
- Business tone
- Respectful communication
- Concise writing
- Enterprise consistency

Factual Validation

- Verified business references
- Verified website findings
- Verified modernization opportunities
- Verified prototype references

- Source consistency

Formatting Validation

- Section ordering
- Output formatting
- Schema compliance
- Required field formatting
- Output completeness

8.9.7 Validation Workflow

The Email Validation Layer shall execute the following sequence:

1. Load the Generated Email Package.
2. Verify package integrity.
3. Validate mandatory sections.
4. Validate structural completeness.
5. Validate personalization.
6. Validate factual consistency.
7. Validate communication quality.
8. Validate formatting compliance.
9. Validate output schema.
10. Generate Validation Report.
11. Publish Validated Email Package.

Execution order shall remain fixed.

8.9.8 Validation Rules

The Email Validation Layer shall ensure:

- Every mandatory section is present.
- Every personalization reference originates from validated intelligence.
- Every business reference is verified.
- Every website reference is verified.
- Every modernization reference is supported.
- Enterprise communication standards are maintained.

8.9.9 Quality Rules

The generated email shall:

- Maintain professional language.
- Maintain business relevance.
- Maintain factual consistency.
- Maintain logical structure.
- Maintain readability.
- Maintain concise communication.
- Maintain enterprise presentation standards.

8.9.10 Rejection Criteria

The generated email shall be rejected if any of the following conditions are detected:

- Missing mandatory sections.
- Unsupported business references.
- Unsupported website findings.
- Unsupported modernization claims.
- Fabricated information.
- Inconsistent personalization.
- Structural corruption.
- Output schema violation.
- Validation contract failure.

Rejected emails shall not proceed to downstream processing.

8.9.11 Validation Report Structure

The Validation Report shall contain:

- Validation Identifier
- Processing Status
- Structural Validation Status
- Personalization Validation Status
- Communication Validation Status

- Factual Validation Status
- Formatting Validation Status
- Overall Validation Result

8.9.12 Output Structure

The Email Validation Layer produces:

Validated Email Package

The package shall contain:

- Approved Email
- Validation Report
- Validation Status

Only validated email packages shall be published.

8.9.13 Failure Conditions

Validation shall terminate if:

- Generated Email Package is unavailable.
- Mandatory sections are missing.
- Structural validation fails.
- Personalization validation fails.
- Factual validation fails.
- Formatting validation fails.
- Output schema validation fails.

Failed validation packages shall not be published.

8.9.14 Downstream Interface

The Validated Email Package shall be transferred to the Output Formatting Layer.

No additional validation shall occur after publication by the Email Validation Layer.

8.9.15 Processing Constraints

The Email Validation Layer shall not:

- Modify generated email content.
- Rewrite generated text.
- Execute AI inference.
- Invent validation results.
- Alter validated upstream intelligence.
- Generate business assumptions.

8.9.16 Traceability

Every validation result shall maintain traceability to:

- Generated Email Package
- AI Prompt Package
- Personalization Context Package
- Business Context Package
- Upstream validated intelligence

Traceability shall remain preserved throughout downstream processing.

8.9.17 Completion Requirements

This phase is considered complete when:

- Validation objectives are documented.
- Validation categories are defined.
- Validation workflow is established.
- Validation rules are specified.
- Quality rules are documented.
- Rejection criteria are defined.
- Validation report structure is documented.
- Output structure is defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.10

AI PERSONALIZED EMAIL ENGINE OUTPUT FORMATTING LAYER

8.10.1 Overview

The Output Formatting Layer is responsible for transforming the Validated Email Package into a standardized Final Email Package suitable for downstream enterprise processing.

The layer performs formatting and packaging only.

The layer does not generate content, validate content, execute AI inference, or modify approved communication.

8.10.2 Objectives

The Output Formatting Layer shall:

- Standardize validated email output.
- Normalize document formatting.
- Assemble standardized output packages.
- Preserve validated content integrity.
- Prepare downstream-compatible output.
- Produce immutable publication-ready packages.

8.10.3 Architectural Position

The Output Formatting Layer executes immediately after successful completion of the Email Validation Layer.

Execution Flow

...

Input Aggregation Layer



Context Assembly Layer



Personalization Layer



Prompt Construction Layer



AI Generation Layer



Email Validation Layer



Output Formatting Layer



Metadata Generation Layer

...

The layer shall complete successfully before metadata generation begins.

8.10.4 Responsibilities

The Output Formatting Layer is responsible for:

- Reading the Validated Email Package.
- Standardizing approved content.

- Applying output formatting rules.
- Normalizing document structure.
- Packaging approved communication.
- Producing standardized downstream output.

8.10.5 Input Source

The Output Formatting Layer consumes:

- Validated Email Package

The package shall be validated before formatting begins.

8.10.6 Formatting Categories

The Output Formatting Layer shall standardize the following categories:

Document Structure

- Subject section
- Greeting section
- Email body
- Call-to-action
- Professional closing
- Signature block

Text Formatting

- Line separation
- Paragraph separation
- Whitespace normalization
- Character encoding
- Text normalization

Structural Formatting

- Section ordering
- Required field ordering
- Output consistency
- Schema organization
- Package organization

Package Formatting

- Email package structure
- Validation report inclusion
- Processing status inclusion
- Packaging consistency

8.10.7 Formatting Workflow

The Output Formatting Layer shall execute the following sequence:

1. Load the Validated Email Package.
2. Verify package integrity.
3. Normalize document structure.
4. Normalize text formatting.
5. Normalize package organization.
6. Assemble standardized output.
7. Verify formatting completeness.
8. Publish Formatted Email Package.

Execution order shall remain fixed.

8.10.8 Formatting Rules

The Output Formatting Layer shall ensure:

- Approved content remains unchanged.
- Section ordering remains standardized.
- Formatting remains consistent.
- Output complies with package schema.
- Character encoding remains normalized.
- Document structure remains deterministic.

8.10.9 Content Preservation Rules

The Output Formatting Layer shall preserve:

- Approved subject.
- Approved greeting.
- Approved email body.
- Approved call-to-action.
- Approved professional closing.
- Approved signature.
- Validation results.

No approved content shall be modified during formatting.

8.10.10 Standardization Rules

Every formatted email package shall:

- Follow identical structural organization.
- Use standardized section ordering.
- Use normalized spacing.
- Use normalized line formatting.
- Maintain schema consistency.
- Maintain deterministic output formatting.

8.10.11 Output Structure

The Output Formatting Layer produces:

Formatted Email Package

The package shall contain:

- Approved Subject
- Approved Email Body
- Approved Signature
- Validation Report

- Validation Status
- Processing Status

8.10.12 Formatting Validation

Before publication, the formatted package shall be verified for:

- Structural completeness.
- Formatting consistency.
- Package completeness.
- Schema compliance.
- Section ordering.
- Output integrity.

Only correctly formatted packages shall be published.

8.10.13 Failure Conditions

Formatting shall terminate if:

- Validated Email Package is unavailable.
- Package integrity verification fails.
- Formatting normalization fails.
- Package assembly fails.
- Schema validation fails.
- Output validation fails.

No partially formatted package shall be published.

8.10.14 Downstream Interface

The Formatted Email Package shall be transferred to the Metadata Generation Layer.

No additional formatting shall occur after publication by the Output Formatting Layer.

8.10.15 Processing Constraints

The Output Formatting Layer shall not:

- Generate email content.
- Modify approved communication.
- Execute AI inference.
- Alter validation results.
- Modify upstream intelligence.
- Generate business information.

8.10.16 Traceability

Every formatted package shall maintain traceability to:

- Validated Email Package
- Validation Report
- Generated Email Package
- AI Prompt Package
- Personalization Context Package
- Business Context Package
- Upstream validated intelligence

Traceability shall remain preserved throughout downstream processing.

8.10.17 Completion Requirements

This phase is considered complete when:

- Formatting objectives are documented.
- Formatting categories are defined.
- Formatting workflow is established.
- Formatting rules are specified.
- Content preservation rules are documented.
- Standardization rules are defined.
- Output structure is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.11

AI PERSONALIZED EMAIL ENGINE METADATA GENERATION LAYER

8.11.1 Overview

The Metadata Generation Layer is responsible for generating standardized execution metadata for every successfully formatted email package.

The metadata provides execution traceability, processing status, validation status, version information, and audit records required by downstream enterprise systems.

The layer performs metadata generation only.

The layer does not generate email content, validate communication, modify formatted output, or execute AI inference.

8.11.2 Objectives

The Metadata Generation Layer shall:

- Generate execution metadata.
- Record processing information.
- Record validation information.
- Preserve execution traceability.
- Produce standardized metadata packages.
- Prepare publication-ready output for downstream systems.

8.11.3 Architectural Position

The Metadata Generation Layer executes immediately after successful completion of the Output Formatting Layer.

Execution Flow

...

Input Aggregation Layer



Context Assembly Layer



Personalization Layer



Prompt Construction Layer



AI Generation Layer



Email Validation Layer



Output Formatting Layer



Metadata Generation Layer



Final Email Package

...

The layer shall complete successfully before final package publication.

8.11.4 Responsibilities

The Metadata Generation Layer is responsible for:

- Reading the Formatted Email Package.
- Generating execution identifiers.
- Recording engine information.
- Recording processing information.
- Recording validation information.
- Recording schema information.
- Producing standardized metadata.

8.11.5 Input Source

The Metadata Generation Layer consumes:

- Formatted Email Package

The package shall be verified before metadata generation begins.

8.11.6 Metadata Categories

The Metadata Package shall contain the following categories:

Execution Metadata

- Execution Identifier
- Processing Identifier
- Session Identifier
- Correlation Identifier

Engine Metadata

- Engine Name
- Engine Version
- Module Version
- Workflow Version

Contract Metadata

- Contract Version
- Schema Version
- Package Version

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

Validation Metadata

- Validation Identifier
- Validation Status
- Validation Timestamp
- Validation Result

Publication Metadata

- Publication Status
- Publication Timestamp
- Output Package Identifier

8.11.7 Metadata Generation Workflow

The Metadata Generation Layer shall execute the following sequence:

1. Load the Formatted Email Package.
2. Verify package integrity.
3. Generate execution metadata.

4. Generate engine metadata.
5. Generate contract metadata.
6. Generate processing metadata.
7. Generate validation metadata.
8. Generate publication metadata.
9. Assemble Metadata Package.
10. Validate metadata completeness.
11. Publish Final Email Package.

Execution order shall remain fixed.

8.11.8 Metadata Rules

The Metadata Generation Layer shall ensure:

- Metadata accurately represents execution.
- Metadata remains immutable after publication.
- Metadata preserves execution traceability.
- Metadata follows standardized schema.
- Metadata remains version consistent.
- Metadata contains no fabricated values.

8.11.9 Integrity Rules

The Metadata Package shall:

- Preserve execution identifiers.
- Preserve validation identifiers.
- Preserve processing history.
- Preserve version information.
- Preserve package integrity.
- Preserve downstream compatibility.

8.11.10 Output Structure

The Metadata Generation Layer produces:

Metadata Package

The package shall contain:

- Execution Metadata
- Engine Metadata
- Contract Metadata
- Processing Metadata
- Validation Metadata
- Publication Metadata

8.11.11 Final Email Package

The Metadata Generation Layer assembles the Final Email Package.

The Final Email Package shall contain:

- Formatted Email Package
- Metadata Package
- Validation Report
- Processing Status
- Publication Status

The package shall be immutable after publication.

8.11.12 Metadata Validation

Before publication, the Metadata Package shall be verified for:

- Structural completeness.
- Required metadata fields.
- Schema compliance.
- Version consistency.
- Identifier consistency.
- Processing consistency.
- Publication readiness.

Only validated metadata shall be published.

8.11.13 Failure Conditions

Metadata generation shall terminate if:

- Formatted Email Package is unavailable.
- Package integrity verification fails.
- Metadata generation fails.
- Identifier generation fails.
- Metadata validation fails.
- Final package assembly fails.

No incomplete metadata package shall be published.

8.11.14 Downstream Interface

The completed Final Email Package shall be published for downstream consumption by the Follow-Up Engine.

The Metadata Generation Layer represents the final processing stage of the AI Personalized Email Engine.

No additional processing shall occur within this engine after publication.

8.11.15 Processing Constraints

The Metadata Generation Layer shall not:

- Generate email content.
- Modify formatted communication.
- Modify validation results.
- Execute AI inference.
- Alter upstream intelligence.
- Generate business information.
- Reformat published output.

8.11.16 Traceability

Every metadata record shall maintain traceability to:

- Formatted Email Package
- Validated Email Package
- Generated Email Package
- AI Prompt Package
- Personalization Context Package
- Business Context Package
- Upstream validated intelligence

End-to-end traceability shall remain preserved throughout the complete execution lifecycle.

8.11.17 Completion Requirements

This phase is considered complete when:

- Metadata objectives are documented.
- Metadata categories are defined.
- Metadata workflow is established.
- Metadata rules are specified.
- Integrity rules are documented.
- Output structure is defined.
- Final Email Package is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.12

AI PERSONALIZED EMAIL ENGINE OUTPUT SCHEMA SPECIFICATION

8.12.1 Overview

The Output Schema Specification defines the standardized structure of the Final Email Package produced by the AI Personalized Email Engine.

The schema establishes a deterministic output contract for downstream enterprise systems.

All generated output shall conform to this specification before publication.

8.12.2 Objectives

The Output Schema Specification shall:

- Define standardized output structures.
- Define required output sections.
- Define schema hierarchy.
- Define mandatory fields.
- Define validation requirements.
- Define downstream compatibility requirements.

8.12.3 Schema Design Principles

The output schema shall comply with the following principles:

- Deterministic structure.
- Immutable after publication.
- Version controlled.
- Self-describing.
- Platform independent.
- Strongly structured.
- Enterprise compatible.
- Backward compatible within the same major version.

8.12.4 Top-Level Output Schema

The Final Email Package shall contain the following top-level sections:

...

Final Email Package

- |— Formatted Email Package
- |— Metadata Package
- |— Validation Report
- |— Processing Status

...

No additional top-level sections shall be introduced.

8.12.5 Formatted Email Package Schema

The Formatted Email Package shall contain:

...

Formatted Email Package

- |— Subject
- |— Greeting
- |— Opening Statement
- |— Email Body
- |— Call To Action
- |— Professional Closing
- |— Signature
- |— Formatting Status

...

All sections are mandatory.

8.12.6 Metadata Package Schema

The Metadata Package shall contain:

...

Metadata Package

- |— Execution Metadata
- |— Engine Metadata
- |— Contract Metadata
- |— Processing Metadata
- |— Validation Metadata
- |— Publication Metadata

...

All metadata categories are mandatory.

8.12.7 Execution Metadata Schema

Execution Metadata shall contain:

...

Execution Metadata

- |— Execution Identifier
- |— Processing Identifier
- |— Session Identifier
- |— Correlation Identifier

...

8.12.8 Engine Metadata Schema

Engine Metadata shall contain:

...

Engine Metadata

- |— Engine Name
- |— Engine Version
- |— Module Version
- |— Workflow Version

...

8.12.9 Contract Metadata Schema

Contract Metadata shall contain:

...

Contract Metadata

- |— Contract Version

- Schema Version
- Package Version

...

8.12.10 Processing Metadata Schema

Processing Metadata shall contain:

...

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

...

8.12.11 Validation Metadata Schema

Validation Metadata shall contain:

...

Validation Metadata

- Validation Identifier
- Validation Timestamp
- Validation Status
- Validation Result

...

8.12.12 Publication Metadata Schema

Publication Metadata shall contain:

...

Publication Metadata

- Publication Status

- Publication Timestamp
- Output Package Identifier

...

8.12.13 Validation Report Schema

The Validation Report shall contain:

...

Validation Report

- Validation Identifier
- Structural Validation
- Personalization Validation
- Communication Validation
- Factual Validation
- Formatting Validation
- Overall Validation Result

...

All validation categories are mandatory.

8.12.14 Processing Status Schema

Processing Status shall contain:

...

Processing Status

- Current Status
- Completion Status
- Publication Status
- Processing Result

...

8.12.15 Schema Validation Rules

Every Final Email Package shall satisfy the following requirements:

- All mandatory sections shall exist.
- All mandatory fields shall exist.
- Section hierarchy shall remain unchanged.
- Parent-child relationships shall remain valid.
- Version identifiers shall be present.
- Schema integrity shall be maintained.

8.12.16 Schema Integrity Rules

The Output Schema shall:

- Preserve deterministic ordering.
- Preserve immutable identifiers.
- Preserve validation results.
- Preserve metadata integrity.
- Preserve downstream compatibility.
- Preserve structural consistency.

8.12.17 Schema Compatibility

The Output Schema shall support:

- Internal enterprise services.
- Workflow orchestration.
- Follow-Up Engine integration.
- CRM integration.
- Reporting systems.
- Audit systems.
- Long-term archival.

Compatibility shall remain stable within the same major schema version.

8.12.18 Schema Publication Rules

The Final Email Package shall be published only when:

- Schema validation has passed.

- Metadata validation has passed.
- Validation Report is complete.
- Processing Status indicates successful completion.
- All mandatory schema sections are present.

8.12.19 Schema Failure Conditions

Publication shall be rejected if:

- Mandatory sections are missing.
- Schema hierarchy is invalid.
- Required metadata is incomplete.
- Validation Report is incomplete.
- Processing Status is invalid.
- Schema validation fails.

No invalid schema shall be published.

8.12.20 Completion Requirements

This phase is considered complete when:

- Top-level schema is defined.
- All subordinate schemas are documented.
- Mandatory sections are specified.
- Validation rules are documented.
- Integrity rules are established.
- Compatibility requirements are defined.
- Publication rules are documented.
- Failure conditions are specified.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-3

PHASE 8.13

AI PERSONALIZED EMAIL ENGINE ENTERPRISE VALIDATION & PHASE COMPLETION

8.13.1 Overview

This phase defines the enterprise validation framework and formal completion requirements for the AI Personalized Email Engine.

Enterprise validation confirms that every architectural component, processing layer, execution workflow, interface, and output specification defined within Phase 8 has been successfully documented and internally validated.

Successful completion authorizes the phase to enter the LOCKED state.

8.13.2 Validation Objectives

Enterprise validation shall verify:

- Architectural completeness.
- Documentation completeness.
- Workflow completeness.
- Module completeness.
- Interface completeness.
- Contract completeness.
- Schema completeness.
- Enterprise compliance.
- Internal consistency.

8.13.3 Validation Scope

The validation scope includes every document produced within Phase 8.

Validated documents include:

- Phase 8 — AI Personalized Email Engine
- Phase 8.1 — Architecture
- Phase 8.2 — Internal Modules

- Phase 8.3 — Data Contracts
- Phase 8.4 — Execution Workflow
- Phase 8.5 — Context Assembly Layer
- Phase 8.6 — Personalization Layer
- Phase 8.7 — Prompt Construction Layer
- Phase 8.8 — AI Generation Layer
- Phase 8.9 — Email Validation Layer
- Phase 8.10 — Output Formatting Layer
- Phase 8.11 — Metadata Generation Layer
- Phase 8.12 — Output Schema Specification

No additional documents are included within this validation scope.

8.13.4 Architectural Validation

Enterprise validation shall confirm that:

- Architectural boundaries are documented.
- Layer responsibilities are documented.
- Module responsibilities are documented.
- Processing order is fixed.
- Upstream dependencies are documented.
- Downstream interfaces are documented.
- Separation of responsibilities is maintained.

8.13.5 Workflow Validation

The execution workflow shall be verified for:

- Sequential execution.
- Fixed execution order.
- Stage completeness.
- Entry conditions.
- Exit conditions.
- Failure handling.
- Publication sequence.

No undocumented execution path shall exist.

8.13.6 Module Validation

Each internal module shall be verified for:

- Defined responsibilities.
- Defined inputs.
- Defined outputs.
- Processing constraints.
- Validation requirements.
- Downstream interfaces.

All modules shall maintain single-responsibility architecture.

8.13.7 Layer Validation

Each processing layer shall be verified for:

- Architectural position.
- Responsibilities.
- Input requirements.
- Output requirements.
- Workflow integration.
- Validation rules.
- Failure conditions.

All documented layers shall integrate without architectural conflict.

8.13.8 Contract Validation

Enterprise validation shall confirm that:

- Input contracts are documented.
- Output contracts are documented.
- Contract hierarchy is complete.
- Contract integrity rules are defined.
- Versioning rules are defined.
- Validation rules are defined.

All contract interfaces shall remain deterministic.

8.13.9 Schema Validation

The Final Email Package schema shall be verified for:

- Structural completeness.
- Mandatory sections.
- Schema hierarchy.
- Metadata integration.
- Validation report integration.
- Processing status integration.

The published schema shall remain immutable after validation.

8.13.10 Traceability Validation

End-to-end traceability shall be verified across:

- Business Context Package.
- Personalization Context Package.
- AI Prompt Package.
- Generated Email Package.
- Validated Email Package.
- Formatted Email Package.
- Metadata Package.
- Final Email Package.

Every downstream artifact shall maintain traceability to validated upstream intelligence.

8.13.11 Enterprise Compliance Validation

The AI Personalized Email Engine shall comply with the following enterprise principles:

- Modular architecture.
- Deterministic processing.
- Immutable contracts.
- Standardized interfaces.
- Enterprise documentation standards.

- Fixed execution workflow.
- Read-only upstream consumption.
- Immutable published outputs.

8.13.12 Documentation Completeness Validation

Enterprise documentation shall verify that every Phase 8 document contains:

- Defined purpose.
- Architectural position.
- Responsibilities.
- Processing workflow.
- Input definition.
- Output definition.
- Validation requirements.
- Failure conditions.
- Completion requirements.

No mandatory documentation section shall be omitted.

8.13.13 Validation Result

Enterprise validation shall confirm that:

- All Phase 8 documentation has been completed.
- All architectural components have been documented.
- All processing layers have been documented.
- All internal modules have been documented.
- All execution workflows have been documented.
- All data contracts have been documented.
- All output schemas have been documented.
- All enterprise validation requirements have been satisfied.

8.13.14 Phase Completion Criteria

Phase 8 shall be considered complete when:

- Enterprise validation is successful.

- Documentation completeness is verified.
- Architectural validation is successful.
- Workflow validation is successful.
- Module validation is successful.
- Contract validation is successful.
- Schema validation is successful.
- Traceability validation is successful.

Only after all completion criteria have passed shall Phase 8 be considered complete.

8.13.15 Phase Status

Phase Name

AI Personalized Email Engine

Phase Identifier

PART 2B-3 — Phase 8

Phase State

COMPLETED

Validation Status

PASSED

Architecture Status

VERIFIED

Documentation Status

COMPLETE

Workflow Status

VERIFIED

Contract Status

VERIFIED

Schema Status

VERIFIED

Publication Status

APPROVED

8.13.16 Phase Lock

Following successful enterprise validation:

- Phase 8 is declared COMPLETE.
- Phase 8 enters the LOCKED state.
- Architectural changes are prohibited.
- Workflow modifications are prohibited.
- Module modifications are prohibited.
- Contract modifications are prohibited.
- Schema modifications are prohibited.
- Documentation modifications are prohibited unless a formal change request is approved.

8.13.17 Completion Summary

The AI Personalized Email Engine has been fully specified as an enterprise-grade subsystem.

The completed documentation defines:

- Enterprise architecture.
- Internal modules.
- Data contracts.
- Execution workflow.
- Context Assembly Layer.
- Personalization Layer.
- Prompt Construction Layer.
- AI Generation Layer.
- Email Validation Layer.
- Output Formatting Layer.
- Metadata Generation Layer.

- Output Schema Specification.
- Enterprise validation framework.

This concludes the complete documentation scope for PART 2B-3 — Phase 8.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9

FOLLOW-UP ENGINE FOUNDATION

9.1 Purpose

The Follow-Up Engine is responsible for managing structured follow-up communication after successful completion of the AI Personalized Email Engine.

The engine operates exclusively on validated Final Email Packages and associated execution metadata.

The engine does not perform lead discovery, website analysis, contact extraction, email validation, email generation, or email delivery.

Its sole responsibility is to prepare and manage enterprise follow-up workflow information for downstream execution.

9.2 Objectives

The Follow-Up Engine shall:

- Consume validated Final Email Packages.
- Consume execution metadata.
- Consume follow-up configuration.
- Prepare structured follow-up workflows.

- Maintain follow-up state information.
- Generate follow-up execution plans.
- Produce enterprise-ready follow-up packages.

9.3 Scope

Included Responsibilities:

- Follow-up workflow preparation
- Follow-up state management
- Follow-up scheduling preparation
- Follow-up sequence preparation
- Follow-up metadata generation
- Follow-up package generation

Excluded Responsibilities:

- Lead discovery
- Search Engine execution
- Website auditing
- Prototype generation
- Contact extraction
- Email validation
- AI email generation
- Email delivery
- CRM synchronization
- External messaging
- Calendar management

9.4 Dependencies

The Follow-Up Engine depends on successful completion of:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine

- AI Personalized Email Engine

Execution shall not begin unless all dependencies have completed successfully.

9.5 Execution Position

Enterprise Workflow

Phase 0



Search Engine



Website Audit Engine



Niche Detection Engine



Lead Scoring Engine



Contact Extraction Engine



Email Validation Engine



Prototype Intelligence Engine



AI Personalized Email Engine

↓

Follow-Up Engine

↓

CRM Engine

9.6 Engine Responsibilities

The engine is responsible for:

- Reading validated email packages.
- Reading execution metadata.
- Preparing follow-up workflows.
- Preparing follow-up sequences.
- Maintaining follow-up processing state.
- Preparing downstream execution packages.
- Producing standardized follow-up outputs.

9.7 Inputs

The engine consumes:

Final Email Package

Execution Metadata

Validation Report

Processing Status

Follow-Up Configuration

Agency Configuration

Brand Voice Configuration

Workflow Configuration

9.8 Outputs

The engine produces:

Follow-Up Workflow

Follow-Up Sequence

Follow-Up State

Follow-Up Metadata

Follow-Up Execution Package

Quality Report

9.9 Functional Responsibilities

The engine shall:

Prepare one follow-up workflow for each successfully published Final Email Package.

Each workflow shall remain associated with its originating email package.

Every follow-up workflow shall preserve execution traceability.

Every generated workflow shall comply with enterprise workflow standards.

9.10 Processing Principles

The engine shall never:

Generate follow-up communication.

Deliver follow-up emails.

Modify validated email content.

Modify execution metadata.

Modify upstream intelligence.

Only validated upstream outputs may be consumed.

9.11 Workflow Foundation

The Follow-Up Engine shall establish:

- Follow-up workflow identity.
- Follow-up execution context.
- Follow-up state information.
- Follow-up sequence definition.
- Follow-up processing metadata.
- Downstream execution package.

9.12 Enterprise Constraints

The engine shall never:

Invent communication history.

Invent recipient actions.

Invent delivery status.

Invent engagement metrics.

Invent CRM information.

Invent scheduling results.

Modify published Final Email Packages.

9.13 Error Handling

If mandatory upstream information is unavailable:

The engine shall terminate processing.

An execution failure report shall be generated.

No partial Follow-Up Execution Package shall be published.

9.14 Completion Criteria

Phase completion requires:

- Successful input validation.
- Successful workflow preparation.
- Successful sequence preparation.
- Successful metadata generation.
- Successful output validation.
- Successful package generation.
- Successful enterprise verification.

Only after all completion criteria have passed shall the engine be considered complete.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.1

FOLLOW-UP ENGINE ARCHITECTURE

9.1.1 Overview

The Follow-Up Engine is an enterprise subsystem responsible for preparing structured follow-up workflows from validated Final Email Packages produced by the AI Personalized Email Engine.

The engine operates exclusively on validated internal system outputs.

The engine does not perform communication generation, message delivery, scheduling execution, CRM synchronization, or modification of upstream intelligence.

Its responsibility is limited to preparing deterministic follow-up execution packages for downstream enterprise processing.

9.1.2 Architectural Position

The Follow-Up Engine is positioned immediately after successful completion of the AI Personalized Email Engine.

System Execution Order

...

Foundation Framework

↓

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine



AI Personalized Email Engine



Follow-Up Engine



CRM Engine

...

The engine shall not execute before all upstream engines have completed successfully.

9.1.3 Architectural Principles

The architecture shall comply with the following principles:

- Single responsibility.
- Modular execution.
- Deterministic workflow.
- Read-only consumption of upstream outputs.
- Stateless processing.
- Immutable execution packages.
- Enterprise-grade reliability.
- Strict separation of processing layers.

9.1.4 High-Level Architecture

...

Final Email Package



Input Aggregation Layer

|



Workflow Assembly Layer

|



Sequence Preparation Layer

|



State Management Layer

|



Output Validation Layer

|



Packaging Layer

|



Follow-Up Execution Package

...

Each architectural layer performs one defined responsibility and passes structured output to the next processing stage.

9.1.5 Architectural Layers

The engine consists of the following logical layers:

Layer 1

Input Aggregation Layer

Responsibilities:

- Load validated Final Email Package.
- Load execution metadata.
- Load workflow configuration.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Layer 2

Workflow Assembly Layer

Responsibilities:

- Build follow-up execution context.
- Assemble workflow information.
- Consolidate execution metadata.
- Normalize workflow structure.
- Produce unified workflow context.

Layer 3

Sequence Preparation Layer

Responsibilities:

- Prepare follow-up sequence.
- Prepare workflow stages.
- Prepare sequence metadata.
- Produce deterministic sequence structure.

Layer 4

State Management Layer

Responsibilities:

- Initialize follow-up state.
- Record workflow status.
- Record processing state.
- Prepare state package.

Layer 5

Output Validation Layer

Responsibilities:

- Validate workflow completeness.
- Validate sequence integrity.
- Validate state information.
- Validate package consistency.
- Validate required sections.

Layer 6

Packaging Layer

Responsibilities:

- Assemble execution package.
- Normalize package structure.
- Prepare downstream output.
- Produce standardized Follow-Up Execution Package.

9.1.6 Data Flow

...

Validated Final Email Package



Input Aggregation



Workflow Assembly



Sequence Preparation



State Management



Validation



Packaging



Follow-Up Execution Package

...

Processing shall occur sequentially.

No architectural layer may be skipped.

9.1.7 Upstream Dependencies

The architecture consumes outputs from:

- AI Personalized Email Engine
- Final Email Package
- Execution Metadata

- Validation Report
- Processing Status

No direct interaction with external communication systems is permitted.

9.1.8 Downstream Dependency

The engine publishes structured Follow-Up Execution Packages for consumption by the CRM Engine.

The engine does not perform CRM synchronization.

The engine does not execute follow-up communication.

9.1.9 State Management

The architecture is stateless.

Each execution shall:

- Load validated inputs.
- Prepare workflow information.
- Generate execution package.
- Return results.
- Release execution state.

No persistent execution session shall be maintained within the engine.

9.1.10 Failure Handling

Execution shall terminate when:

- Required inputs are missing.
- Mandatory packages are unavailable.
- Workflow assembly fails.
- Sequence preparation fails.
- Validation fails.
- Packaging fails.

No partial Follow-Up Execution Package shall be published.

9.1.11 Scalability Requirements

The architecture shall support:

- Independent execution per email package.
- Parallel processing across multiple workflows.
- Isolated execution contexts.
- Horizontal scaling without workflow modification.
- Deterministic output generation for identical inputs.

9.1.12 Security Principles

The architecture shall:

- Operate on validated internal packages only.
- Prevent modification of upstream outputs.
- Prevent unauthorized data injection.
- Maintain strict separation between processing layers.
- Preserve integrity of generated execution packages.

9.1.13 Completion Requirements

The architecture is considered complete when:

- All architectural layers are defined.
- Data flow is established.
- Processing order is fixed.
- Dependencies are documented.
- Failure handling is specified.
- Scalability requirements are defined.
- Security principles are documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.2

FOLLOW-UP ENGINE INTERNAL MODULES

9.2.1 Overview

The Follow-Up Engine is composed of independent internal modules.

Each module performs a single well-defined responsibility.

Modules execute sequentially according to the fixed processing workflow.

No module shall perform responsibilities assigned to another module.

9.2.2 Module Architecture

...

Input Manager

|



Workflow Builder

|



Sequence Manager

|



State Manager

|



Output Validator

|



Package Formatter

|



Metadata Generator

|



Follow-Up Execution Package

...

9.2.3 Input Manager

Purpose

The Input Manager loads all validated upstream packages required for Follow-Up Engine processing.

Responsibilities

- Load Final Email Package.
- Load Execution Metadata.
- Load Validation Report.

- Load Processing Status.
- Load Follow-Up Configuration.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Inputs

- Final Email Package
- Execution Metadata
- Validation Report
- Follow-Up Configuration
- Workflow Configuration

Outputs

Unified Input Package

9.2.4 Workflow Builder

Purpose

The Workflow Builder constructs the unified follow-up workflow context.

Responsibilities

- Consolidate workflow information.
- Consolidate execution metadata.
- Consolidate processing status.
- Normalize workflow structure.
- Prepare execution context.

Inputs

Unified Input Package

Outputs

Workflow Context Package

9.2.5 Sequence Manager

Purpose

The Sequence Manager prepares the deterministic follow-up sequence.

Responsibilities

- Build follow-up sequence.
- Define workflow stages.
- Generate sequence identifiers.
- Prepare sequence metadata.
- Normalize sequence structure.

Inputs

Workflow Context Package

Outputs

Follow-Up Sequence Package

9.2.6 State Manager

Purpose

The State Manager initializes and maintains follow-up processing state.

Responsibilities

- Initialize workflow state.
- Initialize processing state.
- Record workflow status.
- Record execution status.
- Prepare state package.

Inputs

Follow-Up Sequence Package

Outputs

Follow-Up State Package

9.2.7 Output Validator

Purpose

The Output Validator verifies that all follow-up workflow components satisfy enterprise requirements.

Responsibilities

- Validate workflow completeness.
- Validate sequence integrity.
- Validate state information.
- Validate package consistency.
- Validate required sections.
- Validate enterprise workflow standards.

Inputs

Follow-Up State Package

Outputs

Validated Follow-Up Package

Validation Report

9.2.8 Package Formatter

Purpose

The Package Formatter converts validated workflow information into a standardized downstream package.

Responsibilities

- Normalize package formatting.
- Normalize workflow structure.
- Standardize package schema.
- Prepare downstream package.

Inputs

Validated Follow-Up Package

Outputs

Formatted Follow-Up Package

9.2.9 Metadata Generator

Purpose

The Metadata Generator records execution metadata associated with the Follow-Up Engine execution.

Responsibilities

- Generate execution identifier.
- Record engine version.
- Record processing status.
- Record validation status.
- Record execution metrics.
- Produce metadata package.

Inputs

Formatted Follow-Up Package

Validation Report

Outputs

Metadata Package

Follow-Up Execution Package

9.2.10 Module Interaction Sequence

...

Input Manager

↓

Workflow Builder



Sequence Manager



State Manager



Output Validator



Package Formatter



Metadata Generator



Follow-Up Execution Package

...

Module execution order is fixed.

No module may execute before completion of its predecessor.

9.2.11 Module Independence

Each module shall:

- Perform one responsibility only.
- Consume defined inputs.
- Produce defined outputs.
- Avoid direct dependency on non-adjacent modules.
- Preserve upstream package integrity.
- Avoid modification of validated upstream outputs.

9.2.12 Error Propagation

If any module reports failure:

- Execution shall stop immediately.
- Remaining modules shall not execute.
- Partial outputs shall be discarded.
- An execution failure report shall be generated.

9.2.13 Extensibility

Internal modules shall remain independently maintainable.

Future implementation updates shall preserve:

- Module boundaries.
- Processing order.
- Input contracts.
- Output contracts.
- Public interfaces.

9.2.14 Completion Requirements

This phase is considered complete when:

- All internal modules are defined.
- Module responsibilities are documented.
- Input contracts are documented.
- Output contracts are documented.
- Execution sequence is fixed.
- Failure propagation rules are specified.
- Module interaction is fully documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.3

FOLLOW-UP ENGINE DATA CONTRACTS

9.3.1 Overview

The Follow-Up Engine exchanges information exclusively through structured data contracts.

Data contracts define the required inputs, generated outputs, mandatory fields, validation requirements, and interface boundaries between the Follow-Up Engine and other enterprise subsystems.

All contracts are immutable during execution.

The engine shall consume upstream contracts in read-only mode.

9.3.2 Contract Design Principles

Every data contract shall comply with the following principles:

- Strongly structured.
- Version controlled.
- Immutable during execution.
- Self-describing.
- Deterministic.
- Platform independent.
- Schema validated.
- Backward compatible within the same major version.

9.3.3 Upstream Input Contracts

The Follow-Up Engine consumes the following validated contracts:

- Final Email Package Contract

- Execution Metadata Contract
- Validation Report Contract
- Processing Status Contract
- Follow-Up Configuration Contract
- Workflow Configuration Contract
- Agency Configuration Contract
- Brand Voice Configuration Contract

Each contract shall be validated prior to processing.

9.3.4 Unified Input Contract

Purpose

The Unified Input Contract represents the normalized dataset consumed by the Follow-Up Engine after successful input aggregation.

Required Sections

- Final Email Information
- Execution Metadata
- Validation Information
- Processing Information
- Follow-Up Configuration
- Workflow Configuration
- Agency Configuration
- Brand Voice Configuration

Validation Requirements

The Unified Input Contract shall:

- Contain all mandatory sections.
- Preserve upstream identifiers.
- Preserve source integrity.
- Contain normalized field formats.
- Exclude duplicate records.

9.3.5 Workflow Context Contract

Purpose

The Workflow Context Contract represents the consolidated workflow information required for follow-up preparation.

Required Sections

- Workflow Identity
- Email Reference
- Execution Reference
- Validation Summary
- Processing Summary
- Configuration Summary
- Workflow Context
- Follow-Up Context

Validation Requirements

The contract shall contain verified information only.

No inferred or fabricated information shall be included.

9.3.6 Follow-Up Sequence Contract

Purpose

The Follow-Up Sequence Contract contains the structured sequence information required for downstream processing.

Required Sections

- Sequence Identifier

- Workflow Identifier
- Sequence Definition
- Sequence Stages
- Sequence Metadata
- Processing Context

Validation Requirements

Every sequence element shall originate from validated workflow information.

9.3.7 Follow-Up State Contract

Purpose

The Follow-Up State Contract represents the initialized processing state of the follow-up workflow.

Required Sections

- Workflow State
- Processing State
- Validation State
- Publication State
- State Metadata

Validation Requirements

The state contract shall contain only verified processing information.

9.3.8 Validated Follow-Up Contract

Purpose

The Validated Follow-Up Contract represents the approved workflow after successful enterprise validation.

Required Sections

- Approved Workflow
- Approved Sequence
- Approved State
- Validation Status
- Validation Identifier

Validation Requirements

The contract shall contain only approved workflow information.

Rejected workflow information shall not be retained within this contract.

9.3.9 Metadata Contract

Purpose

The Metadata Contract records execution information associated with Follow-Up Engine processing.

Required Sections

- Execution Identifier
- Engine Version
- Contract Version
- Processing Timestamp
- Validation Timestamp
- Processing Status
- Validation Status
- Execution Metrics

Validation Requirements

Metadata shall accurately represent the completed execution.

9.3.10 Follow-Up Execution Package Contract

Purpose

The Follow-Up Execution Package Contract represents the complete output produced by the Follow-Up Engine.

Required Sections

- Formatted Follow-Up Package
- Metadata Package
- Validation Report
- Processing Status

Validation Requirements

The package shall be complete before publication to downstream systems.

9.3.11 Contract Versioning

Every contract shall include:

- Contract Identifier
- Major Version
- Minor Version
- Schema Version

Version identifiers shall remain consistent throughout a single execution.

9.3.12 Contract Validation

Before processing, every contract shall be verified for:

- Structural integrity.
- Required sections.
- Mandatory fields.
- Data consistency.
- Schema compliance.
- Version compatibility.

Contracts failing validation shall be rejected.

9.3.13 Contract Integrity Rules

The Follow-Up Engine shall:

- Preserve upstream identifiers.
- Preserve contract ownership.
- Preserve immutable fields.
- Reject unauthorized modifications.
- Prevent duplicate contract publication.
- Prevent incomplete contract generation.

9.3.14 Downstream Publication Contract

The Follow-Up Execution Package Contract shall be published for downstream consumption by the CRM Engine.

No additional transformation shall occur within the Follow-Up Engine after publication.

9.3.15 Completion Requirements

This phase is considered complete when:

- All input contracts are defined.
- All output contracts are defined.
- Mandatory sections are documented.

- Validation requirements are specified.
- Versioning rules are documented.
- Integrity rules are established.
- Downstream publication contract is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.4

FOLLOW-UP ENGINE EXECUTION WORKFLOW

9.4.1 Overview

The Follow-Up Engine executes through a fixed sequential workflow.

Each execution stage has a single defined responsibility.

Execution shall proceed only after successful completion of the preceding stage.

No execution stage may be skipped, reordered, executed conditionally, or executed in parallel within a single Follow-Up Engine request.

9.4.2 Workflow Objectives

The execution workflow shall:

- Consume validated upstream contracts.
- Assemble a unified follow-up workflow context.
- Prepare deterministic follow-up sequences.
- Initialize workflow state.
- Validate generated workflow information.
- Produce standardized Follow-Up Execution Packages.

9.4.3 Workflow Entry Conditions

Execution shall begin only when all of the following conditions are satisfied:

- AI Personalized Email Engine has completed successfully.
- Final Email Package is available.
- Execution Metadata is available.
- Validation Report is available.
- Processing Status is available.
- Follow-Up Configuration is available.
- Workflow Configuration is available.
- Input contract validation has passed.

Failure to satisfy any entry condition shall terminate execution.

9.4.4 Stage 1 — Input Acquisition

Purpose

Acquire all validated contracts required for Follow-Up Engine execution.

Activities

- Load Unified Input Contract.
- Verify contract availability.
- Verify contract versions.
- Verify mandatory sections.
- Initialize execution context.

Output

Execution Input Context

9.4.5 Stage 2 — Workflow Assembly

Purpose

Construct the unified workflow context for follow-up processing.

Activities

- Merge Final Email information.
- Merge execution metadata.
- Merge validation information.
- Merge processing status.
- Merge workflow configuration.
- Normalize workflow context.

Output

Workflow Context Package

9.4.6 Stage 3 — Sequence Preparation

Purpose

Prepare the deterministic follow-up sequence.

Activities

- Create workflow identity.
- Create sequence identity.
- Prepare workflow stages.
- Prepare sequence metadata.
- Normalize sequence structure.
- Assemble sequence package.

Output

Follow-Up Sequence Package

9.4.7 Stage 4 — State Initialization

Purpose

Initialize follow-up workflow state.

Activities

- Initialize workflow state.
- Initialize processing state.
- Initialize validation state.
- Initialize publication state.
- Assemble state package.

Output

Follow-Up State Package

9.4.8 Stage 5 — Output Validation

Purpose

Validate workflow information against enterprise requirements.

Activities

- Validate workflow structure.
- Validate sequence integrity.
- Validate state information.
- Validate package completeness.
- Validate schema compliance.
- Generate Validation Report.

Output

Validated Follow-Up Package

Validation Report

9.4.9 Stage 6 — Package Formatting

Purpose

Transform validated workflow information into standardized downstream output.

Activities

- Normalize package structure.
- Normalize workflow formatting.
- Standardize package schema.
- Assemble formatted package.
- Verify formatting completeness.

Output

Formatted Follow-Up Package

9.4.10 Stage 7 — Metadata Generation

Purpose

Generate standardized execution metadata.

Activities

- Generate execution identifiers.
- Record engine version.
- Record processing metadata.
- Record validation metadata.
- Record execution metrics.
- Assemble Metadata Package.

Output

Metadata Package

9.4.11 Stage 8 — Final Publication

Purpose

Publish the completed Follow-Up Execution Package for downstream processing.

Activities

- Combine formatted package.
- Attach Metadata Package.
- Attach Validation Report.

- Attach Processing Status.
- Verify package completeness.
- Publish Follow-Up Execution Package.

Output

Follow-Up Execution Package

9.4.12 Execution Sequence

...

Entry Validation

↓

Input Acquisition

↓

Workflow Assembly

↓

Sequence Preparation

↓

State Initialization

↓

Output Validation

↓

Package Formatting

↓

Metadata Generation

↓

Final Publication

↓

Execution Complete

The execution sequence is fixed and immutable.

9.4.13 Workflow Control Rules

The execution workflow shall enforce the following rules:

- Execute stages sequentially.
- Prevent stage reordering.
- Prevent stage skipping.
- Prevent duplicate execution.
- Prevent partial publication.
- Preserve immutable upstream contracts.
- Preserve deterministic processing.

9.4.14 Failure Handling Workflow

If any stage fails:

- Terminate execution immediately.
- Discard intermediate outputs.
- Prevent downstream execution.
- Record failure status.
- Generate execution failure report.
- Do not publish a Follow-Up Execution Package.

9.4.15 Retry Policy

Execution may be retried only after:

- Failed validation has been resolved.

- Missing contracts have been supplied.
- Version incompatibilities have been corrected.
- Required upstream information has become available.

Each retry shall initiate a new execution cycle.

9.4.16 Execution Completion Criteria

Execution is considered successful only when:

- All workflow stages have completed.
- Validation has passed.
- Metadata has been generated.
- Follow-Up Execution Package has been assembled.
- Publication has completed successfully.

9.4.17 Workflow Integrity

The execution workflow shall maintain:

- Fixed execution order.
- Deterministic processing.
- Immutable upstream contracts.
- Consistent contract usage.
- Complete traceability across all execution stages.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.5

FOLLOW-UP ENGINE WORKFLOW ASSEMBLY LAYER

9.5.1 Overview

The Workflow Assembly Layer is responsible for transforming validated upstream contracts into a unified Follow-Up Workflow Context for downstream processing.

The layer performs structured workflow aggregation only.

The layer does not generate follow-up communication, perform scheduling, execute downstream actions, or modify validated upstream contracts.

9.5.2 Objectives

The Workflow Assembly Layer shall:

- Aggregate validated workflow information.
- Normalize heterogeneous contract data.
- Consolidate execution context.
- Preserve upstream contract integrity.
- Build a unified Follow-Up Workflow Context.
- Prepare deterministic input for downstream processing.

9.5.3 Architectural Position

The Workflow Assembly Layer executes immediately after successful completion of the Input Aggregation Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

...

The layer shall complete successfully before sequence preparation begins.

9.5.4 Responsibilities

The Workflow Assembly Layer is responsible for:

- Reading validated input contracts.
- Consolidating Final Email information.
- Consolidating execution metadata.
- Consolidating validation information.
- Consolidating processing status.
- Consolidating workflow configuration.
- Normalizing structured workflow data.
- Eliminating duplicated information.
- Producing a unified Follow-Up Workflow Context.

9.5.5 Input Sources

The Workflow Assembly Layer consumes the following validated inputs:

- Unified Input Contract
- Final Email Package Contract
- Execution Metadata Contract
- Validation Report Contract
- Processing Status Contract
- Follow-Up Configuration Contract
- Workflow Configuration Contract
- Agency Configuration Contract
- Brand Voice Configuration Contract

All input contracts shall be validated before processing.

9.5.6 Workflow Context Categories

The assembled workflow context shall contain the following categories:

Email Context

- Email identifier
- Email reference
- Publication status
- Validation status
- Processing summary

Execution Context

- Execution identifier
- Processing metadata
- Validation metadata
- Engine information
- Contract information

Workflow Context

- Workflow identity
- Workflow configuration
- Workflow processing rules
- Workflow execution context

Follow-Up Context

- Follow-up configuration
- Sequence definition
- Processing parameters
- State initialization information

Agency Context

- Agency configuration
- Brand voice configuration
- Enterprise communication standards

9.5.7 Workflow Assembly Process

The Workflow Assembly Layer shall execute the following sequence:

1. Load validated contracts.
2. Verify mandatory sections.
3. Normalize field structures.
4. Consolidate Final Email information.
5. Consolidate execution metadata.
6. Consolidate validation information.
7. Consolidate workflow configuration.
8. Consolidate follow-up configuration.
9. Remove duplicated information.
10. Validate assembled workflow context.
11. Publish Follow-Up Workflow Context.

Execution order shall remain fixed.

9.5.8 Normalization Rules

The Workflow Assembly Layer shall:

- Preserve original identifiers.
- Preserve upstream ownership.
- Normalize field formats.
- Normalize naming conventions.
- Normalize text encoding.
- Normalize structural organization.

Normalization shall not alter business meaning or workflow intent.

9.5.9 Duplicate Resolution

Duplicate information shall be resolved according to the following principles:

- Preserve authoritative source data.
- Remove redundant records.
- Eliminate repeated metadata.
- Eliminate repeated workflow information.

- Retain a single normalized representation.

No conflicting information shall remain within the assembled workflow context.

9.5.10 Workflow Integrity Rules

The assembled workflow context shall:

- Contain verified information only.
- Preserve upstream traceability.
- Exclude fabricated information.
- Exclude inferred workflow state.
- Exclude unsupported processing data.
- Maintain deterministic structure.

9.5.11 Output Structure

The Workflow Assembly Layer produces a single structured output:

Follow-Up Workflow Context Package

The package shall contain:

- Email Context
- Execution Context
- Workflow Context
- Follow-Up Context
- Agency Context

9.5.12 Validation Requirements

Before publication, the assembled workflow context shall be verified for:

- Structural completeness.
- Mandatory sections.
- Source consistency.
- Duplicate elimination.
- Schema compliance.

- Version compatibility.
- Data integrity.

Only validated workflow context shall be published.

9.5.13 Failure Conditions

Workflow assembly shall terminate if:

- Mandatory contracts are unavailable.
- Required sections are missing.
- Schema validation fails.
- Contract versions are incompatible.
- Duplicate resolution fails.
- Workflow context validation fails.

No partial workflow context shall be published.

9.5.14 Downstream Interface

The validated Follow-Up Workflow Context Package shall be transferred to the Sequence Preparation Layer.

No additional transformation shall occur after publication by the Workflow Assembly Layer.

9.5.15 Processing Constraints

The Workflow Assembly Layer shall not:

- Generate follow-up communication.
- Execute scheduling.
- Modify upstream contracts.
- Create workflow assumptions.
- Invent processing information.
- Perform AI inference.
- Alter validated source information.

9.5.16 Traceability

Every workflow context element shall maintain traceability to its originating upstream contract.

Traceability shall be preserved throughout downstream processing.

9.5.17 Completion Requirements

This phase is considered complete when:

- Input sources are documented.
- Workflow context categories are defined.
- Assembly workflow is established.
- Normalization rules are specified.
- Duplicate resolution rules are documented.
- Validation requirements are defined.
- Output structure is documented.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.6

FOLLOW-UP ENGINE SEQUENCE PREPARATION LAYER

9.6.1 Overview

The Sequence Preparation Layer is responsible for transforming the validated Follow-Up Workflow Context Package into a deterministic Follow-Up Sequence Package.

The layer prepares structured follow-up sequences only.

The layer does not generate follow-up communication, execute scheduling, initiate delivery, perform CRM synchronization, or modify validated upstream workflow information.

9.6.2 Objectives

The Sequence Preparation Layer shall:

- Consume validated workflow context.
- Prepare deterministic follow-up sequences.
- Build workflow stage definitions.
- Generate sequence metadata.
- Preserve workflow integrity.
- Produce standardized Follow-Up Sequence Packages.

9.6.3 Architectural Position

The Sequence Preparation Layer executes immediately after successful completion of the Workflow Assembly Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

↓

State Management Layer

...

The layer shall complete successfully before state initialization begins.

9.6.4 Responsibilities

The Sequence Preparation Layer is responsible for:

- Reading the Follow-Up Workflow Context Package.
- Building workflow sequence structures.
- Defining sequence stages.
- Generating sequence identifiers.
- Preparing sequence metadata.
- Normalizing sequence organization.
- Producing the Follow-Up Sequence Package.

9.6.5 Input Source

The Sequence Preparation Layer consumes:

- Follow-Up Workflow Context Package

The package shall be validated before processing.

9.6.6 Sequence Categories

The Follow-Up Sequence Package shall include the following categories:

Workflow Identity

- Workflow identifier
- Sequence identifier
- Parent execution reference
- Package reference

Sequence Definition

- Sequence structure
- Stage ordering
- Processing rules
- Execution context

Stage Information

- Stage identifiers
- Stage definitions
- Stage dependencies
- Stage metadata

Processing Context

- Processing configuration
- Workflow configuration
- Follow-up configuration
- Enterprise processing rules

Sequence Metadata

- Sequence version
- Schema version
- Generation metadata
- Validation reference

9.6.7 Sequence Preparation Process

The Sequence Preparation Layer shall execute the following sequence:

1. Load the Follow-Up Workflow Context Package.
2. Verify mandatory sections.
3. Initialize workflow identity.
4. Generate sequence identifier.
5. Construct sequence definition.
6. Define workflow stages.
7. Generate sequence metadata.
8. Normalize sequence structure.
9. Validate sequence completeness.
10. Publish Follow-Up Sequence Package.

Execution order shall remain fixed.

9.6.8 Sequence Construction Rules

The Sequence Preparation Layer shall:

- Use validated workflow information only.
- Preserve workflow identity.
- Preserve execution traceability.
- Preserve deterministic stage ordering.
- Preserve enterprise workflow standards.
- Preserve upstream references.

No unsupported workflow information shall be introduced.

9.6.9 Sequence Integrity Rules

The Follow-Up Sequence Package shall:

- Preserve workflow consistency.
- Preserve sequence consistency.
- Preserve identifier integrity.
- Preserve stage relationships.
- Preserve upstream traceability.
- Maintain deterministic organization.

9.6.10 Sequence Standardization

Every Follow-Up Sequence Package shall:

- Follow identical structural organization.
- Use standardized section names.
- Use normalized field representations.
- Maintain schema consistency.
- Maintain version compatibility.

Sequence structure shall remain deterministic across all executions.

9.6.11 Output Structure

The Sequence Preparation Layer produces:

Follow-Up Sequence Package

The package shall contain:

- Workflow Identity
- Sequence Definition
- Stage Information
- Processing Context
- Sequence Metadata

9.6.12 Validation Requirements

Before publication, the Follow-Up Sequence Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- Sequence consistency.
- Source integrity.
- Schema compliance.
- Version compatibility.
- Workflow integrity.

Only validated sequence packages shall be published.

9.6.13 Failure Conditions

Sequence preparation shall terminate if:

- Follow-Up Workflow Context Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- Sequence construction fails.
- Schema validation fails.

- Sequence validation fails.

No partial sequence package shall be published.

9.6.14 Downstream Interface

The validated Follow-Up Sequence Package shall be transferred to the State Management Layer.

No additional transformation shall occur after publication by the Sequence Preparation Layer.

9.6.15 Processing Constraints

The Sequence Preparation Layer shall not:

- Generate follow-up communication.
- Execute workflow stages.
- Perform scheduling.
- Modify upstream workflow information.
- Invent workflow data.
- Perform AI inference.
- Alter validated source information.

9.6.16 Traceability

Every sequence component shall maintain traceability to its originating workflow context.

Traceability shall remain preserved throughout downstream processing.

9.6.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Sequence categories are defined.
- Sequence preparation process is documented.

- Sequence construction rules are established.
- Sequence integrity rules are specified.
- Output structure is documented.
- Validation requirements are defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.7

FOLLOW-UP ENGINE STATE MANAGEMENT LAYER

9.7.1 Overview

The State Management Layer is responsible for initializing and maintaining the structured processing state of the Follow-Up Engine during execution.

The layer transforms the validated Follow-Up Sequence Package into a standardized Follow-Up State Package.

The layer performs state initialization and state management only.

The layer does not generate follow-up communication, execute workflow stages, perform scheduling, synchronize with external systems, or modify validated upstream workflow information.

9.7.2 Objectives

The State Management Layer shall:

- Consume validated Follow-Up Sequence Packages.
- Initialize workflow state.
- Initialize processing state.

- Initialize validation state.
- Initialize publication state.
- Generate deterministic state information.
- Produce standardized Follow-Up State Packages.

9.7.3 Architectural Position

The State Management Layer executes immediately after successful completion of the Sequence Preparation Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

↓

State Management Layer

↓

Output Validation Layer

...

The layer shall complete successfully before output validation begins.

9.7.4 Responsibilities

The State Management Layer is responsible for:

- Reading the Follow-Up Sequence Package.
- Initializing workflow state.

- Initializing processing state.
- Initializing validation state.
- Initializing publication state.
- Recording state metadata.
- Producing the Follow-Up State Package.

9.7.5 Input Source

The State Management Layer consumes:

- Follow-Up Sequence Package

The package shall be validated before processing.

9.7.6 State Categories

The Follow-Up State Package shall contain the following categories:

Workflow State

- Workflow identifier
- Current workflow status
- Workflow lifecycle state
- Workflow completion status

Processing State

- Processing identifier
- Processing status
- Processing stage
- Processing result

Validation State

- Validation status
- Validation reference

- Validation lifecycle
- Validation result

Publication State

- Publication status
- Publication readiness
- Publication lifecycle
- Publication result

State Metadata

- State version
- Schema version
- State timestamp
- Processing reference

9.7.7 State Initialization Process

The State Management Layer shall execute the following sequence:

1. Load the Follow-Up Sequence Package.
2. Verify mandatory sections.
3. Initialize Workflow State.
4. Initialize Processing State.
5. Initialize Validation State.
6. Initialize Publication State.
7. Generate State Metadata.
8. Normalize state structure.
9. Validate state completeness.
10. Publish Follow-Up State Package.

Execution order shall remain fixed.

9.7.8 State Management Rules

The State Management Layer shall:

- Use validated sequence information only.
- Preserve workflow identity.
- Preserve processing consistency.
- Preserve validation consistency.
- Preserve publication consistency.
- Preserve execution traceability.

No unsupported state information shall be introduced.

9.7.9 State Integrity Rules

The Follow-Up State Package shall:

- Preserve workflow consistency.
- Preserve processing consistency.
- Preserve validation consistency.
- Preserve publication consistency.
- Preserve identifier integrity.
- Maintain deterministic organization.

9.7.10 State Standardization

Every Follow-Up State Package shall:

- Follow identical structural organization.
- Use standardized section names.
- Use normalized field representations.
- Maintain schema consistency.
- Maintain version compatibility.

State structure shall remain deterministic across all executions.

9.7.11 Output Structure

The State Management Layer produces:

Follow-Up State Package

The package shall contain:

- Workflow State
- Processing State
- Validation State
- Publication State
- State Metadata

9.7.12 Validation Requirements

Before publication, the Follow-Up State Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- State consistency.
- Source integrity.
- Schema compliance.
- Version compatibility.
- Workflow integrity.

Only validated state packages shall be published.

9.7.13 Failure Conditions

State initialization shall terminate if:

- Follow-Up Sequence Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- State initialization fails.
- Schema validation fails.
- State validation fails.

No partial state package shall be published.

9.7.14 Downstream Interface

The validated Follow-Up State Package shall be transferred to the Output Validation Layer.

No additional transformation shall occur after publication by the State Management Layer.

9.7.15 Processing Constraints

The State Management Layer shall not:

- Generate follow-up communication.
- Execute workflow stages.
- Perform scheduling.
- Modify upstream workflow information.
- Invent workflow state.
- Perform AI inference.
- Alter validated source information.

9.7.16 Traceability

Every state component shall maintain traceability to:

- Follow-Up Sequence Package
- Workflow Context Package
- Final Email Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

9.7.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- State categories are defined.
- State initialization process is documented.
- State management rules are established.
- State integrity rules are specified.
- Output structure is documented.

- Validation requirements are defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.8

FOLLOW-UP ENGINE OUTPUT VALIDATION LAYER

9.8.1 Overview

The Output Validation Layer is responsible for verifying that every Follow-Up State Package complies with enterprise workflow standards, structural requirements, processing requirements, and output contract specifications before publication.

The layer performs validation only.

The layer does not generate workflow content, modify workflow information, execute workflow stages, perform scheduling, or execute AI inference.

9.8.2 Objectives

The Output Validation Layer shall:

- Validate structural completeness.
- Validate workflow consistency.
- Validate sequence integrity.
- Validate state integrity.
- Validate formatting consistency.
- Validate output contract compliance.
- Produce validation results for downstream processing.

9.8.3 Architectural Position

The Output Validation Layer executes immediately after successful completion of the State Management Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

↓

State Management Layer

↓

Output Validation Layer

↓

Packaging Layer

...

The layer shall complete successfully before package formatting begins.

9.8.4 Responsibilities

The Output Validation Layer is responsible for:

- Reading the Follow-Up State Package.
- Verifying structural integrity.
- Verifying workflow consistency.
- Verifying sequence integrity.

- Verifying state consistency.
- Verifying formatting compliance.
- Verifying enterprise workflow standards.
- Producing a Validation Report.

9.8.5 Input Source

The Output Validation Layer consumes:

- Follow-Up State Package

The package shall be validated for structural integrity before processing.

9.8.6 Validation Categories

The Output Validation Layer shall validate the following categories:

Structural Validation

- Workflow structure
- Sequence structure
- State structure
- Metadata structure
- Required sections
- Package completeness

Workflow Validation

- Workflow identity
- Workflow consistency
- Workflow configuration
- Workflow dependencies
- Workflow traceability

Sequence Validation

- Sequence identifiers
- Stage ordering
- Sequence integrity
- Sequence metadata
- Sequence consistency

State Validation

- Workflow state
- Processing state
- Validation state
- Publication state
- State consistency

Formatting Validation

- Section ordering
- Output formatting
- Schema compliance
- Required field formatting
- Package consistency

9.8.7 Validation Workflow

The Output Validation Layer shall execute the following sequence:

1. Load the Follow-Up State Package.
2. Verify package integrity.
3. Validate mandatory sections.
4. Validate workflow structure.
5. Validate sequence integrity.
6. Validate state consistency.
7. Validate formatting compliance.
8. Validate output schema.
9. Generate Validation Report.
10. Publish Validated Follow-Up Package.

Execution order shall remain fixed.

9.8.8 Validation Rules

The Output Validation Layer shall ensure:

- Every mandatory section is present.
- Every workflow reference is verified.
- Every sequence reference is verified.
- Every state reference is verified.
- Enterprise workflow standards are maintained.
- Output contracts remain consistent.

9.8.9 Quality Rules

The validated workflow package shall:

- Maintain structural integrity.
- Maintain workflow consistency.
- Maintain sequence consistency.
- Maintain state consistency.
- Maintain deterministic organization.
- Maintain enterprise processing quality.
- Maintain downstream compatibility.

9.8.10 Rejection Criteria

The workflow package shall be rejected if any of the following conditions are detected:

- Missing mandatory sections.
- Invalid workflow references.
- Invalid sequence relationships.
- Invalid state information.
- Structural corruption.
- Schema violations.
- Validation contract failure.
- Output package inconsistency.

Rejected packages shall not proceed to downstream processing.

9.8.11 Validation Report Structure

The Validation Report shall contain:

- Validation Identifier
- Processing Status
- Structural Validation Status
- Workflow Validation Status
- Sequence Validation Status
- State Validation Status
- Formatting Validation Status
- Overall Validation Result

9.8.12 Output Structure

The Output Validation Layer produces:

Validated Follow-Up Package

The package shall contain:

- Approved Workflow Package
- Validation Report
- Validation Status

Only validated workflow packages shall be published.

9.8.13 Failure Conditions

Validation shall terminate if:

- Follow-Up State Package is unavailable.
- Mandatory sections are missing.
- Structural validation fails.
- Workflow validation fails.
- Sequence validation fails.
- State validation fails.

- Formatting validation fails.
- Output schema validation fails.

Failed validation packages shall not be published.

9.8.14 Downstream Interface

The Validated Follow-Up Package shall be transferred to the Packaging Layer.

No additional validation shall occur after publication by the Output Validation Layer.

9.8.15 Processing Constraints

The Output Validation Layer shall not:

- Modify workflow information.
- Generate workflow content.
- Execute workflow stages.
- Perform scheduling.
- Execute AI inference.
- Alter validated upstream contracts.
- Generate workflow assumptions.

9.8.16 Traceability

Every validation result shall maintain traceability to:

- Follow-Up State Package
- Follow-Up Sequence Package
- Workflow Context Package
- Final Email Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

9.8.17 Completion Requirements

This phase is considered complete when:

- Validation objectives are documented.
- Validation categories are defined.
- Validation workflow is established.
- Validation rules are specified.
- Quality rules are documented.
- Rejection criteria are defined.
- Validation report structure is documented.
- Output structure is defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.9

FOLLOW-UP ENGINE PACKAGING LAYER

9.9.1 Overview

The Packaging Layer is responsible for transforming the Validated Follow-Up Package into a standardized Formatted Follow-Up Package suitable for downstream enterprise processing.

The layer performs packaging and formatting only.

The layer does not generate workflow content, validate workflow information, execute workflow stages, perform scheduling, or execute AI inference.

9.9.2 Objectives

The Packaging Layer shall:

- Standardize validated workflow output.
- Normalize package formatting.
- Assemble standardized execution packages.
- Preserve validated workflow integrity.
- Prepare downstream-compatible output.
- Produce immutable publication-ready packages.

9.9.3 Architectural Position

The Packaging Layer executes immediately after successful completion of the Output Validation Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

↓

State Management Layer

↓

Output Validation Layer

↓

Packaging Layer

↓

Metadata Generation Layer

...

The layer shall complete successfully before metadata generation begins.

9.9.4 Responsibilities

The Packaging Layer is responsible for:

- Reading the Validated Follow-Up Package.
- Standardizing approved workflow information.
- Applying package formatting rules.
- Normalizing package structure.
- Packaging approved workflow components.
- Producing standardized downstream output.

9.9.5 Input Source

The Packaging Layer consumes:

- Validated Follow-Up Package

The package shall be validated before packaging begins.

9.9.6 Packaging Categories

The Packaging Layer shall standardize the following categories:

Workflow Structure

- Workflow information
- Sequence information
- State information
- Validation information
- Processing information

Package Formatting

- Section organization

- Package hierarchy
- Whitespace normalization
- Character encoding
- Structural normalization

Schema Formatting

- Package schema
- Section ordering
- Required field ordering
- Output consistency
- Schema organization

Publication Package

- Workflow package
- Validation report
- Processing status
- Publication readiness

9.9.7 Packaging Workflow

The Packaging Layer shall execute the following sequence:

1. Load the Validated Follow-Up Package.
2. Verify package integrity.
3. Normalize workflow structure.
4. Normalize package formatting.
5. Normalize schema organization.
6. Assemble standardized package.
7. Verify package completeness.
8. Publish Formatted Follow-Up Package.

Execution order shall remain fixed.

9.9.8 Packaging Rules

The Packaging Layer shall ensure:

- Approved workflow information remains unchanged.
- Section ordering remains standardized.
- Package formatting remains consistent.
- Output complies with package schema.
- Character encoding remains normalized.
- Package structure remains deterministic.

9.9.9 Content Preservation Rules

The Packaging Layer shall preserve:

- Approved workflow information.
- Approved sequence information.
- Approved state information.
- Approved validation report.
- Approved processing status.

No approved information shall be modified during packaging.

9.9.10 Standardization Rules

Every Formatted Follow-Up Package shall:

- Follow identical structural organization.
- Use standardized section ordering.
- Use normalized spacing.
- Use normalized formatting.
- Maintain schema consistency.
- Maintain deterministic package organization.

9.9.11 Output Structure

The Packaging Layer produces:

Formatted Follow-Up Package

The package shall contain:

- Approved Workflow Information
- Approved Sequence Information
- Approved State Information
- Validation Report
- Validation Status
- Processing Status

9.9.12 Packaging Validation

Before publication, the formatted package shall be verified for:

- Structural completeness.
- Formatting consistency.
- Package completeness.
- Schema compliance.
- Section ordering.
- Output integrity.

Only correctly formatted packages shall be published.

9.9.13 Failure Conditions

Packaging shall terminate if:

- Validated Follow-Up Package is unavailable.
- Package integrity verification fails.
- Packaging normalization fails.
- Package assembly fails.
- Schema validation fails.
- Output validation fails.

No partially packaged output shall be published.

9.9.14 Downstream Interface

The Formatted Follow-Up Package shall be transferred to the Metadata Generation Layer.

No additional packaging shall occur after publication by the Packaging Layer.

9.9.15 Processing Constraints

The Packaging Layer shall not:

- Generate workflow content.
- Modify approved workflow information.
- Execute workflow stages.
- Perform scheduling.
- Execute AI inference.
- Alter validation results.
- Modify upstream contracts.

9.9.16 Traceability

Every formatted package shall maintain traceability to:

- Validated Follow-Up Package
- Validation Report
- Follow-Up State Package
- Follow-Up Sequence Package
- Workflow Context Package
- Final Email Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

9.9.17 Completion Requirements

This phase is considered complete when:

- Packaging objectives are documented.
- Packaging categories are defined.
- Packaging workflow is established.
- Packaging rules are specified.

- Content preservation rules are documented.
- Standardization rules are defined.
- Output structure is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.10

FOLLOW-UP ENGINE METADATA GENERATION LAYER

9.10.1 Overview

The Metadata Generation Layer is responsible for generating standardized execution metadata for every successfully formatted Follow-Up Package.

The metadata provides execution traceability, processing status, validation status, version information, and audit records required by downstream enterprise systems.

The layer performs metadata generation only.

The layer does not generate workflow content, validate workflow information, modify formatted packages, execute workflow stages, perform scheduling, or execute AI inference.

9.10.2 Objectives

The Metadata Generation Layer shall:

- Generate execution metadata.
- Record processing information.
- Record validation information.
- Preserve execution traceability.

- Produce standardized metadata packages.
- Prepare publication-ready output for downstream enterprise systems.

9.10.3 Architectural Position

The Metadata Generation Layer executes immediately after successful completion of the Packaging Layer.

Execution Flow

...

Input Aggregation Layer

↓

Workflow Assembly Layer

↓

Sequence Preparation Layer

↓

State Management Layer

↓

Output Validation Layer

↓

Packaging Layer

↓

Metadata Generation Layer

↓

Follow-Up Execution Package

...

The layer shall complete successfully before final package publication.

9.10.4 Responsibilities

The Metadata Generation Layer is responsible for:

- Reading the Formatted Follow-Up Package.
- Generating execution identifiers.
- Recording engine information.
- Recording processing information.
- Recording validation information.
- Recording schema information.
- Producing standardized metadata.

9.10.5 Input Source

The Metadata Generation Layer consumes:

- Formatted Follow-Up Package

The package shall be verified before metadata generation begins.

9.10.6 Metadata Categories

The Metadata Package shall contain the following categories:

Execution Metadata

- Execution Identifier
- Processing Identifier
- Session Identifier
- Correlation Identifier

Engine Metadata

- Engine Name

- Engine Version
- Module Version
- Workflow Version

Contract Metadata

- Contract Version
- Schema Version
- Package Version

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

Validation Metadata

- Validation Identifier
- Validation Status
- Validation Timestamp
- Validation Result

Publication Metadata

- Publication Status
- Publication Timestamp
- Output Package Identifier

9.10.7 Metadata Generation Workflow

The Metadata Generation Layer shall execute the following sequence:

1. Load the Formatted Follow-Up Package.
2. Verify package integrity.
3. Generate execution metadata.
4. Generate engine metadata.
5. Generate contract metadata.
6. Generate processing metadata.
7. Generate validation metadata.
8. Generate publication metadata.
9. Assemble Metadata Package.
10. Validate metadata completeness.
11. Publish Follow-Up Execution Package.

Execution order shall remain fixed.

9.10.8 Metadata Rules

The Metadata Generation Layer shall ensure:

- Metadata accurately represents execution.
- Metadata remains immutable after publication.
- Metadata preserves execution traceability.
- Metadata follows standardized schema.
- Metadata remains version consistent.
- Metadata contains no fabricated values.

9.10.9 Integrity Rules

The Metadata Package shall:

- Preserve execution identifiers.
- Preserve validation identifiers.
- Preserve processing history.
- Preserve version information.
- Preserve package integrity.
- Preserve downstream compatibility.

9.10.10 Output Structure

The Metadata Generation Layer produces:

Metadata Package

The package shall contain:

- Execution Metadata
- Engine Metadata
- Contract Metadata
- Processing Metadata
- Validation Metadata
- Publication Metadata

9.10.11 Follow-Up Execution Package

The Metadata Generation Layer assembles the Follow-Up Execution Package.

The Follow-Up Execution Package shall contain:

- Formatted Follow-Up Package
- Metadata Package
- Validation Report
- Processing Status
- Publication Status

The package shall be immutable after publication.

9.10.12 Metadata Validation

Before publication, the Metadata Package shall be verified for:

- Structural completeness.
- Required metadata fields.
- Schema compliance.
- Version consistency.
- Identifier consistency.
- Processing consistency.
- Publication readiness.

Only validated metadata shall be published.

9.10.13 Failure Conditions

Metadata generation shall terminate if:

- Formatted Follow-Up Package is unavailable.
- Package integrity verification fails.
- Metadata generation fails.
- Identifier generation fails.
- Metadata validation fails.
- Final package assembly fails.

No incomplete metadata package shall be published.

9.10.14 Downstream Interface

The completed Follow-Up Execution Package shall be published for downstream consumption by the CRM Engine.

The Metadata Generation Layer represents the final processing stage of the Follow-Up Engine.

No additional processing shall occur within this engine after publication.

9.10.15 Processing Constraints

The Metadata Generation Layer shall not:

- Generate workflow content.
- Modify formatted workflow information.
- Modify validation results.
- Execute workflow stages.
- Perform scheduling.
- Execute AI inference.
- Alter upstream contracts.
- Reformat published output.

9.10.16 Traceability

Every metadata record shall maintain traceability to:

- Formatted Follow-Up Package
- Validated Follow-Up Package
- Follow-Up State Package
- Follow-Up Sequence Package
- Workflow Context Package
- Final Email Package
- Execution Metadata

End-to-end traceability shall remain preserved throughout the complete execution lifecycle.

9.10.17 Completion Requirements

This phase is considered complete when:

- Metadata objectives are documented.
- Metadata categories are defined.
- Metadata workflow is established.
- Metadata rules are specified.
- Integrity rules are documented.
- Output structure is defined.
- Follow-Up Execution Package is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.11

FOLLOW-UP ENGINE OUTPUT SCHEMA SPECIFICATION

9.11.1 Overview

The Output Schema Specification defines the standardized structure of the Follow-Up Execution Package produced by the Follow-Up Engine.

The schema establishes a deterministic output contract for downstream enterprise systems.

All generated output shall conform to this specification before publication.

9.11.2 Objectives

The Output Schema Specification shall:

- Define standardized output structures.
- Define required output sections.
- Define schema hierarchy.
- Define mandatory fields.
- Define validation requirements.
- Define downstream compatibility requirements.

9.11.3 Schema Design Principles

The output schema shall comply with the following principles:

- Deterministic structure.
- Immutable after publication.
- Version controlled.
- Self-describing.
- Platform independent.
- Strongly structured.
- Enterprise compatible.
- Backward compatible within the same major version.

9.11.4 Top-Level Output Schema

The Follow-Up Execution Package shall contain the following top-level sections:

...

Follow-Up Execution Package

- |— Formatted Follow-Up Package
- |— Metadata Package
- |— Validation Report
- |— Processing Status

...

No additional top-level sections shall be introduced.

9.11.5 Formatted Follow-Up Package Schema

The Formatted Follow-Up Package shall contain:

...

Formatted Follow-Up Package

- |— Workflow Information
- |— Sequence Information
- |— State Information
- |— Validation Information
- |— Processing Information
- |— Packaging Status

...

All sections are mandatory.

9.11.6 Metadata Package Schema

The Metadata Package shall contain:

...

Metadata Package

- |— Execution Metadata
- |— Engine Metadata
- |— Contract Metadata
- |— Processing Metadata
- |— Validation Metadata

└─ Publication Metadata
...

All metadata categories are mandatory.

9.11.7 Execution Metadata Schema

Execution Metadata shall contain:

...

Execution Metadata

└─ Execution Identifier
└─ Processing Identifier
└─ Session Identifier
└─ Correlation Identifier
...

9.11.8 Engine Metadata Schema

Engine Metadata shall contain:

...

Engine Metadata

└─ Engine Name
└─ Engine Version
└─ Module Version
└─ Workflow Version
...

9.11.9 Contract Metadata Schema

Contract Metadata shall contain:

...

Contract Metadata

- Contract Version
- Schema Version
- Package Version

...

9.11.10 Processing Metadata Schema

Processing Metadata shall contain:

...

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

...

9.11.11 Validation Metadata Schema

Validation Metadata shall contain:

...

Validation Metadata

- Validation Identifier
- Validation Timestamp
- Validation Status
- Validation Result

...

9.11.12 Publication Metadata Schema

Publication Metadata shall contain:

...

Publication Metadata

- Publication Status
- Publication Timestamp
- Output Package Identifier

...

9.11.13 Validation Report Schema

The Validation Report shall contain:

...

Validation Report

- Validation Identifier
- Structural Validation
- Workflow Validation
- Sequence Validation
- State Validation
- Formatting Validation
- Overall Validation Result

...

All validation categories are mandatory.

9.11.14 Processing Status Schema

Processing Status shall contain:

...

Processing Status

- Current Status
- Completion Status
- Publication Status
- Processing Result

...

9.11.15 Schema Validation Rules

Every Follow-Up Execution Package shall satisfy the following requirements:

- All mandatory sections shall exist.
- All mandatory fields shall exist.
- Section hierarchy shall remain unchanged.
- Parent-child relationships shall remain valid.
- Version identifiers shall be present.
- Schema integrity shall be maintained.

9.11.16 Schema Integrity Rules

The Output Schema shall:

- Preserve deterministic ordering.
- Preserve immutable identifiers.
- Preserve validation results.
- Preserve metadata integrity.
- Preserve downstream compatibility.
- Preserve structural consistency.

9.11.17 Schema Compatibility

The Output Schema shall support:

- Internal enterprise services.
- Workflow orchestration.
- CRM Engine integration.
- Reporting systems.
- Audit systems.
- Analytics systems.
- Long-term archival.

Compatibility shall remain stable within the same major schema version.

9.11.18 Schema Publication Rules

The Follow-Up Execution Package shall be published only when:

- Schema validation has passed.
- Metadata validation has passed.
- Validation Report is complete.
- Processing Status indicates successful completion.
- All mandatory schema sections are present.

9.11.19 Schema Failure Conditions

Publication shall be rejected if:

- Mandatory sections are missing.
- Schema hierarchy is invalid.
- Required metadata is incomplete.
- Validation Report is incomplete.
- Processing Status is invalid.
- Schema validation fails.

No invalid schema shall be published.

9.11.20 Completion Requirements

This phase is considered complete when:

- Top-level schema is defined.
- All subordinate schemas are documented.
- Mandatory sections are specified.
- Validation rules are documented.
- Integrity rules are established.
- Compatibility requirements are defined.
- Publication rules are documented.
- Failure conditions are specified.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-4

PHASE 9.12

FOLLOW-UP ENGINE ENTERPRISE VALIDATION & PHASE COMPLETION

9.12.1 Overview

This phase defines the enterprise validation framework and formal completion requirements for the Follow-Up Engine.

Enterprise validation confirms that every architectural component, processing layer, execution workflow, interface, contract, and output specification defined within Phase 9 has been successfully documented and internally validated.

Successful completion authorizes the phase to enter the LOCKED state.

9.12.2 Validation Objectives

Enterprise validation shall verify:

- Architectural completeness.
- Documentation completeness.
- Workflow completeness.
- Module completeness.
- Interface completeness.
- Contract completeness.
- Schema completeness.
- Enterprise compliance.
- Internal consistency.

9.12.3 Validation Scope

The validation scope includes every document produced within Phase 9.

Validated documents include:

- Phase 9 — Follow-Up Engine Foundation
- Phase 9.1 — Follow-Up Engine Architecture

- Phase 9.2 — Follow-Up Engine Internal Modules
- Phase 9.3 — Follow-Up Engine Data Contracts
- Phase 9.4 — Follow-Up Engine Execution Workflow
- Phase 9.5 — Follow-Up Engine Workflow Assembly Layer
- Phase 9.6 — Follow-Up Engine Sequence Preparation Layer
- Phase 9.7 — Follow-Up Engine State Management Layer
- Phase 9.8 — Follow-Up Engine Output Validation Layer
- Phase 9.9 — Follow-Up Engine Packaging Layer
- Phase 9.10 — Follow-Up Engine Metadata Generation Layer
- Phase 9.11 — Follow-Up Engine Output Schema Specification

No additional documents are included within this validation scope.

9.12.4 Architectural Validation

Enterprise validation shall confirm that:

- Architectural boundaries are documented.
- Layer responsibilities are documented.
- Module responsibilities are documented.
- Processing order is fixed.
- Upstream dependencies are documented.
- Downstream interfaces are documented.
- Separation of responsibilities is maintained.

9.12.5 Workflow Validation

The execution workflow shall be verified for:

- Sequential execution.
- Fixed execution order.
- Stage completeness.
- Entry conditions.
- Exit conditions.
- Failure handling.
- Publication sequence.

No undocumented execution path shall exist.

9.12.6 Module Validation

Each internal module shall be verified for:

- Defined responsibilities.
- Defined inputs.
- Defined outputs.
- Processing constraints.
- Validation requirements.
- Downstream interfaces.

All modules shall maintain single-responsibility architecture.

9.12.7 Layer Validation

Each processing layer shall be verified for:

- Architectural position.
- Responsibilities.
- Input requirements.
- Output requirements.
- Workflow integration.
- Validation rules.
- Failure conditions.

All documented layers shall integrate without architectural conflict.

9.12.8 Contract Validation

Enterprise validation shall confirm that:

- Input contracts are documented.
- Output contracts are documented.
- Contract hierarchy is complete.
- Contract integrity rules are defined.
- Versioning rules are defined.
- Validation rules are defined.

All contract interfaces shall remain deterministic.

9.12.9 Schema Validation

The Follow-Up Execution Package schema shall be verified for:

- Structural completeness.
- Mandatory sections.
- Schema hierarchy.
- Metadata integration.
- Validation report integration.
- Processing status integration.

The published schema shall remain immutable after validation.

9.12.10 Traceability Validation

End-to-end traceability shall be verified across:

- Workflow Context Package.
- Follow-Up Sequence Package.
- Follow-Up State Package.
- Validated Follow-Up Package.
- Formatted Follow-Up Package.
- Metadata Package.
- Follow-Up Execution Package.

Every downstream artifact shall maintain traceability to validated upstream contracts.

9.12.11 Enterprise Compliance Validation

The Follow-Up Engine shall comply with the following enterprise principles:

- Modular architecture.
- Deterministic processing.
- Immutable contracts.
- Standardized interfaces.
- Enterprise documentation standards.
- Fixed execution workflow.

- Read-only upstream consumption.
- Immutable published outputs.

9.12.12 Documentation Completeness Validation

Enterprise documentation shall verify that every Phase 9 document contains:

- Defined purpose.
- Architectural position.
- Responsibilities.
- Processing workflow.
- Input definition.
- Output definition.
- Validation requirements.
- Failure conditions.
- Completion requirements.

No mandatory documentation section shall be omitted.

9.12.13 Validation Result

Enterprise validation shall confirm that:

- All Phase 9 documentation has been completed.
- All architectural components have been documented.
- All processing layers have been documented.
- All internal modules have been documented.
- All execution workflows have been documented.
- All data contracts have been documented.
- All output schemas have been documented.
- All enterprise validation requirements have been satisfied.

9.12.14 Phase Completion Criteria

Phase 9 shall be considered complete when:

- Enterprise validation is successful.
- Documentation completeness is verified.

- Architectural validation is successful.
- Workflow validation is successful.
- Module validation is successful.
- Contract validation is successful.
- Schema validation is successful.
- Traceability validation is successful.

Only after all completion criteria have passed shall Phase 9 be considered complete.

9.12.15 Phase Status

Phase Name

Follow-Up Engine

Phase Identifier

PART 2B-4 — Phase 9

Phase State

COMPLETED

Validation Status

PASSED

Architecture Status

VERIFIED

Documentation Status

COMPLETE

Workflow Status

VERIFIED

Contract Status

VERIFIED

Schema Status

VERIFIED

Publication Status

APPROVED

9.12.16 Phase Lock

Following successful enterprise validation:

- Phase 9 is declared COMPLETE.
- Phase 9 enters the LOCKED state.
- Architectural changes are prohibited.
- Workflow modifications are prohibited.
- Module modifications are prohibited.
- Contract modifications are prohibited.
- Schema modifications are prohibited.
- Documentation modifications are prohibited unless a formal change request is approved.

9.12.17 Completion Summary

The Follow-Up Engine has been fully specified as an enterprise-grade subsystem.

The completed documentation defines:

- Enterprise architecture.
- Internal modules.
- Data contracts.
- Execution workflow.
- Workflow Assembly Layer.
- Sequence Preparation Layer.
- State Management Layer.
- Output Validation Layer.
- Packaging Layer.
- Metadata Generation Layer.
- Output Schema Specification.
- Enterprise validation framework.

This concludes the complete documentation scope for PART 2B-4 — Phase 9.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10

CRM ENGINE FOUNDATION

10.1 Purpose

The CRM Engine is responsible for managing structured customer relationship records after successful completion of the Follow-Up Engine.

The engine operates exclusively on validated Follow-Up Execution Packages and associated execution metadata.

The engine does not perform lead discovery, website analysis, contact extraction, email generation, follow-up workflow generation, communication delivery, or external CRM synchronization.

Its sole responsibility is to create, maintain, organize, and publish enterprise customer relationship records for downstream business operations.

10.2 Objectives

The CRM Engine shall:

- Consume validated Follow-Up Execution Packages.
- Consume execution metadata.
- Consume workflow metadata.
- Maintain customer relationship records.
- Maintain lead lifecycle information.

- Maintain communication history references.
- Maintain workflow status.
- Produce enterprise CRM records.

10.3 Scope

Included Responsibilities:

- CRM record creation
- CRM record maintenance
- Customer lifecycle management
- Relationship state management
- Workflow tracking
- CRM metadata generation
- CRM package generation

Excluded Responsibilities:

- Lead discovery
- Search Engine execution
- Website auditing
- Prototype generation
- Contact extraction
- Email validation
- AI email generation
- Follow-up generation
- Communication delivery
- External CRM synchronization
- Calendar management

10.4 Dependencies

The CRM Engine depends on successful completion of:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine

- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine

Execution shall not begin unless all dependencies have completed successfully.

10.5 Execution Position

Enterprise Workflow

Phase 0

↓

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine

↓

AI Personalized Email Engine



Follow-Up Engine



CRM Engine



Reporting Engine

10.6 Engine Responsibilities

The engine is responsible for:

- Reading validated Follow-Up Execution Packages.
- Reading execution metadata.
- Creating CRM records.
- Maintaining customer relationship state.
- Maintaining workflow history.
- Preparing downstream CRM packages.
- Producing standardized CRM outputs.

10.7 Inputs

The engine consumes:

Follow-Up Execution Package

Execution Metadata

Validation Report

Processing Status

CRM Configuration

Agency Configuration

Brand Voice Configuration

Workflow Configuration

10.8 Outputs

The engine produces:

CRM Record

Customer Relationship Record

Lead Lifecycle Record

Workflow History

CRM Metadata

CRM Execution Package

Quality Report

10.9 Functional Responsibilities

The engine shall:

Create one CRM record for each successfully published Follow-Up Execution Package.

Each CRM record shall remain associated with its originating workflow.

Every CRM record shall preserve execution traceability.

Every generated CRM record shall comply with enterprise CRM standards.

10.10 Processing Principles

The engine shall never:

Generate communication.

Deliver emails.

Execute follow-up workflows.

Modify validated upstream packages.

Modify execution metadata.

Modify workflow history.

Only validated upstream outputs may be consumed.

10.11 CRM Foundation

The CRM Engine shall establish:

- CRM record identity.
- Customer relationship context.
- Lead lifecycle context.
- Workflow history context.
- CRM processing metadata.
- Downstream CRM execution package.

10.12 Enterprise Constraints

The engine shall never:

Invent customer interactions.

Invent communication history.

Invent workflow activity.

Invent engagement metrics.

Invent delivery status.

Modify published Follow-Up Execution Packages.

10.13 Error Handling

If mandatory upstream information is unavailable:

The engine shall terminate processing.

An execution failure report shall be generated.

No partial CRM Execution Package shall be published.

10.14 Completion Criteria

Phase completion requires:

- Successful input validation.
- Successful CRM record creation.
- Successful relationship context preparation.
- Successful metadata generation.
- Successful output validation.
- Successful package generation.
- Successful enterprise verification.

Only after all completion criteria have passed shall the engine be considered complete.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.1

CRM ENGINE ARCHITECTURE

10.1.1 Overview

The CRM Engine is an enterprise subsystem responsible for creating and maintaining structured customer relationship records from validated Follow-Up Execution Packages produced by the Follow-Up Engine.

The engine operates exclusively on validated internal system outputs.

The engine does not perform communication generation, communication delivery, workflow execution, external CRM synchronization, or modification of upstream intelligence.

Its responsibility is limited to preparing deterministic CRM Execution Packages for downstream enterprise processing.

10.1.2 Architectural Position

The CRM Engine is positioned immediately after successful completion of the Follow-Up Engine.

System Execution Order

...

Foundation Framework

↓

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine

↓

AI Personalized Email Engine

↓

Follow-Up Engine

↓

CRM Engine

↓

Reporting Engine

...

The engine shall not execute before all upstream engines have completed successfully.

10.1.3 Architectural Principles

The architecture shall comply with the following principles:

- Single responsibility.
- Modular execution.
- Deterministic workflow.
- Read-only consumption of upstream outputs.
- Stateless processing.
- Immutable execution packages.
- Enterprise-grade reliability.

- Strict separation of processing layers.

10.1.4 High-Level Architecture

...

Follow-Up Execution Package

|



Input Aggregation Layer

|



CRM Context Assembly Layer

|



Relationship Management Layer

|



Lifecycle Management Layer

|



Output Validation Layer

|



Packaging Layer

|



CRM Execution Package

...

Each architectural layer performs one defined responsibility and passes structured output to the next processing stage.

10.1.5 Architectural Layers

The engine consists of the following logical layers:

Layer 1

Input Aggregation Layer

Responsibilities:

- Load validated Follow-Up Execution Package.
- Load execution metadata.
- Load CRM configuration.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Layer 2

CRM Context Assembly Layer

Responsibilities:

- Build CRM execution context.
- Assemble relationship information.
- Consolidate execution metadata.
- Normalize CRM structure.
- Produce unified CRM context.

Layer 3

Relationship Management Layer

Responsibilities:

- Create customer relationship records.
- Associate workflow references.
- Associate communication references.
- Prepare relationship metadata.
- Produce deterministic relationship structures.

Layer 4

Lifecycle Management Layer

Responsibilities:

- Initialize lifecycle state.
- Record customer lifecycle status.
- Record workflow lifecycle information.
- Prepare lifecycle package.

Layer 5

Output Validation Layer

Responsibilities:

- Validate CRM completeness.
- Validate relationship integrity.
- Validate lifecycle information.
- Validate package consistency.
- Validate required sections.

Layer 6

Packaging Layer

Responsibilities:

- Assemble CRM execution package.
- Normalize package structure.
- Prepare downstream output.
- Produce standardized CRM Execution Package.

10.1.6 Data Flow

...

Validated Follow-Up Execution Package

↓

Input Aggregation

↓

CRM Context Assembly

↓

Relationship Management

↓

Lifecycle Management

↓

Validation

↓

Packaging

↓

CRM Execution Package

...

Processing shall occur sequentially.

No architectural layer may be skipped.

10.1.7 Upstream Dependencies

The architecture consumes outputs from:

- Follow-Up Engine
- Follow-Up Execution Package
- Execution Metadata
- Validation Report
- Processing Status

No direct interaction with external CRM systems is permitted.

10.1.8 Downstream Dependency

The engine publishes structured CRM Execution Packages for consumption by the Reporting Engine.

The engine does not perform reporting.

The engine does not synchronize with external CRM platforms.

10.1.9 State Management

The architecture is stateless.

Each execution shall:

- Load validated inputs.
- Prepare CRM records.
- Generate execution package.
- Return results.
- Release execution state.

No persistent execution session shall be maintained within the engine.

10.1.10 Failure Handling

Execution shall terminate when:

- Required inputs are missing.
- Mandatory packages are unavailable.
- CRM context assembly fails.
- Relationship management fails.
- Validation fails.
- Packaging fails.

No partial CRM Execution Package shall be published.

10.1.11 Scalability Requirements

The architecture shall support:

- Independent execution per CRM record.
- Parallel processing across multiple CRM records.
- Isolated execution contexts.
- Horizontal scaling without workflow modification.
- Deterministic output generation for identical inputs.

10.1.12 Security Principles

The architecture shall:

- Operate on validated internal packages only.
- Prevent modification of upstream outputs.
- Prevent unauthorized data injection.
- Maintain strict separation between processing layers.
- Preserve integrity of generated CRM execution packages.

10.1.13 Completion Requirements

The architecture is considered complete when:

- All architectural layers are defined.
- Data flow is established.
- Processing order is fixed.
- Dependencies are documented.
- Failure handling is specified.
- Scalability requirements are defined.
- Security principles are documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.2

CRM ENGINE INTERNAL MODULES

10.2.1 Overview

The CRM Engine is composed of independent internal modules.

Each module performs a single well-defined responsibility.

Modules execute sequentially according to the fixed processing workflow.

No module shall perform responsibilities assigned to another module.

10.2.2 Module Architecture

...

Input Manager

|



CRM Context Builder



Relationship Manager



Lifecycle Manager



Output Validator



Package Formatter



Metadata Generator



CRM Execution Package

...

10.2.3 Input Manager

Purpose

The Input Manager loads all validated upstream packages required for CRM Engine processing.

Responsibilities

- Load Follow-Up Execution Package.
- Load Execution Metadata.
- Load Validation Report.
- Load Processing Status.
- Load CRM Configuration.
- Verify mandatory inputs.
- Reject incomplete execution requests.

Inputs

- Follow-Up Execution Package
- Execution Metadata
- Validation Report
- CRM Configuration
- Workflow Configuration

Outputs

Unified Input Package

10.2.4 CRM Context Builder

Purpose

The CRM Context Builder constructs the unified CRM execution context.

Responsibilities

- Consolidate CRM information.
- Consolidate relationship information.
- Consolidate execution metadata.
- Consolidate processing status.
- Normalize CRM structure.
- Prepare execution context.

Inputs

Unified Input Package

Outputs

CRM Context Package

10.2.5 Relationship Manager

Purpose

The Relationship Manager creates deterministic customer relationship records.

Responsibilities

- Create customer relationship records.
- Associate workflow references.
- Associate communication references.
- Associate execution references.
- Prepare relationship metadata.

Inputs

CRM Context Package

Outputs

Relationship Package

10.2.6 Lifecycle Manager

Purpose

The Lifecycle Manager initializes and maintains customer lifecycle information.

Responsibilities

- Initialize lifecycle state.
- Initialize processing state.
- Record workflow lifecycle.
- Record relationship status.
- Prepare lifecycle package.

Inputs

Relationship Package

Outputs

Lifecycle Package

10.2.7 Output Validator

Purpose

The Output Validator verifies that all CRM components satisfy enterprise requirements.

Responsibilities

- Validate CRM completeness.
- Validate relationship integrity.
- Validate lifecycle information.
- Validate package consistency.
- Validate required sections.
- Validate enterprise CRM standards.

Inputs

Lifecycle Package

Outputs

Validated CRM Package

Validation Report

10.2.8 Package Formatter

Purpose

The Package Formatter converts validated CRM information into a standardized downstream package.

Responsibilities

- Normalize package formatting.
- Normalize CRM structure.
- Standardize package schema.
- Prepare downstream package.

Inputs

Validated CRM Package

Outputs

Formatted CRM Package

10.2.9 Metadata Generator

Purpose

The Metadata Generator records execution metadata associated with CRM Engine execution.

Responsibilities

- Generate execution identifier.
- Record engine version.
- Record processing status.
- Record validation status.
- Record execution metrics.
- Produce metadata package.

Inputs

Formatted CRM Package

Validation Report

Outputs

Metadata Package

CRM Execution Package

10.2.10 Module Interaction Sequence

...

Input Manager



CRM Context Builder



Relationship Manager



Lifecycle Manager



Output Validator



Package Formatter



Metadata Generator



CRM Execution Package

...

Module execution order is fixed.

No module may execute before completion of its predecessor.

10.2.11 Module Independence

Each module shall:

- Perform one responsibility only.
- Consume defined inputs.
- Produce defined outputs.
- Avoid direct dependency on non-adjacent modules.
- Preserve upstream package integrity.
- Avoid modification of validated upstream outputs.

10.2.12 Error Propagation

If any module reports failure:

- Execution shall stop immediately.
- Remaining modules shall not execute.
- Partial outputs shall be discarded.
- An execution failure report shall be generated.

10.2.13 Extensibility

Internal modules shall remain independently maintainable.

Future implementation updates shall preserve:

- Module boundaries.
- Processing order.
- Input contracts.
- Output contracts.
- Public interfaces.

10.2.14 Completion Requirements

This phase is considered complete when:

- All internal modules are defined.

- Module responsibilities are documented.
- Input contracts are documented.
- Output contracts are documented.
- Execution sequence is fixed.
- Failure propagation rules are specified.
- Module interaction is fully documented.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.3

CRM ENGINE DATA CONTRACTS

10.3.1 Overview

The CRM Engine exchanges information exclusively through structured data contracts.

Data contracts define the required inputs, generated outputs, mandatory fields, validation requirements, and interface boundaries between the CRM Engine and other enterprise subsystems.

All contracts are immutable during execution.

The engine shall consume upstream contracts in read-only mode.

10.3.2 Contract Design Principles

Every data contract shall comply with the following principles:

- Strongly structured.
- Version controlled.
- Immutable during execution.
- Self-describing.

- Deterministic.
- Platform independent.
- Schema validated.
- Backward compatible within the same major version.

10.3.3 Upstream Input Contracts

The CRM Engine consumes the following validated contracts:

- Follow-Up Execution Package Contract
- Execution Metadata Contract
- Validation Report Contract
- Processing Status Contract
- CRM Configuration Contract
- Workflow Configuration Contract
- Agency Configuration Contract
- Brand Voice Configuration Contract

Each contract shall be validated prior to processing.

10.3.4 Unified Input Contract

Purpose

The Unified Input Contract represents the normalized dataset consumed by the CRM Engine after successful input aggregation.

Required Sections

- Follow-Up Information
- Execution Metadata
- Validation Information
- Processing Information
- CRM Configuration
- Workflow Configuration
- Agency Configuration
- Brand Voice Configuration

Validation Requirements

The Unified Input Contract shall:

- Contain all mandatory sections.
- Preserve upstream identifiers.
- Preserve source integrity.
- Contain normalized field formats.
- Exclude duplicate records.

10.3.5 CRM Context Contract

Purpose

The CRM Context Contract represents the consolidated customer relationship information required for CRM record creation.

Required Sections

- CRM Identity
- Customer Information
- Relationship Context
- Workflow Context
- Communication References
- Lifecycle Context
- Configuration Summary
- CRM Processing Context

Validation Requirements

The contract shall contain verified information only.

No inferred or fabricated information shall be included.

10.3.6 Relationship Contract

Purpose

The Relationship Contract contains the structured customer relationship information required for downstream CRM processing.

Required Sections

- Relationship Identifier
- CRM Identifier
- Customer Reference
- Workflow Reference
- Communication Reference
- Relationship Metadata

Validation Requirements

Every relationship element shall originate from validated CRM context information.

10.3.7 Lifecycle Contract

Purpose

The Lifecycle Contract represents the initialized customer lifecycle information managed by the CRM Engine.

Required Sections

- Lifecycle State
- Relationship State
- Workflow State
- Processing State
- Lifecycle Metadata

Validation Requirements

The lifecycle contract shall contain only verified processing information.

10.3.8 Validated CRM Contract

Purpose

The Validated CRM Contract represents the approved CRM record after successful enterprise validation.

Required Sections

- Approved CRM Record
- Approved Relationship Information
- Approved Lifecycle Information
- Validation Status
- Validation Identifier

Validation Requirements

The contract shall contain only approved CRM information.

Rejected CRM information shall not be retained within this contract.

10.3.9 Metadata Contract

Purpose

The Metadata Contract records execution information associated with CRM Engine processing.

Required Sections

- Execution Identifier
- Engine Version
- Contract Version
- Processing Timestamp
- Validation Timestamp
- Processing Status
- Validation Status
- Execution Metrics

Validation Requirements

Metadata shall accurately represent the completed execution.

10.3.10 CRM Execution Package Contract

Purpose

The CRM Execution Package Contract represents the complete output produced by the CRM Engine.

Required Sections

- Formatted CRM Package
- Metadata Package
- Validation Report
- Processing Status

Validation Requirements

The package shall be complete before publication to downstream systems.

10.3.11 Contract Versioning

Every contract shall include:

- Contract Identifier
- Major Version
- Minor Version
- Schema Version

Version identifiers shall remain consistent throughout a single execution.

10.3.12 Contract Validation

Before processing, every contract shall be verified for:

- Structural integrity.
- Required sections.
- Mandatory fields.
- Data consistency.
- Schema compliance.
- Version compatibility.

Contracts failing validation shall be rejected.

10.3.13 Contract Integrity Rules

The CRM Engine shall:

- Preserve upstream identifiers.
- Preserve contract ownership.
- Preserve immutable fields.
- Reject unauthorized modifications.
- Prevent duplicate contract publication.
- Prevent incomplete contract generation.

10.3.14 Downstream Publication Contract

The CRM Execution Package Contract shall be published for downstream consumption by the Reporting Engine.

No additional transformation shall occur within the CRM Engine after publication.

10.3.15 Completion Requirements

This phase is considered complete when:

- All input contracts are defined.
- All output contracts are defined.
- Mandatory sections are documented.
- Validation requirements are specified.
- Versioning rules are documented.
- Integrity rules are established.
- Downstream publication contract is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.4

CRM ENGINE EXECUTION WORKFLOW

10.4.1 Overview

The CRM Engine executes through a fixed sequential workflow.

Each execution stage has a single defined responsibility.

Execution shall proceed only after successful completion of the preceding stage.

No execution stage may be skipped, reordered, executed conditionally, or executed in parallel within a single CRM Engine request.

10.4.2 Workflow Objectives

The execution workflow shall:

- Consume validated upstream contracts.
- Assemble a unified CRM context.
- Prepare deterministic customer relationship records.
- Initialize lifecycle information.
- Validate generated CRM information.
- Produce standardized CRM Execution Packages.

10.4.3 Workflow Entry Conditions

Execution shall begin only when all of the following conditions are satisfied:

- Follow-Up Engine has completed successfully.
- Follow-Up Execution Package is available.
- Execution Metadata is available.
- Validation Report is available.
- Processing Status is available.
- CRM Configuration is available.
- Workflow Configuration is available.
- Input contract validation has passed.

Failure to satisfy any entry condition shall terminate execution.

10.4.4 Stage 1 — Input Acquisition

Purpose

Acquire all validated contracts required for CRM Engine execution.

Activities

- Load Unified Input Contract.
- Verify contract availability.
- Verify contract versions.
- Verify mandatory sections.
- Initialize execution context.

Output

Execution Input Context

10.4.5 Stage 2 — CRM Context Assembly

Purpose

Construct the unified CRM context for customer relationship processing.

Activities

- Merge Follow-Up information.
- Merge execution metadata.
- Merge validation information.
- Merge processing status.
- Merge CRM configuration.
- Normalize CRM context.

Output

CRM Context Package

10.4.6 Stage 3 — Relationship Management

Purpose

Prepare deterministic customer relationship records.

Activities

- Create CRM identity.
- Create relationship identity.
- Associate customer references.
- Associate workflow references.
- Associate communication references.
- Assemble relationship package.

Output

Relationship Package

10.4.7 Stage 4 — Lifecycle Initialization

Purpose

Initialize customer lifecycle information.

Activities

- Initialize lifecycle state.
- Initialize relationship state.
- Initialize workflow state.
- Initialize processing state.
- Assemble lifecycle package.

Output

Lifecycle Package

10.4.8 Stage 5 — Output Validation

Purpose

Validate CRM information against enterprise requirements.

Activities

- Validate CRM structure.
- Validate relationship integrity.
- Validate lifecycle information.
- Validate package completeness.
- Validate schema compliance.
- Generate Validation Report.

Output

Validated CRM Package

Validation Report

10.4.9 Stage 6 — Package Formatting

Purpose

Transform validated CRM information into standardized downstream output.

Activities

- Normalize package structure.
- Normalize CRM formatting.
- Standardize package schema.
- Assemble formatted package.
- Verify formatting completeness.

Output

Formatted CRM Package

10.4.10 Stage 7 — Metadata Generation

Purpose

Generate standardized execution metadata.

Activities

- Generate execution identifiers.
- Record engine version.
- Record processing metadata.
- Record validation metadata.
- Record execution metrics.
- Assemble Metadata Package.

Output

Metadata Package

10.4.11 Stage 8 — Final Publication

Purpose

Publish the completed CRM Execution Package for downstream processing.

Activities

- Combine formatted package.
- Attach Metadata Package.
- Attach Validation Report.
- Attach Processing Status.
- Verify package completeness.
- Publish CRM Execution Package.

Output

CRM Execution Package

10.4.12 Execution Sequence

...

Entry Validation

↓

Input Acquisition

↓

CRM Context Assembly

↓

Relationship Management

↓

Lifecycle Initialization

↓

Output Validation

↓

Package Formatting

↓

Metadata Generation

↓

Final Publication

↓

Execution Complete

...

The execution sequence is fixed and immutable.

10.4.13 Workflow Control Rules

The execution workflow shall enforce the following rules:

- Execute stages sequentially.
- Prevent stage reordering.
- Prevent stage skipping.
- Prevent duplicate execution.
- Prevent partial publication.
- Preserve immutable upstream contracts.
- Preserve deterministic processing.

10.4.14 Failure Handling Workflow

If any stage fails:

- Terminate execution immediately.
- Discard intermediate outputs.
- Prevent downstream execution.
- Record failure status.

- Generate execution failure report.
- Do not publish a CRM Execution Package.

10.4.15 Retry Policy

Execution may be retried only after:

- Failed validation has been resolved.
- Missing contracts have been supplied.
- Version incompatibilities have been corrected.
- Required upstream information has become available.

Each retry shall initiate a new execution cycle.

10.4.16 Execution Completion Criteria

Execution is considered successful only when:

- All workflow stages have completed.
- Validation has passed.
- Metadata has been generated.
- CRM Execution Package has been assembled.
- Publication has completed successfully.

10.4.17 Workflow Integrity

The execution workflow shall maintain:

- Fixed execution order.
- Deterministic processing.
- Immutable upstream contracts.
- Consistent contract usage.
- Complete traceability across all execution stages.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.5

CRM ENGINE CONTEXT ASSEMBLY LAYER

10.5.1 Overview

The CRM Context Assembly Layer is responsible for transforming validated upstream contracts into a unified CRM Context Package for downstream processing.

The layer performs structured CRM context aggregation only.

The layer does not generate customer relationship records, perform lifecycle management, execute downstream operations, synchronize with external CRM systems, or modify validated upstream contracts.

10.5.2 Objectives

The CRM Context Assembly Layer shall:

- Aggregate validated CRM information.
- Normalize heterogeneous contract data.
- Consolidate customer relationship information.
- Preserve upstream contract integrity.
- Build a unified CRM Context Package.
- Prepare deterministic input for downstream processing.

10.5.3 Architectural Position

The CRM Context Assembly Layer executes immediately after successful completion of the Input Aggregation Layer.

Execution Flow

...

Input Aggregation Layer



CRM Context Assembly Layer



Relationship Management Layer

...

The layer shall complete successfully before relationship management begins.

10.5.4 Responsibilities

The CRM Context Assembly Layer is responsible for:

- Reading validated input contracts.
- Consolidating Follow-Up information.
- Consolidating execution metadata.
- Consolidating validation information.
- Consolidating processing status.
- Consolidating CRM configuration.
- Normalizing structured CRM data.
- Eliminating duplicated information.
- Producing a unified CRM Context Package.

10.5.5 Input Sources

The CRM Context Assembly Layer consumes the following validated inputs:

- Unified Input Contract
- Follow-Up Execution Package Contract
- Execution Metadata Contract
- Validation Report Contract
- Processing Status Contract
- CRM Configuration Contract
- Workflow Configuration Contract
- Agency Configuration Contract

- Brand Voice Configuration Contract

All input contracts shall be validated before processing.

10.5.6 CRM Context Categories

The assembled CRM context shall contain the following categories:

Customer Context

- Customer identifier
- Customer references
- Relationship references
- Communication references
- Processing summary

Execution Context

- Execution identifier
- Processing metadata
- Validation metadata
- Engine information
- Contract information

CRM Context

- CRM identity
- CRM configuration
- CRM processing rules
- CRM execution context

Lifecycle Context

- Lifecycle configuration
- Relationship context
- Workflow context

- State initialization information

Agency Context

- Agency configuration
- Brand voice configuration
- Enterprise CRM standards

10.5.7 Context Assembly Process

The CRM Context Assembly Layer shall execute the following sequence:

1. Load validated contracts.
2. Verify mandatory sections.
3. Normalize field structures.
4. Consolidate Follow-Up information.
5. Consolidate execution metadata.
6. Consolidate validation information.
7. Consolidate CRM configuration.
8. Consolidate lifecycle configuration.
9. Remove duplicated information.
10. Validate assembled CRM context.
11. Publish CRM Context Package.

Execution order shall remain fixed.

10.5.8 Normalization Rules

The CRM Context Assembly Layer shall:

- Preserve original identifiers.
- Preserve upstream ownership.
- Normalize field formats.
- Normalize naming conventions.
- Normalize text encoding.
- Normalize structural organization.

Normalization shall not alter customer relationship meaning or CRM processing intent.

10.5.9 Duplicate Resolution

Duplicate information shall be resolved according to the following principles:

- Preserve authoritative source data.
- Remove redundant records.
- Eliminate repeated metadata.
- Eliminate repeated CRM information.
- Retain a single normalized representation.

No conflicting information shall remain within the assembled CRM context.

10.5.10 Context Integrity Rules

The assembled CRM context shall:

- Contain verified information only.
- Preserve upstream traceability.
- Exclude fabricated information.
- Exclude inferred customer relationships.
- Exclude unsupported processing data.
- Maintain deterministic structure.

10.5.11 Output Structure

The CRM Context Assembly Layer produces a single structured output:

CRM Context Package

The package shall contain:

- Customer Context
- Execution Context
- CRM Context
- Lifecycle Context
- Agency Context

10.5.12 Validation Requirements

Before publication, the assembled CRM context shall be verified for:

- Structural completeness.
- Mandatory sections.
- Source consistency.
- Duplicate elimination.
- Schema compliance.
- Version compatibility.
- Data integrity.

Only validated CRM context shall be published.

10.5.13 Failure Conditions

CRM context assembly shall terminate if:

- Mandatory contracts are unavailable.
- Required sections are missing.
- Schema validation fails.
- Contract versions are incompatible.
- Duplicate resolution fails.
- CRM context validation fails.

No partial CRM context shall be published.

10.5.14 Downstream Interface

The validated CRM Context Package shall be transferred to the Relationship Management Layer.

No additional transformation shall occur after publication by the CRM Context Assembly Layer.

10.5.15 Processing Constraints

The CRM Context Assembly Layer shall not:

- Generate customer relationship records.
- Execute lifecycle management.
- Modify upstream contracts.
- Create customer relationship assumptions.
- Invent processing information.
- Perform AI inference.
- Alter validated source information.

10.5.16 Traceability

Every CRM context element shall maintain traceability to its originating upstream contract.

Traceability shall be preserved throughout downstream processing.

10.5.17 Completion Requirements

This phase is considered complete when:

- Input sources are documented.
- CRM context categories are defined.
- Assembly workflow is established.
- Normalization rules are specified.
- Duplicate resolution rules are documented.
- Validation requirements are defined.
- Output structure is documented.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.6

CRM ENGINE RELATIONSHIP MANAGEMENT LAYER

10.6.1 Overview

The Relationship Management Layer is responsible for transforming the validated CRM Context Package into deterministic customer relationship records.

The layer creates and organizes structured relationship information only.

The layer does not generate communication, execute follow-up workflows, perform lifecycle management, synchronize with external CRM systems, or modify validated upstream CRM context.

10.6.2 Objectives

The Relationship Management Layer shall:

- Consume validated CRM Context Packages.
- Create deterministic customer relationship records.
- Associate workflow references.
- Associate communication references.
- Associate execution references.
- Produce standardized Relationship Packages.

10.6.3 Architectural Position

The Relationship Management Layer executes immediately after successful completion of the CRM Context Assembly Layer.

Execution Flow

...

Input Aggregation Layer

↓

CRM Context Assembly Layer

↓

Relationship Management Layer

↓

Lifecycle Management Layer

...

The layer shall complete successfully before lifecycle initialization begins.

10.6.4 Responsibilities

The Relationship Management Layer is responsible for:

- Reading the CRM Context Package.
- Creating CRM identifiers.
- Creating relationship identifiers.
- Associating customer references.
- Associating workflow references.
- Associating communication references.
- Associating execution references.
- Producing the Relationship Package.

10.6.5 Input Source

The Relationship Management Layer consumes:

- CRM Context Package

The package shall be validated before processing.

10.6.6 Relationship Categories

The Relationship Package shall contain the following categories:

CRM Identity

- CRM identifier
- Relationship identifier
- Customer identifier
- Parent execution reference

Customer References

- Customer reference
- Workflow reference
- Communication reference
- Follow-Up reference

Relationship Context

- Relationship definition
- Relationship classification
- Relationship processing rules
- Relationship execution context

Relationship Metadata

- Relationship version
- Schema version
- Processing metadata
- Validation reference

Execution References

- Execution identifier
- Processing identifier
- Validation identifier
- Package reference

10.6.7 Relationship Creation Process

The Relationship Management Layer shall execute the following sequence:

1. Load the CRM Context Package.
2. Verify mandatory sections.
3. Initialize CRM identity.
4. Generate relationship identifier.
5. Associate customer references.
6. Associate workflow references.
7. Associate communication references.
8. Associate execution references.
9. Generate relationship metadata.
10. Validate relationship completeness.
11. Publish Relationship Package.

Execution order shall remain fixed.

10.6.8 Relationship Rules

The Relationship Management Layer shall:

- Use validated CRM context only.
- Preserve CRM identity.
- Preserve execution traceability.
- Preserve relationship consistency.
- Preserve enterprise CRM standards.
- Preserve upstream references.

No unsupported relationship information shall be introduced.

10.6.9 Relationship Integrity Rules

The Relationship Package shall:

- Preserve CRM consistency.
- Preserve relationship consistency.
- Preserve identifier integrity.
- Preserve reference relationships.
- Preserve upstream traceability.
- Maintain deterministic organization.

10.6.10 Relationship Standardization

Every Relationship Package shall:

- Follow identical structural organization.
- Use standardized section names.
- Use normalized field representations.
- Maintain schema consistency.
- Maintain version compatibility.

Relationship structure shall remain deterministic across all executions.

10.6.11 Output Structure

The Relationship Management Layer produces:

Relationship Package

The package shall contain:

- CRM Identity
- Customer References
- Relationship Context
- Relationship Metadata
- Execution References

10.6.12 Validation Requirements

Before publication, the Relationship Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- Relationship consistency.
- Source integrity.
- Schema compliance.
- Version compatibility.
- CRM integrity.

Only validated relationship packages shall be published.

10.6.13 Failure Conditions

Relationship creation shall terminate if:

- CRM Context Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- Relationship generation fails.
- Schema validation fails.
- Relationship validation fails.

No partial relationship package shall be published.

10.6.14 Downstream Interface

The validated Relationship Package shall be transferred to the Lifecycle Management Layer.

No additional transformation shall occur after publication by the Relationship Management Layer.

10.6.15 Processing Constraints

The Relationship Management Layer shall not:

- Generate communication.
- Execute workflow stages.
- Perform lifecycle management.
- Modify upstream CRM context.
- Invent relationship information.
- Perform AI inference.
- Alter validated source information.

10.6.16 Traceability

Every relationship component shall maintain traceability to its originating CRM context.

Traceability shall remain preserved throughout downstream processing.

10.6.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Relationship categories are defined.
- Relationship creation process is documented.
- Relationship rules are established.
- Relationship integrity rules are specified.
- Output structure is documented.
- Validation requirements are defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.7

CRM ENGINE LIFECYCLE MANAGEMENT LAYER

10.7.1 Overview

The Lifecycle Management Layer is responsible for transforming the validated Relationship Package into a deterministic Lifecycle Package.

The layer initializes and manages structured customer lifecycle information only.

The layer does not generate communication, execute follow-up workflows, synchronize with external CRM systems, perform reporting, or modify validated upstream relationship information.

10.7.2 Objectives

The Lifecycle Management Layer shall:

- Consume validated Relationship Packages.
- Initialize customer lifecycle information.
- Initialize relationship state.
- Initialize workflow state.
- Initialize processing state.
- Produce standardized Lifecycle Packages.

10.7.3 Architectural Position

The Lifecycle Management Layer executes immediately after successful completion of the Relationship Management Layer.

Execution Flow

...

Input Aggregation Layer

↓

CRM Context Assembly Layer

↓

Relationship Management Layer

↓

Lifecycle Management Layer

↓

Output Validation Layer

...

The layer shall complete successfully before output validation begins.

10.7.4 Responsibilities

The Lifecycle Management Layer is responsible for:

- Reading the Relationship Package.
- Initializing lifecycle state.
- Initializing relationship state.
- Initializing workflow state.
- Initializing processing state.
- Recording lifecycle metadata.
- Producing the Lifecycle Package.

10.7.5 Input Source

The Lifecycle Management Layer consumes:

- Relationship Package

The package shall be validated before processing.

10.7.6 Lifecycle Categories

The Lifecycle Package shall contain the following categories:

Lifecycle State

- Lifecycle identifier
- Current lifecycle status
- Lifecycle stage
- Lifecycle completion status

Relationship State

- Relationship identifier
- Current relationship status
- Relationship lifecycle state
- Relationship processing result

Workflow State

- Workflow identifier
- Workflow status
- Workflow lifecycle
- Workflow processing state

Processing State

- Processing identifier
- Processing status
- Processing stage
- Processing result

Lifecycle Metadata

- Lifecycle version
- Schema version
- Processing timestamp
- Validation reference

10.7.7 Lifecycle Initialization Process

The Lifecycle Management Layer shall execute the following sequence:

1. Load the Relationship Package.
2. Verify mandatory sections.
3. Initialize Lifecycle State.
4. Initialize Relationship State.
5. Initialize Workflow State.

6. Initialize Processing State.
7. Generate Lifecycle Metadata.
8. Normalize lifecycle structure.
9. Validate lifecycle completeness.
10. Publish Lifecycle Package.

Execution order shall remain fixed.

10.7.8 Lifecycle Rules

The Lifecycle Management Layer shall:

- Use validated relationship information only.
- Preserve CRM identity.
- Preserve lifecycle consistency.
- Preserve workflow consistency.
- Preserve processing consistency.
- Preserve execution traceability.

No unsupported lifecycle information shall be introduced.

10.7.9 Lifecycle Integrity Rules

The Lifecycle Package shall:

- Preserve lifecycle consistency.
- Preserve relationship consistency.
- Preserve workflow consistency.
- Preserve identifier integrity.
- Preserve upstream traceability.
- Maintain deterministic organization.

10.7.10 Lifecycle Standardization

Every Lifecycle Package shall:

- Follow identical structural organization.
- Use standardized section names.

- Use normalized field representations.
- Maintain schema consistency.
- Maintain version compatibility.

Lifecycle structure shall remain deterministic across all executions.

10.7.11 Output Structure

The Lifecycle Management Layer produces:

Lifecycle Package

The package shall contain:

- Lifecycle State
- Relationship State
- Workflow State
- Processing State
- Lifecycle Metadata

10.7.12 Validation Requirements

Before publication, the Lifecycle Package shall be verified for:

- Structural completeness.
- Mandatory sections.
- Lifecycle consistency.
- Source integrity.
- Schema compliance.
- Version compatibility.
- CRM integrity.

Only validated lifecycle packages shall be published.

10.7.13 Failure Conditions

Lifecycle initialization shall terminate if:

- Relationship Package is unavailable.
- Mandatory sections are missing.
- Source validation fails.
- Lifecycle initialization fails.
- Schema validation fails.
- Lifecycle validation fails.

No partial lifecycle package shall be published.

10.7.14 Downstream Interface

The validated Lifecycle Package shall be transferred to the Output Validation Layer.

No additional transformation shall occur after publication by the Lifecycle Management Layer.

10.7.15 Processing Constraints

The Lifecycle Management Layer shall not:

- Generate communication.
- Execute workflow stages.
- Synchronize with external CRM systems.
- Modify upstream relationship information.
- Invent lifecycle information.
- Perform AI inference.
- Alter validated source information.

10.7.16 Traceability

Every lifecycle component shall maintain traceability to:

- Relationship Package
- CRM Context Package
- Follow-Up Execution Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

10.7.17 Completion Requirements

This phase is considered complete when:

- Input source is documented.
- Lifecycle categories are defined.
- Lifecycle initialization process is documented.
- Lifecycle rules are established.
- Lifecycle integrity rules are specified.
- Output structure is documented.
- Validation requirements are defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.8

CRM ENGINE OUTPUT VALIDATION LAYER

10.8.1 Overview

The Output Validation Layer is responsible for verifying that every Lifecycle Package complies with enterprise CRM standards, structural requirements, processing requirements, and output contract specifications before publication.

The layer performs validation only.

The layer does not generate CRM records, modify CRM information, execute CRM workflows, synchronize with external CRM systems, or execute AI inference.

10.8.2 Objectives

The Output Validation Layer shall:

- Validate structural completeness.
- Validate CRM consistency.
- Validate relationship integrity.
- Validate lifecycle integrity.
- Validate formatting consistency.
- Validate output contract compliance.
- Produce validation results for downstream processing.

10.8.3 Architectural Position

The Output Validation Layer executes immediately after successful completion of the Lifecycle Management Layer.

Execution Flow

...

Input Aggregation Layer

↓

CRM Context Assembly Layer

↓

Relationship Management Layer

↓

Lifecycle Management Layer

↓

Output Validation Layer

↓

Packaging Layer

...

The layer shall complete successfully before package formatting begins.

10.8.4 Responsibilities

The Output Validation Layer is responsible for:

- Reading the Lifecycle Package.
- Verifying structural integrity.
- Verifying CRM consistency.
- Verifying relationship integrity.
- Verifying lifecycle consistency.
- Verifying formatting compliance.
- Verifying enterprise CRM standards.
- Producing a Validation Report.

10.8.5 Input Source

The Output Validation Layer consumes:

- Lifecycle Package

The package shall be validated for structural integrity before processing.

10.8.6 Validation Categories

The Output Validation Layer shall validate the following categories:

Structural Validation

- CRM structure
- Relationship structure
- Lifecycle structure
- Metadata structure
- Required sections
- Package completeness

CRM Validation

- CRM identity
- CRM consistency
- CRM configuration
- CRM dependencies
- CRM traceability

Relationship Validation

- Relationship identifiers
- Relationship references
- Relationship integrity
- Relationship metadata
- Relationship consistency

Lifecycle Validation

- Lifecycle state
- Relationship state
- Workflow state
- Processing state
- Lifecycle consistency

Formatting Validation

- Section ordering
- Output formatting
- Schema compliance
- Required field formatting
- Package consistency

10.8.7 Validation Workflow

The Output Validation Layer shall execute the following sequence:

1. Load the Lifecycle Package.
2. Verify package integrity.
3. Validate mandatory sections.
4. Validate CRM structure.
5. Validate relationship integrity.
6. Validate lifecycle consistency.
7. Validate formatting compliance.
8. Validate output schema.
9. Generate Validation Report.
10. Publish Validated CRM Package.

Execution order shall remain fixed.

10.8.8 Validation Rules

The Output Validation Layer shall ensure:

- Every mandatory section is present.
- Every CRM reference is verified.
- Every relationship reference is verified.
- Every lifecycle reference is verified.
- Enterprise CRM standards are maintained.
- Output contracts remain consistent.

10.8.9 Quality Rules

The validated CRM package shall:

- Maintain structural integrity.
- Maintain CRM consistency.
- Maintain relationship consistency.
- Maintain lifecycle consistency.
- Maintain deterministic organization.
- Maintain enterprise processing quality.
- Maintain downstream compatibility.

10.8.10 Rejection Criteria

The CRM package shall be rejected if any of the following conditions are detected:

- Missing mandatory sections.
- Invalid CRM references.
- Invalid relationship references.
- Invalid lifecycle information.
- Structural corruption.
- Schema violations.
- Validation contract failure.
- Output package inconsistency.

Rejected packages shall not proceed to downstream processing.

10.8.11 Validation Report Structure

The Validation Report shall contain:

- Validation Identifier
- Processing Status
- Structural Validation Status
- CRM Validation Status
- Relationship Validation Status
- Lifecycle Validation Status
- Formatting Validation Status
- Overall Validation Result

10.8.12 Output Structure

The Output Validation Layer produces:

Validated CRM Package

The package shall contain:

- Approved CRM Package
- Validation Report
- Validation Status

Only validated CRM packages shall be published.

10.8.13 Failure Conditions

Validation shall terminate if:

- Lifecycle Package is unavailable.
- Mandatory sections are missing.
- Structural validation fails.
- CRM validation fails.
- Relationship validation fails.
- Lifecycle validation fails.
- Formatting validation fails.
- Output schema validation fails.

Failed validation packages shall not be published.

10.8.14 Downstream Interface

The Validated CRM Package shall be transferred to the Packaging Layer.

No additional validation shall occur after publication by the Output Validation Layer.

10.8.15 Processing Constraints

The Output Validation Layer shall not:

- Modify CRM information.
- Generate CRM content.
- Execute CRM workflows.
- Synchronize with external CRM systems.
- Execute AI inference.
- Alter validated upstream contracts.
- Generate CRM assumptions.

10.8.16 Traceability

Every validation result shall maintain traceability to:

- Lifecycle Package
- Relationship Package
- CRM Context Package
- Follow-Up Execution Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

10.8.17 Completion Requirements

This phase is considered complete when:

- Validation objectives are documented.
- Validation categories are defined.
- Validation workflow is established.
- Validation rules are specified.
- Quality rules are documented.
- Rejection criteria are defined.
- Validation report structure is documented.
- Output structure is defined.
- Downstream interface is established.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.9

CRM ENGINE PACKAGING LAYER

10.9.1 Overview

The Packaging Layer is responsible for transforming the Validated CRM Package into a standardized Formatted CRM Package suitable for downstream enterprise processing.

The layer performs packaging and formatting only.

The layer does not generate CRM content, validate CRM information, execute CRM workflows, synchronize with external CRM systems, or execute AI inference.

10.9.2 Objectives

The Packaging Layer shall:

- Standardize validated CRM output.
- Normalize package formatting.
- Assemble standardized execution packages.
- Preserve validated CRM integrity.
- Prepare downstream-compatible output.
- Produce immutable publication-ready packages.

10.9.3 Architectural Position

The Packaging Layer executes immediately after successful completion of the Output Validation Layer.

Execution Flow

...

Input Aggregation Layer

↓

CRM Context Assembly Layer

↓

Relationship Management Layer

↓

Lifecycle Management Layer

↓

Output Validation Layer



Packaging Layer



Metadata Generation Layer

...

The layer shall complete successfully before metadata generation begins.

10.9.4 Responsibilities

The Packaging Layer is responsible for:

- Reading the Validated CRM Package.
- Standardizing approved CRM information.
- Applying package formatting rules.
- Normalizing package structure.
- Packaging approved CRM components.
- Producing standardized downstream output.

10.9.5 Input Source

The Packaging Layer consumes:

- Validated CRM Package

The package shall be validated before packaging begins.

10.9.6 Packaging Categories

The Packaging Layer shall standardize the following categories:

CRM Structure

- CRM information
- Relationship information
- Lifecycle information
- Validation information
- Processing information

Package Formatting

- Section organization
- Package hierarchy
- Whitespace normalization
- Character encoding
- Structural normalization

Schema Formatting

- Package schema
- Section ordering
- Required field ordering
- Output consistency
- Schema organization

Publication Package

- CRM package
- Validation report
- Processing status
- Publication readiness

10.9.7 Packaging Workflow

The Packaging Layer shall execute the following sequence:

1. Load the Validated CRM Package.
2. Verify package integrity.

3. Normalize CRM structure.
4. Normalize package formatting.
5. Normalize schema organization.
6. Assemble standardized package.
7. Verify package completeness.
8. Publish Formatted CRM Package.

Execution order shall remain fixed.

10.9.8 Packaging Rules

The Packaging Layer shall ensure:

- Approved CRM information remains unchanged.
- Section ordering remains standardized.
- Package formatting remains consistent.
- Output complies with package schema.
- Character encoding remains normalized.
- Package structure remains deterministic.

10.9.9 Content Preservation Rules

The Packaging Layer shall preserve:

- Approved CRM information.
- Approved relationship information.
- Approved lifecycle information.
- Approved validation report.
- Approved processing status.

No approved information shall be modified during packaging.

10.9.10 Standardization Rules

Every Formatted CRM Package shall:

- Follow identical structural organization.
- Use standardized section ordering.

- Use normalized spacing.
- Use normalized formatting.
- Maintain schema consistency.
- Maintain deterministic package organization.

10.9.11 Output Structure

The Packaging Layer produces:

Formatted CRM Package

The package shall contain:

- Approved CRM Information
- Approved Relationship Information
- Approved Lifecycle Information
- Validation Report
- Validation Status
- Processing Status

10.9.12 Packaging Validation

Before publication, the formatted package shall be verified for:

- Structural completeness.
- Formatting consistency.
- Package completeness.
- Schema compliance.
- Section ordering.
- Output integrity.

Only correctly formatted packages shall be published.

10.9.13 Failure Conditions

Packaging shall terminate if:

- Validated CRM Package is unavailable.

- Package integrity verification fails.
- Packaging normalization fails.
- Package assembly fails.
- Schema validation fails.
- Output validation fails.

No partially packaged output shall be published.

10.9.14 Downstream Interface

The Formatted CRM Package shall be transferred to the Metadata Generation Layer.

No additional packaging shall occur after publication by the Packaging Layer.

10.9.15 Processing Constraints

The Packaging Layer shall not:

- Generate CRM content.
- Modify approved CRM information.
- Execute CRM workflows.
- Synchronize with external CRM systems.
- Execute AI inference.
- Alter validation results.
- Modify upstream contracts.

10.9.16 Traceability

Every formatted package shall maintain traceability to:

- Validated CRM Package
- Validation Report
- Lifecycle Package
- Relationship Package
- CRM Context Package
- Follow-Up Execution Package
- Execution Metadata

Traceability shall remain preserved throughout downstream processing.

10.9.17 Completion Requirements

This phase is considered complete when:

- Packaging objectives are documented.
- Packaging categories are defined.
- Packaging workflow is established.
- Packaging rules are specified.
- Content preservation rules are documented.
- Standardization rules are defined.
- Output structure is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.10

CRM ENGINE METADATA GENERATION LAYER

10.10.1 Overview

The Metadata Generation Layer is responsible for generating standardized execution metadata for every successfully formatted CRM Package.

The metadata provides execution traceability, processing status, validation status, version information, and audit records required by downstream enterprise systems.

The layer performs metadata generation only.

The layer does not generate CRM content, validate CRM information, modify formatted CRM packages, execute CRM workflows, synchronize with external CRM systems, or execute AI inference.

10.10.2 Objectives

The Metadata Generation Layer shall:

- Generate execution metadata.
- Record processing information.
- Record validation information.
- Preserve execution traceability.
- Produce standardized metadata packages.
- Prepare publication-ready output for downstream enterprise systems.

10.10.3 Architectural Position

The Metadata Generation Layer executes immediately after successful completion of the Packaging Layer.

Execution Flow

...

Input Aggregation Layer

↓

CRM Context Assembly Layer

↓

Relationship Management Layer

↓

Lifecycle Management Layer

↓

Output Validation Layer

↓

Packaging Layer

↓

Metadata Generation Layer

↓

CRM Execution Package

...

The layer shall complete successfully before final package publication.

10.10.4 Responsibilities

The Metadata Generation Layer is responsible for:

- Reading the Formatted CRM Package.
- Generating execution identifiers.
- Recording engine information.
- Recording processing information.
- Recording validation information.
- Recording schema information.
- Producing standardized metadata.

10.10.5 Input Source

The Metadata Generation Layer consumes:

- Formatted CRM Package

The package shall be verified before metadata generation begins.

10.10.6 Metadata Categories

The Metadata Package shall contain the following categories:

Execution Metadata

- Execution Identifier
- Processing Identifier
- Session Identifier
- Correlation Identifier

Engine Metadata

- Engine Name
- Engine Version
- Module Version
- Workflow Version

Contract Metadata

- Contract Version
- Schema Version
- Package Version

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

Validation Metadata

- Validation Identifier
- Validation Status
- Validation Timestamp
- Validation Result

Publication Metadata

- Publication Status
- Publication Timestamp
- Output Package Identifier

10.10.7 Metadata Generation Workflow

The Metadata Generation Layer shall execute the following sequence:

1. Load the Formatted CRM Package.
2. Verify package integrity.
3. Generate execution metadata.
4. Generate engine metadata.
5. Generate contract metadata.
6. Generate processing metadata.
7. Generate validation metadata.
8. Generate publication metadata.
9. Assemble Metadata Package.
10. Validate metadata completeness.
11. Publish CRM Execution Package.

Execution order shall remain fixed.

10.10.8 Metadata Rules

The Metadata Generation Layer shall ensure:

- Metadata accurately represents execution.
- Metadata remains immutable after publication.
- Metadata preserves execution traceability.
- Metadata follows standardized schema.
- Metadata remains version consistent.
- Metadata contains no fabricated values.

10.10.9 Integrity Rules

The Metadata Package shall:

- Preserve execution identifiers.
- Preserve validation identifiers.
- Preserve processing history.
- Preserve version information.
- Preserve package integrity.
- Preserve downstream compatibility.

10.10.10 Output Structure

The Metadata Generation Layer produces:

Metadata Package

The package shall contain:

- Execution Metadata
- Engine Metadata
- Contract Metadata
- Processing Metadata
- Validation Metadata
- Publication Metadata

10.10.11 CRM Execution Package

The Metadata Generation Layer assembles the CRM Execution Package.

The CRM Execution Package shall contain:

- Formatted CRM Package
- Metadata Package
- Validation Report
- Processing Status
- Publication Status

The package shall be immutable after publication.

10.10.12 Metadata Validation

Before publication, the Metadata Package shall be verified for:

- Structural completeness.
- Required metadata fields.
- Schema compliance.
- Version consistency.
- Identifier consistency.
- Processing consistency.
- Publication readiness.

Only validated metadata shall be published.

10.10.13 Failure Conditions

Metadata generation shall terminate if:

- Formatted CRM Package is unavailable.
- Package integrity verification fails.
- Metadata generation fails.
- Identifier generation fails.
- Metadata validation fails.
- Final package assembly fails.

No incomplete metadata package shall be published.

10.10.14 Downstream Interface

The completed CRM Execution Package shall be published for downstream consumption by the Reporting Engine.

The Metadata Generation Layer represents the final processing stage of the CRM Engine.

No additional processing shall occur within this engine after publication.

10.10.15 Processing Constraints

The Metadata Generation Layer shall not:

- Generate CRM content.
- Modify formatted CRM information.
- Modify validation results.
- Execute CRM workflows.
- Synchronize with external CRM systems.
- Execute AI inference.
- Alter upstream contracts.
- Reformat published output.

10.10.16 Traceability

Every metadata record shall maintain traceability to:

- Formatted CRM Package
- Validated CRM Package
- Lifecycle Package
- Relationship Package
- CRM Context Package
- Follow-Up Execution Package
- Execution Metadata

End-to-end traceability shall remain preserved throughout the complete execution lifecycle.

10.10.17 Completion Requirements

This phase is considered complete when:

- Metadata objectives are documented.
- Metadata categories are defined.
- Metadata workflow is established.
- Metadata rules are specified.
- Integrity rules are documented.
- Output structure is defined.
- CRM Execution Package is documented.
- Validation requirements are established.
- Downstream interface is defined.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.11

CRM ENGINE OUTPUT SCHEMA SPECIFICATION

10.11.1 Overview

The Output Schema Specification defines the standardized structure of the CRM Execution Package produced by the CRM Engine.

The schema establishes a deterministic output contract for downstream enterprise systems.

All generated output shall conform to this specification before publication.

10.11.2 Objectives

The Output Schema Specification shall:

- Define standardized output structures.
- Define required output sections.
- Define schema hierarchy.
- Define mandatory fields.
- Define validation requirements.
- Define downstream compatibility requirements.

10.11.3 Schema Design Principles

The output schema shall comply with the following principles:

- Deterministic structure.

- Immutable after publication.
- Version controlled.
- Self-describing.
- Platform independent.
- Strongly structured.
- Enterprise compatible.
- Backward compatible within the same major version.

10.11.4 Top-Level Output Schema

The CRM Execution Package shall contain the following top-level sections:

...

CRM Execution Package

- |— Formatted CRM Package
- |— Metadata Package
- |— Validation Report
- |— Processing Status

...

No additional top-level sections shall be introduced.

10.11.5 Formatted CRM Package Schema

The Formatted CRM Package shall contain:

...

Formatted CRM Package

- |— CRM Information
- |— Relationship Information
- |— Lifecycle Information
- |— Validation Information
- |— Processing Information
- |— Packaging Status

...

All sections are mandatory.

10.11.6 Metadata Package Schema

The Metadata Package shall contain:

...

Metadata Package

- |— Execution Metadata
- |— Engine Metadata
- |— Contract Metadata
- |— Processing Metadata
- |— Validation Metadata
- |— Publication Metadata

...

All metadata categories are mandatory.

10.11.7 Execution Metadata Schema

Execution Metadata shall contain:

...

Execution Metadata

- |— Execution Identifier
- |— Processing Identifier
- |— Session Identifier
- |— Correlation Identifier

...

10.11.8 Engine Metadata Schema

Engine Metadata shall contain:

...

Engine Metadata

- |— Engine Name

- Engine Version
- Module Version
- Workflow Version

...

10.11.9 Contract Metadata Schema

Contract Metadata shall contain:

...

Contract Metadata

- Contract Version
- Schema Version
- Package Version

...

10.11.10 Processing Metadata Schema

Processing Metadata shall contain:

...

Processing Metadata

- Processing Timestamp
- Processing Status
- Execution Duration
- Completion Status

...

10.11.11 Validation Metadata Schema

Validation Metadata shall contain:

...

Validation Metadata

- Validation Identifier

- Validation Timestamp
- Validation Status
- Validation Result

...

10.11.12 Publication Metadata Schema

Publication Metadata shall contain:

...

Publication Metadata

- Publication Status
- Publication Timestamp
- Output Package Identifier

...

10.11.13 Validation Report Schema

The Validation Report shall contain:

...

Validation Report

- Validation Identifier
- Structural Validation
- CRM Validation
- Relationship Validation
- Lifecycle Validation
- Formatting Validation
- Overall Validation Result

...

All validation categories are mandatory.

10.11.14 Processing Status Schema

Processing Status shall contain:

...

Processing Status

- |— Current Status
- |— Completion Status
- |— Publication Status
- |— Processing Result

...

10.11.15 Schema Validation Rules

Every CRM Execution Package shall satisfy the following requirements:

- All mandatory sections shall exist.
- All mandatory fields shall exist.
- Section hierarchy shall remain unchanged.
- Parent-child relationships shall remain valid.
- Version identifiers shall be present.
- Schema integrity shall be maintained.

10.11.16 Schema Integrity Rules

The Output Schema shall:

- Preserve deterministic ordering.
- Preserve immutable identifiers.
- Preserve validation results.
- Preserve metadata integrity.
- Preserve downstream compatibility.
- Preserve structural consistency.

10.11.17 Schema Compatibility

The Output Schema shall support:

- Internal enterprise services.
- Workflow orchestration.

- Reporting Engine integration.
- Analytics systems.
- Audit systems.
- Business intelligence systems.
- Long-term archival.

Compatibility shall remain stable within the same major schema version.

10.11.18 Schema Publication Rules

The CRM Execution Package shall be published only when:

- Schema validation has passed.
- Metadata validation has passed.
- Validation Report is complete.
- Processing Status indicates successful completion.
- All mandatory schema sections are present.

10.11.19 Schema Failure Conditions

Publication shall be rejected if:

- Mandatory sections are missing.
- Schema hierarchy is invalid.
- Required metadata is incomplete.
- Validation Report is incomplete.
- Processing Status is invalid.
- Schema validation fails.

No invalid schema shall be published.

10.11.20 Completion Requirements

This phase is considered complete when:

- Top-level schema is defined.
- All subordinate schemas are documented.
- Mandatory sections are specified.

- Validation rules are documented.
- Integrity rules are established.
- Compatibility requirements are defined.
- Publication rules are documented.
- Failure conditions are specified.

END OF DOCUMENT

AI WEBSITE UPGRADE AGENCY

ENTERPRISE DEVELOPMENT KIT (EDK)

PART 2B-5

PHASE 10.12

CRM ENGINE ENTERPRISE VALIDATION & PHASE COMPLETION

10.12.1 Overview

This phase defines the enterprise validation framework and formal completion requirements for the CRM Engine.

Enterprise validation confirms that every architectural component, processing layer, execution workflow, interface, contract, and output specification defined within Phase 10 has been successfully documented and internally validated.

Successful completion authorizes the phase to enter the LOCKED state.

10.12.2 Validation Objectives

Enterprise validation shall verify:

- Architectural completeness.
- Documentation completeness.
- Workflow completeness.
- Module completeness.
- Interface completeness.
- Contract completeness.

- Schema completeness.
- Enterprise compliance.
- Internal consistency.

10.12.3 Validation Scope

The validation scope includes every document produced within Phase 10.

Validated documents include:

- Phase 10 — CRM Engine Foundation
- Phase 10.1 — CRM Engine Architecture
- Phase 10.2 — CRM Engine Internal Modules
- Phase 10.3 — CRM Engine Data Contracts
- Phase 10.4 — CRM Engine Execution Workflow
- Phase 10.5 — CRM Engine Context Assembly Layer
- Phase 10.6 — CRM Engine Relationship Management Layer
- Phase 10.7 — CRM Engine Lifecycle Management Layer
- Phase 10.8 — CRM Engine Output Validation Layer
- Phase 10.9 — CRM Engine Packaging Layer
- Phase 10.10 — CRM Engine Metadata Generation Layer
- Phase 10.11 — CRM Engine Output Schema Specification

No additional documents are included within this validation scope.

10.12.4 Architectural Validation

Enterprise validation shall confirm that:

- Architectural boundaries are documented.
- Layer responsibilities are documented.
- Module responsibilities are documented.
- Processing order is fixed.
- Upstream dependencies are documented.
- Downstream interfaces are documented.
- Separation of responsibilities is maintained.

10.12.5 Workflow Validation

The execution workflow shall be verified for:

- Sequential execution.
- Fixed execution order.
- Stage completeness.
- Entry conditions.
- Exit conditions.
- Failure handling.
- Publication sequence.

No undocumented execution path shall exist.

10.12.6 Module Validation

Each internal module shall be verified for:

- Defined responsibilities.
- Defined inputs.
- Defined outputs.
- Processing constraints.
- Validation requirements.
- Downstream interfaces.

All modules shall maintain single-responsibility architecture.

10.12.7 Layer Validation

Each processing layer shall be verified for:

- Architectural position.
- Responsibilities.
- Input requirements.
- Output requirements.
- Workflow integration.
- Validation rules.
- Failure conditions.

All documented layers shall integrate without architectural conflict.

10.12.8 Contract Validation

Enterprise validation shall confirm that:

- Input contracts are documented.
- Output contracts are documented.
- Contract hierarchy is complete.
- Contract integrity rules are defined.
- Versioning rules are defined.
- Validation rules are defined.

All contract interfaces shall remain deterministic.

10.12.9 Schema Validation

The CRM Execution Package schema shall be verified for:

- Structural completeness.
- Mandatory sections.
- Schema hierarchy.
- Metadata integration.
- Validation Report integration.
- Processing Status integration.

The published schema shall remain immutable after validation.

10.12.10 Traceability Validation

End-to-end traceability shall be verified across:

- CRM Context Package.
- Relationship Package.
- Lifecycle Package.
- Validated CRM Package.
- Formatted CRM Package.
- Metadata Package.
- CRM Execution Package.

Every downstream artifact shall maintain traceability to validated upstream contracts.

10.12.11 Enterprise Compliance Validation

The CRM Engine shall comply with the following enterprise principles:

- Modular architecture.
- Deterministic processing.
- Immutable contracts.
- Standardized interfaces.
- Enterprise documentation standards.
- Fixed execution workflow.
- Read-only upstream consumption.
- Immutable published outputs.

10.12.12 Documentation Completeness Validation

Enterprise documentation shall verify that every Phase 10 document contains:

- Defined purpose.
- Architectural position.
- Responsibilities.
- Processing workflow.
- Input definition.
- Output definition.
- Validation requirements.
- Failure conditions.
- Completion requirements.

No mandatory documentation section shall be omitted.

10.12.13 Validation Result

Enterprise validation shall confirm that:

- All Phase 10 documentation has been completed.
- All architectural components have been documented.
- All processing layers have been documented.

- All internal modules have been documented.
- All execution workflows have been documented.
- All data contracts have been documented.
- All output schemas have been documented.
- All enterprise validation requirements have been satisfied.

10.12.14 Phase Completion Criteria

Phase 10 shall be considered complete when:

- Enterprise validation is successful.
- Documentation completeness is verified.
- Architectural validation is successful.
- Workflow validation is successful.
- Module validation is successful.
- Contract validation is successful.
- Schema validation is successful.
- Traceability validation is successful.

Only after all completion criteria have passed shall Phase 10 be considered complete.

10.12.15 Phase Status

Phase Name

CRM Engine

Phase Identifier

PART 2B-5 — Phase 10

Phase State

COMPLETED

Validation Status

PASSED

Architecture Status

VERIFIED

Documentation Status

COMPLETE

Workflow Status

VERIFIED

Contract Status

VERIFIED

Schema Status

VERIFIED

Publication Status

APPROVED

10.12.16 Phase Lock

Following successful enterprise validation:

- Phase 10 is declared COMPLETE.
- Phase 10 enters the LOCKED state.
- Architectural changes are prohibited.
- Workflow modifications are prohibited.
- Module modifications are prohibited.
- Contract modifications are prohibited.
- Schema modifications are prohibited.
- Documentation modifications are prohibited unless a formal change request is approved.

10.12.17 Completion Summary

The CRM Engine has been fully specified as an enterprise-grade subsystem.

The completed documentation defines:

- Enterprise architecture.
- Internal modules.
- Data contracts.
- Execution workflow.
- CRM Context Assembly Layer.
- Relationship Management Layer.
- Lifecycle Management Layer.
- Output Validation Layer.
- Packaging Layer.
- Metadata Generation Layer.
- Output Schema Specification.
- Enterprise validation framework.

This concludes the complete documentation scope for PART 2B-5 — Phase 10.

END OF DOCUMENT

PART 2B-6 — Phase 11

Reporting Engine Foundation

11.1 Purpose

The Reporting Engine is responsible for transforming structured outputs generated by all upstream engines into standardized business reports for internal users, administrators, operators, and enterprise stakeholders.

The Reporting Engine does not perform data collection, business analysis, AI generation, validation, or CRM management. Its responsibility begins only after upstream engines have completed execution and produced validated output packages.

The Reporting Engine provides a unified reporting layer capable of producing consistent, traceable, and auditable reports across the entire AI Website Upgrade Agency platform.

11.2 Objectives

The Reporting Engine shall provide:

- Enterprise reporting foundation
- Standardized report generation

- Cross-engine report aggregation
- Business intelligence presentation
- Operational reporting
- Executive reporting
- Historical reporting support
- Audit-ready reporting
- Consistent report formatting
- Enterprise metadata generation

11.3 Scope

The Reporting Engine operates after completion of:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine

The Reporting Engine consumes validated outputs from these engines and produces enterprise-grade reports.

11.4 Responsibilities

The Reporting Engine is responsible for:

- Reading validated engine outputs
- Aggregating report data
- Organizing business metrics
- Creating standardized report structures
- Producing executive summaries
- Producing operational reports
- Producing campaign reports
- Producing pipeline reports
- Producing audit reports

- Packaging report artifacts

11.5 Out of Scope

The Reporting Engine shall not:

- Execute searches
- Audit websites
- Detect niches
- Score leads
- Extract contacts
- Validate emails
- Generate prototypes
- Generate AI emails
- Execute follow-up workflows
- Manage CRM lifecycle
- Modify upstream data
- Rewrite engine outputs
- Perform AI inference
- Perform business decisions

11.6 Position within System Architecture

The Reporting Engine is positioned after all operational engines.

Execution Flow

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine

↓

AI Personalized Email Engine

↓

Follow-Up Engine

↓

CRM Engine

↓

Reporting Engine

11.7 Reporting Philosophy

The Reporting Engine follows a read-only reporting model.

It never changes source information.

It never recalculates upstream results.

It only organizes validated information into standardized reporting structures.

All business facts originate from upstream engines.

11.8 Enterprise Design Principles

The Reporting Engine follows the following principles:

- Immutable reporting
- Deterministic output
- Traceable report lineage
- Consistent formatting
- Modular architecture
- Enterprise scalability
- Metadata completeness
- Audit compatibility
- Version-controlled schemas
- Non-destructive processing

11.9 High-Level Responsibilities

The Reporting Engine provides the following functional layers:

1. Report Request Processing
2. Data Aggregation
3. Report Assembly
4. Metrics Compilation
5. Executive Summary Generation
6. Operational Report Generation
7. Report Validation
8. Report Packaging
9. Metadata Generation
10. Report Export Preparation

11.10 Input Sources

The Reporting Engine consumes validated outputs from:

Search Engine

Website Audit Engine

Niche Detection Engine

Lead Scoring Engine

Contact Extraction Engine

Email Validation Engine

Prototype Intelligence Engine

AI Personalized Email Engine

Follow-Up Engine

CRM Engine

No direct communication with external systems occurs inside the Reporting Engine.

11.11 Output Consumers

Reporting outputs may be consumed by:

Administrative Dashboard

Operations Dashboard

Management Dashboard

Campaign Dashboard

Business Analytics Layer

Export Services

Audit Systems

Enterprise Archive

Future Analytics Modules

11.12 Core Characteristics

The Reporting Engine shall be:

Deterministic

Stateless

Read-only

Versioned

Schema-driven

Extensible

Auditable

Traceable

Scalable

Enterprise compliant

11.13 Enterprise Reporting Categories

The engine supports generation of standardized report categories including:

Executive Reports

Operational Reports

Campaign Reports

Pipeline Reports

Lead Reports

Website Reports

Audit Reports

CRM Reports

Communication Reports

System Reports

Historical Reports

Compliance Reports

11.14 Report Lifecycle

Every report follows the same lifecycle:

Input Collection



Data Aggregation



Report Assembly



Validation



Packaging



Metadata Generation

↓

Export Preparation

↓

Final Report

11.15 Architectural Constraints

The Reporting Engine shall never:

Modify source records

Overwrite engine outputs

Generate business logic

Change CRM state

Trigger workflows

Send communications

Perform AI generation

Replace operational engines

11.16 Foundation Summary

The Reporting Engine establishes the enterprise reporting layer for the AI Website Upgrade Agency platform.

It provides a standardized mechanism for aggregating validated outputs produced by upstream engines into consistent, auditable, and enterprise-ready reports while preserving complete traceability and maintaining strict separation from operational processing.

DOCUMENT STATUS

Phase 11 — Reporting Engine Foundation

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.1 — Reporting Engine Architecture

END OF DOCUMENT

PART 2B-6 — Phase 11.1

Reporting Engine Architecture

11.1.1 Overview

The Reporting Engine Architecture defines the structural organization of the enterprise reporting subsystem.

Its purpose is to transform validated outputs generated by upstream engines into standardized reporting artifacts through a deterministic, modular, and read-only processing pipeline.

The architecture separates report generation responsibilities into independent layers to ensure maintainability, scalability, traceability, and audit compliance.

11.1.2 Architectural Objectives

The Reporting Engine architecture is designed to achieve the following objectives:

- Modular report generation
- Separation of reporting responsibilities
- Enterprise scalability
- Deterministic processing
- Read-only data handling
- Standardized report construction
- Schema-driven execution
- Complete report traceability
- Version-controlled reporting
- Audit-ready architecture

11.1.3 Architectural Position

The Reporting Engine is the terminal operational layer within the Enterprise Development Kit workflow.

All upstream engines complete execution before report generation begins.

Execution Order

Search Engine



Website Audit Engine



Niche Detection Engine



Lead Scoring Engine



Contact Extraction Engine



Email Validation Engine



Prototype Intelligence Engine



AI Personalized Email Engine



Follow-Up Engine



CRM Engine



Reporting Engine

11.1.4 Architectural Layers

The Reporting Engine architecture is composed of the following logical layers:

Layer 1

Report Request Layer

Responsible for accepting standardized report generation requests.

Layer 2

Data Aggregation Layer

Collects validated outputs from all upstream engines.

Layer 3

Normalization Layer

Transforms aggregated information into standardized reporting models.

Layer 4

Metrics Compilation Layer

Compiles operational, business, campaign, and executive metrics.

Layer 5

Report Assembly Layer

Constructs complete enterprise report structures.

Layer 6

Validation Layer

Verifies structural integrity and reporting completeness.

Layer 7

Packaging Layer

Packages reports into standardized internal artifacts.

Layer 8

Metadata Layer

Generates reporting metadata and lineage information.

Layer 9

Export Preparation Layer

Prepares validated reports for downstream export services.

11.1.5 Architectural Principles

The architecture follows these principles:

- Single responsibility
- Read-only execution
- Immutable processing
- Layer isolation
- Deterministic execution
- Standardized interfaces
- Version compatibility
- Schema enforcement
- Complete traceability
- Enterprise consistency

11.1.6 Processing Direction

Information flows through the Reporting Engine in a single direction.

Report Request



Data Aggregation



Normalization



Metrics Compilation



Report Assembly



Validation



Packaging



Metadata Generation



Export Preparation



Completed Report

No reverse processing is permitted.

11.1.7 Data Ownership

The Reporting Engine owns only reporting artifacts.

Ownership excludes:

- Source datasets
- Operational records
- CRM records
- Search results
- Website audits
- AI outputs
- Email workflows
- Follow-up workflows

These remain under the ownership of their originating engines.

11.1.8 Module Independence

Each architectural layer operates independently.

A layer:

- Receives validated input
- Performs its assigned responsibility
- Produces standardized output
- Does not modify upstream data
- Does not depend on internal implementation details of other layers

11.1.9 Fault Isolation

Failures within one architectural layer shall not modify upstream engine outputs.

Fault handling is limited to:

- Processing termination
- Validation failure reporting
- Error propagation
- Diagnostic metadata generation

No recovery logic modifies source information.

11.1.10 Scalability Model

The architecture supports independent scaling of:

- Report generation
- Aggregation processing
- Metrics compilation
- Validation
- Packaging
- Metadata generation
- Export preparation

Each layer may scale independently without altering reporting behavior.

11.1.11 Interface Boundaries

The Reporting Engine exposes only standardized internal interfaces.

Interfaces exist between:

- Request Layer and Aggregation Layer
- Aggregation Layer and Normalization Layer
- Normalization Layer and Metrics Layer
- Metrics Layer and Assembly Layer
- Assembly Layer and Validation Layer
- Validation Layer and Packaging Layer
- Packaging Layer and Metadata Layer
- Metadata Layer and Export Preparation Layer

No layer bypasses another layer.

11.1.12 Architectural Constraints

The architecture shall never:

- Execute business logic
- Modify upstream outputs
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Initiate communications
- Replace operational engines
- Store operational state
- Recalculate upstream metrics

11.1.13 Traceability

Every generated report maintains complete traceability through:

- Source engine references
- Processing lineage
- Schema version
- Report version
- Metadata identifiers
- Validation status
- Packaging information
- Export preparation status

11.1.14 Architectural Summary

The Reporting Engine Architecture establishes a layered, deterministic, and read-only reporting subsystem that converts validated enterprise outputs into standardized reporting artifacts while preserving complete traceability, schema consistency, and strict separation from operational processing.

DOCUMENT STATUS

Phase 11.1 — Reporting Engine Architecture

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.2 — Reporting Engine Internal Modules

END OF DOCUMENT

PART 2B-6 — Phase 11.2

Reporting Engine Internal Modules

11.2.1 Overview

The Reporting Engine is composed of independent internal modules responsible for transforming validated enterprise data into standardized reporting artifacts.

Each module performs a single well-defined responsibility.

Modules communicate only through standardized internal contracts.

No module directly modifies upstream engine outputs.

11.2.2 Internal Module Hierarchy

The Reporting Engine consists of the following internal modules:

1. Report Request Manager
2. Data Aggregation Manager
3. Data Normalization Manager
4. Metrics Compilation Manager
5. Executive Summary Manager
6. Report Assembly Manager
7. Report Validation Manager
8. Report Packaging Manager
9. Report Metadata Manager
10. Export Preparation Manager

11.2.3 Report Request Manager

Purpose

Receives standardized report generation requests from internal platform components.

Responsibilities

- Receive report requests
- Validate request structure
- Verify supported report type
- Initialize reporting workflow
- Forward validated requests to the aggregation module

Inputs

- Report generation request

Outputs

- Validated reporting request

11.2.4 Data Aggregation Manager

Purpose

Collects validated information from upstream engine outputs.

Responsibilities

- Read Search Engine output
- Read Website Audit output
- Read Niche Detection output
- Read Lead Scoring output
- Read Contact Extraction output
- Read Email Validation output
- Read Prototype Intelligence output
- Read AI Personalized Email output
- Read Follow-Up output
- Read CRM output
- Aggregate reporting datasets

Inputs

- Validated engine outputs

Outputs

- Aggregated reporting dataset

11.2.5 Data Normalization Manager

Purpose

Converts aggregated datasets into standardized reporting models.

Responsibilities

- Normalize field names
- Normalize value structures
- Standardize reporting objects

- Remove structural inconsistencies
- Prepare reporting models

Inputs

- Aggregated dataset

Outputs

- Normalized reporting model

11.2.6 Metrics Compilation Manager

Purpose

Compiles standardized business metrics used throughout enterprise reports.

Responsibilities

- Compile operational metrics
- Compile campaign metrics
- Compile pipeline metrics
- Compile lead metrics
- Compile communication metrics
- Compile CRM metrics
- Compile audit metrics
- Compile reporting statistics

Inputs

- Normalized reporting model

Outputs

- Enterprise metrics model

11.2.7 Executive Summary Manager

Purpose

Creates standardized executive reporting summaries.

Responsibilities

- Organize executive highlights
- Summarize enterprise metrics
- Prepare management overview
- Consolidate reporting insights
- Generate executive reporting section

Inputs

- Enterprise metrics model

Outputs

- Executive summary model

11.2.8 Report Assembly Manager

Purpose

Constructs the complete enterprise report.

Responsibilities

- Assemble report sections
- Merge executive summary
- Merge business metrics
- Merge operational information
- Build standardized report hierarchy
- Produce complete report object

Inputs

- Executive summary
- Metrics model
- Normalized reporting model

Outputs

- Complete enterprise report

11.2.9 Report Validation Manager

Purpose

Ensures the generated report satisfies enterprise reporting standards.

Responsibilities

- Validate structure
- Validate completeness
- Validate schema
- Validate mandatory sections
- Validate metadata references
- Verify report consistency

Inputs

- Enterprise report

Outputs

- Validated enterprise report

11.2.10 Report Packaging Manager

Purpose

Packages validated reports into standardized internal artifacts.

Responsibilities

- Organize report package
- Attach supporting sections
- Prepare internal package structure
- Finalize reporting artifact

Inputs

- Validated report

Outputs

- Packaged report artifact

11.2.11 Report Metadata Manager

Purpose

Generates standardized metadata for every report.

Responsibilities

- Generate report identifier
- Generate version information
- Generate processing metadata
- Generate lineage metadata
- Generate audit metadata
- Generate timestamps
- Generate reporting descriptors

Inputs

- Packaged report

Outputs

- Report metadata

11.2.12 Export Preparation Manager

Purpose

Prepares completed reports for downstream export services.

Responsibilities

- Verify export readiness
- Attach metadata
- Prepare standardized export object

- Finalize reporting payload
- Publish export-ready package

Inputs

- Packaged report
- Report metadata

Outputs

- Export-ready report package

11.2.13 Module Interaction Flow

Report Request Manager



Data Aggregation Manager



Data Normalization Manager



Metrics Compilation Manager



Executive Summary Manager



Report Assembly Manager



Report Validation Manager



Report Packaging Manager

↓

Report Metadata Manager

↓

Export Preparation Manager

11.2.14 Module Design Principles

Every internal module shall:

- Perform one responsibility only
- Be independently testable
- Be deterministic
- Be read-only
- Produce standardized outputs
- Consume standardized inputs
- Maintain schema compliance
- Preserve traceability
- Avoid side effects
- Support enterprise scalability

11.2.15 Dependency Rules

Internal modules shall never:

- Skip execution order
- Access downstream modules directly
- Modify upstream outputs
- Execute operational workflows
- Trigger AI generation
- Execute CRM operations
- Perform business decision making
- Alter source records

11.2.16 Internal Module Summary

The Reporting Engine Internal Modules establish a modular processing architecture in which each component performs a distinct reporting responsibility. Together they transform validated enterprise outputs into standardized, validated, metadata-enriched reporting artifacts while maintaining deterministic execution, complete traceability, and strict separation from operational processing.

DOCUMENT STATUS

Phase 11.2 — Reporting Engine Internal Modules

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.3 — Reporting Engine Data Contracts

END OF DOCUMENT

PART 2B-6 — Phase 11.3

Reporting Engine Data Contracts

11.3.1 Purpose

The Reporting Engine Data Contracts define the standardized data exchange specifications between the Reporting Engine and its internal modules.

These contracts ensure deterministic processing, schema consistency, interoperability, and complete traceability throughout the reporting lifecycle.

All internal modules shall exchange data exclusively through the contracts defined in this document.

11.3.2 Objectives

The Data Contracts are designed to achieve the following objectives:

- Standardized data exchange
- Schema consistency
- Version compatibility
- Module interoperability
- Deterministic processing
- Complete traceability
- Validation readiness
- Enterprise scalability
- Audit compliance
- Non-destructive data handling

11.3.3 Contract Principles

Every Reporting Engine data contract shall adhere to the following principles:

- Immutable after creation
- Strongly structured
- Version-controlled
- Schema-driven
- Self-describing
- Deterministic
- Read-only
- Traceable
- Extensible
- Independently verifiable

11.3.4 Contract Lifecycle

Every data contract progresses through the following lifecycle:

Contract Creation

↓

Schema Validation

↓

Module Processing

↓

Output Validation

↓

Metadata Attachment

↓

Contract Completion

No contract may bypass any lifecycle stage.

11.3.5 Primary Contract Types

The Reporting Engine defines the following primary contract types:

- Report Request Contract
- Aggregated Data Contract
- Normalized Data Contract
- Metrics Contract
- Executive Summary Contract
- Report Assembly Contract
- Validation Contract
- Packaging Contract
- Metadata Contract
- Export Preparation Contract

11.3.6 Report Request Contract

Purpose

Represents a standardized request for report generation.

Producer

Report Request Manager

Consumer

Data Aggregation Manager

Contents

- Request identifier
- Requested report category
- Report parameters
- Processing options
- Schema version
- Request timestamp

11.3.7 Aggregated Data Contract

Purpose

Represents consolidated information collected from upstream engine outputs.

Producer

Data Aggregation Manager

Consumer

Data Normalization Manager

Contents

- Aggregated datasets
- Source engine references
- Dataset identifiers
- Collection metadata
- Processing references

11.3.8 Normalized Data Contract

Purpose

Represents standardized reporting models derived from aggregated data.

Producer

Data Normalization Manager

Consumer

Metrics Compilation Manager

Contents

- Normalized reporting objects
- Standardized field mappings
- Canonical data structures
- Normalization metadata
- Schema references

11.3.9 Metrics Contract

Purpose

Represents compiled business and operational metrics.

Producer

Metrics Compilation Manager

Consumer

Executive Summary Manager

Contents

- Business metrics
- Operational metrics
- Campaign metrics
- Pipeline metrics
- CRM metrics
- Reporting statistics

11.3.10 Executive Summary Contract

Purpose

Represents the executive reporting overview.

Producer

Executive Summary Manager

Consumer

Report Assembly Manager

Contents

- Executive highlights
- Business overview
- Operational summary
- Reporting synopsis
- Summary metadata

11.3.11 Report Assembly Contract

Purpose

Represents the fully assembled enterprise report.

Producer

Report Assembly Manager

Consumer

Report Validation Manager

Contents

- Report sections
- Executive summary
- Metrics
- Supporting information

- Structural hierarchy
- Assembly references

11.3.12 Validation Contract

Purpose

Represents the validation status of the assembled report.

Producer

Report Validation Manager

Consumer

Report Packaging Manager

Contents

- Validation status
- Validation results
- Schema verification
- Structural verification
- Completeness indicators
- Validation metadata

11.3.13 Packaging Contract

Purpose

Represents the packaged reporting artifact.

Producer

Report Packaging Manager

Consumer

Report Metadata Manager

Contents

- Packaged report
- Packaging descriptors
- Artifact references
- Internal package identifiers
- Packaging metadata

11.3.14 Metadata Contract

Purpose

Represents enterprise reporting metadata.

Producer

Report Metadata Manager

Consumer

Export Preparation Manager

Contents

- Report identifier
- Metadata version
- Processing lineage
- Source references
- Audit information
- Timestamp information

11.3.15 Export Preparation Contract

Purpose

Represents the finalized report package prepared for downstream export services.

Producer

Export Preparation Manager

Consumer

Downstream Export Services

Contents

- Export-ready report
- Attached metadata
- Packaging information
- Export descriptors
- Export readiness status

11.3.16 Contract Validation Rules

Every contract shall satisfy the following requirements before being accepted by the receiving module:

- Valid schema version
- Mandatory fields present
- Structural integrity verified
- Required metadata attached
- Traceability information available
- Producer identification present
- Consumer compatibility verified
- Contract completeness confirmed

Contracts failing validation shall not proceed to subsequent modules.

11.3.17 Version Management

Each contract shall include version information sufficient to ensure compatibility across Reporting Engine releases.

Version information shall remain immutable throughout the contract lifecycle.

11.3.18 Traceability Requirements

Every contract shall preserve complete processing lineage, including:

- Contract identifier
- Producing module
- Consuming module
- Source references
- Schema version
- Processing sequence
- Metadata references
- Validation status

11.3.19 Contract Constraints

Reporting Engine data contracts shall never:

- Modify source information
- Execute business logic
- Trigger operational workflows
- Perform AI inference
- Alter upstream outputs
- Store mutable operational state
- Omit mandatory metadata
- Bypass validation procedures

11.3.20 Data Contract Summary

The Reporting Engine Data Contracts establish a standardized, immutable, and schema-driven communication framework between internal reporting modules. These contracts ensure deterministic execution, interoperability, complete traceability, audit readiness, and enterprise-wide consistency while preserving the integrity of validated upstream outputs.

DOCUMENT STATUS

Phase 11.3 — Reporting Engine Data Contracts

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.4 — Reporting Engine Execution Workflow

END OF DOCUMENT

PART 2B-6 — Phase 11.4

Reporting Engine Execution Workflow

11.4.1 Purpose

The Reporting Engine Execution Workflow defines the deterministic processing sequence used to generate standardized enterprise reports from validated upstream engine outputs.

The workflow establishes the execution order, processing boundaries, module interactions, validation checkpoints, and final report preparation process.

Every Reporting Engine execution shall follow the workflow defined in this document without deviation.

11.4.2 Workflow Objectives

The execution workflow is designed to provide:

- Deterministic report generation
- Standardized processing sequence
- Complete data traceability
- Read-only execution
- Enterprise consistency
- Validation-driven processing
- Modular execution
- Repeatable report generation
- Audit-ready processing
- Export-ready report preparation

11.4.3 Workflow Preconditions

Execution may begin only when all of the following conditions are satisfied:

- Upstream engine execution has completed
- Required engine outputs have been validated
- Report generation request has been accepted
- Required schemas are available
- Internal reporting modules are available
- Processing metadata can be generated

If any precondition fails, workflow execution shall terminate before report processing begins.

11.4.4 High-Level Execution Flow

The Reporting Engine executes the following sequence:

Report Request



Request Validation



Data Aggregation



Data Normalization



Metrics Compilation



Executive Summary Generation



Report Assembly



Report Validation



Report Packaging



Metadata Generation



Export Preparation



Completed Report

11.4.5 Stage 1 — Report Request Processing

Purpose

Initialize enterprise report generation.

Activities

- Receive report request
- Verify request structure
- Verify supported report category
- Assign request identifier
- Initialize workflow context

Output

Validated Report Request

11.4.6 Stage 2 — Data Aggregation

Purpose

Collect validated information from all supported upstream engines.

Activities

- Read Search Engine output
- Read Website Audit output
- Read Niche Detection output
- Read Lead Scoring output
- Read Contact Extraction output
- Read Email Validation output
- Read Prototype Intelligence output
- Read AI Personalized Email output
- Read Follow-Up output
- Read CRM output
- Consolidate reporting datasets

Output

Aggregated Reporting Dataset

11.4.7 Stage 3 — Data Normalization

Purpose

Transform aggregated information into standardized reporting structures.

Activities

- Normalize reporting fields
- Standardize object hierarchy
- Normalize value formats
- Prepare canonical reporting models
- Verify structural consistency

Output

Normalized Reporting Model

11.4.8 Stage 4 — Metrics Compilation

Purpose

Compile standardized enterprise metrics.

Activities

- Compile operational metrics
- Compile campaign metrics
- Compile lead metrics
- Compile communication metrics
- Compile CRM metrics
- Compile reporting statistics
- Organize enterprise measurements

Output

Enterprise Metrics Model

11.4.9 Stage 5 — Executive Summary Generation

Purpose

Generate standardized executive reporting information.

Activities

- Prepare executive overview
- Summarize key metrics
- Consolidate business highlights
- Organize management information
- Produce executive reporting section

Output

Executive Summary

11.4.10 Stage 6 — Report Assembly

Purpose

Construct the complete enterprise report.

Activities

- Assemble report sections
- Integrate executive summary
- Integrate enterprise metrics
- Organize supporting information
- Build standardized report hierarchy

Output

Enterprise Report

11.4.11 Stage 7 — Report Validation

Purpose

Verify report integrity and enterprise compliance.

Activities

- Validate report schema
- Validate report structure
- Verify mandatory sections
- Verify completeness
- Validate metadata references
- Confirm report consistency

Output

Validated Enterprise Report

11.4.12 Stage 8 — Report Packaging

Purpose

Package validated reports into standardized reporting artifacts.

Activities

- Build report package
- Organize internal artifacts
- Attach supporting information
- Finalize package structure

Output

Packaged Report

11.4.13 Stage 9 — Metadata Generation

Purpose

Generate enterprise reporting metadata.

Activities

- Generate report identifier
- Generate report version
- Generate lineage metadata
- Generate processing metadata
- Generate audit metadata
- Generate timestamps

Output

Report Metadata

11.4.14 Stage 10 — Export Preparation

Purpose

Prepare completed reports for downstream export services.

Activities

- Attach metadata
- Verify export readiness
- Prepare standardized export package

- Finalize reporting payload

Output

Export-Ready Report Package

11.4.15 Workflow Validation Checkpoints

Validation shall occur at the following checkpoints:

Checkpoint 1

Report Request Validation

Checkpoint 2

Aggregated Dataset Validation

Checkpoint 3

Normalized Data Validation

Checkpoint 4

Metrics Validation

Checkpoint 5

Executive Summary Validation

Checkpoint 6

Report Structure Validation

Checkpoint 7

Packaging Validation

Checkpoint 8

Metadata Validation

Checkpoint 9

Export Readiness Validation

Execution shall not continue if any checkpoint fails.

11.4.16 Workflow Failure Handling

If execution fails during any processing stage:

- Workflow execution shall terminate
- Source information shall remain unchanged
- Completed upstream outputs shall remain immutable
- Failure information shall be recorded
- Processing diagnostics shall be generated
- Validation status shall be updated

No automatic recovery shall modify reporting data.

11.4.17 Workflow Characteristics

The Reporting Engine execution workflow is:

- Deterministic
- Sequential
- Read-only
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-driven
- Enterprise compliant

11.4.18 Execution Constraints

The execution workflow shall never:

- Modify upstream engine outputs
- Execute operational business logic
- Perform AI inference
- Trigger CRM operations
- Initiate communications
- Execute follow-up workflows
- Recalculate source metrics
- Bypass validation checkpoints
- Skip execution stages

11.4.19 Workflow Completion Criteria

Workflow execution is considered complete only when:

- Every processing stage has completed successfully
- All validation checkpoints have passed
- Report packaging has completed
- Metadata has been generated
- Export preparation has completed
- Final reporting artifacts have been produced

11.4.20 Workflow Summary

The Reporting Engine Execution Workflow defines a deterministic, sequential, and validation-driven reporting process that transforms validated upstream engine outputs into standardized enterprise reporting artifacts. The workflow preserves complete traceability, enforces immutable processing, and ensures every generated report satisfies enterprise reporting standards before export preparation.

DOCUMENT STATUS

Phase 11.4 — Reporting Engine Execution Workflow

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.5 — Reporting Engine Data Aggregation Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.5

Reporting Engine Data Aggregation Layer

11.5.1 Purpose

The Data Aggregation Layer is responsible for collecting validated outputs produced by all upstream engines and consolidating them into a unified reporting dataset.

This layer serves as the single point of data acquisition for the Reporting Engine.

It performs only data collection and organization.

It does not modify, recalculate, enrich, or interpret source information.

11.5.2 Objectives

The Data Aggregation Layer is designed to achieve the following objectives:

- Centralized data collection
- Standardized dataset aggregation
- Source integrity preservation
- Deterministic aggregation
- Schema consistency
- Complete source traceability
- Read-only processing
- Enterprise scalability
- Validation readiness
- Audit compatibility

11.5.3 Layer Position

The Data Aggregation Layer is executed immediately after successful report request validation.

Execution Flow

Report Request Validation



Data Aggregation Layer



Data Normalization Layer

11.5.4 Responsibilities

The Data Aggregation Layer is responsible for:

- Reading validated engine outputs
- Verifying source availability
- Collecting reporting datasets
- Organizing source information
- Maintaining source references
- Preserving engine ownership
- Consolidating reporting inputs
- Preparing aggregated datasets
- Maintaining deterministic ordering
- Producing aggregation metadata

11.5.5 Source Systems

The Data Aggregation Layer collects information from the following validated engine outputs:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine

Only validated outputs are eligible for aggregation.

11.5.6 Aggregation Workflow

The Data Aggregation Layer executes the following sequence:

Source Discovery

↓

Source Verification

↓

Schema Verification

↓

Data Collection

↓

Dataset Consolidation

↓

Source Reference Mapping

↓

Aggregation Validation

↓

Aggregated Dataset Generation

11.5.7 Source Discovery

Purpose

Identify all upstream engine outputs required for the requested report.

Activities

- Determine required source engines
- Locate validated outputs
- Identify report dependencies
- Register available datasets

Output

Source Inventory

11.5.8 Source Verification

Purpose

Verify that every required source dataset is available and valid.

Activities

- Verify dataset existence
- Verify validation status
- Verify version compatibility
- Verify accessibility
- Verify ownership references

Output

Verified Source Inventory

11.5.9 Schema Verification

Purpose

Ensure all source datasets conform to supported schema versions.

Activities

- Validate schema identifiers
- Verify contract compatibility
- Confirm structural integrity
- Reject unsupported schemas

Output

Schema-Verified Sources

11.5.10 Data Collection

Purpose

Read validated source information without modification.

Activities

- Read source records
- Preserve source ordering
- Preserve identifiers
- Preserve structural hierarchy
- Preserve validation information

Output

Collected Source Data

11.5.11 Dataset Consolidation

Purpose

Combine collected datasets into a unified reporting collection.

Activities

- Organize datasets
- Preserve source boundaries
- Consolidate reporting inputs
- Maintain deterministic ordering
- Retain dataset lineage

Output

Aggregated Reporting Dataset

11.5.12 Source Reference Mapping

Purpose

Maintain complete traceability between aggregated data and originating engines.

Activities

- Associate source engine identifiers
- Associate dataset identifiers
- Record ownership references
- Record processing lineage
- Preserve origin metadata

Output

Source Reference Map

11.5.13 Aggregation Validation

Purpose

Verify the integrity of the aggregated reporting dataset.

Activities

- Validate dataset completeness
- Validate required sources
- Verify structural consistency
- Verify reference integrity
- Confirm aggregation success

Output

Validated Aggregated Dataset

11.5.14 Aggregated Dataset

The completed aggregated dataset shall contain:

- Source engine references
- Aggregated reporting records
- Dataset identifiers
- Validation references
- Schema references
- Processing metadata
- Collection metadata
- Lineage information

No source information shall be modified during aggregation.

11.5.15 Traceability Requirements

The Data Aggregation Layer shall preserve:

- Source engine identity
- Dataset identity
- Processing sequence
- Validation references
- Schema versions
- Collection timestamps
- Ownership information
- Aggregation identifiers

Traceability shall remain intact throughout subsequent reporting stages.

11.5.16 Validation Rules

The aggregated dataset shall satisfy all of the following conditions:

- All required sources collected
- All datasets validated
- Supported schema versions confirmed

- Mandatory metadata available
- Source references preserved
- Processing sequence recorded
- Structural consistency verified

Datasets failing validation shall not proceed to the Data Normalization Layer.

11.5.17 Failure Handling

If aggregation fails:

- Processing shall terminate
- Source datasets shall remain unchanged
- Partial aggregation results shall not be published
- Failure metadata shall be generated
- Diagnostic information shall be recorded
- Validation status shall indicate failure

No automatic correction of source information shall occur.

11.5.18 Layer Characteristics

The Data Aggregation Layer is:

- Read-only
- Deterministic
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-aware
- Enterprise compliant
- Non-destructive

11.5.19 Architectural Constraints

The Data Aggregation Layer shall never:

- Modify source datasets
- Recalculate business metrics
- Execute AI inference
- Trigger workflows
- Perform CRM operations
- Generate reports
- Normalize data
- Package reporting artifacts
- Alter validation results

11.5.20 Layer Summary

The Data Aggregation Layer establishes the enterprise data acquisition foundation of the Reporting Engine by collecting validated outputs from all upstream engines into a unified reporting dataset. It preserves complete source integrity, deterministic ordering, schema consistency, and end-to-end traceability while ensuring that all subsequent reporting stages operate on validated, immutable information.

DOCUMENT STATUS

Phase 11.5 — Reporting Engine Data Aggregation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.6 — Reporting Engine Metrics Compilation Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.6

Reporting Engine Metrics Compilation Layer

11.6.1 Purpose

The Metrics Compilation Layer is responsible for compiling standardized enterprise metrics from normalized reporting data.

This layer transforms normalized reporting models into structured business, operational, campaign, and system metrics that are consumed by downstream reporting components.

The Metrics Compilation Layer performs metric organization only.

It does not modify source information, execute business logic, or alter upstream engine outputs.

11.6.2 Objectives

The Metrics Compilation Layer is designed to achieve the following objectives:

- Standardized metric compilation
- Consistent enterprise measurements
- Deterministic processing
- Read-only execution
- Schema-driven metric generation
- Business metric organization
- Operational metric organization
- Reporting consistency
- Complete traceability
- Audit readiness

11.6.3 Layer Position

The Metrics Compilation Layer executes after successful completion of the Data Normalization Layer.

Execution Flow

Data Normalization Layer

↓

Metrics Compilation Layer

↓

Executive Summary Generation Layer

11.6.4 Responsibilities

The Metrics Compilation Layer is responsible for:

- Reading normalized reporting models
- Identifying reportable metrics
- Compiling standardized metric collections
- Organizing metrics by reporting category
- Maintaining metric traceability
- Preserving source references
- Preparing enterprise metric models
- Generating metric metadata
- Validating compiled metrics
- Producing a standardized metrics package

11.6.5 Input Sources

The Metrics Compilation Layer consumes:

- Normalized reporting models
- Source reference mappings
- Validation metadata
- Schema metadata
- Processing metadata

Only validated normalized data shall be processed.

11.6.6 Metric Categories

The Metrics Compilation Layer organizes metrics into the following categories:

- Executive Metrics
- Operational Metrics
- Campaign Metrics
- Pipeline Metrics
- Lead Metrics

- Contact Metrics
- Email Metrics
- CRM Metrics
- Website Metrics
- Audit Metrics
- Communication Metrics
- System Metrics

Each metric category remains logically independent while contributing to the unified enterprise metrics model.

11.6.7 Compilation Workflow

The Metrics Compilation Layer executes the following sequence:

Metric Source Identification



Metric Extraction



Metric Classification



Metric Organization



Metric Consolidation



Metric Validation



Metric Metadata Generation



Enterprise Metrics Package

11.6.8 Metric Source Identification

Purpose

Identify all normalized reporting elements eligible for metric compilation.

Activities

- Identify reporting objects
- Identify measurable values
- Verify reporting eligibility
- Associate source references
- Register metric candidates

Output

Metric Source Inventory

11.6.9 Metric Extraction

Purpose

Extract standardized metric values from normalized reporting models.

Activities

- Read normalized fields
- Preserve source identifiers
- Preserve structural relationships
- Preserve validation references
- Extract reportable measurements

Output

Extracted Metrics

11.6.10 Metric Classification

Purpose

Classify extracted metrics into predefined enterprise categories.

Activities

- Assign reporting category
- Group related measurements
- Associate business context
- Maintain category hierarchy
- Preserve source lineage

Output

Classified Metrics

11.6.11 Metric Organization

Purpose

Organize classified metrics into standardized reporting structures.

Activities

- Build metric collections
- Organize category hierarchy
- Preserve deterministic ordering
- Associate processing references
- Prepare reporting structures

Output

Organized Metric Collections

11.6.12 Metric Consolidation

Purpose

Create a unified enterprise metrics model.

Activities

- Consolidate categorized metrics
- Merge reporting collections
- Preserve category boundaries
- Attach source references
- Build enterprise metric hierarchy

Output

Enterprise Metrics Model

11.6.13 Metric Validation

Purpose

Verify the integrity and completeness of compiled metrics.

Activities

- Validate metric structure
- Verify required categories
- Verify schema compatibility
- Verify source traceability
- Confirm compilation completeness

Output

Validated Enterprise Metrics

11.6.14 Metric Metadata Generation

Purpose

Generate metadata describing the compiled metrics.

Activities

- Generate metric identifiers
- Generate compilation metadata
- Generate processing references
- Generate schema references
- Generate lineage metadata
- Generate timestamps

Output

Metric Metadata

11.6.15 Enterprise Metrics Package

The completed metrics package shall contain:

- Categorized enterprise metrics
- Metric hierarchy
- Source references
- Validation information
- Processing metadata
- Schema metadata
- Compilation metadata
- Lineage information

11.6.16 Validation Rules

Compiled metrics shall satisfy all of the following requirements:

- Mandatory metric categories present
- Required metric values compiled
- Source references preserved
- Structural integrity verified
- Schema compliance confirmed
- Metadata attached
- Processing sequence recorded
- Validation successfully completed

Metrics failing validation shall not proceed to the Executive Summary Generation Layer.

11.6.17 Traceability Requirements

The Metrics Compilation Layer shall preserve complete lineage for every compiled metric, including:

- Source engine reference
- Source dataset reference
- Normalized data reference
- Metric identifier
- Compilation identifier
- Processing sequence
- Schema version
- Validation status

Traceability information shall remain immutable throughout the reporting lifecycle.

11.6.18 Failure Handling

If metric compilation fails:

- Processing shall terminate
- Source information shall remain unchanged
- Normalized reporting models shall remain immutable
- Partial metric packages shall not be published
- Diagnostic metadata shall be generated
- Validation status shall indicate failure

No automatic recalculation or modification of source information shall occur.

11.6.19 Architectural Constraints

The Metrics Compilation Layer shall never:

- Modify normalized reporting data
- Execute operational workflows
- Perform AI inference
- Trigger CRM operations
- Generate executive summaries

- Assemble reports
- Package reporting artifacts
- Export reports
- Alter upstream validation results

11.6.20 Layer Summary

The Metrics Compilation Layer establishes the standardized enterprise measurement foundation of the Reporting Engine by transforming normalized reporting models into validated, categorized, and traceable enterprise metrics. It ensures deterministic processing, schema consistency, immutable data handling, and complete audit readiness while preserving the integrity of all upstream reporting information.

DOCUMENT STATUS

Phase 11.6 — Reporting Engine Metrics Compilation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.7 — Reporting Engine Executive Summary Generation Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.7

Reporting Engine Executive Summary Generation Layer

11.7.1 Purpose

The Executive Summary Generation Layer is responsible for producing a standardized executive-level overview from validated enterprise metrics.

This layer organizes compiled reporting information into a concise, structured summary suitable for management and executive consumption.

The Executive Summary Generation Layer does not generate new business information, perform AI inference, modify metrics, or alter upstream reporting data.

11.7.2 Objectives

The Executive Summary Generation Layer is designed to achieve the following objectives:

- Standardized executive reporting
- Consistent summary generation
- Deterministic processing
- Read-only execution
- Structured business overview
- Enterprise reporting consistency
- Complete traceability
- Schema-driven execution
- Validation readiness
- Audit compliance

11.7.3 Layer Position

The Executive Summary Generation Layer executes immediately after successful completion of the Metrics Compilation Layer.

Execution Flow

Metrics Compilation Layer

↓

Executive Summary Generation Layer

↓

Report Assembly Layer

11.7.4 Responsibilities

The Executive Summary Generation Layer is responsible for:

- Reading validated enterprise metrics
- Organizing executive information
- Consolidating reporting highlights
- Structuring management overview
- Preserving metric traceability
- Maintaining reporting consistency
- Preparing executive summary models
- Generating summary metadata
- Validating executive summaries
- Producing standardized executive summary packages

11.7.5 Input Sources

The Executive Summary Generation Layer consumes:

- Validated enterprise metrics
- Metric metadata
- Source reference mappings
- Validation metadata
- Processing metadata

Only validated metric packages shall be processed.

11.7.6 Executive Summary Components

Every executive summary shall consist of the following standardized components:

- Executive Overview
- Business Highlights
- Operational Highlights
- Campaign Highlights
- Pipeline Overview
- CRM Overview
- Communication Overview
- Audit Overview
- Reporting Status
- Processing Summary

The structure of these components shall remain standardized across all reports.

11.7.7 Generation Workflow

The Executive Summary Generation Layer executes the following sequence:

Summary Initialization

↓

Metric Review

↓

Information Organization

↓

Summary Construction

↓

Summary Validation

↓

Metadata Generation

↓

Executive Summary Package

11.7.8 Summary Initialization

Purpose

Initialize executive summary generation for the current reporting request.

Activities

- Create summary context

- Register report identifier
- Associate reporting metadata
- Associate processing references
- Initialize summary structure

Output

Executive Summary Context

11.7.9 Metric Review

Purpose

Review validated enterprise metrics for executive reporting.

Activities

- Read compiled metrics
- Preserve metric identifiers
- Preserve category hierarchy
- Preserve source references
- Verify reporting eligibility

Output

Reviewed Enterprise Metrics

11.7.10 Information Organization

Purpose

Organize reviewed metrics into standardized executive reporting sections.

Activities

- Categorize executive information
- Organize reporting highlights
- Build summary hierarchy
- Maintain deterministic ordering
- Preserve reporting lineage

Output

Organized Executive Information

11.7.11 Summary Construction

Purpose

Construct the standardized executive summary.

Activities

- Build executive overview
- Assemble reporting sections
- Organize management information
- Preserve reporting consistency
- Produce executive summary structure

Output

Executive Summary Model

11.7.12 Summary Validation

Purpose

Verify structural integrity and completeness of the executive summary.

Activities

- Validate summary structure
- Verify mandatory components
- Verify schema compliance
- Verify metric references
- Confirm reporting completeness

Output

Validated Executive Summary

11.7.13 Metadata Generation

Purpose

Generate metadata describing the executive summary.

Activities

- Generate summary identifier
- Generate processing metadata
- Generate lineage metadata
- Generate schema references
- Generate timestamps
- Generate validation metadata

Output

Executive Summary Metadata

11.7.14 Executive Summary Package

The completed executive summary package shall contain:

- Executive summary model
- Structured reporting sections
- Metric references
- Validation information
- Processing metadata
- Schema metadata
- Summary metadata
- Lineage information

11.7.15 Validation Rules

The executive summary shall satisfy all of the following requirements:

- Mandatory summary components present

- Required metric references preserved
- Structural integrity verified
- Schema compatibility confirmed
- Metadata attached
- Processing sequence recorded
- Validation completed successfully
- Traceability maintained

Executive summaries failing validation shall not proceed to the Report Assembly Layer.

11.7.16 Traceability Requirements

The Executive Summary Generation Layer shall preserve complete lineage for every generated summary, including:

- Source engine references
- Source dataset references
- Metric package references
- Executive summary identifier
- Processing sequence
- Schema version
- Validation status
- Metadata references

Traceability information shall remain immutable throughout the reporting lifecycle.

11.7.17 Failure Handling

If executive summary generation fails:

- Processing shall terminate
- Enterprise metrics shall remain unchanged
- Partial summaries shall not be published
- Diagnostic metadata shall be generated
- Validation status shall indicate failure
- Source reporting information shall remain immutable

No automatic modification of enterprise metrics shall occur.

11.7.18 Architectural Constraints

The Executive Summary Generation Layer shall never:

- Modify enterprise metrics
- Perform AI inference
- Execute operational workflows
- Trigger CRM operations
- Assemble complete reports
- Package reporting artifacts
- Export reports
- Alter validation results
- Recalculate source information

11.7.19 Layer Characteristics

The Executive Summary Generation Layer is:

- Deterministic
- Read-only
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-aware
- Enterprise compliant
- Non-destructive

11.7.20 Layer Summary

The Executive Summary Generation Layer establishes the executive reporting foundation of the Reporting Engine by transforming validated enterprise metrics into standardized executive summaries. It preserves deterministic execution, complete traceability, schema consistency, and immutable processing while preparing management-level reporting information for the Report Assembly Layer.

DOCUMENT STATUS

Phase 11.7 — Reporting Engine Executive Summary Generation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.8 — Reporting Engine Report Validation Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.8

Reporting Engine Report Validation Layer

11.8.1 Purpose

The Report Validation Layer is responsible for verifying the structural integrity, schema compliance, completeness, consistency, and enterprise readiness of assembled reports before packaging.

This layer serves as the final quality assurance checkpoint within the Reporting Engine.

The Report Validation Layer performs validation only.

It does not modify report content, generate new information, perform AI inference, or alter upstream processing results.

11.8.2 Objectives

The Report Validation Layer is designed to achieve the following objectives:

- Enterprise report validation
- Structural verification
- Schema compliance verification
- Reporting consistency verification
- Completeness verification
- Deterministic validation
- Read-only processing

- Validation traceability
- Audit readiness
- Packaging readiness

11.8.3 Layer Position

The Report Validation Layer executes immediately after successful completion of the Report Assembly Layer.

Execution Flow

Report Assembly Layer

↓

Report Validation Layer

↓

Report Packaging Layer

11.8.4 Responsibilities

The Report Validation Layer is responsible for:

- Reading assembled enterprise reports
- Verifying report structure
- Verifying mandatory sections
- Verifying schema compliance
- Verifying report completeness
- Verifying metadata consistency
- Preserving report traceability
- Recording validation results
- Generating validation metadata
- Producing validated report packages

11.8.5 Input Sources

The Report Validation Layer consumes:

- Assembled enterprise reports
- Executive summary packages
- Enterprise metrics packages
- Report metadata
- Processing metadata
- Schema metadata

Only fully assembled reports shall be eligible for validation.

11.8.6 Validation Categories

The Report Validation Layer performs validation across the following categories:

- Structural Validation
- Schema Validation
- Completeness Validation
- Consistency Validation
- Metadata Validation
- Traceability Validation
- Packaging Readiness Validation
- Enterprise Compliance Validation

Each category shall complete successfully before report packaging begins.

11.8.7 Validation Workflow

The Report Validation Layer executes the following sequence:

Validation Initialization

↓

Structure Verification

↓

Schema Verification

↓

Completeness Verification

↓

Consistency Verification

↓

Metadata Verification

↓

Traceability Verification

↓

Packaging Readiness Verification

↓

Validation Result Generation

↓

Validated Report

11.8.8 Validation Initialization

Purpose

Initialize report validation for the assembled enterprise report.

Activities

- Register validation request
- Associate report identifier
- Initialize validation context
- Associate schema references
- Associate processing metadata

Output

Validation Context

11.8.9 Structure Verification

Purpose

Verify that the assembled report conforms to the standardized reporting structure.

Activities

- Verify section hierarchy
- Verify document organization
- Verify required report components
- Verify structural consistency
- Verify report integrity

Output

Structure Verification Result

11.8.10 Schema Verification

Purpose

Verify compliance with supported reporting schemas.

Activities

- Validate schema version
- Verify required fields
- Verify contract compatibility
- Verify schema integrity
- Confirm structural compliance

Output

Schema Verification Result

11.8.11 Completeness Verification

Purpose

Verify that all mandatory reporting information is present.

Activities

- Verify mandatory sections
- Verify executive summary
- Verify enterprise metrics
- Verify supporting information
- Verify report descriptors

Output

Completeness Verification Result

11.8.12 Consistency Verification

Purpose

Verify internal consistency throughout the report.

Activities

- Verify reporting relationships
- Verify section consistency
- Verify metadata consistency
- Verify reference integrity
- Verify reporting alignment

Output

Consistency Verification Result

11.8.13 Metadata Verification

Purpose

Verify the completeness and integrity of report metadata.

Activities

- Verify report identifiers
- Verify processing metadata
- Verify lineage metadata
- Verify timestamps
- Verify version information

Output

Metadata Verification Result

11.8.14 Traceability Verification

Purpose

Verify complete reporting lineage.

Activities

- Verify source references
- Verify engine references
- Verify processing sequence
- Verify validation references
- Verify ownership information

Output

Traceability Verification Result

11.8.15 Packaging Readiness Verification

Purpose

Verify that the report is ready for packaging.

Activities

- Confirm successful validation
- Verify report completeness
- Verify metadata availability
- Verify packaging prerequisites
- Confirm export eligibility

Output

Packaging Readiness Result

11.8.16 Validation Result Generation

Purpose

Generate the final validation outcome for the report.

Activities

- Consolidate validation results
- Assign validation status
- Generate validation metadata
- Record validation identifiers
- Produce validated report package

Output

Validated Enterprise Report

11.8.17 Validation Rules

Every report shall satisfy the following requirements:

- Standardized structure verified
- Required sections present
- Schema compliance confirmed
- Metadata verified
- Traceability preserved
- Processing lineage complete

- Version compatibility confirmed
- Packaging readiness confirmed

Reports failing any validation requirement shall not proceed to the Report Packaging Layer.

11.8.18 Failure Handling

If validation fails:

- Report processing shall terminate
- Report content shall remain unchanged
- Source information shall remain immutable
- Partial validation results shall not be published as completed reports
- Validation diagnostics shall be generated
- Failure metadata shall be recorded

No automatic modification of report content shall occur.

11.8.19 Architectural Constraints

The Report Validation Layer shall never:

- Modify report contents
- Generate business information
- Execute AI inference
- Trigger workflows
- Recalculate enterprise metrics
- Alter executive summaries
- Package reports
- Export reports
- Modify upstream processing results

11.8.20 Layer Summary

The Report Validation Layer establishes the enterprise quality assurance foundation of the Reporting Engine by verifying structural integrity, schema compliance, completeness, consistency, metadata integrity, and traceability of assembled reports. It ensures that only

validated, immutable, and enterprise-compliant reporting artifacts proceed to the Report Packaging Layer.

DOCUMENT STATUS

Phase 11.8 — Reporting Engine Report Validation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.9 — Reporting Engine Packaging Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.9

Reporting Engine Packaging Layer

11.9.1 Purpose

The Reporting Engine Packaging Layer is responsible for transforming validated enterprise reports into standardized reporting packages for downstream consumption.

This layer assembles validated report artifacts, supporting metadata, lineage information, and packaging descriptors into a unified enterprise reporting package.

The Packaging Layer performs packaging only.

It does not modify report content, execute business logic, perform AI inference, or alter validation results.

11.9.2 Objectives

The Reporting Engine Packaging Layer is designed to achieve the following objectives:

- Standardized report packaging
- Enterprise artifact construction

- Packaging consistency
- Read-only processing
- Deterministic execution
- Metadata integration
- Traceability preservation
- Schema compliance
- Export readiness
- Audit compatibility

11.9.3 Layer Position

The Reporting Engine Packaging Layer executes immediately after successful completion of the Report Validation Layer.

Execution Flow

Report Validation Layer

↓

Reporting Engine Packaging Layer

↓

Metadata Generation Layer

11.9.4 Responsibilities

The Reporting Engine Packaging Layer is responsible for:

- Reading validated enterprise reports
- Constructing standardized report packages
- Integrating report components
- Preserving validation results
- Preserving source references
- Preserving processing lineage
- Maintaining package integrity
- Generating packaging descriptors
- Validating package completeness
- Producing packaged reporting artifacts

11.9.5 Input Sources

The Reporting Engine Packaging Layer consumes:

- Validated enterprise reports
- Validation results
- Executive summary packages
- Enterprise metrics packages
- Report metadata
- Processing metadata
- Schema metadata

Only successfully validated reports shall be eligible for packaging.

11.9.6 Package Components

Every reporting package shall contain the following standardized components:

- Report Header
- Executive Summary
- Enterprise Metrics
- Report Sections
- Supporting Information
- Validation Information
- Source References
- Processing Lineage
- Packaging Descriptors
- Metadata References

The composition of the package shall remain consistent across all report categories.

11.9.7 Packaging Workflow

The Reporting Engine Packaging Layer executes the following sequence:

Package Initialization

↓

Component Collection

↓

Package Assembly

↓

Package Integrity Verification

↓

Packaging Descriptor Generation

↓

Package Validation

↓

Packaged Report Generation

11.9.8 Package Initialization

Purpose

Initialize packaging for a validated enterprise report.

Activities

- Register packaging request
- Associate report identifier
- Initialize package context
- Associate validation references
- Associate processing metadata

Output

Packaging Context

11.9.9 Component Collection

Purpose

Collect all validated report components required for package construction.

Activities

- Collect executive summary
- Collect enterprise metrics
- Collect report sections
- Collect validation information
- Collect metadata references
- Collect lineage information

Output

Package Components

11.9.10 Package Assembly

Purpose

Assemble all collected components into a standardized reporting package.

Activities

- Build package hierarchy
- Organize package sections
- Preserve component ordering
- Preserve validation references
- Preserve report integrity

Output

Assembled Report Package

11.9.11 Package Integrity Verification

Purpose

Verify the structural integrity of the assembled reporting package.

Activities

- Verify package hierarchy
- Verify component completeness
- Verify structural consistency
- Verify validation references
- Verify metadata references

Output

Package Integrity Result

11.9.12 Packaging Descriptor Generation

Purpose

Generate descriptors describing the packaged reporting artifact.

Activities

- Generate package identifier
- Generate package version
- Generate packaging metadata
- Generate packaging timestamps
- Generate package descriptors
- Generate lineage references

Output

Packaging Descriptors

11.9.13 Package Validation

Purpose

Verify that the completed package satisfies enterprise packaging standards.

Activities

- Verify package completeness
- Verify descriptor integrity
- Verify schema compatibility
- Verify metadata availability
- Confirm downstream readiness

Output

Validated Reporting Package

11.9.14 Packaged Report Artifact

The completed reporting package shall contain:

- Validated enterprise report
- Executive summary
- Enterprise metrics
- Supporting report sections
- Validation information
- Packaging descriptors
- Metadata references
- Processing lineage
- Source references
- Package identifiers

The packaged artifact represents the official internal reporting package consumed by downstream processing stages.

11.9.15 Validation Rules

Every reporting package shall satisfy the following requirements:

- All mandatory package components present
- Report integrity preserved
- Validation results included
- Source references preserved

- Metadata references attached
- Processing lineage maintained
- Package descriptors generated
- Schema compliance confirmed

Packages failing validation shall not proceed to the Metadata Generation Layer.

11.9.16 Traceability Requirements

The Reporting Engine Packaging Layer shall preserve complete package lineage, including:

- Package identifier
- Report identifier
- Source engine references
- Validation references
- Processing sequence
- Packaging metadata
- Schema version
- Version identifiers

Traceability shall remain immutable after package creation.

11.9.17 Failure Handling

If packaging fails:

- Processing shall terminate
- Validated reports shall remain unchanged
- Partial packages shall not be published
- Packaging diagnostics shall be generated
- Failure metadata shall be recorded
- Validation results shall remain intact

No automatic modification of validated report content shall occur.

11.9.18 Architectural Constraints

The Reporting Engine Packaging Layer shall never:

- Modify validated report contents
- Generate business information
- Execute AI inference
- Trigger operational workflows
- Recalculate enterprise metrics
- Alter validation results
- Generate export artifacts
- Modify upstream processing outputs
- Perform report normalization

11.9.19 Layer Characteristics

The Reporting Engine Packaging Layer is:

- Deterministic
- Read-only
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-aware
- Enterprise compliant
- Non-destructive

11.9.20 Layer Summary

The Reporting Engine Packaging Layer establishes the standardized packaging foundation of the Reporting Engine by assembling validated reports, executive summaries, enterprise metrics, validation information, metadata references, and processing lineage into consistent enterprise reporting packages. It preserves report integrity, deterministic execution, complete traceability, and schema compliance while preparing packaged reporting artifacts for metadata generation.

DOCUMENT STATUS

Phase 11.9 — Reporting Engine Packaging Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.10 — Reporting Engine Metadata Generation Layer

END OF DOCUMENT

PART 2B-6 — Phase 11.10

Reporting Engine Metadata Generation Layer

11.10.1 Purpose

The Reporting Engine Metadata Generation Layer is responsible for generating standardized metadata for every packaged reporting artifact produced by the Reporting Engine.

This layer creates immutable metadata that identifies, describes, classifies, versions, and traces each reporting package throughout its lifecycle.

The Metadata Generation Layer performs metadata creation only.

It does not modify report contents, validation results, packaging structures, or upstream processing outputs.

11.10.2 Objectives

The Reporting Engine Metadata Generation Layer is designed to achieve the following objectives:

- Standardized metadata generation
- Enterprise report identification
- Version management
- Processing traceability
- Source lineage preservation
- Schema compatibility
- Audit readiness
- Read-only processing
- Deterministic execution
- Enterprise metadata consistency

11.10.3 Layer Position

The Reporting Engine Metadata Generation Layer executes immediately after successful completion of the Reporting Engine Packaging Layer.

Execution Flow

Reporting Engine Packaging Layer

↓

Reporting Engine Metadata Generation Layer

↓

Export Preparation Layer

11.10.4 Responsibilities

The Reporting Engine Metadata Generation Layer is responsible for:

- Reading packaged reporting artifacts
- Generating report identifiers
- Generating package identifiers
- Generating metadata descriptors
- Recording processing lineage
- Recording schema information
- Recording validation references
- Recording version information
- Recording timestamps
- Producing metadata-enriched reporting packages

11.10.5 Input Sources

The Reporting Engine Metadata Generation Layer consumes:

- Packaged reporting artifacts

- Packaging descriptors
- Validation information
- Source references
- Processing metadata
- Schema metadata

Only validated reporting packages shall be eligible for metadata generation.

11.10.6 Metadata Categories

The Reporting Engine generates the following standardized metadata categories:

- Report Metadata
- Package Metadata
- Processing Metadata
- Validation Metadata
- Version Metadata
- Schema Metadata
- Lineage Metadata
- Source Metadata
- Timestamp Metadata
- Audit Metadata

Each category shall remain independently identifiable within the metadata model.

11.10.7 Metadata Generation Workflow

The Reporting Engine Metadata Generation Layer executes the following sequence:

Metadata Initialization

↓

Identifier Generation

↓

Version Registration

↓

Schema Registration



Lineage Registration



Metadata Assembly



Metadata Validation



Metadata Attachment



Metadata-Enriched Reporting Package

11.10.8 Metadata Initialization

Purpose

Initialize metadata generation for the packaged reporting artifact.

Activities

- Register metadata request
- Associate package identifier
- Initialize metadata context
- Associate processing references
- Associate validation references

Output

Metadata Context

11.10.9 Identifier Generation

Purpose

Generate unique identifiers for reporting artifacts.

Activities

- Generate report identifier
- Generate package identifier
- Generate metadata identifier
- Associate identifier hierarchy
- Preserve identifier uniqueness

Output

Identifier Set

11.10.10 Version Registration

Purpose

Register version information for the reporting artifact.

Activities

- Register report version
- Register package version
- Register schema version
- Associate version references
- Preserve version consistency

Output

Version Metadata

11.10.11 Schema Registration

Purpose

Associate reporting artifacts with supported schema definitions.

Activities

- Register schema identifiers
- Associate contract versions
- Record schema compatibility
- Preserve schema references
- Validate schema registration

Output

Schema Metadata

11.10.12 Lineage Registration

Purpose

Record complete processing lineage for the reporting artifact.

Activities

- Register source engine references
- Register dataset references
- Register processing sequence
- Register validation lineage
- Register packaging lineage

Output

Lineage Metadata

11.10.13 Metadata Assembly

Purpose

Construct the complete enterprise metadata model.

Activities

- Assemble metadata categories
- Organize metadata hierarchy
- Preserve processing references
- Preserve validation references
- Build standardized metadata structure

Output

Enterprise Metadata Model

11.10.14 Metadata Validation

Purpose

Verify completeness and integrity of generated metadata.

Activities

- Verify mandatory metadata
- Verify identifier integrity
- Verify version consistency
- Verify schema references
- Verify lineage completeness

Output

Validated Metadata

11.10.15 Metadata Attachment

Purpose

Attach validated metadata to the packaged reporting artifact.

Activities

- Attach enterprise metadata
- Associate metadata identifiers
- Preserve package integrity

- Preserve reporting structure
- Finalize metadata-enriched package

Output

Metadata-Enriched Reporting Package

11.10.16 Metadata Contents

Every metadata-enriched reporting package shall contain:

- Report identifier
- Package identifier
- Metadata identifier
- Version information
- Schema references
- Validation references
- Processing lineage
- Source references
- Timestamp information
- Audit descriptors

Metadata shall remain immutable after attachment.

11.10.17 Validation Rules

Generated metadata shall satisfy all of the following requirements:

- Mandatory metadata present
- Unique identifiers generated
- Version information verified
- Schema references validated
- Processing lineage complete
- Source references preserved
- Timestamp information recorded
- Validation successfully completed

Metadata failing validation shall not proceed to the Export Preparation Layer.

11.10.18 Failure Handling

If metadata generation fails:

- Processing shall terminate
- Packaged reporting artifacts shall remain unchanged
- Partial metadata shall not be attached
- Failure metadata shall be recorded
- Diagnostic information shall be generated
- Validation status shall indicate failure

No automatic modification of reporting packages shall occur.

11.10.19 Architectural Constraints

The Reporting Engine Metadata Generation Layer shall never:

- Modify report contents
- Modify packaged artifacts
- Execute business logic
- Perform AI inference
- Trigger operational workflows
- Recalculate enterprise metrics
- Alter validation results
- Generate export artifacts
- Modify upstream processing outputs

11.10.20 Layer Characteristics

The Reporting Engine Metadata Generation Layer is:

- Deterministic
- Read-only
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-aware

- Enterprise compliant
- Non-destructive

11.10.21 Layer Summary

The Reporting Engine Metadata Generation Layer establishes the enterprise metadata foundation of the Reporting Engine by generating standardized, immutable metadata for every packaged reporting artifact. It preserves complete identification, version control, schema compatibility, processing lineage, and audit traceability while ensuring metadata consistency before export preparation.

DOCUMENT STATUS

Phase 11.10 — Reporting Engine Metadata Generation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.11 — Reporting Engine Output Schema Specification

END OF DOCUMENT

PART 2B-6 — Phase 11.11

Reporting Engine Output Schema Specification

11.11.1 Purpose

The Reporting Engine Output Schema Specification defines the standardized enterprise schema for all reporting artifacts produced by the Reporting Engine.

This specification establishes the canonical structure of reporting outputs, ensuring consistency, interoperability, validation compatibility, audit readiness, and downstream integration.

All reporting artifacts generated by the Reporting Engine shall conform to the schema defined in this document.

11.11.2 Objectives

The Output Schema Specification is designed to achieve the following objectives:

- Standardized report structure
- Enterprise-wide schema consistency
- Deterministic output representation
- Validation compatibility
- Metadata integration
- Version management
- Traceability preservation
- Export readiness
- Audit compliance
- Future schema extensibility

11.11.3 Schema Principles

The Reporting Engine output schema shall adhere to the following principles:

- Strongly typed
- Deterministic
- Immutable
- Version-controlled
- Self-describing
- Schema-driven
- Read-only
- Traceable
- Enterprise compliant
- Backward compatible

11.11.4 Schema Hierarchy

Every Reporting Engine output shall follow the standardized hierarchy below.

Reporting Package

↓

Package Header



Executive Summary



Enterprise Metrics



Report Sections



Supporting Information



Validation Information



Metadata



Processing Lineage



Package Footer

11.11.5 Root Schema Structure

Every reporting artifact shall contain the following top-level sections:

- Package Header
- Report Body
- Executive Summary
- Enterprise Metrics

- Supporting Sections
- Validation Information
- Metadata
- Processing Lineage
- Package Footer

No additional root-level structures shall be introduced without schema version updates.

11.11.6 Package Header Schema

The Package Header shall contain:

- Package Identifier
- Report Identifier
- Report Category
- Schema Version
- Package Version
- Generation Timestamp
- Processing Identifier
- Report Status

The Package Header uniquely identifies every reporting artifact.

11.11.7 Executive Summary Schema

The Executive Summary section shall contain:

- Executive Overview
- Business Highlights
- Operational Highlights
- Campaign Highlights
- Pipeline Overview
- CRM Overview
- Communication Overview
- Audit Overview
- Reporting Status
- Processing Summary

This section provides the standardized management-level overview.

11.11.8 Enterprise Metrics Schema

The Enterprise Metrics section shall contain:

- Executive Metrics
- Operational Metrics
- Campaign Metrics
- Pipeline Metrics
- Lead Metrics
- Contact Metrics
- Email Metrics
- CRM Metrics
- Website Metrics
- Audit Metrics
- Communication Metrics
- System Metrics

Each metric category shall remain independently identifiable.

11.11.9 Report Sections Schema

The Report Sections shall contain standardized reporting content organized according to the requested report category.

Each section shall include:

- Section Identifier
- Section Name
- Section Contents
- Source References
- Validation References
- Processing References

Section ordering shall remain deterministic.

11.11.10 Supporting Information Schema

Supporting Information shall contain:

- Supplemental Data
- Supporting References
- Report Notes
- Processing Context
- Additional Reporting Information

Supporting information shall remain separate from primary report content.

11.11.11 Validation Information Schema

Validation Information shall contain:

- Validation Identifier
- Validation Status
- Validation Timestamp
- Validation Results
- Schema Verification
- Completeness Verification
- Consistency Verification
- Packaging Readiness Status

Validation information shall remain immutable after successful validation.

11.11.12 Metadata Schema

Metadata shall contain:

- Metadata Identifier
- Report Identifier
- Package Identifier
- Schema Version
- Package Version
- Report Version
- Processing Metadata
- Version Metadata
- Timestamp Metadata
- Audit Metadata

Metadata shall uniquely describe every reporting artifact.

11.11.13 Processing Lineage Schema

Processing Lineage shall contain:

- Source Engine References
- Source Dataset References
- Processing Sequence
- Validation References
- Packaging References
- Metadata References
- Report Generation References
- Lineage Identifier

Complete lineage shall be preserved throughout the reporting lifecycle.

11.11.14 Package Footer Schema

The Package Footer shall contain:

- Package Completion Status
- Export Readiness Status
- Final Validation Status
- Package Timestamp
- Package Integrity Indicator

The footer represents the completion state of the reporting package.

11.11.15 Mandatory Schema Requirements

Every reporting artifact shall include:

- Package Header
- Executive Summary
- Enterprise Metrics
- Report Sections
- Validation Information
- Metadata

- Processing Lineage
- Package Footer

Reports missing mandatory sections shall be considered invalid.

11.11.16 Schema Validation Rules

Every output schema shall satisfy the following requirements:

- Root hierarchy verified
- Mandatory sections present
- Required identifiers available
- Metadata attached
- Validation information preserved
- Schema version supported
- Processing lineage complete
- Structural integrity confirmed

Schema validation shall complete successfully before export preparation.

11.11.17 Version Compatibility

Every reporting artifact shall include explicit version references for:

- Schema Version
- Report Version
- Package Version
- Metadata Version

Version information shall remain immutable after report generation.

11.11.18 Traceability Requirements

Every schema instance shall preserve complete traceability, including:

- Report Identifier
- Package Identifier
- Metadata Identifier

- Source Engine References
- Dataset References
- Processing Sequence
- Validation References
- Version References

Traceability information shall remain intact throughout downstream processing.

11.11.19 Schema Constraints

The Reporting Engine Output Schema shall never:

- Modify report content
- Omit mandatory metadata
- Omit processing lineage
- Alter validation information
- Introduce unsupported structures
- Remove required identifiers
- Bypass schema validation
- Store mutable operational state

11.11.20 Output Schema Summary

The Reporting Engine Output Schema Specification establishes the canonical enterprise structure for all reporting artifacts generated by the Reporting Engine. It defines a deterministic, immutable, and schema-driven representation that preserves structural consistency, complete traceability, validation integrity, metadata completeness, and enterprise interoperability across all downstream consumers.

DOCUMENT STATUS

Phase 11.11 — Reporting Engine Output Schema Specification

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-6 — Phase 11.12 — Reporting Engine Enterprise Validation & Phase Completion

END OF DOCUMENT

PART 2B-6 — Phase 11.12

Reporting Engine Enterprise Validation & Phase Completion

11.12.1 Purpose

The Enterprise Validation phase performs the final architectural, functional, structural, and documentation verification of the Reporting Engine.

Its purpose is to confirm that every Reporting Engine document, module, workflow, contract, layer, schema, and processing specification defined throughout Phase 11 forms a complete, internally consistent, enterprise-grade reporting subsystem.

Successful completion of this phase signifies the formal completion of the Reporting Engine specification.

11.12.2 Validation Objectives

Enterprise Validation verifies that the Reporting Engine satisfies the following objectives:

- Architectural completeness
- Documentation completeness
- Functional completeness
- Structural consistency
- Module consistency
- Workflow consistency
- Contract consistency
- Schema consistency
- Metadata consistency
- Enterprise readiness

11.12.3 Validation Scope

This validation applies to the complete Reporting Engine documentation produced during Part 2B-6.

The following documents are included within the validation scope:

- Phase 11 — Reporting Engine Foundation
- Phase 11.1 — Reporting Engine Architecture
- Phase 11.2 — Reporting Engine Internal Modules
- Phase 11.3 — Reporting Engine Data Contracts
- Phase 11.4 — Reporting Engine Execution Workflow
- Phase 11.5 — Reporting Engine Data Aggregation Layer
- Phase 11.6 — Reporting Engine Metrics Compilation Layer
- Phase 11.7 — Reporting Engine Executive Summary Generation Layer
- Phase 11.8 — Reporting Engine Report Validation Layer
- Phase 11.9 — Reporting Engine Packaging Layer
- Phase 11.10 — Reporting Engine Metadata Generation Layer
- Phase 11.11 — Reporting Engine Output Schema Specification

No additional documentation is included within this validation scope.

11.12.4 Architectural Validation

The Reporting Engine architecture has been verified for:

- Layered architecture consistency
- Module separation
- Responsibility isolation
- Deterministic execution
- Read-only processing
- Immutable reporting
- Standardized interfaces
- Enterprise scalability
- Traceability preservation
- Audit compatibility

Validation Result

PASS

11.12.5 Module Validation

The following internal modules have been validated:

- Report Request Manager
- Data Aggregation Manager
- Data Normalization Manager
- Metrics Compilation Manager
- Executive Summary Manager
- Report Assembly Manager
- Report Validation Manager
- Report Packaging Manager
- Report Metadata Manager
- Export Preparation Manager

Validation Result

PASS

11.12.6 Workflow Validation

The Reporting Engine execution workflow has been verified for:

- Sequential execution
- Stage completeness
- Deterministic processing
- Validation checkpoints
- Failure handling
- Processing boundaries
- Export preparation readiness

Validation Result

PASS

11.12.7 Data Contract Validation

The Reporting Engine data contracts have been verified for:

- Contract completeness
- Producer and consumer definitions
- Schema compatibility
- Validation requirements

- Version management
- Traceability preservation
- Immutable processing

Validation Result

PASS

11.12.8 Layer Validation

The following reporting layers have been validated:

- Data Aggregation Layer
- Metrics Compilation Layer
- Executive Summary Generation Layer
- Report Validation Layer
- Packaging Layer
- Metadata Generation Layer

Each layer has been verified for:

- Defined responsibilities
- Input consistency
- Output consistency
- Validation rules
- Failure handling
- Architectural constraints

Validation Result

PASS

11.12.9 Schema Validation

The Reporting Engine Output Schema has been verified for:

- Canonical structure
- Root hierarchy
- Mandatory sections
- Metadata integration

- Validation integration
- Lineage preservation
- Version compatibility
- Enterprise interoperability

Validation Result

PASS

11.12.10 Metadata Validation

The Reporting Engine metadata model has been verified for:

- Unique identification
- Package identification
- Version registration
- Schema registration
- Processing metadata
- Validation metadata
- Audit metadata
- Timestamp management
- Lineage preservation

Validation Result

PASS

11.12.11 Traceability Validation

Complete end-to-end traceability has been verified across:

- Source engine references
- Source datasets
- Report identifiers
- Package identifiers
- Validation identifiers
- Metadata identifiers
- Processing sequence
- Version references

Validation Result

PASS

11.12.12 Enterprise Compliance Validation

The Reporting Engine has been verified for compliance with Enterprise Development Kit documentation standards.

Compliance verification includes:

- Professional documentation structure
- Consistent terminology
- Modular organization
- Enterprise architecture alignment
- Documentation consistency
- Deterministic processing model
- Read-only execution model
- Immutable data handling
- Schema-driven design
- Audit readiness

Validation Result

PASS

11.12.13 Cross-Phase Compatibility Validation

The Reporting Engine has been verified for compatibility with all previously completed and locked phases.

Validated integrations include:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine

- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine

No architectural conflicts were identified.

Validation Result

PASS

11.12.14 Documentation Completeness Validation

The Reporting Engine documentation has been verified for:

- Foundation specification
- Architecture specification
- Internal modules
- Data contracts
- Execution workflow
- Processing layers
- Packaging specification
- Metadata specification
- Output schema
- Enterprise validation

Documentation completeness has been confirmed.

Validation Result

PASS

11.12.15 Final Enterprise Assessment

The Reporting Engine satisfies the following enterprise characteristics:

- Complete
- Modular
- Deterministic
- Read-only
- Immutable

- Traceable
- Schema-driven
- Version-controlled
- Validation-driven
- Audit-ready
- Enterprise scalable
- Enterprise compliant

Overall Assessment

PASS

11.12.16 Phase Completion Summary

Reporting Engine documentation has been completed successfully.

All planned Phase 11 documentation has been produced.

All validation checkpoints have passed.

No unresolved architectural inconsistencies remain within the scope of Phase 11.

The Reporting Engine is approved as the enterprise reporting subsystem specification for the AI Website Upgrade Agency Enterprise Development Kit.

11.12.17 Phase Lock Status

Phase

Part 2B-6

Reporting Engine

Status

COMPLETED

Enterprise Validation

PASS

Documentation

COMPLETE

Architecture

VERIFIED

Modules

VERIFIED

Workflow

VERIFIED

Contracts

VERIFIED

Layers

VERIFIED

Schema

VERIFIED

Metadata

VERIFIED

Traceability

VERIFIED

Enterprise Compliance

VERIFIED

Overall Result

PASS

11.12.18 Phase Closure

Part 2B-6 — Reporting Engine is formally completed.

The complete Reporting Engine specification is now considered stable and suitable for downstream implementation, testing, and enterprise integration.

This phase is designated as ****LOCKED**** and shall not be modified unless a critical architectural defect requiring formal revision is identified.

DOCUMENT STATUS

Phase 11.12 — Reporting Engine Enterprise Validation & Phase Completion

Status: COMPLETED

Locked: READY FOR PHASE LOCK

Next Document (LOCKED):

PART 2B-7 — Phase 12 — Dashboard Engine Foundation

END OF DOCUMENT

PART 2B-7 — Phase 12

Dashboard Engine Foundation

12.1 Purpose

The Dashboard Engine is responsible for transforming validated enterprise outputs into standardized, interactive dashboard views for operational users, administrators, managers, and executive stakeholders.

The Dashboard Engine provides the presentation orchestration layer for enterprise data visualization while consuming only validated outputs produced by upstream engines.

The Dashboard Engine does not execute business logic, perform AI inference, modify operational records, or alter validated reporting data.

Its responsibility begins after upstream engines have completed processing and produced validated enterprise artifacts.

12.2 Objectives

The Dashboard Engine is designed to provide:

- Enterprise dashboard foundation
- Standardized dashboard generation
- Cross-engine data visualization
- Operational dashboards
- Executive dashboards
- Management dashboards
- Campaign dashboards
- CRM dashboards
- Real-time dashboard assembly
- Enterprise dashboard metadata generation

12.3 Scope

The Dashboard Engine operates after successful completion of:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine
- Reporting Engine

The Dashboard Engine consumes validated enterprise outputs and transforms them into standardized dashboard models suitable for downstream presentation systems.

12.4 Responsibilities

The Dashboard Engine is responsible for:

- Reading validated enterprise outputs
- Aggregating dashboard data
- Preparing visualization models
- Organizing dashboard sections
- Generating dashboard layouts
- Building widget-ready datasets
- Producing executive dashboards
- Producing operational dashboards
- Producing management dashboards
- Packaging dashboard artifacts

12.5 Out of Scope

The Dashboard Engine shall not:

- Execute searches
- Audit websites
- Detect niches
- Score leads
- Extract contacts
- Validate emails
- Generate prototypes
- Generate AI emails
- Execute follow-up workflows
- Manage CRM lifecycle
- Modify reporting outputs
- Modify source datasets
- Perform AI inference
- Execute business decisions

12.6 Position within System Architecture

The Dashboard Engine executes after completion of the Reporting Engine.

Execution Flow

Search Engine



Website Audit Engine



Niche Detection Engine



Lead Scoring Engine



Contact Extraction Engine



Email Validation Engine



Prototype Intelligence Engine



AI Personalized Email Engine



Follow-Up Engine



CRM Engine



Reporting Engine



Dashboard Engine

12.7 Dashboard Philosophy

The Dashboard Engine follows a presentation-oriented, read-only execution model.

Dashboards represent visual views of validated enterprise information.

The Dashboard Engine never modifies reporting artifacts or operational records.

Every dashboard is generated exclusively from validated enterprise outputs.

12.8 Enterprise Design Principles

The Dashboard Engine follows these principles:

- Read-only processing
- Deterministic execution
- Modular dashboard construction
- Standardized dashboard models
- Complete traceability
- Schema-driven architecture
- Immutable source preservation
- Enterprise scalability
- Consistent dashboard behavior
- Audit compatibility

12.9 High-Level Responsibilities

The Dashboard Engine provides the following functional layers:

1. Dashboard Request Processing
2. Dashboard Data Aggregation
3. Dashboard Model Construction
4. Widget Assembly

5. Dashboard Layout Assembly
6. Dashboard Validation
7. Dashboard Packaging
8. Dashboard Metadata Generation
9. Dashboard Export Preparation
10. Dashboard Delivery Preparation

12.10 Input Sources

The Dashboard Engine consumes validated outputs from:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine
- Reporting Engine

No direct communication with external systems occurs within the Dashboard Engine.

12.11 Output Consumers

Dashboard outputs may be consumed by:

- Administrative Dashboard
- Executive Dashboard
- Operations Dashboard
- Sales Dashboard
- Campaign Dashboard

- CRM Dashboard
- Analytics Dashboard
- Enterprise Portal
- Internal Monitoring Systems
- Future Visualization Modules

12.12 Core Characteristics

The Dashboard Engine shall be:

- Deterministic
- Stateless
- Read-only
- Version-controlled
- Schema-driven
- Modular
- Traceable
- Scalable
- Enterprise compliant
- Visualization-oriented

12.13 Dashboard Categories

The Dashboard Engine supports standardized dashboard categories including:

- Executive Dashboard
- Operational Dashboard
- Sales Dashboard
- Campaign Dashboard
- CRM Dashboard
- Website Performance Dashboard
- Lead Management Dashboard
- Email Activity Dashboard
- Follow-Up Dashboard
- Reporting Dashboard
- Audit Dashboard
- System Dashboard

12.14 Dashboard Lifecycle

Every dashboard follows the same lifecycle:

Dashboard Request



Data Aggregation



Dashboard Model Construction



Widget Assembly



Layout Assembly



Validation



Packaging



Metadata Generation



Export Preparation



Dashboard Delivery

12.15 Architectural Constraints

The Dashboard Engine shall never:

- Modify enterprise data
- Modify reporting artifacts
- Trigger workflows
- Execute CRM operations
- Perform AI inference
- Send communications
- Execute business logic
- Replace upstream engines

12.16 Foundation Summary

The Dashboard Engine establishes the enterprise dashboard foundation for the AI Website Upgrade Agency platform.

It provides a standardized, deterministic, and read-only visualization layer that transforms validated enterprise outputs into structured dashboard artifacts while preserving complete traceability, schema consistency, and strict separation from operational processing.

DOCUMENT STATUS

Phase 12 — Dashboard Engine Foundation

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.1 — Dashboard Engine Architecture

END OF DOCUMENT

PART 2B-7 — Phase 12

Dashboard Engine Foundation

12.1 Purpose

The Dashboard Engine is responsible for transforming validated enterprise outputs into standardized, interactive dashboard views for operational users, administrators, managers, and executive stakeholders.

The Dashboard Engine provides the presentation orchestration layer for enterprise data visualization while consuming only validated outputs produced by upstream engines.

The Dashboard Engine does not execute business logic, perform AI inference, modify operational records, or alter validated reporting data.

Its responsibility begins after upstream engines have completed processing and produced validated enterprise artifacts.

12.2 Objectives

The Dashboard Engine is designed to provide:

- Enterprise dashboard foundation
- Standardized dashboard generation
- Cross-engine data visualization
- Operational dashboards
- Executive dashboards
- Management dashboards
- Campaign dashboards
- CRM dashboards
- Real-time dashboard assembly
- Enterprise dashboard metadata generation

12.3 Scope

The Dashboard Engine operates after successful completion of:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine

- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine
- Reporting Engine

The Dashboard Engine consumes validated enterprise outputs and transforms them into standardized dashboard models suitable for downstream presentation systems.

12.4 Responsibilities

The Dashboard Engine is responsible for:

- Reading validated enterprise outputs
- Aggregating dashboard data
- Preparing visualization models
- Organizing dashboard sections
- Generating dashboard layouts
- Building widget-ready datasets
- Producing executive dashboards
- Producing operational dashboards
- Producing management dashboards
- Packaging dashboard artifacts

12.5 Out of Scope

The Dashboard Engine shall not:

- Execute searches
- Audit websites
- Detect niches
- Score leads
- Extract contacts
- Validate emails
- Generate prototypes
- Generate AI emails
- Execute follow-up workflows
- Manage CRM lifecycle
- Modify reporting outputs
- Modify source datasets

- Perform AI inference
- Execute business decisions

12.6 Position within System Architecture

The Dashboard Engine executes after completion of the Reporting Engine.

Execution Flow

Search Engine

↓

Website Audit Engine

↓

Niche Detection Engine

↓

Lead Scoring Engine

↓

Contact Extraction Engine

↓

Email Validation Engine

↓

Prototype Intelligence Engine

↓

AI Personalized Email Engine

↓

Follow-Up Engine

↓

CRM Engine

↓

Reporting Engine

↓

Dashboard Engine

12.7 Dashboard Philosophy

The Dashboard Engine follows a presentation-oriented, read-only execution model.

Dashboards represent visual views of validated enterprise information.

The Dashboard Engine never modifies reporting artifacts or operational records.

Every dashboard is generated exclusively from validated enterprise outputs.

12.8 Enterprise Design Principles

The Dashboard Engine follows these principles:

- Read-only processing
- Deterministic execution
- Modular dashboard construction
- Standardized dashboard models
- Complete traceability
- Schema-driven architecture
- Immutable source preservation
- Enterprise scalability
- Consistent dashboard behavior
- Audit compatibility

12.9 High-Level Responsibilities

The Dashboard Engine provides the following functional layers:

1. Dashboard Request Processing
2. Dashboard Data Aggregation
3. Dashboard Model Construction
4. Widget Assembly
5. Dashboard Layout Assembly
6. Dashboard Validation
7. Dashboard Packaging
8. Dashboard Metadata Generation
9. Dashboard Export Preparation
10. Dashboard Delivery Preparation

12.10 Input Sources

The Dashboard Engine consumes validated outputs from:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine
- Reporting Engine

No direct communication with external systems occurs within the Dashboard Engine.

12.11 Output Consumers

Dashboard outputs may be consumed by:

- Administrative Dashboard
- Executive Dashboard
- Operations Dashboard
- Sales Dashboard
- Campaign Dashboard
- CRM Dashboard
- Analytics Dashboard
- Enterprise Portal
- Internal Monitoring Systems
- Future Visualization Modules

12.12 Core Characteristics

The Dashboard Engine shall be:

- Deterministic
- Stateless
- Read-only
- Version-controlled
- Schema-driven
- Modular
- Traceable
- Scalable
- Enterprise compliant
- Visualization-oriented

12.13 Dashboard Categories

The Dashboard Engine supports standardized dashboard categories including:

- Executive Dashboard
- Operational Dashboard
- Sales Dashboard
- Campaign Dashboard

- CRM Dashboard
- Website Performance Dashboard
- Lead Management Dashboard
- Email Activity Dashboard
- Follow-Up Dashboard
- Reporting Dashboard
- Audit Dashboard
- System Dashboard

12.14 Dashboard Lifecycle

Every dashboard follows the same lifecycle:

Dashboard Request



Data Aggregation



Dashboard Model Construction



Widget Assembly



Layout Assembly



Validation



Packaging



Metadata Generation

↓

Export Preparation

↓

Dashboard Delivery

12.15 Architectural Constraints

The Dashboard Engine shall never:

- Modify enterprise data
- Modify reporting artifacts
- Trigger workflows
- Execute CRM operations
- Perform AI inference
- Send communications
- Execute business logic
- Replace upstream engines

12.16 Foundation Summary

The Dashboard Engine establishes the enterprise dashboard foundation for the AI Website Upgrade Agency platform.

It provides a standardized, deterministic, and read-only visualization layer that transforms validated enterprise outputs into structured dashboard artifacts while preserving complete traceability, schema consistency, and strict separation from operational processing.

DOCUMENT STATUS

Phase 12 — Dashboard Engine Foundation

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.1 — Dashboard Engine Architecture

END OF DOCUMENT

PART 2B-7 — Phase 12.1

Dashboard Engine Architecture

12.1.1 Overview

The Dashboard Engine Architecture defines the structural organization of the enterprise dashboard subsystem.

Its purpose is to transform validated enterprise outputs into standardized dashboard artifacts through a deterministic, modular, schema-driven, and read-only processing pipeline.

The architecture separates dashboard construction responsibilities into independent architectural layers to ensure maintainability, scalability, traceability, interoperability, and enterprise compliance.

12.1.2 Architectural Objectives

The Dashboard Engine architecture is designed to achieve the following objectives:

- Modular dashboard construction
- Separation of dashboard responsibilities
- Enterprise scalability
- Deterministic processing
- Read-only execution
- Standardized dashboard generation
- Schema-driven architecture
- Complete dashboard traceability
- Version-controlled processing
- Audit-ready dashboard infrastructure

12.1.3 Architectural Position

The Dashboard Engine is positioned immediately after the Reporting Engine within the Enterprise Development Kit processing pipeline.

All upstream engines shall complete execution before dashboard generation begins.

Execution Order

Search Engine



Website Audit Engine



Niche Detection Engine



Lead Scoring Engine



Contact Extraction Engine



Email Validation Engine



Prototype Intelligence Engine



AI Personalized Email Engine



Follow-Up Engine



CRM Engine

↓

Reporting Engine

↓

Dashboard Engine

12.1.4 Architectural Layers

The Dashboard Engine architecture consists of the following logical layers:

Layer 1

Dashboard Request Layer

Responsible for receiving standardized dashboard generation requests.

Layer 2

Dashboard Data Aggregation Layer

Collects validated enterprise information required for dashboard generation.

Layer 3

Dashboard Model Construction Layer

Transforms aggregated information into standardized dashboard models.

Layer 4

Widget Assembly Layer

Builds standardized dashboard widget models.

Layer 5

Dashboard Layout Assembly Layer

Constructs complete dashboard layouts.

Layer 6

Dashboard Validation Layer

Verifies dashboard integrity and enterprise compliance.

Layer 7

Dashboard Packaging Layer

Packages validated dashboard artifacts.

Layer 8

Dashboard Metadata Layer

Generates dashboard metadata and processing lineage.

Layer 9

Dashboard Export Preparation Layer

Prepares dashboard artifacts for downstream presentation systems.

12.1.5 Architectural Principles

The Dashboard Engine architecture follows these principles:

- Single responsibility
- Layer isolation
- Read-only processing
- Immutable execution
- Deterministic behavior
- Standardized interfaces
- Schema enforcement
- Version compatibility
- Complete traceability
- Enterprise consistency

12.1.6 Processing Direction

Information flows through the Dashboard Engine in a single direction.

Dashboard Request

↓

Dashboard Data Aggregation

↓

Dashboard Model Construction

↓

Widget Assembly

↓

Dashboard Layout Assembly

↓

Dashboard Validation

↓

Dashboard Packaging



Dashboard Metadata Generation



Dashboard Export Preparation



Completed Dashboard

Reverse processing is not permitted.

12.1.7 Data Ownership

The Dashboard Engine owns only dashboard artifacts generated during its execution.

Ownership excludes:

- Search records
- Website audit records
- Niche detection records
- Lead scoring records
- Contact extraction records
- Email validation records
- Prototype intelligence records
- AI email records
- Follow-up records
- CRM records
- Reporting artifacts

These remain under the ownership of their originating engines.

12.1.8 Layer Independence

Each architectural layer shall operate independently.

Each layer:

- Receives standardized input
- Performs a single responsibility
- Produces standardized output
- Preserves upstream integrity
- Maintains schema compliance

No layer shall depend upon internal implementation details of another layer.

12.1.9 Fault Isolation

Failures occurring within one architectural layer shall not affect upstream processing results.

Fault handling is limited to:

- Workflow termination
- Validation failure reporting
- Diagnostic metadata generation
- Error propagation

Source information shall remain unchanged.

12.1.10 Scalability Model

The architecture supports independent scaling of:

- Dashboard generation
- Data aggregation
- Widget assembly
- Layout construction
- Validation
- Packaging
- Metadata generation
- Export preparation

Scaling shall not modify deterministic dashboard behavior.

12.1.11 Interface Boundaries

Standardized internal interfaces exist between:

- Dashboard Request Layer and Dashboard Data Aggregation Layer
- Dashboard Data Aggregation Layer and Dashboard Model Construction Layer
- Dashboard Model Construction Layer and Widget Assembly Layer
- Widget Assembly Layer and Dashboard Layout Assembly Layer
- Dashboard Layout Assembly Layer and Dashboard Validation Layer
- Dashboard Validation Layer and Dashboard Packaging Layer
- Dashboard Packaging Layer and Dashboard Metadata Layer
- Dashboard Metadata Layer and Dashboard Export Preparation Layer

No architectural layer shall bypass another layer.

12.1.12 Architectural Constraints

The Dashboard Engine architecture shall never:

- Modify enterprise data
- Modify reporting artifacts
- Execute business logic
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Store mutable operational state
- Bypass validation
- Recalculate upstream information

12.1.13 Traceability

Every generated dashboard shall maintain complete traceability through:

- Source engine references
- Source dataset references
- Dashboard identifiers
- Processing lineage
- Schema versions
- Dashboard versions
- Metadata identifiers

- Validation status
- Packaging information

12.1.14 Architectural Summary

The Dashboard Engine Architecture establishes a layered, deterministic, schema-driven, and read-only dashboard subsystem that transforms validated enterprise outputs into standardized dashboard artifacts while preserving complete traceability, architectural consistency, enterprise scalability, and strict separation from operational processing.

DOCUMENT STATUS

Phase 12.1 — Dashboard Engine Architecture

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.2 — Dashboard Engine Internal Modules

END OF DOCUMENT

PART 2B-7 — Phase 12.2

Dashboard Engine Internal Modules

12.2.1 Overview

The Dashboard Engine is composed of independent internal modules responsible for transforming validated enterprise outputs into standardized dashboard artifacts.

Each module performs a single, clearly defined responsibility.

All modules communicate exclusively through standardized internal data contracts.

No module modifies upstream engine outputs, reporting artifacts, or operational records.

12.2.2 Internal Module Hierarchy

The Dashboard Engine consists of the following internal modules:

1. Dashboard Request Manager
2. Dashboard Data Aggregation Manager
3. Dashboard Model Manager
4. Widget Assembly Manager
5. Dashboard Layout Manager
6. Dashboard Validation Manager
7. Dashboard Packaging Manager
8. Dashboard Metadata Manager
9. Dashboard Export Preparation Manager
10. Dashboard Delivery Preparation Manager

12.2.3 Dashboard Request Manager

Purpose

Receives standardized dashboard generation requests from authorized enterprise components.

Responsibilities

- Receive dashboard requests
- Validate request structure
- Verify supported dashboard category
- Initialize dashboard workflow
- Generate request context
- Forward validated requests to the Dashboard Data Aggregation Manager

Inputs

- Dashboard generation request

Outputs

- Validated dashboard request

12.2.4 Dashboard Data Aggregation Manager

Purpose

Collects validated enterprise information required for dashboard generation.

Responsibilities

- Read Search Engine outputs
- Read Website Audit Engine outputs
- Read Niche Detection Engine outputs
- Read Lead Scoring Engine outputs
- Read Contact Extraction Engine outputs
- Read Email Validation Engine outputs
- Read Prototype Intelligence Engine outputs
- Read AI Personalized Email Engine outputs
- Read Follow-Up Engine outputs
- Read CRM Engine outputs
- Read Reporting Engine outputs
- Aggregate dashboard datasets

Inputs

- Validated enterprise outputs

Outputs

- Aggregated dashboard dataset

12.2.5 Dashboard Model Manager

Purpose

Transforms aggregated enterprise information into standardized dashboard models.

Responsibilities

- Normalize dashboard structures
- Organize visualization models
- Build dashboard objects
- Prepare dashboard sections
- Maintain standardized hierarchy
- Generate dashboard model

Inputs

- Aggregated dashboard dataset

Outputs

- Standardized dashboard model

12.2.6 Widget Assembly Manager

Purpose

Constructs standardized dashboard widget models from dashboard data.

Responsibilities

- Build widget definitions
- Assemble widget collections
- Organize widget hierarchy
- Associate widget metadata
- Preserve visualization consistency
- Prepare widget-ready structures

Inputs

- Dashboard model

Outputs

- Widget assembly package

12.2.7 Dashboard Layout Manager

Purpose

Constructs complete dashboard layouts using validated widget assemblies.

Responsibilities

- Assemble dashboard layout
- Organize widget placement
- Maintain layout hierarchy
- Preserve dashboard consistency
- Prepare presentation structure
- Produce dashboard layout

Inputs

- Widget assembly package

Outputs

- Dashboard layout

12.2.8 Dashboard Validation Manager

Purpose

Verifies that generated dashboard artifacts satisfy enterprise standards.

Responsibilities

- Validate dashboard structure
- Validate layout integrity
- Validate widget hierarchy
- Validate schema compliance
- Validate metadata references
- Verify dashboard completeness

Inputs

- Dashboard layout

Outputs

- Validated dashboard artifact

12.2.9 Dashboard Packaging Manager

Purpose

Packages validated dashboard artifacts into standardized enterprise packages.

Responsibilities

- Organize dashboard package
- Attach layout information
- Attach widget collections
- Attach validation information
- Finalize dashboard package

Inputs

- Validated dashboard artifact

Outputs

- Packaged dashboard artifact

12.2.10 Dashboard Metadata Manager

Purpose

Generates standardized metadata describing packaged dashboard artifacts.

Responsibilities

- Generate dashboard identifier
- Generate package identifier
- Generate metadata descriptors
- Generate processing metadata
- Generate lineage metadata

- Generate version metadata
- Generate timestamps

Inputs

- Packaged dashboard artifact

Outputs

- Dashboard metadata

12.2.11 Dashboard Export Preparation Manager

Purpose

Prepares dashboard artifacts for downstream presentation systems.

Responsibilities

- Verify export readiness
- Attach metadata
- Prepare standardized export package
- Finalize export payload
- Publish export-ready dashboard package

Inputs

- Packaged dashboard artifact
- Dashboard metadata

Outputs

- Export-ready dashboard package

12.2.12 Dashboard Delivery Preparation Manager

Purpose

Prepares validated dashboard packages for enterprise dashboard delivery systems.

Responsibilities

- Verify delivery readiness
- Associate delivery descriptors
- Prepare delivery package
- Preserve dashboard integrity
- Produce delivery-ready dashboard artifact

Inputs

- Export-ready dashboard package

Outputs

- Delivery-ready dashboard package

12.2.13 Module Interaction Flow

Dashboard Request Manager



Dashboard Data Aggregation Manager



Dashboard Model Manager



Widget Assembly Manager



Dashboard Layout Manager



Dashboard Validation Manager



Dashboard Packaging Manager

↓

Dashboard Metadata Manager

↓

Dashboard Export Preparation Manager

↓

Dashboard Delivery Preparation Manager

12.2.14 Module Design Principles

Every Dashboard Engine module shall:

- Perform one responsibility only
- Operate independently
- Be deterministic
- Be read-only
- Produce standardized outputs
- Consume standardized inputs
- Maintain schema compliance
- Preserve traceability
- Avoid side effects
- Support enterprise scalability

12.2.15 Dependency Rules

Internal modules shall never:

- Skip execution order
- Access downstream modules directly
- Modify upstream engine outputs
- Modify reporting artifacts
- Execute business logic
- Perform AI inference
- Trigger workflows

- Execute CRM operations
- Alter operational records

12.2.16 Internal Module Summary

The Dashboard Engine Internal Modules establish a modular enterprise architecture in which each module performs a distinct dashboard generation responsibility. Together, these modules transform validated enterprise outputs into standardized, validated, metadata-enriched dashboard artifacts while preserving deterministic execution, complete traceability, schema consistency, and strict separation from operational processing.

DOCUMENT STATUS

Phase 12.2 — Dashboard Engine Internal Modules

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.3 — Dashboard Engine Data Contracts

END OF DOCUMENT

PART 2B-7 — Phase 12.3

Dashboard Engine Data Contracts

12.3.1 Purpose

The Dashboard Engine Data Contracts define the standardized data exchange specifications between the Dashboard Engine and its internal modules.

These contracts establish a consistent communication framework that ensures deterministic processing, schema compatibility, module interoperability, complete traceability, and enterprise compliance throughout the dashboard generation lifecycle.

All Dashboard Engine modules shall exchange information exclusively through the contracts defined in this document.

12.3.2 Objectives

The Dashboard Engine Data Contracts are designed to achieve the following objectives:

- Standardized data exchange
- Schema consistency
- Module interoperability
- Deterministic processing
- Enterprise compatibility
- Validation readiness
- Complete traceability
- Version compatibility
- Audit readiness
- Non-destructive processing

12.3.3 Contract Principles

Every Dashboard Engine data contract shall adhere to the following principles:

- Immutable after creation
- Strongly structured
- Version-controlled
- Schema-driven
- Self-describing
- Deterministic
- Read-only
- Traceable
- Extensible
- Independently verifiable

12.3.4 Contract Lifecycle

Every Dashboard Engine data contract shall progress through the following lifecycle:

Contract Creation

↓

Schema Validation



Module Processing



Output Validation



Metadata Attachment



Contract Completion

No contract shall bypass any lifecycle stage.

12.3.5 Primary Contract Types

The Dashboard Engine defines the following primary contract types:

- Dashboard Request Contract
- Dashboard Aggregated Data Contract
- Dashboard Model Contract
- Widget Assembly Contract
- Dashboard Layout Contract
- Dashboard Validation Contract
- Dashboard Packaging Contract
- Dashboard Metadata Contract
- Dashboard Export Contract
- Dashboard Delivery Contract

12.3.6 Dashboard Request Contract

Purpose

Represents a standardized request for dashboard generation.

Producer

Dashboard Request Manager

Consumer

Dashboard Data Aggregation Manager

Contents

- Request identifier
- Dashboard category
- Request parameters
- Processing options
- Schema version
- Request timestamp

12.3.7 Dashboard Aggregated Data Contract

Purpose

Represents consolidated enterprise information collected for dashboard generation.

Producer

Dashboard Data Aggregation Manager

Consumer

Dashboard Model Manager

Contents

- Aggregated enterprise datasets
- Source engine references
- Dataset identifiers
- Collection metadata
- Processing references

12.3.8 Dashboard Model Contract

Purpose

Represents standardized dashboard models generated from aggregated enterprise information.

Producer

Dashboard Model Manager

Consumer

Widget Assembly Manager

Contents

- Dashboard model
- Dashboard sections
- Visualization structures
- Processing metadata
- Schema references

12.3.9 Widget Assembly Contract

Purpose

Represents standardized widget collections prepared for dashboard construction.

Producer

Widget Assembly Manager

Consumer

Dashboard Layout Manager

Contents

- Widget definitions
- Widget hierarchy
- Widget collections

- Widget metadata
- Visualization references

12.3.10 Dashboard Layout Contract

Purpose

Represents the complete dashboard layout.

Producer

Dashboard Layout Manager

Consumer

Dashboard Validation Manager

Contents

- Dashboard layout
- Layout hierarchy
- Widget placements
- Layout metadata
- Processing references

12.3.11 Dashboard Validation Contract

Purpose

Represents validation results for the completed dashboard.

Producer

Dashboard Validation Manager

Consumer

Dashboard Packaging Manager

Contents

- Validation status
- Validation results
- Schema verification
- Layout verification
- Metadata verification
- Validation metadata

12.3.12 Dashboard Packaging Contract

Purpose

Represents the packaged dashboard artifact.

Producer

Dashboard Packaging Manager

Consumer

Dashboard Metadata Manager

Contents

- Packaged dashboard
- Package descriptors
- Package identifiers
- Packaging metadata
- Processing references

12.3.13 Dashboard Metadata Contract

Purpose

Represents enterprise dashboard metadata.

Producer

Dashboard Metadata Manager

Consumer

Dashboard Export Preparation Manager

Contents

- Dashboard identifier
- Package identifier
- Metadata identifier
- Version information
- Processing metadata
- Lineage metadata
- Timestamp metadata

12.3.14 Dashboard Export Contract

Purpose

Represents the finalized dashboard package prepared for downstream presentation systems.

Producer

Dashboard Export Preparation Manager

Consumer

Dashboard Delivery Preparation Manager

Contents

- Export-ready dashboard
- Attached metadata
- Export descriptors
- Packaging information
- Export readiness status

12.3.15 Dashboard Delivery Contract

Purpose

Represents the final dashboard package prepared for enterprise dashboard delivery.

Producer

Dashboard Delivery Preparation Manager

Consumer

Enterprise Dashboard Delivery Systems

Contents

- Delivery-ready dashboard
- Delivery descriptors
- Dashboard metadata
- Validation references
- Delivery readiness status

12.3.16 Contract Validation Rules

Every Dashboard Engine contract shall satisfy the following requirements before being accepted by the receiving module:

- Valid schema version
- Mandatory fields present
- Structural integrity verified
- Required metadata attached
- Producer identification present
- Consumer compatibility verified
- Traceability information available
- Contract completeness confirmed

Contracts failing validation shall not proceed to subsequent modules.

12.3.17 Version Management

Each Dashboard Engine contract shall include version information sufficient to ensure compatibility across Dashboard Engine releases.

Version information shall remain immutable throughout the contract lifecycle.

12.3.18 Traceability Requirements

Every Dashboard Engine contract shall preserve complete processing lineage, including:

- Contract identifier
- Producing module
- Consuming module
- Source engine references
- Source dataset references
- Schema version
- Processing sequence
- Validation status
- Metadata references

12.3.19 Contract Constraints

Dashboard Engine data contracts shall never:

- Modify enterprise information
- Modify reporting artifacts
- Execute business logic
- Perform AI inference
- Trigger workflows
- Alter validation results
- Store mutable operational state
- Omit mandatory metadata

12.3.20 Data Contract Summary

The Dashboard Engine Data Contracts establish a standardized, immutable, schema-driven communication framework between Dashboard Engine modules. These contracts ensure deterministic execution, module interoperability, complete traceability, validation integrity, enterprise consistency, and audit readiness while preserving the integrity of all validated upstream enterprise outputs.

DOCUMENT STATUS

Phase 12.3 — Dashboard Engine Data Contracts

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.4 — Dashboard Engine Execution Workflow

END OF DOCUMENT

PART 2B-7 — Phase 12.4

Dashboard Engine Execution Workflow

12.4.1 Purpose

The Dashboard Engine Execution Workflow defines the deterministic processing sequence used to generate standardized enterprise dashboard artifacts from validated upstream enterprise outputs.

The workflow establishes the execution order, module interactions, processing boundaries, validation checkpoints, packaging procedures, metadata generation, and dashboard delivery preparation.

Every Dashboard Engine execution shall follow the workflow defined in this document without deviation.

12.4.2 Workflow Objectives

The execution workflow is designed to provide:

- Deterministic dashboard generation
- Standardized execution sequence
- Read-only processing
- Enterprise consistency
- Complete data traceability
- Validation-driven execution
- Modular processing

- Dashboard integrity
- Audit readiness
- Delivery-ready dashboard preparation

12.4.3 Workflow Preconditions

Dashboard generation may begin only when all of the following conditions are satisfied:

- All upstream engines have completed execution
- Reporting Engine outputs have been validated
- Dashboard generation request has been accepted
- Required dashboard schemas are available
- Dashboard Engine modules are operational
- Processing metadata can be generated

If any precondition fails, workflow execution shall terminate before dashboard processing begins.

12.4.4 High-Level Execution Flow

The Dashboard Engine executes the following sequence:

Dashboard Request

↓

Request Validation

↓

Dashboard Data Aggregation

↓

Dashboard Model Construction

↓

Widget Assembly

↓

Dashboard Layout Assembly

↓

Dashboard Validation

↓

Dashboard Packaging

↓

Metadata Generation

↓

Export Preparation

↓

Delivery Preparation

↓

Completed Dashboard

12.4.5 Stage 1 — Dashboard Request Processing

Purpose

Initialize dashboard generation.

Activities

- Receive dashboard request
- Verify request structure
- Verify dashboard category
- Assign request identifier
- Initialize dashboard workflow context

Output

Validated Dashboard Request

12.4.6 Stage 2 — Dashboard Data Aggregation

Purpose

Collect validated enterprise information required for dashboard generation.

Activities

- Read Search Engine outputs
- Read Website Audit Engine outputs
- Read Niche Detection Engine outputs
- Read Lead Scoring Engine outputs
- Read Contact Extraction Engine outputs
- Read Email Validation Engine outputs
- Read Prototype Intelligence Engine outputs
- Read AI Personalized Email Engine outputs
- Read Follow-Up Engine outputs
- Read CRM Engine outputs
- Read Reporting Engine outputs
- Consolidate dashboard datasets

Output

Aggregated Dashboard Dataset

12.4.7 Stage 3 — Dashboard Model Construction

Purpose

Transform aggregated enterprise information into standardized dashboard models.

Activities

- Normalize dashboard structures
- Build dashboard objects
- Organize dashboard sections

- Prepare visualization models
- Verify structural consistency

Output

Dashboard Model

12.4.8 Stage 4 — Widget Assembly

Purpose

Construct standardized widget collections.

Activities

- Generate widget definitions
- Assemble widget hierarchy
- Associate widget metadata
- Preserve dashboard consistency
- Build widget collections

Output

Widget Assembly Package

12.4.9 Stage 5 — Dashboard Layout Assembly

Purpose

Construct the complete dashboard layout.

Activities

- Organize widget placement
- Build layout hierarchy
- Preserve dashboard organization
- Associate visualization references
- Produce dashboard layout

Output

Dashboard Layout

12.4.10 Stage 6 — Dashboard Validation

Purpose

Verify dashboard integrity and enterprise compliance.

Activities

- Validate dashboard structure
- Validate widget hierarchy
- Validate layout integrity
- Verify schema compliance
- Verify metadata references
- Confirm dashboard completeness

Output

Validated Dashboard Artifact

12.4.11 Stage 7 — Dashboard Packaging

Purpose

Package validated dashboard artifacts into standardized enterprise packages.

Activities

- Build dashboard package
- Organize package hierarchy
- Attach validation information
- Attach supporting artifacts
- Finalize package structure

Output

Packaged Dashboard Artifact

12.4.12 Stage 8 — Metadata Generation

Purpose

Generate standardized metadata describing the dashboard package.

Activities

- Generate dashboard identifier
- Generate package identifier
- Generate metadata descriptors
- Generate lineage metadata
- Generate version metadata
- Generate timestamps

Output

Dashboard Metadata

12.4.13 Stage 9 — Export Preparation

Purpose

Prepare dashboard artifacts for downstream presentation systems.

Activities

- Attach metadata
- Verify export readiness
- Prepare export package
- Finalize export payload

Output

Export-Ready Dashboard Package

12.4.14 Stage 10 — Delivery Preparation

Purpose

Prepare validated dashboard packages for enterprise dashboard delivery.

Activities

- Verify delivery readiness
- Generate delivery descriptors
- Associate delivery metadata
- Finalize delivery package

Output

Delivery-Ready Dashboard Package

12.4.15 Workflow Validation Checkpoints

Validation shall occur at the following checkpoints:

Checkpoint 1

Dashboard Request Validation

Checkpoint 2

Aggregated Dataset Validation

Checkpoint 3

Dashboard Model Validation

Checkpoint 4

Widget Assembly Validation

Checkpoint 5

Dashboard Layout Validation

Checkpoint 6

Dashboard Validation Verification

Checkpoint 7

Packaging Validation

Checkpoint 8

Metadata Validation

Checkpoint 9

Export Readiness Validation

Checkpoint 10

Delivery Readiness Validation

Execution shall not continue if any checkpoint fails.

12.4.16 Workflow Failure Handling

If execution fails during any stage:

- Workflow execution shall terminate
- Source information shall remain unchanged
- Reporting outputs shall remain immutable
- Dashboard artifacts shall not be published
- Failure diagnostics shall be generated
- Validation status shall indicate failure

No automatic recovery shall modify enterprise information.

12.4.17 Workflow Characteristics

The Dashboard Engine execution workflow is:

- Deterministic
- Sequential
- Read-only
- Immutable

- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-driven
- Enterprise compliant

12.4.18 Execution Constraints

The Dashboard Engine workflow shall never:

- Modify enterprise data
- Modify reporting artifacts
- Execute business logic
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Recalculate upstream information
- Bypass validation checkpoints
- Skip execution stages

12.4.19 Workflow Completion Criteria

Workflow execution is considered complete only when:

- Every execution stage has completed successfully
- All validation checkpoints have passed
- Dashboard packaging has completed
- Metadata has been generated
- Export preparation has completed
- Delivery preparation has completed
- Final dashboard artifact has been produced

12.4.20 Workflow Summary

The Dashboard Engine Execution Workflow defines a deterministic, sequential, and validation-driven process that transforms validated enterprise outputs into standardized dashboard artifacts. The workflow preserves immutable processing, complete traceability,

schema consistency, and enterprise compliance while ensuring every dashboard is fully validated, packaged, metadata-enriched, and prepared for downstream delivery.

DOCUMENT STATUS

Phase 12.4 — Dashboard Engine Execution Workflow

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.5 — Dashboard Engine Data Aggregation Layer

END OF DOCUMENT

PART 2B-7 — Phase 12.5

Dashboard Engine Data Aggregation Layer

12.5.1 Purpose

The Dashboard Engine Data Aggregation Layer is responsible for collecting validated enterprise outputs from all supported upstream engines and consolidating them into a unified dashboard dataset.

This layer serves as the centralized data acquisition component of the Dashboard Engine.

The Data Aggregation Layer performs only data collection and organization.

It does not modify, recalculate, enrich, transform, or interpret source information.

12.5.2 Objectives

The Dashboard Engine Data Aggregation Layer is designed to achieve the following objectives:

- Centralized dashboard data collection
- Standardized enterprise aggregation
- Source integrity preservation

- Deterministic aggregation
- Schema consistency
- Complete source traceability
- Read-only processing
- Enterprise scalability
- Validation readiness
- Audit compatibility

12.5.3 Layer Position

The Dashboard Engine Data Aggregation Layer executes immediately after successful dashboard request validation.

Execution Flow

Dashboard Request Validation

↓

Dashboard Engine Data Aggregation Layer

↓

Dashboard Model Construction Layer

12.5.4 Responsibilities

The Dashboard Engine Data Aggregation Layer is responsible for:

- Reading validated enterprise outputs
- Verifying source availability
- Collecting dashboard datasets
- Organizing source information
- Maintaining source references
- Preserving engine ownership
- Consolidating dashboard inputs
- Preparing aggregated dashboard datasets
- Maintaining deterministic ordering
- Producing aggregation metadata

12.5.5 Source Systems

The Dashboard Engine Data Aggregation Layer collects information from the following validated enterprise outputs:

- Search Engine
- Website Audit Engine
- Niche Detection Engine
- Lead Scoring Engine
- Contact Extraction Engine
- Email Validation Engine
- Prototype Intelligence Engine
- AI Personalized Email Engine
- Follow-Up Engine
- CRM Engine
- Reporting Engine

Only validated outputs shall be eligible for aggregation.

12.5.6 Aggregation Workflow

The Dashboard Engine Data Aggregation Layer executes the following sequence:

Source Discovery

↓

Source Verification

↓

Schema Verification

↓

Data Collection

↓

Dataset Consolidation

↓

Source Reference Mapping

↓

Aggregation Validation

↓

Aggregated Dashboard Dataset Generation

12.5.7 Source Discovery

Purpose

Identify all enterprise outputs required for dashboard generation.

Activities

- Determine required source engines
- Locate validated outputs
- Identify dashboard dependencies
- Register available datasets

Output

Source Inventory

12.5.8 Source Verification

Purpose

Verify that all required source datasets are available and valid.

Activities

- Verify dataset availability
- Verify validation status

- Verify version compatibility
- Verify accessibility
- Verify ownership references

Output

Verified Source Inventory

12.5.9 Schema Verification

Purpose

Ensure that all collected datasets conform to supported schema versions.

Activities

- Validate schema identifiers
- Verify contract compatibility
- Confirm structural integrity
- Reject unsupported schema versions

Output

Schema-Verified Sources

12.5.10 Data Collection

Purpose

Read validated enterprise information without modification.

Activities

- Read enterprise records
- Preserve source ordering
- Preserve identifiers
- Preserve structural hierarchy
- Preserve validation information

Output

Collected Enterprise Data

12.5.11 Dataset Consolidation

Purpose

Combine collected enterprise datasets into a unified dashboard dataset.

Activities

- Organize enterprise datasets
- Preserve source boundaries
- Consolidate dashboard inputs
- Maintain deterministic ordering
- Retain dataset lineage

Output

Aggregated Dashboard Dataset

12.5.12 Source Reference Mapping

Purpose

Maintain complete traceability between dashboard datasets and originating enterprise engines.

Activities

- Associate source engine identifiers
- Associate dataset identifiers
- Record ownership references
- Record processing lineage
- Preserve origin metadata

Output

Source Reference Map

12.5.13 Aggregation Validation

Purpose

Verify the integrity of the aggregated dashboard dataset.

Activities

- Validate dataset completeness
- Validate required sources
- Verify structural consistency
- Verify reference integrity
- Confirm aggregation success

Output

Validated Aggregated Dashboard Dataset

12.5.14 Aggregated Dashboard Dataset

The completed aggregated dashboard dataset shall contain:

- Source engine references
- Aggregated enterprise records
- Dataset identifiers
- Validation references
- Schema references
- Processing metadata
- Collection metadata
- Lineage information

No source information shall be modified during aggregation.

12.5.15 Traceability Requirements

The Dashboard Engine Data Aggregation Layer shall preserve:

- Source engine identity
- Dataset identity

- Processing sequence
- Validation references
- Schema versions
- Collection timestamps
- Ownership information
- Aggregation identifiers

Traceability shall remain intact throughout subsequent dashboard processing stages.

12.5.16 Validation Rules

The aggregated dashboard dataset shall satisfy all of the following requirements:

- All required sources collected
- All datasets validated
- Supported schema versions confirmed
- Mandatory metadata available
- Source references preserved
- Processing sequence recorded
- Structural consistency verified

Datasets failing validation shall not proceed to the Dashboard Model Construction Layer.

12.5.17 Failure Handling

If aggregation fails:

- Dashboard processing shall terminate
- Source datasets shall remain unchanged
- Partial aggregation results shall not be published
- Failure metadata shall be generated
- Diagnostic information shall be recorded
- Validation status shall indicate failure

No automatic correction or modification of source information shall occur.

12.5.18 Layer Characteristics

The Dashboard Engine Data Aggregation Layer is:

- Read-only
- Deterministic
- Immutable
- Schema-driven
- Modular
- Traceable
- Version-controlled
- Validation-aware
- Enterprise compliant
- Non-destructive

12.5.19 Architectural Constraints

The Dashboard Engine Data Aggregation Layer shall never:

- Modify enterprise datasets
- Recalculate business metrics
- Execute business logic
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Generate dashboard layouts
- Package dashboard artifacts
- Alter validation results

12.5.20 Layer Summary

The Dashboard Engine Data Aggregation Layer establishes the enterprise data acquisition foundation of the Dashboard Engine by collecting validated outputs from all supported upstream engines into a unified dashboard dataset. It preserves complete source integrity, deterministic ordering, schema consistency, immutable processing, and end-to-end traceability while preparing validated enterprise information for dashboard model construction.

DOCUMENT STATUS

Phase 12.5 — Dashboard Engine Data Aggregation Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.6 — Dashboard Engine Widget Assembly Layer

END OF DOCUMENT

PART 2B-7 — Phase 12.6

Dashboard Engine Widget Assembly Layer

12.6.1 Purpose

The Dashboard Engine Widget Assembly Layer is responsible for transforming standardized dashboard models into reusable, standardized dashboard widget structures.

This layer constructs widget definitions, widget collections, and widget hierarchies that are consumed by the Dashboard Layout Assembly Layer.

The Widget Assembly Layer performs widget construction only.

It does not modify enterprise information, execute business logic, perform AI inference, or alter upstream dashboard models.

12.6.2 Objectives

The Dashboard Engine Widget Assembly Layer is designed to achieve the following objectives:

- Standardized widget construction
- Consistent dashboard visualization
- Deterministic processing
- Read-only execution
- Reusable widget generation
- Widget hierarchy organization
- Enterprise scalability
- Complete traceability
- Schema consistency
- Audit readiness

12.6.3 Layer Position

The Widget Assembly Layer executes after successful completion of the Dashboard Model Construction Layer.

Execution Flow

Dashboard Model Construction Layer

↓

Widget Assembly Layer

↓

Dashboard Layout Assembly Layer

12.6.4 Responsibilities

The Widget Assembly Layer is responsible for:

- Reading standardized dashboard models
- Constructing widget definitions
- Organizing widget collections
- Building widget hierarchy
- Associating visualization metadata
- Preserving dashboard consistency
- Preparing widget packages
- Generating widget metadata
- Validating widget assemblies
- Producing standardized widget collections

12.6.5 Input Sources

The Widget Assembly Layer consumes:

- Standardized dashboard models

- Source reference mappings
- Validation metadata
- Schema metadata
- Processing metadata

Only validated dashboard models shall be processed.

12.6.6 Widget Categories

The Widget Assembly Layer supports standardized widget categories including:

- Summary Widgets
- KPI Widgets
- Metric Widgets
- Chart Widgets
- Table Widgets
- Pipeline Widgets
- CRM Widgets
- Campaign Widgets
- Activity Widgets
- Status Widgets
- Audit Widgets
- System Widgets

Each widget category shall remain independently identifiable within the widget hierarchy.

12.6.7 Widget Assembly Workflow

The Widget Assembly Layer executes the following sequence:

Widget Initialization

↓

Dashboard Model Analysis

↓

Widget Definition Generation

↓

Widget Classification

↓

Widget Organization

↓

Widget Validation

↓

Widget Metadata Generation

↓

Widget Assembly Package

12.6.8 Widget Initialization

Purpose

Initialize widget assembly for the current dashboard generation request.

Activities

- Register widget assembly request
- Associate dashboard identifier
- Initialize widget context
- Associate processing metadata
- Associate schema references

Output

Widget Assembly Context

12.6.9 Dashboard Model Analysis

Purpose

Analyze standardized dashboard models to identify required widget structures.

Activities

- Read dashboard model
- Identify dashboard sections
- Identify visualization requirements
- Preserve source references
- Register widget candidates

Output

Widget Source Inventory

12.6.10 Widget Definition Generation

Purpose

Generate standardized widget definitions.

Activities

- Build widget definitions
- Associate widget identifiers
- Generate widget descriptors
- Preserve visualization references
- Build reusable widget structures

Output

Widget Definitions

12.6.11 Widget Classification

Purpose

Classify widgets into standardized enterprise categories.

Activities

- Assign widget categories
- Organize widget groups
- Preserve category hierarchy
- Associate processing references
- Preserve dashboard consistency

Output

Classified Widget Collection

12.6.12 Widget Organization

Purpose

Organize classified widgets into standardized dashboard collections.

Activities

- Build widget hierarchy
- Organize widget collections
- Preserve deterministic ordering
- Associate visualization metadata
- Prepare widget package

Output

Widget Collection

12.6.13 Widget Validation

Purpose

Verify structural integrity and completeness of the widget collection.

Activities

- Validate widget definitions
- Validate hierarchy

- Verify schema compatibility
- Verify metadata references
- Confirm widget completeness

Output

Validated Widget Collection

12.6.14 Widget Metadata Generation

Purpose

Generate standardized metadata describing assembled widgets.

Activities

- Generate widget identifiers
- Generate widget metadata
- Generate processing references
- Generate schema metadata
- Generate lineage metadata
- Generate timestamps

Output

Widget Metadata

12.6.15 Widget Assembly Package

The completed widget assembly package shall contain:

- Widget definitions
- Widget collections
- Widget hierarchy
- Widget metadata
- Validation information
- Processing metadata
- Schema references
- Lineage information

12.6.16 Validation Rules

The widget assembly package shall satisfy all of the following requirements:

- Mandatory widget definitions present
- Widget hierarchy verified
- Category assignments complete
- Schema compliance confirmed
- Metadata attached
- Processing sequence recorded
- Traceability preserved
- Validation completed successfully

Widget assemblies failing validation shall not proceed to the Dashboard Layout Assembly Layer.

12.6.17 Traceability Requirements

The Widget Assembly Layer shall preserve complete lineage for every generated widget, including:

- Source engine references
- Source dataset references
- Dashboard model reference
- Widget identifier
- Widget assembly identifier
- Processing sequence
- Schema version
- Validation status

Traceability information shall remain immutable throughout the dashboard lifecycle.

12.6.18 Failure Handling

If widget assembly fails:

- Dashboard processing shall terminate
- Dashboard models shall remain unchanged
- Partial widget collections shall not be published

- Diagnostic metadata shall be generated
- Validation status shall indicate failure
- Source enterprise information shall remain immutable

No automatic modification of dashboard models shall occur.

12.6.19 Architectural Constraints

The Widget Assembly Layer shall never:

- Modify dashboard models
- Modify enterprise information
- Execute business logic
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Assemble dashboard layouts
- Package dashboard artifacts
- Alter upstream validation results

12.6.20 Layer Summary

The Dashboard Engine Widget Assembly Layer establishes the reusable visualization foundation of the Dashboard Engine by transforming standardized dashboard models into validated widget definitions, organized widget collections, and structured widget hierarchies. It ensures deterministic execution, schema consistency, complete traceability, immutable processing, and enterprise-standard visualization preparation for the Dashboard Layout Assembly Layer.

DOCUMENT STATUS

Phase 12.6 — Dashboard Engine Widget Assembly Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.7 — Dashboard Engine Layout Assembly Layer

END OF DOCUMENT

PART 2B-7 — Phase 12.7

Dashboard Engine Layout Assembly Layer

12.7.1 Purpose

The Dashboard Engine Layout Assembly Layer is responsible for transforming validated widget collections into standardized enterprise dashboard layouts.

This layer constructs the complete structural organization of dashboard artifacts by arranging validated widget assemblies into predefined layout models suitable for enterprise presentation systems.

The Layout Assembly Layer performs layout construction only.

It does not modify enterprise information, reporting artifacts, dashboard models, widget definitions, or upstream processing outputs.

12.7.2 Objectives

The Dashboard Engine Layout Assembly Layer is designed to achieve the following objectives:

- Standardized dashboard layout construction
- Consistent widget organization
- Deterministic processing
- Read-only execution
- Enterprise layout consistency
- Reusable layout structures
- Schema-driven organization
- Complete traceability
- Validation readiness
- Audit compatibility

12.7.3 Layer Position

The Dashboard Engine Layout Assembly Layer executes immediately after successful completion of the Widget Assembly Layer.

Execution Flow

Widget Assembly Layer

↓

Dashboard Layout Assembly Layer

↓

Dashboard Validation Layer

12.7.4 Responsibilities

The Dashboard Engine Layout Assembly Layer is responsible for:

- Reading validated widget collections
- Constructing dashboard layouts
- Organizing widget placement
- Building layout hierarchy
- Preserving widget relationships
- Maintaining layout consistency
- Preparing dashboard presentation structures
- Generating layout metadata
- Validating layout integrity
- Producing standardized dashboard layouts

12.7.5 Input Sources

The Dashboard Engine Layout Assembly Layer consumes:

- Validated widget collections
- Widget metadata
- Source reference mappings
- Validation metadata
- Schema metadata
- Processing metadata

Only validated widget assemblies shall be processed.

12.7.6 Layout Components

Every dashboard layout shall consist of the following standardized components:

- Dashboard Header
- Navigation Area
- Summary Section
- Primary Widget Area
- Secondary Widget Area
- Detail Section
- Status Section
- Footer Area
- Metadata References
- Validation References

Each component shall remain independently identifiable within the dashboard hierarchy.

12.7.7 Layout Assembly Workflow

The Dashboard Engine Layout Assembly Layer executes the following sequence:

Layout Initialization

↓

Widget Collection Analysis

↓

Layout Structure Generation

↓

Widget Placement

↓

Layout Organization

↓

Layout Validation

↓

Layout Metadata Generation

↓

Dashboard Layout Package

12.7.8 Layout Initialization

Purpose

Initialize dashboard layout construction.

Activities

- Register layout request
- Associate dashboard identifier
- Initialize layout context
- Associate processing metadata
- Associate schema references

Output

Layout Context

12.7.9 Widget Collection Analysis

Purpose

Analyze validated widget collections required for dashboard construction.

Activities

- Read widget collections
- Identify widget hierarchy
- Verify widget dependencies
- Preserve visualization references
- Register layout components

Output

Layout Component Inventory

12.7.10 Layout Structure Generation

Purpose

Generate standardized dashboard layout structures.

Activities

- Build layout hierarchy
- Define layout sections
- Associate structural identifiers
- Preserve deterministic organization
- Prepare presentation framework

Output

Dashboard Layout Structure

12.7.11 Widget Placement

Purpose

Organize validated widgets within the standardized dashboard layout.

Activities

- Position widgets
- Associate layout regions
- Preserve widget hierarchy
- Preserve dashboard consistency

- Record placement references

Output

Positioned Dashboard Layout

12.7.12 Layout Organization

Purpose

Construct the finalized dashboard presentation hierarchy.

Activities

- Organize layout components
- Maintain section ordering
- Preserve layout integrity
- Associate processing references
- Finalize dashboard organization

Output

Organized Dashboard Layout

12.7.13 Layout Validation

Purpose

Verify the integrity and completeness of the dashboard layout.

Activities

- Validate layout hierarchy
- Verify widget placement
- Verify structural consistency
- Verify schema compatibility
- Confirm layout completeness

Output

Validated Dashboard Layout

12.7.14 Layout Metadata Generation

Purpose

Generate metadata describing the completed dashboard layout.

Activities

- Generate layout identifier
- Generate layout metadata
- Generate processing references
- Generate schema metadata
- Generate lineage metadata
- Generate timestamps

Output

Layout Metadata

12.7.15 Dashboard Layout Package

The completed dashboard layout package shall contain:

- Dashboard layout
- Layout hierarchy
- Positioned widget collections
- Layout metadata
- Validation information
- Processing metadata
- Schema references
- Lineage information

12.7.16 Validation Rules

The dashboard layout package shall satisfy all of the following requirements:

- Mandatory layout components present
- Widget placement verified
- Layout hierarchy validated
- Structural integrity confirmed
- Schema compliance verified
- Metadata attached
- Processing sequence recorded
- Traceability preserved

Dashboard layouts failing validation shall not proceed to the Dashboard Validation Layer.

12.7.17 Traceability Requirements

The Dashboard Engine Layout Assembly Layer shall preserve complete lineage for every dashboard layout, including:

- Source engine references
- Source dataset references
- Dashboard model reference
- Widget assembly reference
- Layout identifier
- Processing sequence
- Schema version
- Validation status

Traceability information shall remain immutable throughout the dashboard lifecycle.

12.7.18 Failure Handling

If layout assembly fails:

- Dashboard processing shall terminate
- Widget collections shall remain unchanged
- Partial layouts shall not be published
- Diagnostic metadata shall be generated
- Validation status shall indicate failure
- Source enterprise information shall remain immutable

No automatic modification of widget collections or dashboard models shall occur.

12.7.19 Architectural Constraints

The Dashboard Engine Layout Assembly Layer shall never:

- Modify widget collections
- Modify dashboard models
- Modify enterprise information
- Execute business logic
- Perform AI inference
- Trigger workflows
- Execute CRM operations
- Package dashboard artifacts
- Alter upstream validation results

12.7.20 Layer Summary

The Dashboard Engine Layout Assembly Layer establishes the structural presentation foundation of the Dashboard Engine by transforming validated widget collections into standardized enterprise dashboard layouts. It ensures deterministic execution, schema consistency, immutable processing, complete traceability, and enterprise-standard layout organization while preparing validated dashboard layouts for the Dashboard Validation Layer.

DOCUMENT STATUS

Phase 12.7 — Dashboard Engine Layout Assembly Layer

Status: COMPLETED

Locked: Pending User Approval

Next Document (LOCKED):

PART 2B-7 — Phase 12.8 — Dashboard Engine Validation Layer

END OF DOCUMENT

